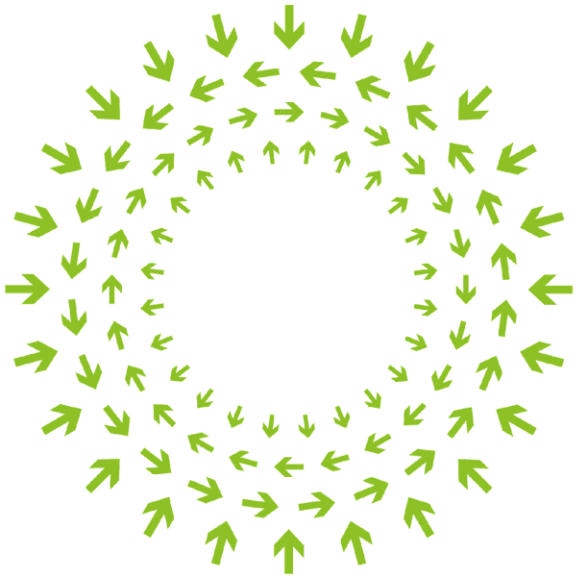


OPEN Compute Project

HPM AND PERIPHERAL PROGRAMMABLE DEVICE (PD) DESIGN SPECIFICATION

Part of the

Modular Hardware Systems (MHS) Rev 1.0 Family

Version 0.7.0 RC1

10/07/2026

M-PNP HPM And Peripheral Programmable Device (PD) Design Specification Authors/Contributors:

Advanced Micro Devices, Inc: Justin King, Duk Kim

Dell, Inc.: Tim Lambert, Jeff Kennedy

Hewlett Packard Enterprise Company: Edgar Campos, David Heinrich, Chuck Hartman, Andrew Klein, Phillip Leech, Jhovel Lopez, Christopher McGuinness, Bill Scherer

Intel Corporation: Nilan Nadoo, Kasper Wszolek

Jabil, Inc.: Todd Rosedahl

Lattice Semiconductor Corporation: Munir Ahmad

Table of Contents

1. License 22
1.1. Open Web Foundation (OWF) CLA 22
1.2. Acknowledgements 23
2. Version Table 23
3. Overview 28
4. MHS Family of Specifications 28
4.1. Related OCP Specifications 30
4.2. Related Industry Specifications 30
5. Terminology 31
5.1. Standards Terminology 31
5.2. M-PNP Specific Standards Terminology 31
6. HPM and Peripheral Programmable Device (PD) Interoperability Vision 32
7. Scope 34
7.1. Typical OCP Sections Not Applicable 34
7.2. In-Scope 34
7.3. Out-Scope 35
7.4. Supplementary Materials 35
8. MHS HPM Primary Programmable Device (PD) Interfaces and Logical IP Blocks Overview 37
8.1. MHS HPM Primary Programmable Device (PD) Common Interfaces 37
8.2. MHS HPM Programmable Device (PD) Logical IP Blocks List 40
8.3. Vendor Defined IP Blocks General Guidance 42
9. Programmable Device (PD) Memory Map Architecture and Requirements 43
9.1. Programmable Device (PD) Memory Map Access Methods 43
9.2. Programmable Device (PD) Memory Map Architecture 45
9.2.1. Memory Map Directory 46
9.3. Programmable Device (PD) Memory Map Access 47

9.3.1. Memory Map Access Interface: LTPI 48

9.3.2. Memory Map Access Interface: SPI 49

9.3.3. Memory Map Access Interface: I2C / I3C 50

9.3.4. Memory Map Access Contention Considerations 50

10. Programmable Device (PD) JSON Schema Representation 51

10.1. Programmable Device (PD) JSON Schema Representation Overview 51

10.2. JSON Schema Reference Documentation 53

11. Programmable Device Internal Signal Virtual Wires Support 53

11.1. Programmable Device Internal Signal Virtual Wire Mapping Framework 54

11.2. Programmable Device Internal Signal Virtual Wire Mapping Description Overview 55

11.3. Programmable Device Internal Signal Virtual Wire Self-Describing JSON Mapping Framework 56

11.4. Programmable Device Internal Signal Virtual Wire Self-Describing JSON Examples 57

11.5. Required Programmable Device Internal Signal Virtual Wires List 57

11.6. PDHPM Primary Programmable Device Internal Signal Virtual Wire Supplementary Materials 58

12. Programmable Device Logical IP Block Descriptions and Requirements 58

12.1. Logical IP Blocks Abstraction 58

12.1.1. Common Required Memory Mapped CSRs 59

12.1.1.1. Required and Optional IP Block CSRs List 60

12.1.1.2. Common IP Block CSR: IP Block ID 60

12.1.1.3. Common IP Block CSR: Version 60

12.1.1.4. Common IP Block CSR: Status 61

12.1.1.5. Common IP Block CSR: Control 62

12.1.1.6. Common IP Block CSR: Interrupt Status 62

12.1.1.7. Common IP Block CSR: Interrupt Enable 62

12.1.1.8. Common IP Block CSR: Extended OEM Defined Status Register 63

12.2. Power Sequencing and Management Logical IP Block 63

12.2.1. Power Sequencing Conceptual Overview 63
12.2.2. System Power States Definition 64
12.2.3. System Power States Transition Diagram 65
12.2.3.1. Power State Transition Description 66
12.2.3.2. Power State Transition vs. Power and Reset States by Component 68
12.2.4. Power Sequencing and Management IP Block Architecture 70
12.2.5. Power Sequencing and Management IP Block Requirements 71
12.2.5.1. Required and Optional Feature Set 71
12.2.5.2. BMC Visible Power Management IP Block Inputs 71
12.2.5.3. BMC Visible Power Management IP Block Outputs 73
12.2.6. Power Sequencing and Management CSRs Memory Map 73
12.2.6.1. Power Management CSR Region Overview 73
12.2.6.2. Power Management IP Block Region 0 Common Registers Memory Map 74
12.2.6.2.1. Power Management IP Block Common Registers Subregion 0 Description 75
12.2.6.2.2. Power Management IP Block Common Registers Subregion 1 Description 76
12.2.6.3. Power Management IP Block Region 1 CSR Memory Map 76
12.2.6.3.1. Power Management IP Block Region 1 Subregion 0 Register List 77
12.2.6.3.2. Power Management IP Block Region 1 Register Offset 0x00: General Power Control 78
12.2.6.3.3. Power Management IP Block Region 1 Register Offset 0x04: General Power Status 80
12.2.6.3.4. Power Management IP Block Region 1 Register Offset 0x08: Power Error Status 81
12.2.6.3.5. Power Management IP Block Region 1 Register Offset 0x0C: Power Management State Machine Error State 82
12.2.6.3.6. Power Management IP Block Region 1 Register Offset 0x10: Aggregated Power Good and Reset State 83
12.2.6.3.7. Power Management IP Block Region 1 Register Offset 0x14: VR Power Good State 84

12.2.6.3.8. Power Management IP Block Region 1 Register Offset 0x18: VR Power Failure State 84

12.2.6.3.9. Power Management IP Block Region 1 Register Offset 0x1C: SoC Presence 85

12.2.6.3.10. Power Management IP Block Region 1 Register Offset 0x20: SoC THERMTRIP Signal Current State 86

12.2.6.3.11. Power Management IP Block Region 1 Register Offset 0x24: SoC THERMTRIP LATCH 87

12.2.6.3.12. Power Management IP Block Region 1 Register Offset 0x28: SoC POWER_THROTTLE 87

12.2.6.4. Power Management CSRs Region 1 Sub-region 1 Vendor Extension CSR Definition 88

12.2.6.5. Power Management CSRs Region 2 Memory Map 88

12.2.7. Platform Silicon Vendor Specific Power Sequencing and Management Support 88

12.2.8. Example Implementations 88

12.3. System / Device Reset Control Logical IP Block 89

12.3.1. System / Device Reset Control Conceptual Overview 89

12.3.2. System / Device Reset Control IP Block Architecture 89

12.3.3. System / Device Reset Control IP Block Requirements 90

12.3.3.1. Required and Optional Feature Set 90

12.3.3.2. Required and Optional Inputs 90

12.3.3.3. Required and Optional Outputs 91

12.3.4. System / Device Reset Control CSRs Requirements 91

12.3.4.1. System Management IP Block CSR Region Map Overview 92

12.3.4.2. Region 0: System / Device Reset Control IP Block Common Registers 92

12.3.4.2.1. Region 0 Subregion 0 Memory Map 93

12.3.4.2.2. Region 0 Subregion 1 Memory Map 94

12.3.4.3. Region 1 System / Device Reset Control IP Block Required CSR Memory Map 94

12.3.4.3.1. Region 1 Subregion 0 Memory Map - MXIO PCIe Reset Enable CSRs 95

12.3.4.3.2. System Management PCIe Reset Control Register Offset 0x00: PCIe Reset Enable Logical MXIO-0 to MXIO-7 95

12.3.4.3.3. System Management PCIe Reset Control Register Offset 0x01: PCIe Reset Enable Logical MXIO-8 to MXIO-15 96

12.3.4.3.4. System Management PCIe Reset Control Register Offset 0x02 – N: PCIe Reset Enable Logical MXIO-16 to MXIO-N 97

12.3.4.3.5. Region 1 Subregion 1 Memory Map– OCP PCIe Reset Enable CSRs 97

12.3.4.3.6. OCP Slot PCIe Reset Control Register Offset 0x00: PCIe Reset Enable Logical OCP-0 to OCP-7 98

12.3.4.3.7. OCP Slot PCIe Reset Control Register Offset 0x01 - N: PCIe Reset Enable Logical OCP-8 to OCP-N 99

12.3.4.4. Region 1 Subregion 2 Memory Map – HPM On-board Device PCIe Reset Enable CSRs 99

12.3.4.4.1. HPM_DEVICE Slot PCIe Reset Control Register Offset 0x00: PCIe Reset Enable Logical HPM_DEVICE -0 to HPM_DEVICE -7 100

12.3.4.4.2. HPM Device PCIe Reset Control Register Offset 0x01 - N: PCIe Reset Enable Logical HPM_DEVICE -8 to HPM_DEVICE - N 101

12.3.4.5. Region 1 Subregion 3 Memory Map – Reserved 101

12.3.4.6. Region 2: OEM CSR Memory Map 101

12.3.5. Vendor Specific Extensions Support 101

12.3.6. Example Implementation 101

12.4. LTPI Standardized Logical IP Block 102

12.4.1. LTPI Interface Conceptual Overview 102

12.4.2. LTPI Interface IP Block Architecture 103

12.4.3. LTPI Interface IP Block Requirements 104

12.4.3.1. Required and Optional Feature Set 104

12.4.3.2. LTPI Error Handling and Failure Reporting 104

12.4.3.3. LTPI GPIO Channel Tunneled GPIOs List 104

12.4.3.3.1. LTPI Data Channel Tunneled GPIOs Mapping PD Internal Virtual Wires Considerations 106

12.4.4. LTPI Interface Control and Status Registers Memory Map 107

12.4.4.1. LTPI CSR Region Overview 107

12.4.4.2. LTPI IP Block Region 0 Common Registers Memory Map 108
12.4.4.2.1. LTPI IP Block Common Registers Sub-Region 0 Description 109
12.4.4.2.2. LTPI IP Block Common Registers Sub-Region 1 Description 110
12.4.4.3. LTPI IP Block Region 1: CPLD/PD Information CSR Memory Map 110
12.4.4.3.1. LTPI Region 1 Sub-region 0 CSR Memory Map 111
12.4.4.3.2. LTPI Region 1 Sub-region 1 CSR Memory Map 111
12.4.4.4. LTPI IP Block Region 2 CSR Memory Map 111
12.4.4.5. LTPI IP Block Region 3 CSR Memory Map 111
12.4.4.6. LTPI IP Block Region 4 CSR Memory Map 112
12.4.5. LTPI Interface Data Channel Tunneling Requirements 112
12.4.6. LTPI Interface Bus Tunneling Requirements 113
12.4.7. LTPI Interface Vendor Specific Extensions Support 113
12.4.8. LTPI Interface Example Implementation 113
12.5. SGPIO Logical IP Block 113
12.5.1. SGPIO Interface Conceptual Overview 114
12.5.2. SGPIO Interface Operation Phases 116
12.5.2.1. SGPIO Capabilities Discovery Phase 117
12.5.2.2. SGPIO Handshake Phase 118
12.5.2.3. SGPIO Interface Operational Phase 121
12.5.3. SGPIO Virtual Wire Packet Format 121
12.5.4. SGPIO CRC and Error Handling 122
12.5.4.1. SGPIO CRC Error Handling 122
12.5.5. SGPIO Interface IP Block Architecture 123
12.5.6. SGPIO Interface IP Block Requirements 124
12.5.6.1. Required and Optional Feature Set 124
12.5.7. SGPIO Interface Control and Status Registers Requirements 124
12.5.7.1. SGPIO CSR Region Overview 124
12.5.7.2. SGPIO IP Block Region 0 Common Registers Memory Map 125
12.5.7.2.1. SGPIO IP Block Common Registers Sub-Region 0 Description 126

12.5.7.2.2. SGPIO IP Block Common Registers Sub-Region 1 Description 127
12.5.7.3. SGPIO IP Block Region 1 CSR Memory Map 127
12.5.7.3.1. SGPIO IP Block Region 1 Registers Sub-Region 0 Description 128
12.5.7.3.2. SGPIO IP Block Region 1 Registers Sub-Region 1 Description 130
12.5.7.4. SGPIO IP Block Region 2 CSR Memory Map 130
12.5.8. SGPIO Tunneled PD Virtual Wires Set Requirements 130
12.5.8.1. SCM To HPM SGPIO Tunneled GPIOs List 130
12.5.8.1.1. SCM To HPM SGPIO Tunneled GPIOs Mapping PD Internal Virtual Wires Considerations 132
12.5.8.2. SGPIO HPM To PDB Required and Optional GPIOs List 133
12.5.8.2.1. HPM To PDB SGPIO Tunneled GPIOs PD Internal Virtual Wires Mapping Considerations 134
12.5.9. SGPIO Interface Vendor Specific Extensions Support 135
12.5.10. SGPIO Interface Example Implementation 136
12.6. DC-SCM to HPM PD SPI Interface IP Logical Block 136
12.6.1. SPI Interface Conceptual Overview 136
12.6.2. SPI Interface IP Block Architecture 137
12.6.3. SPI Interface IP Block Requirements 138
12.6.3.1. SPI Required and Optional Feature Set 138
12.6.3.2. SPI Required and Optional SPI Command Support 138
12.6.3.3. SPI Command Timing Diagrams 139
12.6.4. SPI Interface Control and Status Registers Memory Map 142
12.6.4.1. SPI CSR Region Overview 142
12.6.4.2. SPI IP Block Region 0 Common Registers Memory Map 143
12.6.4.2.1. SPI IP Block Common Registers Sub-Region 0 Description 143
12.6.4.2.2. SPI IP Block Common Registers Sub-Region 1 Description 145
12.6.4.3. SPI IP Block Region 1 CSR Memory Map 145
12.6.5. SPI Interface Vendor Specific Extensions Support 145
12.6.6. SPI Interface Example Implementation 145
12.7. DC-SCM to HPM PD I2C / I3C Basic Interface IP Logical Block 145

12.7.1. I2C / I3C Basic Interface Conceptual Overview 146
12.7.2. I2C / I3C Basic Interface IP Block Architecture 147
12.7.3. I2C / I3C Basic Interface IP Block Requirements 148
12.7.3.1. I2C / I3C Required and Optional Feature Set 148
12.7.3.2. Multi-function I2C / I3C Endpoint Considerations 148
12.7.3.3. I2C Command Timing Diagrams 149
12.7.3.4. I3C Basic Considerations 150
12.7.4. I2C / I3C Basic Interface Control and Status Registers Memory Map 151
12.7.4.1. I2C / I3C Basic CSR Region Overview 151
12.7.4.2. I2C / I3C Basic IP Block Region 0 Common Registers Memory Map 152
12.7.4.2.1. I2C / I3C Basic IP Block Common Registers Sub-Region 0 Description 152
12.7.4.2.2. I2C / I3C Basic IP Block Common Registers Sub-Region 1 Description 154
12.7.4.3. I2C / I3C Basic IP Block Region 1 CSR Memory Map 154
12.7.5. I2C / I3C Basic Interface Vendor Specific Extensions Support 154
12.7.6. I2C / I3C Basic Interface Example Implementation 154
12.8. M-PESTI Interface Logical IP Block 155
12.8.1. M-PESTI Interface Conceptual Overview 155
12.8.2. M-PESTI M-PNP Specific Terminology 155
12.8.3. M-PESTI Interface IP Block Architecture 157
12.8.3.1. M-PESTI Initiator Instance IP Block Implementation Variations 158
12.8.4. M-PESTI Discovery Considerations 160
12.8.5. M-PESTI Interface IP Block Requirements 161
12.8.5.1. M-PESTI Required and Optional Feature Set 161
12.8.6. M-PESTI Control and Status Registers Requirements 161
12.8.6.1. M-PESTI IP BLOCK CSR Region Map Overview 162
12.8.6.2. M-PESTI IP Block Global Registers Section 163
12.8.6.2.1. M-PESTI IP Block Global Required Registers 163
12.8.6.2.2. M-PESTI IP Block Global Required Registers 164
12.8.7. M-PESTI Per Target Required Registers 166

12.8.7.1. M-PESTI Per Target Required Register Set Overview 168
12.8.7.2. M-PESTI Per Target Required Register Set Description 168
12.8.7.2.1. M-PESTI Per Target Register 0x00: Target Error and Status Register 168
12.8.7.2.2. M-PESTI Per Target Register 0x02: Target Control Register 170
12.8.7.2.3. M-PESTI Per Target Register 0x03: Target Configuration Register 171
12.8.8. M-PESTI Per Target Optional Registers 172
12.8.9. M-PESTI Discovery Payload Register Region 173
12.8.9.1. Non-Destructive Per Target Discovery Mailbox Window 173
12.8.9.1.1. Destructive Per Target Discovery Mailbox Window 174
12.8.10. M-PESTI Per Target Virtual Wires Register Region 175
12.8.11. PD M-PESTI JSON File Considerations For CSR Description 177
12.8.12. M-PESTI Interface Vendor Specific Extensions Support 178
12.8.13. M-PESTI Interface Example Implementation 178
12.9. LED Control Logical IP Block 178
12.9.1. LED Control Conceptual Overview 178
12.9.2. LED Control IP Block Architecture 179
12.9.3. LED Control IP Block Requirements 179
12.9.3.1. Required and Optional Feature Set 180
12.9.4. LED Control IP Block CSR Requirements 180
12.9.4.1. LED Control CSR Region Overview 180
12.9.4.2. LED Control IP Block Region 0 Common Registers Memory Map 181
12.9.4.2.1. LED Control IP Block Common Registers Sub-Region 0 Description 182
12.9.4.2.2. LED Control IP Block Common Registers Sub-Region 1 Description 183
12.9.4.3. LED Control IP Block Region 1 CSR Memory Map 183
12.9.4.3.1. LED Control CSRs Region 1 Sub-Region 0 Description 184
12.9.4.3.2. LED Control IP Block CSRs Region 1 Sub-Region 1 Description 185
12.9.4.4. LED Control IP Block Region 2 CSR Memory Map 185
12.9.4.5. LED Control IP Block Region 3 CSR Memory Map 185
12.9.5. LED Control Vendor Specific Extensions Support 185

12.9.6. LED Control Example Implementation 186

12.10. M-XIO FLEX IO and M-PIC Sideband Control Logical IP Block 186

12.10.1. M-XIO FLEX IO and M-PIC PICPWR Sideband Control Conceptual Overview 186

12.10.1.1. M-XIO FLEX IO and M-PIC PICPWR Sideband Control Scope 187

12.10.2. M-XIO FLEX IO and M-PIC PICPWR Sideband Control IP Block Architecture 187

12.10.2.1. M-XIO Flex IO Architectural Block Diagram 187

12.10.2.2. M-XIO Flex IO Alternate Functions List 188

12.10.2.3. M-PIC PICPWR Sideband Architectural Block Diagram 189

12.10.2.4. M-PIC PICPWR Sideband Alternate Functions List 190

12.10.3. M-XIO FLEX IO and M-PIC PICPWR Sideband Control IP Block Requirements 191

12.10.3.1. M-XIO FLEX IO and M-PIC PICPWR Sideband Control IP Block Required and Optional Feature Set 192

12.10.4. M-XIO FLEX IO and M-PIC PICPWR Sideband Control IP Block CSR Requirements 192

12.10.4.1. FlexIO and PICPWR IP BLOCK CSR Region Map Overview 192

12.10.4.2. Region 0: FlexIO / Sideband IP Block Common Registers 194

12.10.4.2.1. Region 0: FlexIO / Sideband IP Block Subregion 0 Memory Map 194

12.10.4.2.2. Region 0: FlexIO / Sideband IP Block Subregion 1 Memory Map 195

12.10.4.2.3. Region 0: FlexIO / Sideband IP Block Subregion 2 Memory Map 196

12.10.4.3. Region 1: FlexIO Per M-XIO Target Memory Map 196

12.10.4.3.1. FlexIO Per M-XIO Target CSRs 197

12.10.4.3.2. FlexIO Register Offset 0x00: General GPIO Status Register 198

12.10.4.3.3. FlexIO Register Offset 0x01: General GPIO Control 199

12.10.4.3.4. FlexIO Register Offset 0x02: Alternate Function Configuration 200

12.10.4.3.5. FlexIO Register 0x03: Reserved 201

12.10.4.4. Region 0: FlexIO / Sideband IP Block Region 2: M-PIC Per PICPWR Sideband CSR Memory Map 201

12.10.4.4.1. PICPWR Sideband Per M-PIC Connector 2x3 Register Definition 202

12.10.4.4.2. PICPWR Sideband Register Offset 0x00: General GPIO Status 203
12.10.4.4.3. PICPWR Sideband Register Offset 0x01: General GPIO Control 204
12.10.4.4.4. PICPWR Side Band Register 0x02: Alternate Function Configuration 205
12.10.4.4.5. PICPWR Sideband 0x03: Reserved 206
12.10.4.5. Region 3: Control Panel Sideband CSR Memory Map 206
12.10.4.5.1. Control Panel Sideband Register Definition 207
12.10.4.5.2. Control Panel Sideband Register Offset 0x00: General GPIO Status 208
12.10.4.5.3. Control Panel Sideband Register Offset 0x01: General GPIO Control 209
12.10.4.5.4. Control Panel Sideband Register 0x02: Alternate Function Configuration 210
12.10.4.5.5. Control Panel Sideband 0x03: Reserved 211
12.10.5. M-XIO FLEX IO and M-PIC PICPWR Sideband Control Vendor Specific Extensions Support 211
12.11. MUX Control Logical IP Block 212
12.11.1. MUX Control Conceptual Overview 212
12.11.2. MUX Control Bus Target Types 212
12.11.3. MUX Control IP Block Architecture 213
12.11.4. MUX Control IP Block Requirements 214
12.11.4.1. Required and Optional Feature Set 214
12.11.5. MUX Control CSRs 215
12.11.5.1. MUX Control CSR Region Map Overview 215
12.11.5.2. Region 0: MUX Control IP Block Common Registers 216
12.11.5.2.1. Region 0: Sub-region 0 Memory Map 216
12.11.5.2.2. Region 0: Sub-region 1 Memory Map 217
12.11.5.3. Region 1: MUX Control Per I2C Bus MUX Target Memory Map 217
12.11.5.4. Mux Control Register Offset 0x00: General MUX Channel Control 217
12.11.6. MUX Control Vendor Specific Extensions Support 219
12.11.7. MUX Control Example Implementation 219
12.12. Cooling Subsystem Control Logical IP Block 219
12.12.1. Cooling Subsystem Control Conceptual Overview 220

12.12.2. Cooling Subsystem Control IP Block Architecture 221
12.12.3. Cooling Subsystem Control IP Block Requirements 222
12.12.3.1. Required and Optional Feature Set 222
12.12.3.2. Required Inputs 223
12.12.3.3. Required Outputs 223
12.12.3.4. Required Bus Support 223
12.12.4. Cooling Subsystem Control IP Block CSR Requirements 224
12.12.4.1. Cooling Control CSR Region Overview 224
12.12.4.2. Cooling Control IP Block Region 0 Common Registers Memory Map 225
12.12.4.2.1. Cooling Control IP Block Common Registers Sub-Region 0 Description 225
12.12.4.2.1.1. Register Offset 0x7: FAN_WATCHDOG_CONTROL 227
12.12.4.2.1.2. Register Offset 0x8: FAN_WATCHDOG_STATUS 228
12.12.4.2.2. Cooling Control IP Block Common Registers Sub-Region 1 Description 228
12.12.4.3. Cooling Control IP Block Region 1 CSR Memory Map 228
12.12.4.4. Cooling Control CSRs Region 1 Sub-Region 0 Description 229
12.12.4.5. Cooling Control IP Block CSRs Region 1 Sub-Region 0 Register Definitions 230
12.12.4.5.1. Register Offset 0x101: FAN_CAPABILITIES 231
12.12.4.5.2. Register Offset 0x110+16n: FANn_CONTROL 231
12.12.4.5.3. Register Offset 0x111+16n: FANn_STATUS 232
12.12.4.5.4. Cooling Modes 232
12.12.4.6. Cooling Control IP Block CSRs Region 1 Sub-Region 1 Description 233
12.12.4.7. Cooling Control IP Block CSRs Region 1 Sub-Region 1 Register Definitions 236
12.12.4.7.1. Register Offset 0x301: FAN_CONTROL 236
12.12.4.7.2. Register Offset 0x302: FAN_ STATUS 236
12.12.4.7.3. Register Offset 0x306: DBG_I2C_CONTROL 236
12.12.4.7.4. Register Offset 0x307: DBG_I2C_ STATUS 237

12.12.4.7.5. Register Offset 0x308: DBG_I2C_ ADDR_CMD 238
12.12.4.7.6. Register Offset 0x321+32n:FANn_CAPABILITIES 239
12.12.4.7.7. Register Offset 0x322+32n: FANn_CONTROL 239
12.12.4.7.8. Register Offset 0x323+32n: FANn_STATUS 239
12.12.4.8. Cooling Control Example Usage 240
12.12.4.9. Cooling Control IP Block CSRs Region 1 Sub-Region 2 Description 243
12.12.4.9.1. Register Offset 0x500: DLC_CAPABILITIES 244
12.12.4.9.2. Register Offset 0x502: DLC_GLOBAL_STATUS 244
12.12.4.9.3. Register Offset 0x504: DLC_GLOBAL_CONTROL 245
12.12.4.9.4. Register Offset 0x506: DLC_SHUTDOWN_MASK 246
12.12.4.9.5. Register Offset 0x508: DLC_FLOW_STATUS 246
12.12.4.9.6. Register Offset 0x50A: DLC_FLOW_CONFIG 246
12.12.4.9.7. Register Offset 0x510+n: DLC_LEAK_ZONE[n] 247
12.12.4.10. Cooling Control IP Block CSRs Region 1 Sub-Region 3 Description 248
12.12.4.11. Cooling Control IP Block Region 2 CSR Memory Map 248
12.12.4.12. Cooling Control IP Block Region 3 CSR Memory Map 248
12.12.5. Cooling Subsystem Control Vendor Specific Extensions Support 248
12.12.6. Cooling Subsystem Control Example Implementation 249
12.13. System Telemetry Collection Logical IP Block 249
12.13.1. System Telemetry Collection Conceptual Overview 249
12.13.2. System Telemetry Collection IP Block Architecture 250
12.13.3. System Telemetry Collection IP Block Requirements 251
12.13.3.1. Required and Optional Feature Set 251
12.13.4. System Telemetry Collection IP Block CSR Requirements 252
12.13.4.1. System Telemetry Collection CSR Region Overview 252
12.13.4.2. System Telemetry Collection IP Block Region 0 Common Registers Memory Map 253
12.13.4.2.1. System Telemetry Collection IP Block Common Registers Sub-Region 0 Description 254

12.13.4.2.2. System Telemetry Collection IP Block Common Registers Sub-Region 1 Description 255

12.13.4.3. System Telemetry Collection IP Block Region 1 CSR Memory Map 255

12.13.4.3.1. System Telemetry Collection CSRs Region 1 Sub-Region 0 Description 255

12.13.4.3.2. System Telemetry Collection IP Block CSRs Region 1 Sub-Region 1 Description 258

12.13.4.4. System Telemetry Collection IP Block Region 2 CSR Memory Map 258

12.13.4.5. System Telemetry Collection IP Block Region 3 CSR Memory Map 258

12.13.5. System Telemetry Collection Vendor Specific Extensions Support 258

12.13.6. System Telemetry Collection Example Implementation 258

12.14. General Use GPIOs IP Block 259

12.14.1. General Use GPIOs IP Block Overview 259

12.14.2. General Use PD GPIOs IP Block Architecture 259

12.14.3. General Use PD GPIOs IP Block Requirements 260

12.14.3.1. Required and Optional Feature Set 260

12.14.3.2. Special Purpose Signals Implemented Within GPIO IP Block 260

12.14.4. General Use PD GPIOs IP Block CSR Requirements 261

12.14.4.1. General Use PD GPIOs CSR Region Overview 261

12.14.4.2. General Use PD GPIOs IP Block Region 0 Common Registers Memory Map 262

12.14.4.2.1. General Use PD GPIOs IP Block Common Registers Region 0 Sub-Region 0 Description 262

12.14.4.2.2. General Use PD GPIOs IP Block Common Registers Region 0 Sub-Region 1 Description 263

12.14.4.3. General Use PD GPIOs IP Block Region 1 CSR Memory Map 263

12.14.4.3.1. General Use PD GPIOs CSRs Region 1 Sub-Region 0 Description 264

12.14.4.3.2. General Use GPIO Register Offset 0x00: General GPIO Status Register 264

12.14.4.3.3. General Use GPIO Register Offset 0x01: General GPIO Control 265

12.14.4.3.4. General Use PD GPIOs IP Block CSRs Region 1 Sub-Region 1 Description 266

12.14.4.4. General Use PD GPIOs IP Block Region 2 CSR Memory Map 266
12.14.4.5. General Use PD GPIOs IP Block Region 3 CSR Memory Map 266
12.14.5. General Use PD GPIOs Vendor Specific Extensions Support 266
12.14.6. General Use PD GPIOs Example Implementation 267
12.15. Host Access IP Block 267
12.15.1. Host Access IP Block Architecture 267
12.15.2. Host Access IP Block Requirements 268
12.15.2.1. General Requirements 268
12.15.2.2. eSPI Interface Usage Requirements 269
12.15.2.3. I2C / I3C Interface Usage Requirements 269
12.15.2.4. SPI Requirements 270
12.15.2.5. PCIe Requirements 270
12.15.2.6. USB Requirements 270
12.15.2.7. Required and Optional Feature Set 271
12.15.3. Host Access IP Block CSR Requirements 271
12.15.3.1. Host Access CSR Region Overview 272
12.15.3.2. Host Access IP Block Region 0 Common Registers Memory Map 273
12.15.3.2.1. Host Access IP Block Common Registers Sub-Region 0 Description 273
12.15.3.2.2. Host Access IP Block Common Registers Sub-Region 1 Description 274
12.15.3.3. Host Access IP Block Region 1 CSR Memory Map 274
12.15.3.3.1. Host Access CSRs Region 1 Sub-Region 0 Description 275
12.15.3.3.2. Host Access IP Block CSRs Region 1 Sub-Region 1 Description 275
12.15.3.4. Host Access IP Block Region 2 CSR Memory Map 275
12.15.3.5. Host Access IP Block Region 3 CSR Memory Map 275
12.15.4. Host Access JSON Requirements 276
12.15.5. BAR Advertisement 276
12.15.6. Host Access Vendor Specific Extensions Support 277
12.15.7. Host Access Example Implementation 277
12.16. eSPI IP Block 277

12.16.1. eSPI IP Block Architecture 278
12.16.2. eSPI IP Block Requirements 279
12.16.2.1. General Requirements 279
12.16.2.2. Required and Optional Feature Set 280
12.16.3. eSPI IP Block CSR Requirements 281
12.16.3.1. eSPI CSR Region Overview 281
12.16.3.2. eSPI LPC Style Host Accessible Register Set Overview 282
12.16.3.2.1. Example Programmatic Flow 283
12.16.3.3. eSPI LPC Style Register Set Definition 284
12.16.3.3.1. eSPI Global Register Set Definition 284
12.16.3.3.2. eSPI Logical Device 0 Register Set Definition 285
12.16.3.4. eSPI BMC Accessible Register Set Overview 287
12.16.3.4.1. eSPI IP Block BMC Accessible Region 0 CSR Memory Map 288
12.16.3.4.1.1. eSPI CSRs Region 0 Sub-Region 0 Description 289
12.16.3.4.2. eSPI IP Block BMC Accessible Region 1 CSR Memory Map 290
12.16.3.4.2.1. eSPI CSRs Region 1 Sub-Region 0 Description 290
12.17. M-PESTI Target IP Block 290
12.17.1. M-PESTI Target IP Block Overview 291
12.17.2. M-PESTI Target IP Block Architecture 291
12.17.3. M-PESTI Target IP Block Requirements 291
12.17.3.1. M-PESTI Target Required and Optional Feature Set 292
12.17.4. M-PESTI Target Discovery Payload Version 3 292
12.17.4.1. M-PESTI Target Discovery Payload Version 3 Definitions 294
12.17.5. M-PESTI Target Virtual Wire Format Type 1 Formatting Definition 295
12.17.5.1. M-PESTI Target Virtual Wire Format Type 1 General Format Definitions 295
12.17.5.2. M-PESTI Target Virtual Wire Format Type 1 General Command List 296
12.17.5.3. M-PESTI Target Virtual Wire Format Type 1 Command List Per Peripheral Device Class and Subclass 297
12.17.5.3.1. Fixed Riser Device Class 0x00 Subclass 0x00 Command List and Definition 297

12.17.5.3.2. Floating Riser Device Class 0x00 Subclass 0x01 Command List and Definition 299

12.17.5.3.3. OCP NIC Floating Riser Device Class 0x00 Subclass 0x02 Command List and Definition 302

12.17.5.3.4. Double Wide OCP NIC Floating Riser Device Class 0x00 Subclass 0x03 Command List and Definition 304

12.17.5.3.5. Non-UBM Backplane Device Class 0x02 Subclass 0x00 Command List and Definition 307

12.17.5.3.6. UBM Backplane Device Class 0x02 Subclass 0x01 Command List and Definition 309

12.17.5.3.7. Primary Control Panel Device Class 0x05 Subclass 0x00 Command List and Definition 311

12.17.5.3.8. Unspecified Peripherals Usage Of Format Type 1 315

12.17.6. M-PESTI Target IP Block CSR Requirements 315

12.17.6.1. M-PESTI Target CSR Region Overview 315

12.17.6.2. M-PESTI Target IP Block Region 0 Common Registers Memory Map 316

12.17.6.2.1. M-PESTI Target IP Block Common Registers Sub-Region 0 Description 316

12.17.6.2.2. M-PESTI Target IP Block Common Registers Sub-Region 1 Description 317

12.17.6.3. M-PESTI Target IP Block Region 1 CSR Memory Map 317

12.17.6.3.1. M-PESTI Target CSRs Region 1 Sub-Region 0 Description 318

12.17.6.3.2. M-PESTI Target IP Block CSRs Region 1 Sub-Region 1 Description 318

12.17.6.4. M-PESTI Target IP Block Region 2 CSR Memory Map 318

12.17.6.5. M-PESTI Target IP Block Region 3 CSR Memory Map 319

12.17.7. M-PESTI Target Vendor Specific Extensions Support 319

12.17.8. M-PESTI Target Example Implementation 319

12.18. Reliability Availability and Serviceability (RAS) IP Block 319

12.18.1. Reliability Availability and Serviceability (RAS) IP Block Overview 319

12.18.2. Reliability Availability and Serviceability (RAS) IP Block Architecture 320

12.18.3. Reliability Availability and Serviceability (RAS) IP Block Requirements 321

12.18.3.1. Required and Optional Feature Set 321

12.18.4. Reliability Availability and Serviceability (RAS) IP Block CSR Requirements 321

12.18.4.1. Reliability Availability and Serviceability (RAS) IP Block Region Overview 322

12.18.4.2. Reliability Availability and Serviceability (RAS) IP Block Region 0 Common Registers Memory Map 322

12.18.4.2.1. Reliability Availability and Serviceability (RAS) IP Block Common Registers Sub-Region 0 Description 323

12.18.4.2.2. Reliability Availability and Serviceability (RAS) IP Block Common Registers Sub-Region 1 Description 324

12.18.4.3. Reliability Availability and Serviceability (RAS) IP Block Region 1 CSR Memory Map 324

12.18.4.3.1. Reliability Availability and Serviceability (RAS) CSRs Region 1 Sub-Region 0 Description 325

12.18.4.4. Reliability Availability and Serviceability (RAS) IP Block Region 2 CSR Memory Map 325

12.18.5. Reliability Availability and Serviceability (RAS) Vendor Specific Extensions Support 326

12.18.6. Reliability Availability and Serviceability (RAS) IP Block Example Implementation 326

12.19. Universal Serial Bus (USB) IP Block 326

12.19.1. Universal Serial Bus (USB) IP Block Overview 326

12.19.2. Universal Serial Bus (USB) IP Block Architecture 327

12.19.3. Universal Serial Bus (USB) IP Block Requirements 327

12.19.3.1. Required and Optional Feature Set 328

12.19.4. Universal Serial Bus (USB) IP Block CSR Requirements 328

12.19.4.1. Universal Serial Bus (USB) IP Block Region Overview 328

12.19.4.2. Universal Serial Bus (USB) IP Block Region 0 Common Registers Memory Map 329

12.19.4.2.1. Universal Serial Bus (USB) IP Block Common Registers Sub-Region 0 Description 329

12.19.4.2.2. Universal Serial Bus (USB) IP Block Common Registers Sub-Region 1 Description 330

12.19.4.3. Universal Serial Bus (USB) IP Block Region 1 CSR Memory Map 331

12.19.4.3.1. Universal Serial Bus (USB) CSRs Region 1 Sub-Region 0 Description 331
12.19.4.4. Universal Serial Bus (USB) IP Block Region 2 CSR Memory Map 332
12.19.5. Universal Serial Bus (USB) Vendor Specific Extensions Support 332
12.19.6. Universal Serial Bus (USB) IP Block Example Implementation 332
13. HPM PD Security Requirements and Considerations 332
14. PD Vendor Extensibility Requirements and Considerations 333
15. HPM PD General Requirements and Considerations 333
15.1. PD Image Considerations and Requirements 334
15.1.1. PD Self-Describing JSON File Considerations 334
15.2. PD Programming Requirements 335
15.3. Electrical Considerations 336
16. Specification Compliance Checklist 337
17. References 337
18. Trademarks 338
19. Future Material 338

1. License

1.1. Open Web Foundation (OWF) CLA

Contributions to this Specification are made under the terms and conditions set forth in Open Web Foundation Modified Contributor License Agreement ("OWF CLA 1.0") ("Contribution License") by:

- Advanced Micro Devices, Inc
- American Megatrends International (AMI)
- Ampere Computing
- Arm Holdings PLC
- Dell, Inc.
- Google LLC
- Hewlett Packard Enterprise Company
- Intel Corporation
- Jabil Corporation
- Lattice Semiconductor Corporation
- Meta Platforms, Inc.
- Microsoft Corporation
- Nvidia Corporation

Usage of this Specification is governed by the terms and conditions set forth in **Open Web Foundation Modified Final Specification Agreement ("OWFa 1.0") ("Specification License").**

You can review the applicable OWFa1.0 Specification License(s) referenced above by the contributors to this Specification on the OCP website at http://www.opencompute.org/participate/legal-documents/. For actual executed copies of either agreement, please contact OCP directly.

Notes:

The above license does not apply to the Appendix or Appendices. The information in the Appendix or Appendices is for reference only and non-normative in nature.

NOTWITHSTANDING THE FOREGOING LICENSES, THIS SPECIFICATION IS PROVIDED BY OCP "AS IS" AND OCP EXPRESSLY DISCLAIMS ANY WARRANTIES (EXPRESS, IMPLIED, OR OTHERWISE), INCLUDING IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, OR TITLE, RELATED TO THE SPECIFICATION. NOTICE IS HEREBY GIVEN, THAT OTHER RIGHTS NOT GRANTED AS SET FORTH ABOVE, INCLUDING WITHOUT LIMITATION, RIGHTS OF THIRD PARTIES WHO DID NOT EXECUTE THE ABOVE LICENSES, MAY BE IMPLICATED BY THE IMPLEMENTATION OF OR COMPLIANCE WITH THIS SPECIFICATION. OCP IS NOT RESPONSIBLE FOR IDENTIFYING RIGHTS FOR WHICH A LICENSE MAY BE REQUIRED IN ORDER TO IMPLEMENT THIS SPECIFICATION. THE ENTIRE RISK AS TO IMPLEMENTING OR OTHERWISE USING THE SPECIFICATION IS ASSUMED BY YOU. IN NO EVENT WILL OCP BE LIABLE TO YOU FOR ANY MONETARY DAMAGES WITH RESPECT TO ANY CLAIMS RELATED TO, OR ARISING OUT OF YOUR USE OF THIS SPECIFICATION, INCLUDING BUT NOT LIMITED TO ANY LIABILITY FOR LOST PROFITS OR ANY CONSEQUENTIAL, INCIDENTAL, INDIRECT, SPECIAL OR PUNITIVE DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS SPECIFICATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND EVEN IF OCP HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

1.2. Acknowledgements

With the hope of making this specification useful for the entire OCP community, we acknowledge and appreciate the contributions, review, and feedback of various individuals and companies that participated in MHS.

2. Version Table

Date	Version #	Description
9/26/24	0.3 RC2	Initial wiki release to the public.
2/21/25	0.3.1	**Changes to sections:** 8.1. MHS HPM FPGA Common Interfaces 9.1. FPGA Memory Map Access Methods 9.3. FPGA Memory Map Access 12.5. SGPIO Logical IP Block
4/1/25	0.3.2	**Changes to sections:** 8.1. MHS HPM FPGA Common Interfaces 8.2. MHS HPM FPGA Logical IP Blocks List 9.3. FPGA Memory Map Access 12.1.1. Common Required Memory Mapped CSRs 12.5. SGPIO Logical IP Block

4/29/25	0.3.3	**Changes to sections:** 12.1.1. Common Required Memory Mapped CSRs 12.2. Power Sequencing and Management Logical IP Block 12.10. M-XIO FLEX IO and M-PIC Sideband Control Logical IP Block
5/30/25	0.3.4	**Changes to many sections including:** 12.10. M-XIO FLEX IO and M-PIC Sideband Control Logical IP Block 12.11. MUX Control Logical IP Block
7/15/25	0.4	**Changes to many sections including:** 7.4. MHS HPM FPGA Specification Supplementary Materials 8.2. MHS HPM FPGA Logical IP Blocks List 10.1. FPGA JSON Schema Representation Overview 10.3. IP Block JSON Schema IP Block Section 10.4. FPGA Memory Map JSON Schema Section 11. Virtual Wires Support 12.1.1. Common Required Memory Mapped CSRs 12.2. Power Sequencing and Management Logical IP Block 12.3. System / Device Management and Reset Control Logical IP Block 12.4. LTPI Standardized Logical IP Block 12.5. SGPIO Logical IP Block 12.6. DC-SCM to HPM FPGA SPI Interface IP Logical Block 12.7. DC-SCM to HPM FPGA I2C / I3C Basic Interface IP Logical Block 12.8. M-PESTI Interface Logical IP Block 12.9. LED Control Logical IP Block 12.10. M-XIO FLEX IO and M-PIC Sideband Control Logical IP Block 12.11. MUX Control Logical IP Block
8/12/25	0.5 DRAFT	**Changes to many sections including:** 12.1.1.6. Common IP Block CSR: Interrupt Status 12.2.6.3.4. Power Management IP Block Region 1 Register 12.3. System / Device Reset Control Logical IP Block 12.4. LTPI Standardized Logical IP Block 12.4.3.2. LTPI GPIO Channel Tunneled GPIOs List 12.4.3.2.1. LTPI Data Channel Tunneled GPIOs Mapping FPGA Internal Virtual 12.4.4.3. LTPI IP Block Region 1: CPLD/FPGA Information CSR Memory Map 12.5.8.1. SCM To HPM SGPIO Tunneled GPIOs List 12.5.8.2. SGPIO HPM To PDB Required and Optional GPIOs List 12.6. DC-SCM to HPM FPGA SPI Interface IP Logical Block

		12.7. DC-SCM to HPM FPGA I2C / I3C Basic Interface IP Logical Block 12.8.7.2.2. M-PESTI Per Target Register 0x02: Target Control Register 12.9. LED Control Logical IP Block 12.12. Cooling Subsystem Control Logical IP Block 12.13. System Telemetry Collection Logical IP Block 12.14. General Use GPIOs IP Block 15.1.1. FPGA Self-Describing JSON File Considerations
9/09/25	0.5.0 RC1	Minor changes to improve grammar and fix spelling errors. Removal of white space. Added table and figure captions where missing. **Changes to sections:** 7.4. Supplementary Materials 12.2. Power Sequencing and Management Logical IP Block 12.4.2. LTPI Interface IP Block Architecture 12.4.5. LTPI Interface Data Channel Tunneling Requirements 12.6. DC-SCM to HPM FPGA SPI Interface IP Logical Block 12.7. DC-SCM to HPM FPGA I2C / I3C Basic Interface IP Logical Block 12.8. M-PESTI Interface Logical IP Block 12.9. LED Control Logical IP Block 12.12. Cooling Subsystem Control Logical IP Block 12.13. System Telemetry Collection Logical IP Block 12.14. General Use GPIOs IP Block 15.1.1. FPGA Self-Describing JSON File Considerations
9/17/25	0.5.0 RC2	Fix missing revision and incorrect word in section 2 revisions table for 0.5.0 RC2. General formatting fixes and removal of white space. Section 12.2.2, changed ‘S0C’ to ‘SoC’ in table. Section 12.2.6.3.2, offset 0x01: General Power Status was missing transition flag R attribute. Section 12.3.4.2.1, changed ‘disabled’ to ‘asserted’ in table. Section 12.8.6.2.1, changed REQ to OPT for register 0x10 in table. Section 12.8.6.2.2, missing sizes on register fields and reserved indication in description. Section 12.8.7.2.2, added the word ‘also’ between ‘that’ and ‘support’. Section 12.9.2, fixed diagram to change ‘remove’ to ‘remote’.
1/16/26	0.5.1	Added host access IP block
3/11/26	0.5.2	Added eSPI IP block.
4/8/26	0.5.3	Added Cooling IP Block CSRs definition. Re-integrated LED IP block verbiage changes.

4/24/26	0.5.4	Minor change to cooling IP block section to add missing table captions. Added LED IP block CSRs.
07/22/26	0.5.5	Added Telemetry CSR definition. Added M-PESTI Target IP Block initial section, MPNP payload, and M-PESTI virtual wires definition.
08/19/2026	0.5.6	Added error / failure path language to chapter 12 IP blocks section. Added common SoC signaling (SOC presence, THERMTRIP, etc.) registers to power management spec. Added PCP virtual wire format. Added RAS IP block. Updates to GPIO IP block.
09/02/26	0.6.0	Added terminology clause for ISO/EAC directives. Fixed formatting. Added USB IP block initial section. Language clarifications. Added optional DPERR bit in MPESTI section. Changed SGPIO CSRs to follow standard IP block format. Removed JSON attributes tables and now referencing schema document. Updated table captions.
09/29/26	0.7.0 Draft	Changed the title and name of the specification to the HPM AND PERIPHERAL PROGRAMMABLE DEVICE (PD) DESIGN SPECIFICATION and filename to OCP_HPM_PERIPHERAL_PD_Spec.docx. Updated CLA members. Updated contributors. Updated TOC. Changed mention of FPGA to either Programmable Device or PD. Added mention of HPM primary Programmable Device to explicitly call out the HPM FPGA. Added this language throughout the entire specification. Expanded scope in chapter 3, “Overview”. Added section 4.2, “Related Industry Specifications”. Added more terminology to table 2. Expanded scope in chapter 6 regarding specification vision statement. Expanded scope description in section 7.2 and adjusted the out-scope in 7.3. Refinement of supplementary materials in table 3 and added schema documentation. Adjusted language in chapter 8. Updated figure 3 to remove mention of I2C/I3C mailbox and substituted ‘HPM Primary PD’ instead of FPGA. Added clarifying language to chapters 9 and 10. Included the use of Internal and Signal when referring to Virtual Wires in chapter 11 to further stress that the virtual wire framework is internal to the programmable device. Added clarifying language throughout all of chapter 12. Switched to usage of ‘Shall’ instead of ‘Will’ throughout the entire document. Changed to ISO language throughout the document for ‘should’ instead of ‘recommended’ and ‘may’ for optional features. Added note to section 12.2.2 regarding state encoding. Fixed table 27. Adjusted power management state machine timeout control register values in table 28. Improved VR fault language in section 12.2.6.3.8. Improved and clarified language for SGPIO Overview in section 12.5.1. Added note for table 70 regarding timeouts and retries. Added notes to table 71 and 73 regarding default values for SGPIO signals. Added ‘CHECKSUM_ERROR’ optional indication in MPESTI registers in table 90. Removed bit 5 TRFDY_INT_EN in table 92 as we decided it was not needed and did not actually

		make sense to support. Added clarifying language in 12.8.3. regarding MPESTI MUX support. Added note to table 93 to for MPESTI virtual wires. Added clarifying language in LED Control Logical IP block chapter 12.9. Added note to table 96. Additional language added to section 12.10.4.3.4 to clarify FlexIO register requirements. Fix for signal integrators in table 115. Clarified language in tables 117 and 118. Missing byte sizes in table 150. Added additional implementation notes in section 12.13.4.3.1. Added section 12.14.3.2 regarding specific special purpose signals within the GPIO IP block for the HPM Primary PD control of the HPM FRU EEPROM Write Protection signal. Changed ESPI to eSPI throughout the document. Added clarifying language in table 171. Added clarifying language in section 12.15.2.3. Clarified table 201 regarding USB support for control panel virtual wires. Changed chapter 14 title. Removed placeholder and non-normative language throughout the document. Format and spacing fixes throughout the document. Minor grammatical fixes throughout the document. Changed to Arial font wherever it was incorrect.
10/07/26	0.7.0 RC1	Replaced reference to FPGA throughout the document and changed to HPM Primary Programmable Device. Changed reference to M-PNP.FPGA specification to M-PNP.HPM and Peripheral Programmable Device specification. Updated terminology in table 2. Updated diagrams to change from FPGA to HPM Primary Programmable Device or added a note regarding the usage of an FPGA in the diagram. Minor language corrections, formatting, grammatical and spelling corrections throughout the document. Added clarifying language to tables 75, 78, and 79. Updated tables 79 and 83 for W1C entry. Added clarifying language to section 12.7.3.2.

3. Overview

This specification defines and standardizes common elements needed for the Host Processor Module (HPM) Programmable Devices (PD)s, Custom Programmable Logic Devices (CPLD)s, Microcontroller Units (MCU)s or other possible devices that support programmability. This specification outlines what is required for the HPM PD to be considered M-PNP compliant.

The main areas of focus within this specification are as follows:

- Standardization of interfaces into the programmable device (PD) for BMC usage
- Standardization of IP blocks and common functionality
- Self-description of the programmable device (PD)PD using JSON to describe the memory map, IP blocks, CSRs, and virtual wires
- General security and other non-functional considerations of the PDPD design

The HPM primary programmable device (PD) is a critical component of the HPM and overall server system, and this specification provides the required and optional BMC interfaces and register sets that shall be used to perform various functions to manage the platform. Additionally, peripherals may have PDs that may present a BMC accessible interface, and this specification is applicable to that use case. The system, as mentioned above, is comprised of all the constituent components within the server. This includes all peripherals as well as Host Processor Modules (HPMs).

4. MHS Family of Specifications

The Modular Hardware System (MHS) Rev 1.0 family of specifications are written to enable interoperability between key elements of datacenter and enterprise infrastructure by providing consistent interfaces and form factors among modular building blocks. At the time of this publication there are the following specification workstreams:

- **M-FLW** (**M**odular Hardware System **F**ull **W**idth Specification) – Host Processor Module (HPM) form factor specification optimized for using the full width of a Standard EIA-310 Rack mountable server. The specification is not limited to use within the EIA-310 Rack but is used to serve as a template for a common target where the design is expected to be utilized.
- **M-DNO** (**M**odular Hardware System Partial Width **De****n**sity **O**ptimized Specification) – Host Processor Module (HPM) specification targeted to partial width (i.e., ½ width or ¾ width) form factors. Such form factors are often depth challenged and found not only in enterprise applications but also in Telecommunications, Cloud and Edge Deployments. While the EIA-310 Rack implementation is chosen as a key test case for use, the specification is not limited to use within the EIA-310 Rack but is used to serve as a template for a common target where the design is expected to be utilized.

- **M-CRPS** (**M**odular Hardware System **C**ommon **R**edundant **P**ower **S**upply Specification) – Specifies the power supply solutions and signaling expected to be utilized by MHS compatible systems.
- **M-PIC** (**M**odular Hardware System **P**latform **I**nfrastructure **C**onnectivity Specification) – Specifies common elements needed to interface a Host Processor Module (HPM) to the platform/chassis infrastructure elements/subsystems. Examples include power management, control panel and cooling amongst others.
- **M-XIO** (**M**odular Hardware System E**x**tensible **I**/**O**) – Specifies the high-speed connector hardware strategy. An M-XIO source connector enables entry and exit points between sources such as Motherboards, Host Processor Modules & RAID Controllers with peripheral subsystems such as PCIe risers, backplanes, etc. M-XIO includes the connector, high speed and management signal interface details and supported pinouts.
- **M-PESTI** (**M**odular Hardware System **Pe**ripheral **S**ideband **T**unneling **I**nterface) – Specifies a standard method for discovery of subsystems, self-describing attributes, and status (i.e., versus a priori knowledge/hard coding firmware and BIOS for fixed/limited configurations). Examples: vendor/module class, physical connectivity descriptions, add-in card presence, precise source to destination cable coupling determination.
- **M-SIF** (**M**odular Hardware System **S**hared **I**n**f**rastructure Specification) – Specifies data and management plane interfaces along with power and cooling connections that are expected to be present within a modular platform utilizing shared elements.
- **M-PNP Interoperability –** Specifies the requirements and conformance criteria a server system and constituent components that component implementors shall meet to satisfy all criteria to be Modular Plug and Play (M-PNP) conformant.
- **M-PNP Discovery and FRU -** Specifies management plane collaterals with respect to common discovery and management of HPMs and peripheral subsystems. Examples include this FRU-related document, HPM PD guidelines, schemas and templates for HW descriptions that ease FW/BIOS platform enablement for new HPMs, peripherals and systems.
- **HPM Common Circuits Type 1 -** Specifies the set of HPM common circuits as interfaced by the SCM and it's BMC as categorized into types. Type 1 refers to a common single node platform.
- **DC-SCM** (Data Center Secure Control Module) 2.1 Specification – Specification for a control module containing the system BMC.
- **DC-SCM** (Data Center Secure Control Module) LVDS Tunneling Protocol & Interface (LTPI) Specification Version 2.1 – Specification of an interface that utilizes LVDS and outlines the tunneling protocol that this interface supports.

To access additional MHS specifications, please visit the OCP Server WIKI.

4.1. Related OCP Specifications

Shown below are the set of related specifications that are either directly or indirectly referenced by this specification.

- **OCP NIC 3.0** - Specification for a Network Interface Card form factor.

4.2. Related Industry Specifications

Shown below are the set of related industry specifications that are either directly or indirectly referenced by this specification.

- **DMTF PCMI DSP0220 v1.0 - Field Replaceable Unit (FRU)** Data Specification. https://www.dmtf.org/sites/default/files/standards/documents/DSP0220_1.0.pdf
- **DMTF DSP0236 v1.3, Management Component Transport Protocol (MCTP)** Base Specification: https://www.dmtf.org/sites/default/files/standards/documents/DSP0236_1.3.pdf
- **DMTF PMCI DSP0257 v2.0 - PLDM** Platform Level Data Model (PLDM) for FRU Data Specification: https://www.dmtf.org/sites/default/files/standards/documents/DSP0257_2.0.pdf
- **DMTF PLDM for File Transfer Specification (DSP0242):** https://www.dmtf.org/dsp/DSP0242
- **DMTF MMBI Specification DSP0282 v1.0** Memory-Mapped Buffer Interface (MMBI) Specification: https://www.dmtf.org/sites/default/files/standards/documents/DSP0282_1.0.pdf
- **IPMI Platform Management FRU Information Storage Definition Specification v1.0 Document Revision 1.3** Specification: https://www.intel.com/content/dam/www/public/us/en/documents/specification-updates/ipmi-platform-mgt-fru-info-storage-def-v1-0-rev-1-3-spec-update.pdf

5. Terminology

5.1. Standards Terminology

The terms in the following table have definitions according to ISO/IEC Directives, Part 2, Clause 7. Those meanings are intended where these terms appear.

TABLE 1: STANDARDS TERMINOLOGY

Standardized Term	Meaning
Shall	Indicates a requirement.
Shall Not	Indicates something that must not be done.
May	Indicates that something is permitted.
Need Not	Not required.
Should	Indicates a recommendation, which is optional.
Should Not	Not recommended.
Can	Used to indicate that something is possible, for example, that an organization or individual is able to do something
Cannot	See ISO/IEC Directives, Part 2, Clause 7.
Normative	See ISO/IEC Directives, Part 2, Clause 3.
Informative	See ISO/IEC Directives, Part 2, Clause 3.

5.2. M-PNP Specific Standards Terminology

This section contains a table of M-PNP specific terminology used in this specification.

TABLE 2: M-PNP SPECIFIC TERMINOLOGY

Standardized Term	Meaning	Alternative Terms

Local Attributes	Descriptive attributes stored in the device being described. e.g., HPM or peripheral EEPROM either in JSON or IPMI format.	
Extended Attributes	Descriptive attributes not stored in the device being described. e.g., BMC or BIOS ROM on behalf of HPM or peripheral and primarily in JSON format.	
Virtual Wire	Internal PD configurable wiring. Allows for mapping of dedicated functional wires to external interfaces. May map to physical GPIOs connected to the PD or Virtual GPIOs (VGPIOs). This is considered a superset of both fixed function and programmable signals.	
GPIO	General Purpose Input Output	
VGPIO	Virtual General Purpose Input Output	
HPM	Host Processor Module	
Non-endpoint Peripheral	An MHS peripheral component that has one or more connectors that support a connection to an endpoint peripheral such as a PCIe CEM card or OCP NIC board. Examples include plug-in card risers, storage backplanes, PCIe switch boards, and others.	
Endpoint Peripheral	A peripheral that does not have any connectors to additional plug-in or cabled components. Examples include PCIe Card Electromechanical (CEM) and OCP NIC boards.	
HPM Primary Programmable Device (PD)	The main programmable device that the SCM BMC primarily interacts with in order to manage the HPM and platform, in general.	FPGA, CPLD

6. HPM and Peripheral Programmable Device (PD) Interoperability Vision

This specification aims to create a uniform approach to HPM and Peripheral Programmable Device design for Host Processor Modules (HPM)s among different vendors. Its primary goal is to reduce the effort needed to integrate the main HPM Programmable Device (PD) with SCM CPLDs, BMC firmware, and the SCM Root of Trust (ROT). Programmable Devices may be embodied in many different forms. Examples include Field Programmable Gate Arrays (FPGA)s, Custom Programmable Logic Devices (CPLD)s, Microcontroller Units (MCU)s and

many others. A secondary goal is to create a set of uniform requirements for non-endpoint peripheral (e.g., backplanes, risers, etc.) PDs such that it eases integration and improves platform component interoperability.

HPM and Peripheral Programmable Device (PD) Interoperability Specification scope and concept is presented in Figure 1. The middle blocks (e.g., LTPI, M-PESTI, etc.) are examples of standardized IPs that are intended to be unified across HPM designs. The IPs expose unified APIs and frameworks that allow both HPM vendors to deliver customizations and inject their custom logic (e.g. to deliver CPU specific power sequencing logic) and this same framework allows the System Integrator to add their custom features without impacting bassline interoperability of primary HPM Programmable Device (PD). This is intended to minimize the number of customized, unique HPM PD features and functionality required, and to reduce the implementation burden on the HPM vendor and System Integrator to achieve baseline functionality and improve HPM interoperability between vendors.

The goal of this specification is to drive open-sourced implementations of these unified IP blocks.

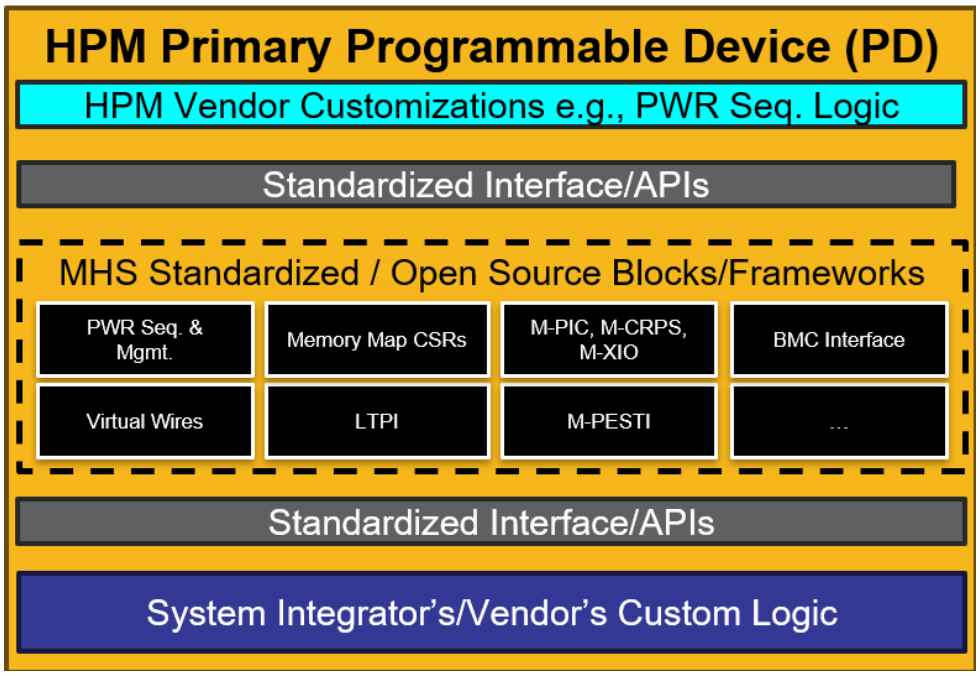


FIGURE 1: HPM PRIMARY PROGRAMMABLE DEVICE (PD) STANDARDIZATION CONCEPT

7. Scope

This document defines technical specifications for the HPM Programmable Device (PD) Design Specification used in the MHS family. This section broadly outlines what is in and out of scope for this specification. In addition, this section lists a set of supplementary materials.

Any supplier seeking OCP recognition for a hardware product based on this specification shall be fully compliant with all features or requirements listed in section 16, “Specification Compliance Checklist”, of this specification.

7.1. Typical OCP Sections Not Applicable

This is a design specification for the HPM PD, and its adherence is necessary to support M-PNP interoperability. The following typical sections of an OCP specification are not included because they are not applicable to this specification.

Rack Compatibility

Physical Spec

Rear Side Power, I/O, Expansion

Mechanical

Onboard Power System

Environmental Regulations/Requirements

Prescribed Materials

Software Support

7.2. In-Scope

This design specification describes a Programmable Device (PD) architecture for the purposes of supporting an interoperable MHS ecosystem of components. This includes implementation details and guidelines. This section outlines what is explicitly within the scope of this specification.

The bulleted list below contains a non-exclusive set of items that are considered in-scope for this specification.

- Standardized interfaces aligned to MHS/DC-SCM specifications e.g., M-PESTI, LTPI, etc.
    - Standardized Building Blocks (IPs)
    - Architecture (States, Interfaces, API)
    - Protocol definitions (e.g. SGPIO) building on existing specifications.

    - References to existing Open-Source IPs (e.g. https://github.com/opencomputeproject/HWMgmt-Module-DCSCM-LTPI)
- Standardized memory map and process for self-description of content definition for use in the plug-and-play discovery process
- Definition of HPM Programmable Device (PD) APIs with BMC FW
    - Definition of Interfaces and Payloads with metadata-based formats e.g. JSON, System RDL, etc.
    - Applicability of IP blocks as defined in this spec to Non-endpoint Peripheral PDs
    - General guidance regarding security enhancements such as SPDM usage
- Integration methodologies: how meta-data is used to automate customization and integration into RTL and BMC FW

7.3. Out-Scope

This section lists what is explicitly outside the scope of this specification.

- Guidance for Programmable Device (PD) physical packaging
- Physical interface (e.g. SGPIO, LTPI, etc.) electrical or timing requirements which are specified elsewhere.
- Explicitly defining and requiring interface (e.g. SGPIO, LTPI, etc.) security mechanisms outside common error detection mechanisms such as a CRC
    - Some recommendations may be provided as optional, such as usage of SPDM or other security functionality
- Programmable Device (PD) resource consumption (e.g. LUTs, memory, etc.) requirements and/or explicit stipulations (e.g. no more than ‘X’ maximum LUTs or memory used for a given IP block)
- Platform and policy level descriptive information and data, encoded or otherwise (e.g., JSON encoding)
    - An example of this would be decisions on LED blink and setting meanings for system diagnostics

7.4. Supplementary Materials

This section outlines a non-exclusive list of materials considered to be supplementary to this design specification and may be packaged within the release package of the specifications.

Revision and documentation control will be outside the scope of this document. Changes to those materials shall not be tracked within this document.

TABLE 3: SUPPLEMENTARY MATERIALS

Material Name	Description
Programmable Device (PD) PD Self-Describing JSON Information File Example	Programmable Device (PD) image deliverable created at build time by vendor which provides the self-describing JSON for Programmable Device (PD) IP block CSRs, virtual wire definitions, and other items.
Self-Describing JSON Schema for HPM and Peripherals	Schema for the HPM and peripherals which includes dynamically and statically discovered Programmable Device (PD) descriptive information.
Self-Describing JSON Mock-up Examples For HPM, HPM Programmable Device (PD)PD, and Peripherals	JSON mock-up examples for the HPM and peripherals which can include both dynamically and statically discovered Programmable Device (PD) descriptive information.
IP Block Examples List	This will be a running list of example IP blocks, such as LTPI, that will be provided as external collateral.
HPM and Peripherals Self-Describing File JSON Schema Documentation Files: hardware-component-documentation.md hardware-component-documentation.pdf hardware-component- developer.md hardware-component- developer.pdf hardware-component-changelog.md	HPM and Peripherals Self-Describing File JSON Schema Documentation both in markdown and PDF format. The document changelog is provided in markdown format.
Programmable Device Self-Describing File JSON Schema Documentation Files: PD-documentation.md PD-documentation.pdf PD-developer.pdf PD-developer.md PD-changelog.md	Programmable Device Self-Describing File JSON Schema Documentation both in markdown and PDF format. The document changelog is provided in markdown format.

8. MHS HPM Primary Programmable Device (PD) Interfaces and Logical IP Blocks Overview

This section provides an overview of the Primary Programmable Device (PD) interfaces and standard logical IP blocks support.

8.1. MHS HPM Primary Programmable Device (PD) Common Interfaces

This section describes a set of possible interfaces between the HPM PD and other components. Below is an example diagram of common interfaces that are connected to the PD. Dotted lines indicate optional connections.

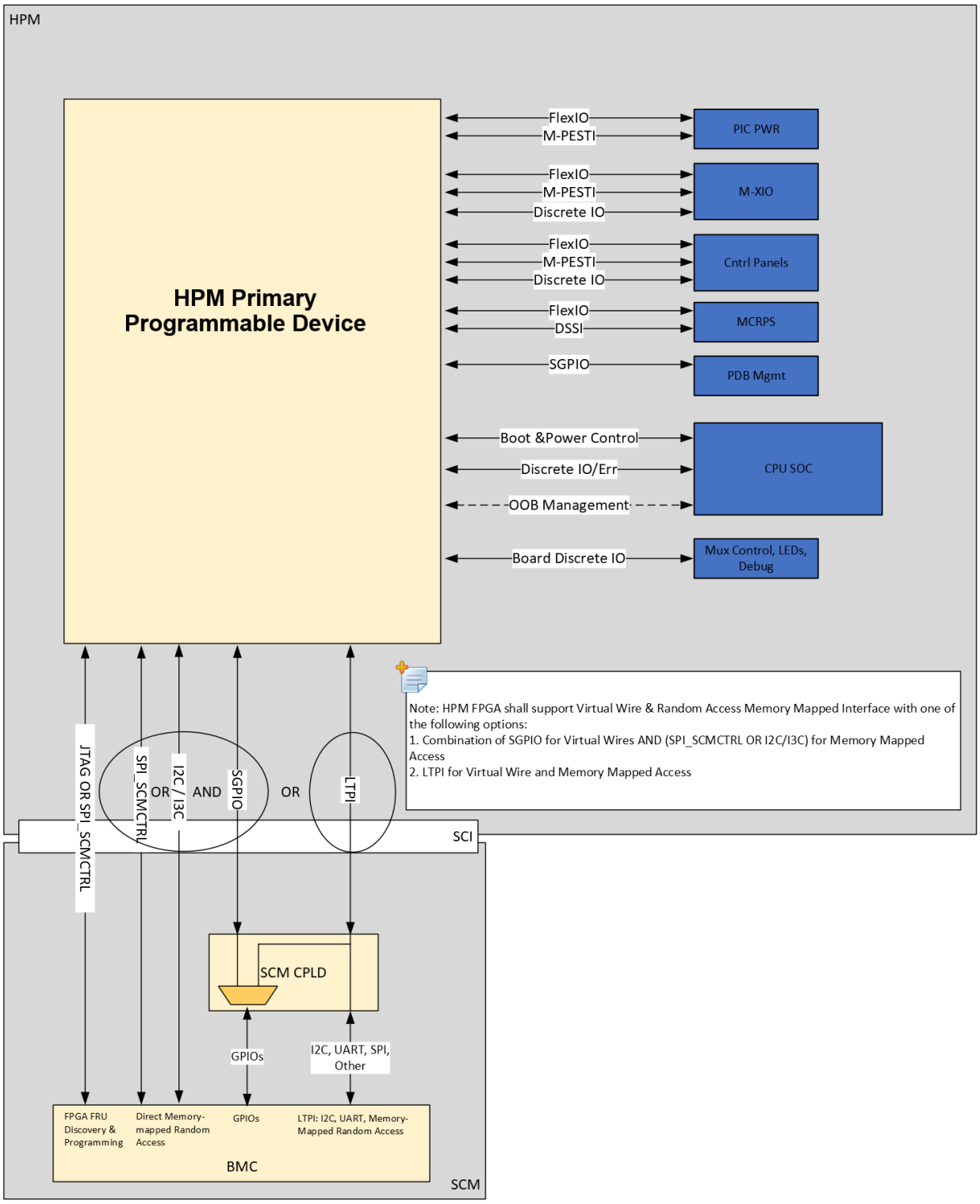


Figure 2: HPM Primary Programmable Device (PD) Common Interfaces Diagram

The above diagram is provided as an example, and vendors may use different buses as they see fit. Common interfaces between the HPM primary Programmable Device (PD) and the SCM are as follows in the table below:

TABLE 4: COMMON INTERFACES

Interface	HPM Type 1	Applicable Specifications
LTPI[1]	Y	OCP DC-SCM LTPI Rev. 2.1 Ver 1.0 or newer OCP DC-SCM Rev. 2.1 Ver1.0 OCP HPM Common Circuits Type 1 Spec Rev. 1.0 Ver 1.0 RC2
SGPIO[2]	Y	OCP DC-SCM Rev. 2.1 Ver1.0 OCP HPM Common Circuits Type 1 Spec Rev. 1.0 Ver 1.0 RC2
SPI	Y	OCP DC-SCM Rev. 2.1 Ver1.0 OCP HPM Common Circuits Type 1 Spec Rev. 1.0 Ver 1.0 RC2
I2C / I3C Basic	Y	OCP DC-SCM Rev. 2.1 Ver1.0 OCP HPM Common Circuits Type 1 Spec Rev. 1.0 Ver 1.0 RC2

Additional interfaces may be used but are outside the scope of this specification.

Common interfaces between the primary Programmable Device (PD) and internal HPM or external HPM connections are as follows in the table below:

TABLE 5: COMMON HPM INTERNAL INTERFACE EXAMPLES

Interface	Applicable Specifications
M-PESTI	OCP MHS M-PESTI Specification
M-XIO Flex IO	OCP MHS M-XIO Specification
M-PIC Sideband	OCP MHS M-PIC Specification
DSSI	OCP MHS M-CRPS Specification
SGPIO (PDB)	OCP M-PNP PD Specification

There may be additional interfaces which are outside the scope of this document. Additional interfaces are not required and are out of scope with this specification. Vendor defined interfaces should provide CSRs that follow the IP block description guidelines. Following the M-PNP framework for vendor defined interfaces provides vendors the benefits of ease of integration with BMC firmware.

[1] Note: The primary HPM PD shall support at least an SGPIO or an LTPI interface (it may support both).
[2] Ibid.

Please see the related documentation list for more specific details regarding interconnection requirements.

8.2. MHS HPM Programmable Device (PD) Logical IP Blocks List

This section itemizes the list of HPM Programmable Device (PD) standard logical IP blocks support. This is not an exclusive list and there may be additional vendor defined IP blocks not listed here. Additionally, if any feature of that IP block is required regardless of how many optional features are required it will be indicated as required in the 'Req' column. For more details as to which features are required vs. optional, please consult the appropriate section on that IP block within the specification.

TABLE 6: COMMON IP BLOCKS LIST

PD IP Block	Req?	Description
Power (Management / Sequencing)	Y	The HPM platform Power Sequencing and Power Control IP block primarily defines the unified HPM Power States and CSR (APIs) to be used by BMC firmware. The specific sequencing logic, state machines, timings and silicon vendor specific power sequencing requirements are not in scope and are expected to be implemented by HPM vendor.
System / Device Reset Control	Y	Required and optional functionality for general system reset control. This includes virtual reset control from BMC for individual MXIO, OCP slot, and on-board device resets.
Programmable Device (PD) Internal Virtual Wires Framework	Y	This IP block is a virtual construct that implements what is described in the Programmable Device (PD) internal virtual framework chapter within this specification. This includes the required and optional set of virtual wires needed for system interoperability as referenced in that section. This is a virtual IP Block that defines a framework for mapping between internal Programmable Device (PD) signals and a set of mappings between that signal and the transport of that signal over SGPIO, LTPI, or any other bus. Additionally, this represents the connection between the virtual wire and memory map interfaces or physical signals of HPM Programmable Device (PD) PD as a package pin.
LTPI	Y[3]	LTPI interface for the BMC and how the CSRs map into the data region and into the overall Programmable Device (PD) memory map. LTPI IP as defined by DC-SCM LTPI Spec as OCP reference implementation. M-PnP HPM Programmable Device (PD) Specification provides details of LTPI IP instantiation in HPM Programmable Device (PD) including mapping of LTPI Channels to

[3] Note: The HPM primary Programmable Device (PD) shall support at least an SGPIO or an LTPI interface (but it may support both).

		HPM Interfaces and virtual wires, usage of Data Channel to access memory map interface and BMC integration interfaces and CSRs.
SGPIO	Y[4]	SGPIO interface for the BMC and includes details regarding protocol, virtual wire mapping. SGPIO IP definition includes the SGPIO Protocol details such as SGPIO interface initialization and discovery, definition of SGPIO payload for virtual wire transport.
M-PESTI	Y	This IP block represents general M-PESTI support that includes mapping CSRs into the Programmable Device (PD) memory map. The PD shall implement the M-PESTI bus with the required M-PESTI CSRs. M-PESTI protocol is defined by MHS M-PESTI spec. M-PnP HPM Primary Programmable Device (PD) Specification provides details of M-PESTI IP implementation and mapping to HPM memory map interfaces and CSRs. The IP block supports different use cases from Full M-PESTI to the simplified Source to Destination mode detection mechanism as per the M-PESTI specification.
FLEX IO MXIO and M-PIC PICPWR Sideband Configuration	Y	Provides mechanism to control and configure the MXIO flexible IO pins for each MXIO and riser connector and M-PIC PICPWR and control panel sideband configuration. IP Block defines unified mechanism for M-XIO Flex-IO pins configuration and support using unified Device Classes defined by M-PnP specifications (or custom extensions using defined framework). It provides an API for BMC to configure the discovered device classes, and it also supports M-PIC PICPWR and control panel sideband configuration framework.
SPI Interface	N	Optional SPI interface for random access support into the Primary Programmable Device (PD) memory map.
Generic Mux Control	N	Provides mechanism to control and configure external muxes for I2C, I3C, and other system buses or general use muxes. Defines IP Block for controlling and configuring external or internal Muxes for I2C, I3C, and other system buses. Defines a memory map and CSR interfaces and mapping to VGPIO for Mux Control.
LED Control	N	Provides guidance and support for controlling system LEDs if implemented including memory mapped CSRs. Defines IP Block for controlling system LEDs, 7-seg displays through memory map interfaces and CSRs.
Cooling Control	N	Provides guidance and support for controlling system cooling hardware if implemented including memory mapped CSRs. Defines IP Block with unified CSRs and memory map API definitions for cooling system management.
Telemetry	N	Provides guidance and support for interfacing with system telemetry hardware if implemented including memory mapped CSRs. Defines IP Block for exposing the telemetry data from HPM Primary Programmable Device (PD) using unified APIs via memory map and CSRs.

[4] ibid

I2C / I3C	N	Provides guidance for usage of I2C or I3C interfaces by the PD if implemented including memory mapped CSRs. Defines method for integration of HPM Primary Programmable Device (PD) with I2C Interfaces by and methods to expose the I2C/I3C HPM Primary Programmable Device (PD) buses through memory map and CSRs.
General Use GPIOs	N	IP block that contains the general use GPIO set that maps to physical pin package signals on the Primary Programmable Device (PD). GPIOs are considered optional.
Host Access Interface	N	Framework for providing the host with Primary Programmable Device (PD) memory space and shared memory space with BMC.
eSPI	N	Provides an eSPI interface for the Host and BMC.
M-PESTI Target	N	M-PESTI target payload, virtual wire definition, and general register space for peripheral PLDs.
RAS	N	Framework for a Reliability, Availability, and Serviceability (RAS) interface from an SoC
USB	N	Provides a USB endpoint interface for BMC or Host access depending on which is the host controller.

8.3. Vendor Defined IP Blocks General Guidance

This section outlines the general guidance for inclusion of vendor defined IP blocks. Vendors should follow the plug-and-play framework outlined in this specification if vendors expect to gain the benefits of ease of integration and interoperability for their custom IP blocks. Following this guidance will help ease the firmware development burden on BMC firmware stacks and overall general manageability of the system.

There are the key aspects of the M-PNP interoperability framework that are necessary for vendors to adhere to when implementing vendor-defined IP blocks.

- CSRs and Memory Map Requirements
- Internal Virtual Wire Framework
- Self-Description Via JSON

CSRs and Memory Map Requirements

Vendor defined IP blocks shall be mapped within the Primary Programmable Device (PD) memory map and added in the directory structure. Vendor defined IP blocks shall adhere to the common CSRs needed for discovery, status, and control.

Internal Virtual Wire Framework

All IP blocks with signaling connected to external interfaces, such as SGPIO or LTPI, shall include their signal mapping from the IP block's internal virtual wire list to the external signal and they shall use the internal virtual wire framework.

Self-Description

Vendor defined IP blocks shall provide a self-describing JSON description of their IP block which includes CSR descriptions for anything that the vendor desires to advertise for M-PNP supporting firmware to utilize it.

Following these frameworks allows for the BMC firmware to ingest the common description thus using common firmware modules to manipulate vendor defined IP block features.

9. Programmable Device (PD) Memory Map Architecture and Requirements

This section contains an overview and architecture and requirements for the PD memory map.

9.1. Programmable Device (PD) Memory Map Access Methods

This section outlines interfaces that can be used to access PD memory.

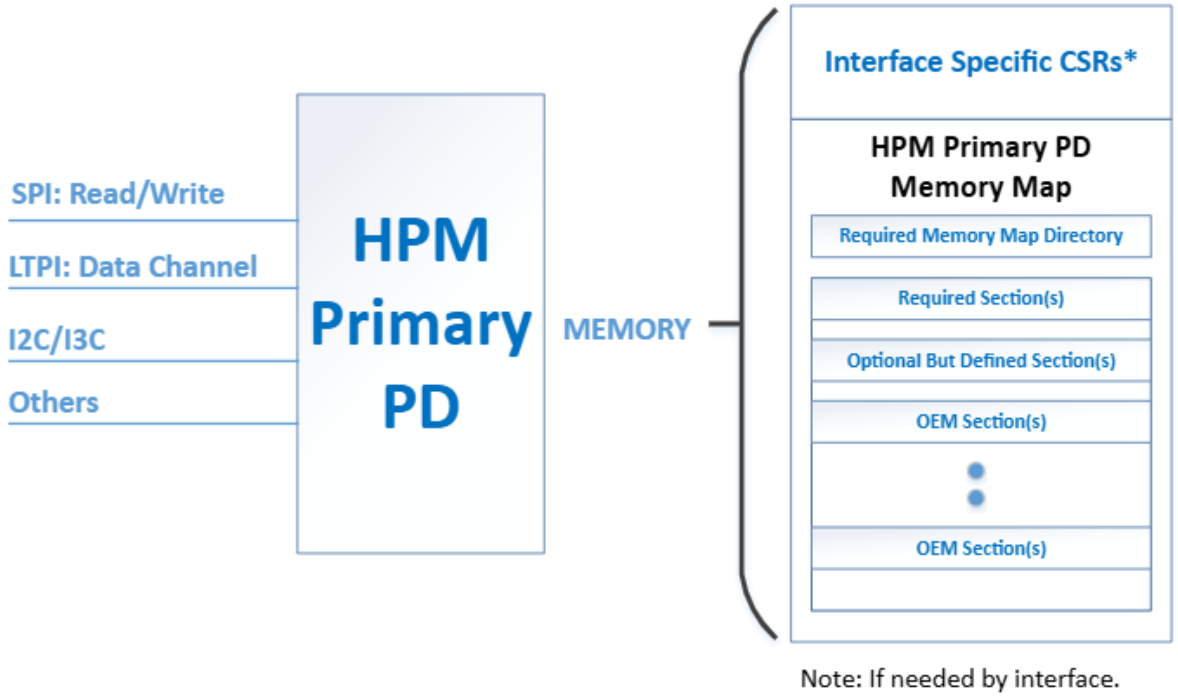


FIGURE 3: PROGRAMMABLE DEVICE (PD) MEMORY RANDOM ACCESS METHODS EXAMPLE

The above diagram is an example and not meant to illustrate every possible bus used to access the PD memory map. The following is a non-exclusive list of interfaces that can be used for random access to PD memory space.

TABLE 7: TYPICAL PRIMARY PROGRAMMABLE DEVICE (PD) AND SCM INTERFACE LIST

PD IP Block	Description
LTPI	Random access via LTPI data channel interface. This is specified in more detail in section 9.3.1.
SPI	Random access through generic SPI read/write. This is specified in more detail in section 9.3.2.
I2C / I3C Basic	Random access via I2C / I3C Basic mailbox interface. This is specified in more detail in section 9.3.3.
OEM Specific / Others	Undefined. Other interfaces may be used but are vendor defined and non-standard.

9.2. Programmable Device (PD) Memory Map Architecture

This section outlines the overall architecture of the memory map for the PD.

An example diagram of the memory map is provided below.

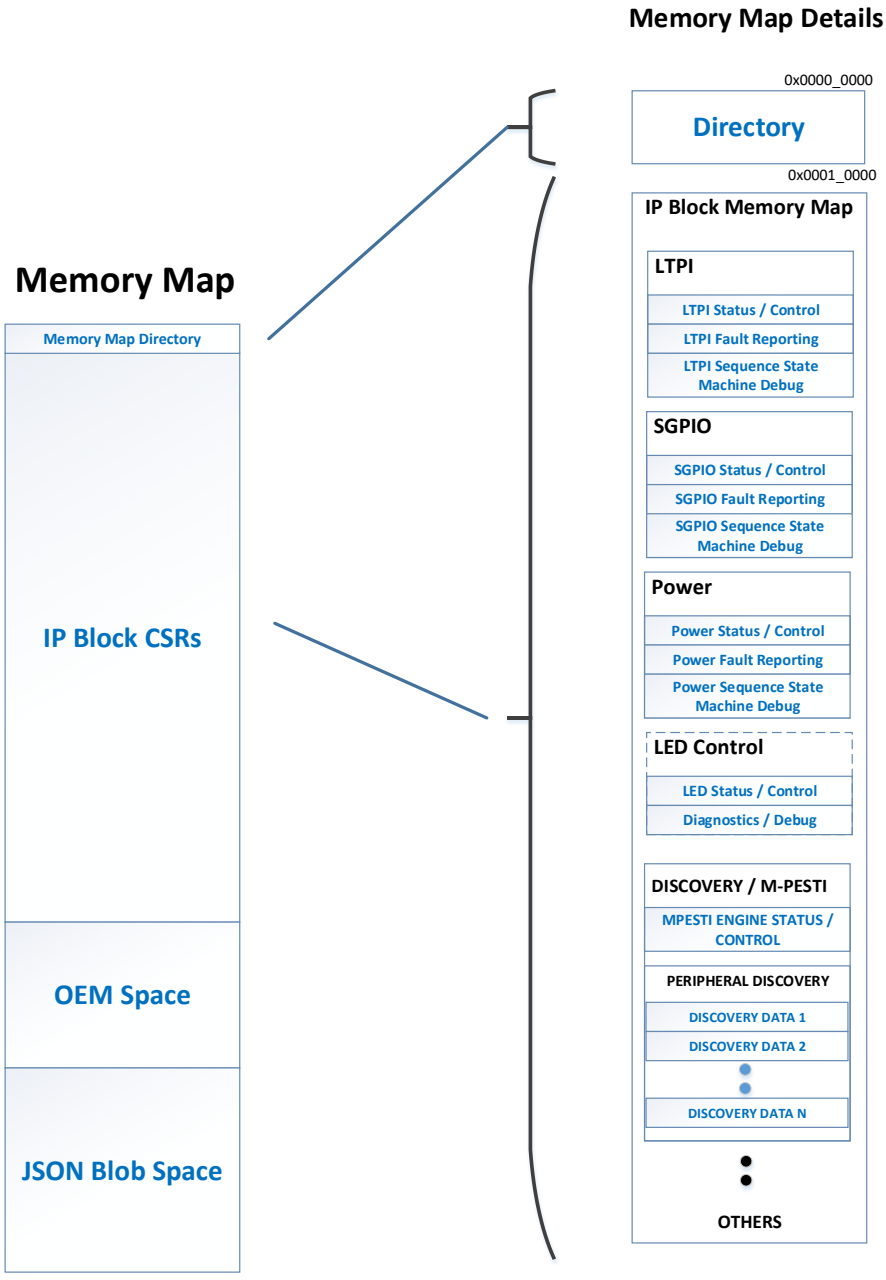


FIGURE 4: EXAMPLE MEMORY MAP ARCHITECTURE DIAGRAM

The memory map is divided into several spaces. Each space has an entry in the memory map directory. Each IP block CSR space also has an entry in the memory map directory.

- Directory: Shall contain a pointer to each space and size. It is assumed it is at offset 0 in the beginning of the memory map. Other than the JSON self-descriptive file location, directory entries may be optional if JSON description of locations of each IP block are provided for CSR locations within the JSON file. The logistical concerns regarding delivery of the JSON file to the BMC are outside of the scope of this specification. It is required that at least one entry that points to a statically mapped internal Programmable Device (PD) JSON file with minimum viable contents, such as IP block CSR locations, be provided for BMC consumption in the event that the full JSON descriptive file is not available. However, if the JSON file is provided as an addendum to the PD image, then the directory is optional.
- IP Block CSR Space: Each IP block entry will have its own CSR space. The CSR space shall have a set of required registers and may contain optional registers. The IP block CSR definition will be provided as part of the self-describing JSON. This may be either provided within an offset in the IP block space, in the file of JSON located in the JSON blob space or provided as a separate deliverable as part of the PD image. The HPM implementor shall provide the JSON file via one of these mechanisms.
- OEM Space: This is an optional space, and its definition is vendor defined. Usage and description are not required and out of scope of this specification.
- JSON Self-Descriptive File Space: This is an optional location where the JSON description of each IP block's CSRs will be located if it is provided as a monolithic file as well as memory map description with IP block offsets.

9.2.1. Memory Map Directory

This section outlines the memory map directory and provides a detailed description of its contents.

The memory map directory is depicted below.

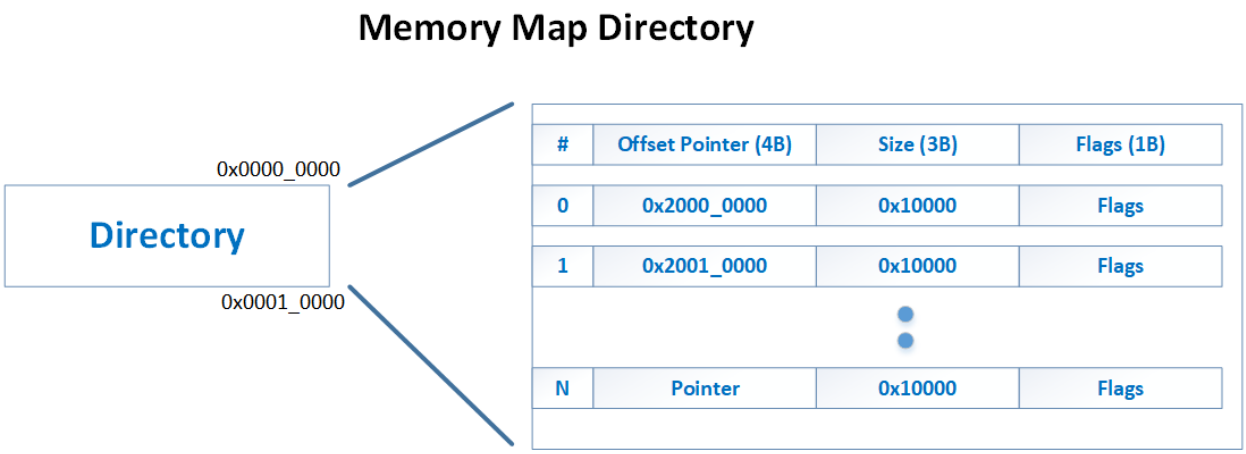


FIGURE 5: MEMORY MAP DIRECTORY

The directory is used by the BMC to assist in the self-discovery process. Various IP blocks will be mapped into the PD memory map space, and a reference pointer and size of space will be added to the directory at build time. The PD implementor shall set the space beyond the final entry to all 0s to indicate the end of the memory map directory since a zero pointer is reserved value.

Note: All register space sizes and offsets are for illustrative purposes only. The values shown are not required register location offsets and ranges. The PD JSON file shall contain the register offsets and ranges for a given implementation.

The contents are described below:

- Offset Pointer: This is a 4B pointer to the specific location of memory typically indicating the start of a specific IP block's CSR space.
- Size: The size of the space reserved for a given IP block CSR space. It is not necessary for the IP block to use all of the reserved space. This value will be in multiples of 4 bytes.
- Flags: Specific bits used as indicators. Currently defined as reserved.

The offset pointer plus size shall NOT overlap into a separate range pointed to by any other offset pointer. Lastly, the endianness for these values will always be in Big Endian format.

9.3. Programmable Device (PD) Memory Map Access

This section outlines the access methods for the PD memory map. Each sub-section outlines a different interface access method.

For each sub-section, the PD memory map in question is the exact same as described in the previous section. The HPM PD implementor shall implement the PD memory map random access through at least one of the interfaces illustrated below or via an interface documented within the self-describing JSON file.

The HPM primary PD implementor may provide access via any or all of these interfaces, including ones not explicitly mentioned. If more than one interface is implemented, all necessary access controls to prevent contention by the BMC are outside of the scope of this specification and are left to the HPM primary PD implementation if desired.

The diagram below provides an example architecture that depicts the hardware components involved when describing the various interfaces that access the PD memory map.

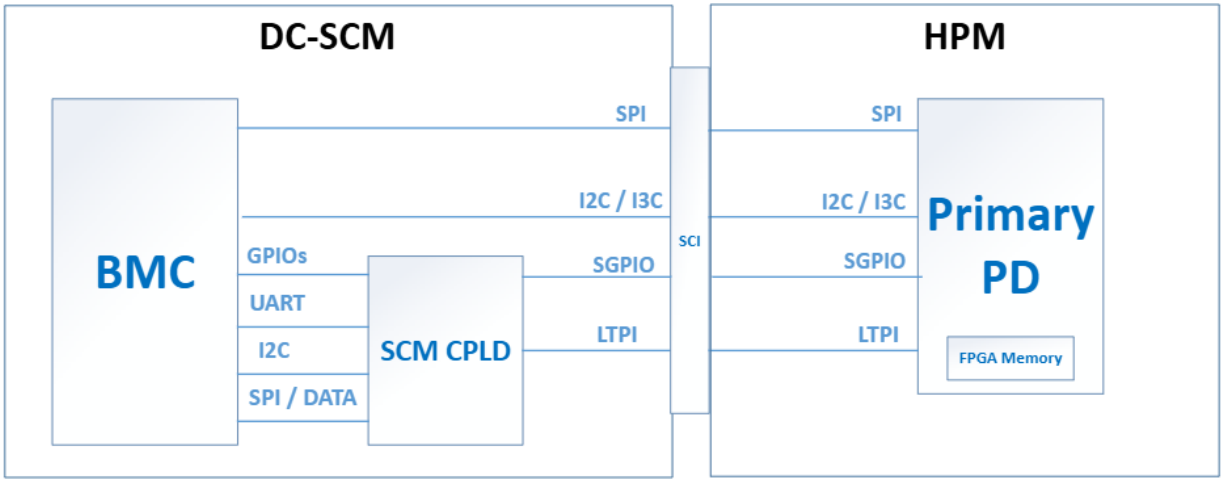


FIGURE 6: EXAMPLE PRIMARY HPM PD ACCESS ARCHITECTURAL BLOCK DIAGRAM

It is mandatory to implement either SGPIO or LTPI to interface with the primary HPM PD. All other interfaces are optional. Any specific DC-SCM implementation of either or all of outlined PD access interfaces, such as usage of an SCM CPLD, is outside the scope of this specification.

9.3.1. Memory Map Access Interface: LTPI

The diagram below depicts how the LTPI interface conceptually accesses the primary HPM Programmable Device (PD) memory map.

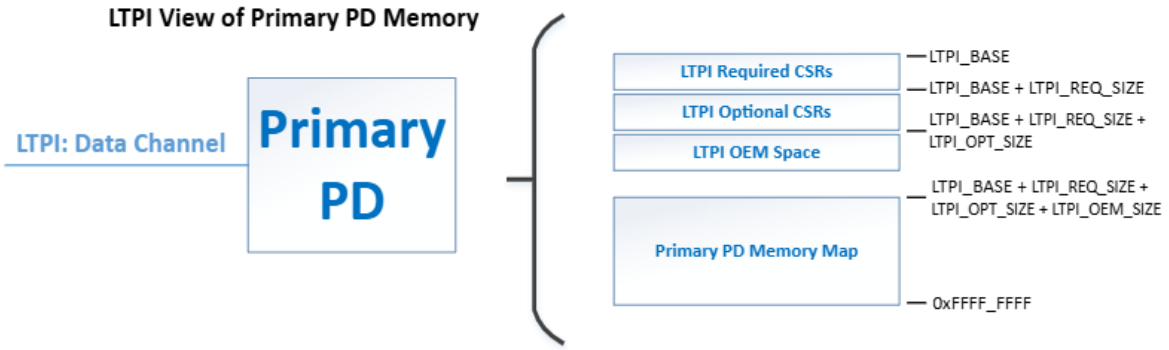


FIGURE 7: LTPI VIEW OF HPM PRIMARY PD MEMORY MAP

The intention for the LTPI data channel interface is to be able to access the PD's LTPI interface CSRs mapped at offset LTPI_BASE and its size is specified within the LTPI specification but will be represented herein as LTPI_REQ_SIZE. Additional spaces dedicated to the LTPI interface can be optionally added as addendum CSR spaces. It is recommended that PD memory map be mapped at an offset with enough space between the end of the LTPI CSRs and the start for the PD memory map.

The example above does not prescribe the specific offsets, and it will be up to the implementor to determine the size and offsets of each region. Please refer to the LTPI specification for further details regarding the LTPI data channel.

9.3.2. Memory Map Access Interface: SPI

The diagram below depicts how the SPI interface conceptually accesses the HPM primary Programmable Device memory map.

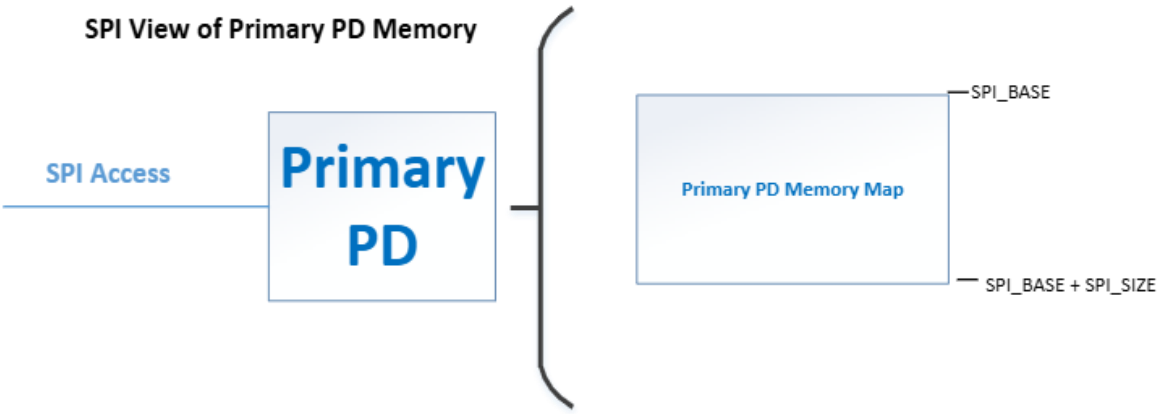


FIGURE 8: SPI VIEW OF HPM PD MEMORY MAP

Random access through the HPM primary PD SPI interface does not have interface specific CSRs located at offset 0 as this is considered a pass-through interface. Thus, the memory map can start at the base of 0x0000_0000 without needing to be located at an offset. However, this is not mandatory. We will reference the base for SPI access to be at SPI_BASE and the size of the space as SPI_SIZE.

9.3.3. Memory Map Access Interface: I2C / I3C

The diagram below depicts how the I2C or I3C interface conceptually accesses the HPM primary Programmable Device (PD) memory map.

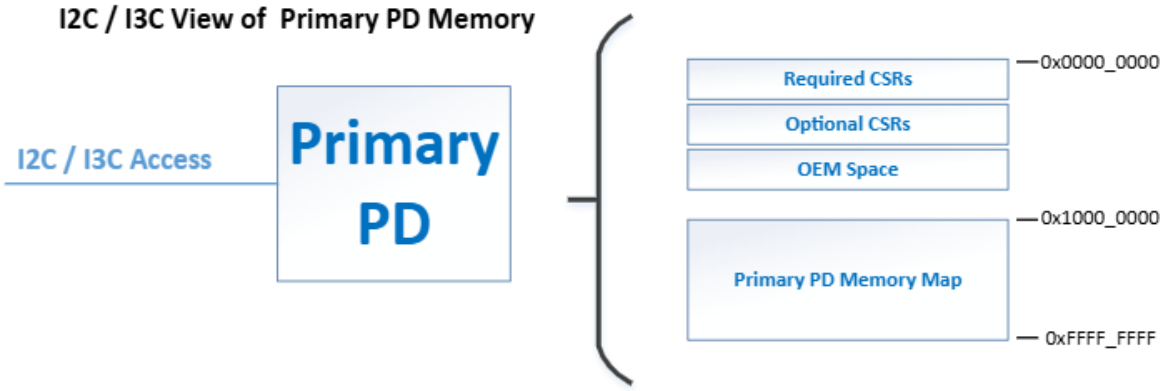


FIGURE 9: I2C / I3C VIEW OF HPM PD MEMORY MAP

The intention for the I2C or I3C interface is to be able to access the PD's I2C or I3C interface CSRs mapped at offset 0 through a simple EEPROM style read/write protocol. It is recommended that HPM primary PD memory map be mapped at an offset with enough space between the end of the interface CSRs and the start for the PD memory map. Additionally, if the implementation does not require specific IP block CSRs for the interface, then this will default to a pass-through link and simply provide flat access to the PD memory map without an offset.

The example above does not prescribe the specific offsets, and it will be up to the implementor to determine the size and offsets of each region.

9.3.4. Memory Map Access Contention Considerations

The HPM primary PD can provide access to the PD memory map using any or all of the interfaces outlined in this specification. It is incumbent on the DC-SCM BMC to choose which

interface it prefers, and this specification does not teach a mechanism to prevent access contention if the BMC uses multiple interfaces to access the same data.

10. Programmable Device (PD) JSON Schema Representation

This section provides details as to how the PD is represented within the JSON schema used for self-description. This includes general information, IP blocks, memory map, CSR descriptions, and virtual wire details and mappings.

10.1. Programmable Device (PD) JSON Schema Representation Overview

This section provides a diagrammatic overview of how the PD is represented within the JSON schema.

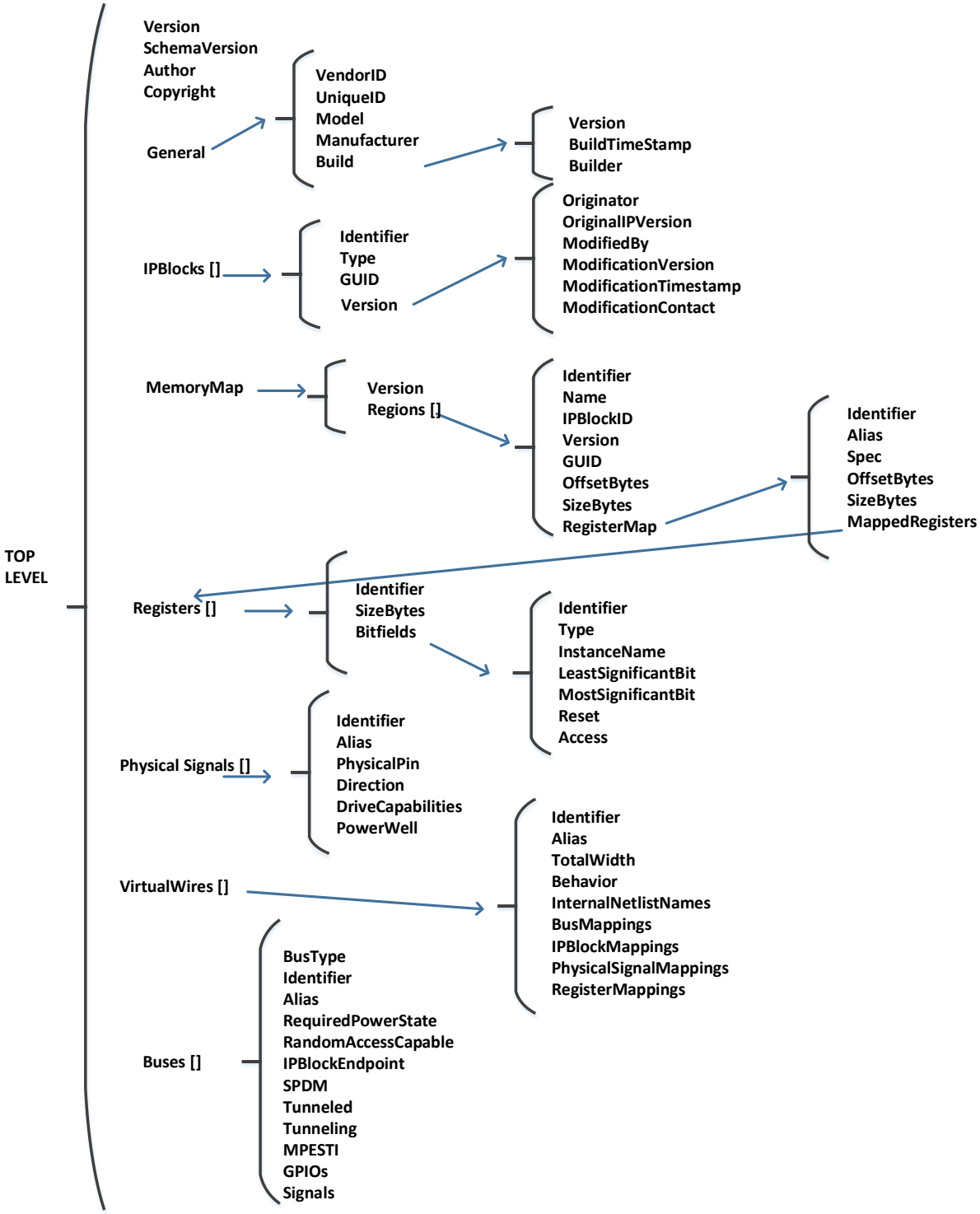


FIGURE 10: PD IP BLOCKS JSON REPRESENTATION

As shown in the above diagram, the top level for the PD JSON schema contains a set of key sections that are discovered and populated by the PD discovery process or provided via a static file either collected during the discovery process or included as a separate JSON file with the PD image.

Please see the document, "**OCP MHS Modular Plug-and-Play Programmable Device Schema Document",** for further details regarding the specific JSON attributes.

10.2. JSON Schema Reference Documentation

For details regarding the JSON attributes, please consult the following documents:

- The "**OCP MHS Modular Plug-and-Play Programmable Device Schema Documentation",** is a document that contains a description of the Programmable Device JSON schema attributes.
- The "**OCP MHS Modular Plug-and-Play Hardware Component Schema Documentation",** is a document that contains a description of the JSON schema attributes for hardware components.

11. Programmable Device Internal Signal Virtual Wires Support

Internal Signal Virtual Wires are the superset of internal fixed function signals and run-time programmable signals produced or consumed by one or more IP blocks are described within the descriptive section of that specific IP block.

For pass-through signal support, certain physical GPIOs may be mapped to one or more tunneling interfaces from IP blocks without having specific internal logic or CSRs. For description of how this is performed, please refer to those interface sections.

11.1. Programmable Device Internal Signal Virtual Wire Mapping Framework

This section provides a description of the virtual wire mapping framework that will be used by the PD to map the set of internal virtual wires as defined in the PD to various physical GPIOs, CSRs, and transport interfaces for consumption by external entities, such as the SCM BMC.

A conceptual diagram is provided below.

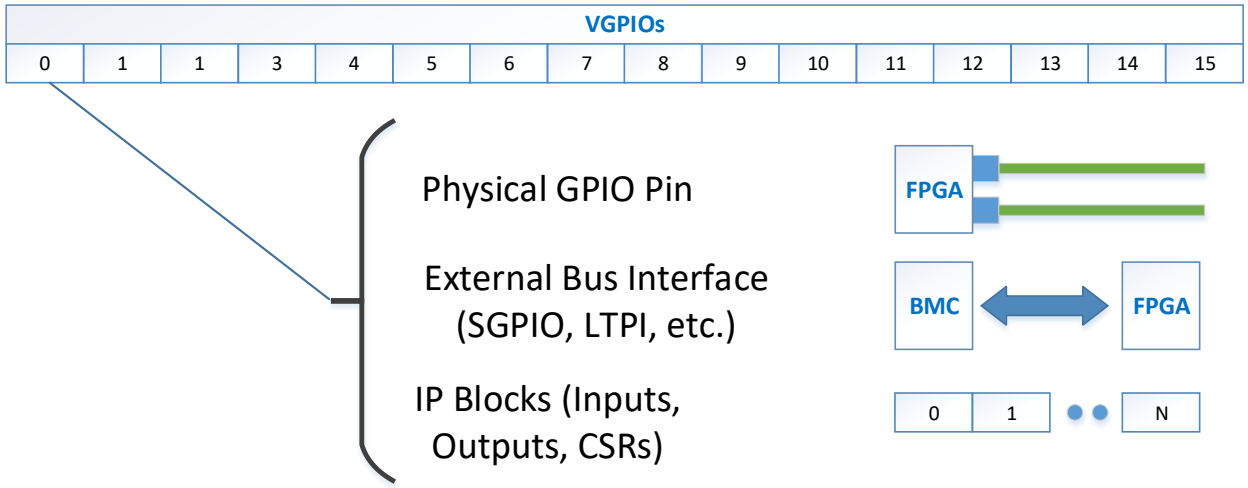


FIGURE 11: INTERNAL VIRTUAL WIRES MAPPING FRAMEWORK

Internal virtual wires can be mapped by the PD in several unique ways. This framework allows flexibility for real physical GPIOs or virtualized GPIOs produced by one or more IP blocks to be mapped through various external interfaces and consumed by the system BMC on the SCM or other consumers on the HPM.

Physical GPIO Pins: The PD can support physical GPIO pins connected to routed physical signal traces on the HPM. A virtual wire is an abstraction that the PD can use to map physical GPIO signals that IP blocks manipulate to virtualize GPIOs that it maintains.

External Bus Interfaces: The PD can transport the state of the virtual wire via external bus interfaces (SGPIO, LTPI, etc.) by creating a mapping between the virtual wire and that interface. See the specific interface's chapter for details on the mechanism by which it can transport virtualized wires as external bus tunneled GPIOs.

IP Blocks: IP Blocks implemented in the PD can produce or consume inputs and output signals. These signals can be mapped to virtual wires which will reflect the state of those signals. Additionally, the IP Block can map the state of the signal into its CSRs.

The mapping for all these cases will be described within the JSON self-describing information file which is provided as additional supplemental material with the PD image. The next section describes how this works.

11.2. Programmable Device Internal Signal Virtual Wire Mapping Description Overview

This section provides an overview of how the virtual wire mapping is described using the self-describing JSON using the mapping framework.

The diagram below conceptually indicates how an internal PD virtual wires can be mapped to external interfaces, registers, IP blocks, and physical signals. This is expressed within the PD JSON self-describing data file provided alongside the PD image file.

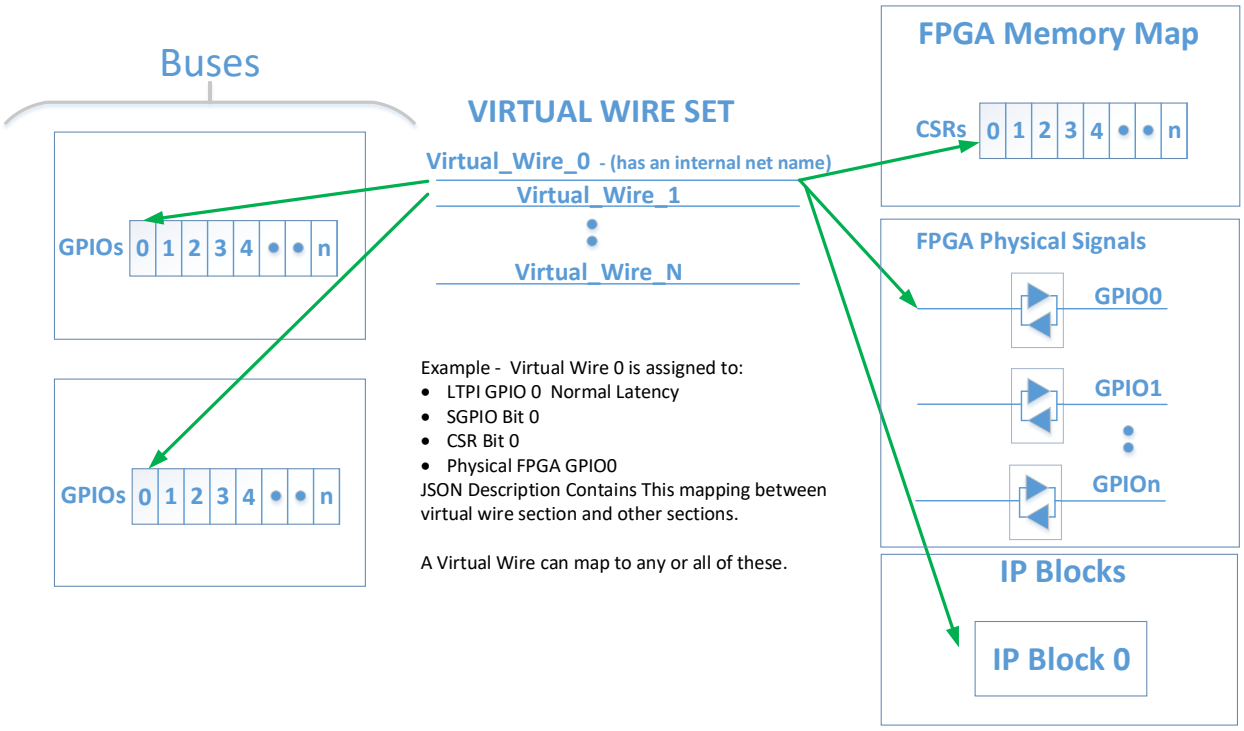


FIGURE 12: INTERNAL SIGNAL VIRTUAL WIRE MAPPING DIAGRAM

Additionally, a set of internal virtual wires with consecutive naming can be treated as a block and represented in block form. The diagram below provides a conceptual depiction of this.

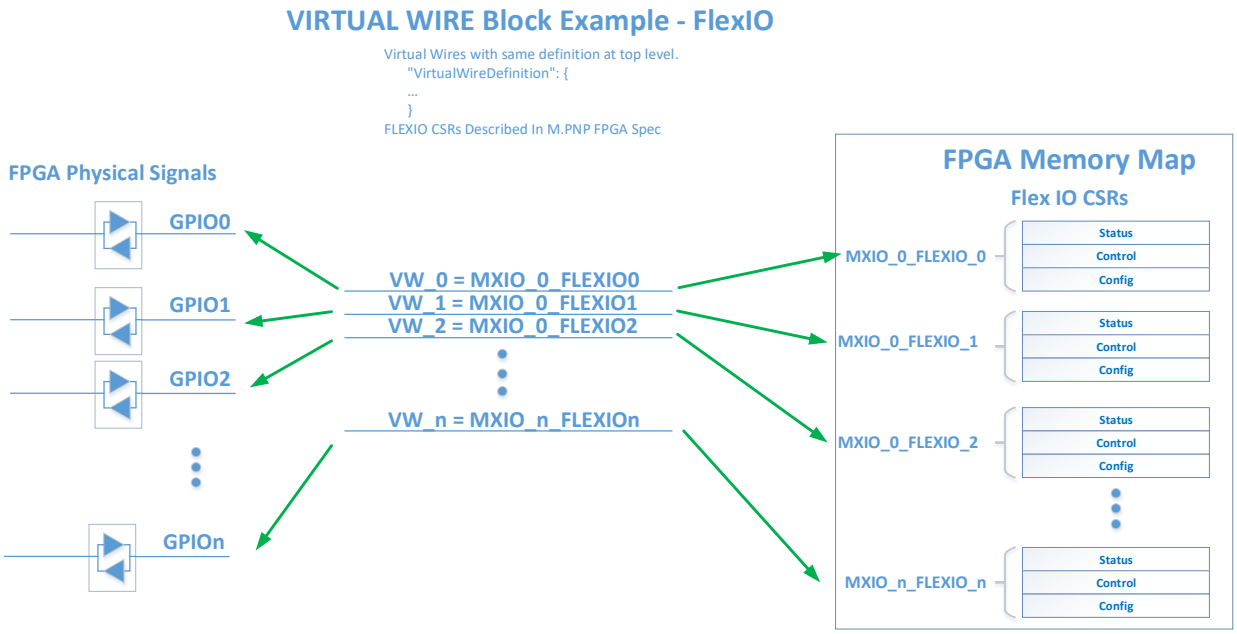


FIGURE 13: INTERNAL SIGNAL VIRTUAL WIRE BLOCK LEVEL MAPPING EXAMPLE

This diagram uses the MXIO FlexIO set of signals as an example of when a block descriptive form would be appropriate to promote the conciseness and efficiency of the JSON representation. This representation makes use of the fact that the FlexIO CSRs are defined within the MPNP.HPM_Peripheral_PD specification and can simply reference the specification instead of defining each CSR and individual mapping within the JSON file.

The next section describes in detail how the JSON representation manages to capture these mappings.

11.3. Programmable Device Internal Signal Virtual Wire Self-Describing JSON Mapping Framework

This section details the PD virtual wire mapping via the self-describing JSON mapping schema which is a sub-section of the PD JSON self-describing virtual wire schema.

Within the JSON schema, a section for each described internal signal virtual wire contains a sub-section which iterates though each of the interfaces that are mapped to the internal signal virtual wire or set of internal signal virtual wires. It is possible that more than one interface is mapped to the internal signal virtual wire. For instance, it may be that a internal signal virtual wire is mapped to a physical signal connected to a specific physical pin on the PD package, and thus HPM planar, as well as being mapped to a specific bit in a packet that is being transported via an external interface, such as the SGPIO interface.

The main mapping sections and mapping support sections are outlined below:

- **InternalNetlistMappingNames:** Set of internally used names for the internal signal virtual wire or set of internal signal virtual wires described as a block.
- **BusMappings:** Set of buses that the internal signal virtual wire or internal signal virtual wire block is mapped to.
- **IPBlockMappings:** References to the IP block identifier supporting this bus interface.
- **PhysicalSignalMappings:** The set of physical signals that the internal signal virtual wire or internal signal virtual wire block is mapped to.
- **RegisterMappings:** The set of CSR regions and specific registers that the internal signal virtual wire or internal signal virtual wire block is mapped to.

Please see the document, "**OCP MHS Modular Plug-and-Play HPM and Peripheral PD Schema Document",** for further details regarding the specific JSON attributes.

11.4. Programmable Device Internal Signal Virtual Wire Self-Describing JSON Examples

Example mock-ups of self-describing JSON for virtual wires are provided in the PD JSON mock-up within the release package. See section 7.4, "MHS HPM and Peripheral Programmable Device Specification Supplementary Materials ", for additional collateral details.

11.5. Required Programmable Device Internal Signal Virtual Wires List

The table below contains the set of required and optional recommended virtual wires defined by the PD. The set of virtual wires are logical, internally represented signals that can be manifested in one of two ways:

- A signal presented to the PD through an external source (physical GPIO pin, tunneled interface, etc.)
- A signal sourced by the PD from one or more of its internal IP blocks (e.g. output from a specific IP block or wire OR 'ed shared signal like an interrupt pin)

TABLE 8: REQUIRED HPM PRIMARY PROGRAMMABLE DEVICE INTERNAL SIGNAL VIRTUAL WIRES

Name	DIR	Sourced	Description
Global Interrupt	OUT	All IP Blocks	Wire OR'ing of all IP block interrupt sources.

Power State	OUT	Power Management	See power management chapter
Power State Transition Status Flag	OUT	Power Management	See power management chapter
Power State Transition Request Flag	IN	Power Management	See power management chapter
Power State Transition Target	IN	Power Management	See power management chapter

11.6. PDHPM Primary Programmable Device Internal Signal Virtual Wire Supplementary Materials

Supplementary materials related to Virtual Wires can be found in section 7.4, “Specification Supplementary Materials”.

12. Programmable Device Logical IP Block Descriptions and Requirements

This section contains the PD logical block requirements regarding required functionality and descriptions thereof. This includes the internal logical block CSRs, and all required inputs and outputs are defined as required for implementation.

12.1. Logical IP Blocks Abstraction

This section contains a logical IP block abstraction diagram and description for use as a template for specifying the logical IP block.

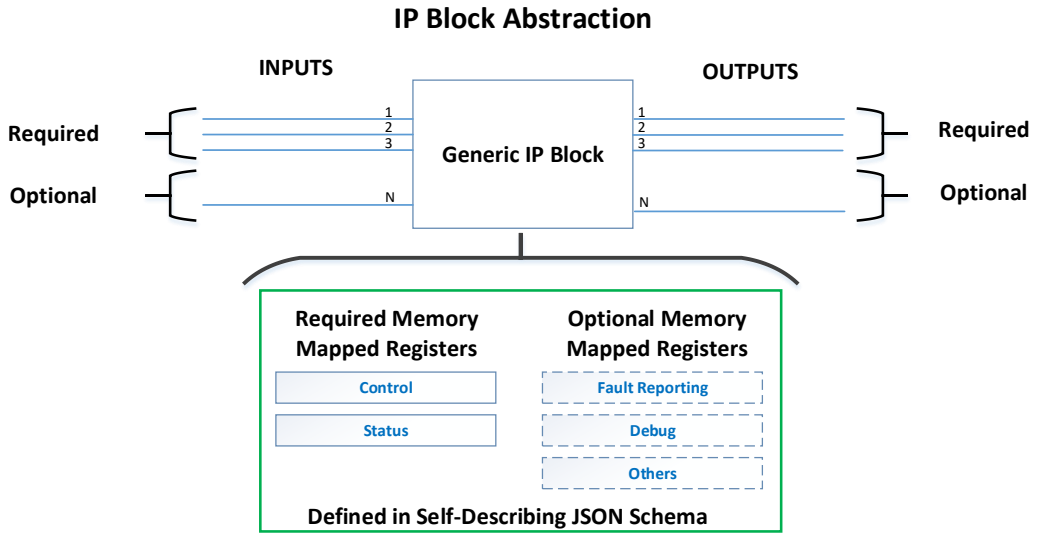


FIGURE 14: GENERIC LOGICAL IP BLOCK ABSTRACTION DIAGRAM

The diagram defines these specific attributes that a logical block specification shall define.

- Inputs: Set of required and optional inputs as consumed by the IP block.
- Outputs: Set of required and optional inputs as produced by the IP block.
- Required CSRs: Control and Status registers that shall be provided by the IP block to allow the BMC to manipulate IP block functionality.
- Optional CSRs: Additional Control and Status registers that can be provided for such uses as debug, performance monitoring, and others. Optional registers are recommended to be defined within the self-describing JSON file.
- Self-describing JSON Data: Provided in the JSON file and adheres to the schema.

It is not required for any IP block to implement inputs or outputs as that is dependent on the specific requirements of the implementation.

12.1.1. Common Required Memory Mapped CSRs

Each logical block shall provide a common set of Control and Status Registers (CSRs).

12.1.1.1. Required and Optional IP Block CSRs List

This section contains the Control and Status Registers (CSR) requirements for standardized IP block functionality.

TABLE 9: REQUIRED AND OPTIONAL IP BLOCK CSR LIST

Name	REQ	OPT	Size (Bytes)	Description
IP Block ID	Y		2 B	Unique ID to identify the IP block using a 2-byte ID format. This is used as a reference into the JSON file as the HWID attribute.
IP Block Version	Y		2 B	Major.Minor into 2 bytes as 1 byte each
Global Status	Y		1 B	Common status register
Global Control		Y	1 B	Common control register. Indicated as optional as not every IP block requires reset control.
Interrupt Status		Y	1 B	IP Block Interrupt Status. Indicated as optional as not every IP block requires interrupt support.
Interrupt Control		Y	1 B	IP Block Interrupt Control. Indicated as optional as not every IP block requires interrupt support.
Extended Status		Y	1B	Optional extended vendor defined status register.

12.1.1.2. Common IP Block CSR: IP Block ID

The intention of the IP block ID is to enable IP block producers to map a locally unique IP block ID to the JSON contents for that IP block. This allows the JSON file to provide additional descriptive information for that IP block to be consumed as JSON self-describing data by the BMC as part of a richer set of data used for broader PD self-description.

The PD shall implement a 2-byte ID for each IP block, and it shall be a unique ID as this is used to reference a specific IP block within the JSON file. This ID is unique only to the specific design and is merely meant as a mapping tool to correlate the IP block to the description within the JSON file.

12.1.1.3. Common IP Block CSR: Version

The IP block version field is provided to enable fine-grained tracking of versioning information.

Version information is encoded in the following manner:

Major ID 1B, Minor ID 1B

IP Block Version Register															
15	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Major Version								Minor Version							

Additional versioning information can be carried in the JSON file.

12.1.1.4. Common IP Block CSR: Status

The common IP block status register is defined below:

TABLE 10: COMMON IP BLOCK CSR STATUS REGISTER DEFINITION

Name	Bitfield	Type	Bit Field Definition
IP Block General Status	0	RWC	**0 = Status OK** **1 = Error Condition** **Write 1 to clear**
IP Block Error Severity	3:1	RO	0x00 = Info, 0x01=Warning, 0x02=Caution, 0x03 =Critical, 0x04 – 0x07 Reserved
IP Block Error Codes	7:4	RO	**Status** **State** / **Encoding** Status OK: 0x00 Unspecified Error State: 0x01 Configuration Error: 0x02 Communication Error: 0x03 Resource Unavailable Error: 0x04 Reserved: 0x05 – 0x0D See Extended Optional OEM Defined Error Code: 0x0x0F

The error code shall represent the most recently reported error condition as detected by the PD.

12.1.1.5. Common IP Block CSR: Control

The common IP block control register is defined below:

TABLE 11: COMMON IP BLOCK CSR CONTROL REGISTER DEFINITION

Name	Bitfield	Type	Bit Field Definition
IP Control	7:1	RW	Reserved – writes shall not take any effect.
	0	RW	**Write 1 to reset IP Block**

12.1.1.6. Common IP Block CSR: Interrupt Status

The common IP block control register is defined below:

TABLE 12 : COMMON IP BLOCK CSR INTERRUPT STATUS REGISTER DEFINITION

Name	Bitfield	Type	Bit Field Definition
Interrupt Status	7:2	R/W	Reserved – writes shall not take any effect.
	1	R	Current Interrupt Signal State
	0	RW	**Latched Interrupt Status Indication - Write 1 to clear**

12.1.1.7. Common IP Block CSR: Interrupt Enable

The common IP block control register is defined below:

TABLE 13: COMMON IP BLOCK CSR INTERRUPT ENABLE REGISTER DEFINITION

Name	Bitfield	Type	Bit Field Definition
Interrupt Enable	7:1	RW	Reserved – writes shall not take any effect.
	0	RW	**1 = Enabled, 0 Disabled**

12.1.1.8. Common IP Block CSR: Extended OEM Defined Status Register

The common IP block extended OEM defined status register is vendor defined. It may contain a bit field for extended OEM defined error conditions.

TABLE 14: COMMON IP BLOCK CSR EXTENDED STATUS REGISTER DEFINITION

Name	Bitfield	Type	Bit Field Definition
Error Code	7:0	RW	Vendor defined code

12.2. Power Sequencing and Management Logical IP Block

This section contains the logical IP block description for the power sequencing requirements and descriptive abstraction for this IP block.

12.2.1. Power Sequencing Conceptual Overview

This section contains an overview of the typical power sequencing functionality for a typical server.

The intention of the power sequencing and management logical IP block is to provide the BMC with a common set of control and status interfaces for controlling the system power state

including HPM, peripherals, and options. This provides a common set of power states that are reported as well as the mechanism by which the BMC can control the generic system power state machine logic.

This specification does not dictate how the PDPD vendor implements the power control and sequencing state machine, nor does it specify how to manage a specific silicon vendor's unique power sequencing requirements.

12.2.2. System Power States Definition

This section contains the system power states used by the HPM primary Programmable Device to communicate the HPM's power status at any given point in the power sequence.

The system power state and status indication will be reflected in the set of CSRs. Additionally, the power state bitfields will be reflected as a virtual wire bitfield such that they can be mapped to an external interface for direct BMC consumption.

The following table outlines the PD power states and descriptions for each power state as well as its encoding.

Note: The state machine encodings below are non-monotonic relative to the documented transition order and not explicitly following a grey coding format.

TABLE 15: SYSTEM POWER STATES DEFINITION

SCM Power State	HPM PD Power Management State	PD Encoding	SCM Encoding	Description
G3	G3	0000000b	000000000b	Mechanical off. State as defined by SCM specification.
Pre-Standby	Pre-Standby	0000000b	000000010b	State as defined by SCM specification.
Standby	Standby Hold-Off	0000000b	000000011b	Standby Hold-Off for PD and other HPM device attestation
	Standby Run	0000001b	000000111b	Standby Run for main standby state
Soft Off (S5)	S5 Hold-Off	0000010b	000001011b	S5 Hold-Off for attestation
	S5 Run	0000011b	000001111b	S5 Run for main S5 state
S0	S0 SoC Ramp	0000101b	000010111b	S0 main power on except SoC
	S0 Hold-Off	0000100b	000010011b	S0 Hold-Off for attestation

	S0 Run BIOS Boot	0000110b	000011011b	S0 Run for SoC enabled to execute code
	S0 Run OS	0000111b	000011111b	S0 Run for allow to boot to OS

The PDHPM primary PD cannot produce the pre-operational early power states from G3 to Standby Hold Off as it is still either powered off or in reset. Thus, all pre-operational states are represented as 0b by the PDPD. The SCM shall keep a separate state variable to keep track of these states before the PDPD is operational. The SCM power state representation is a concatenation of the PDPD power state plus the SCM's pre-operational power state variable. Control of peripheral power states is handled by a combination of control via the system power state variable and the setting of the power policy settings plus the PWRON_ALLOW setting within the M-PESTI target control register. The M-PESTI target control register also is responsible for peripherals that only support simple PESTI / cable presence. Please see section 12.8, "M-PESTI Interface Logical IP Block", for details.

Implementation-specific states shall be mapped to an HPM primary PD power management state. Vendors shall report any vendor defined unique HPM primary PD power management states in JSON file for BMC management firmware to consume.

Additional diagnostic states reflective of internal power sequencing state machines are out of scope of this specification and are at the discretion of the PD implementor and HPM vendor.

12.2.3. System Power States Transition Diagram

This section contains the system power states transition diagram to be followed by the primary HPM PD to implement the required power sequencing.

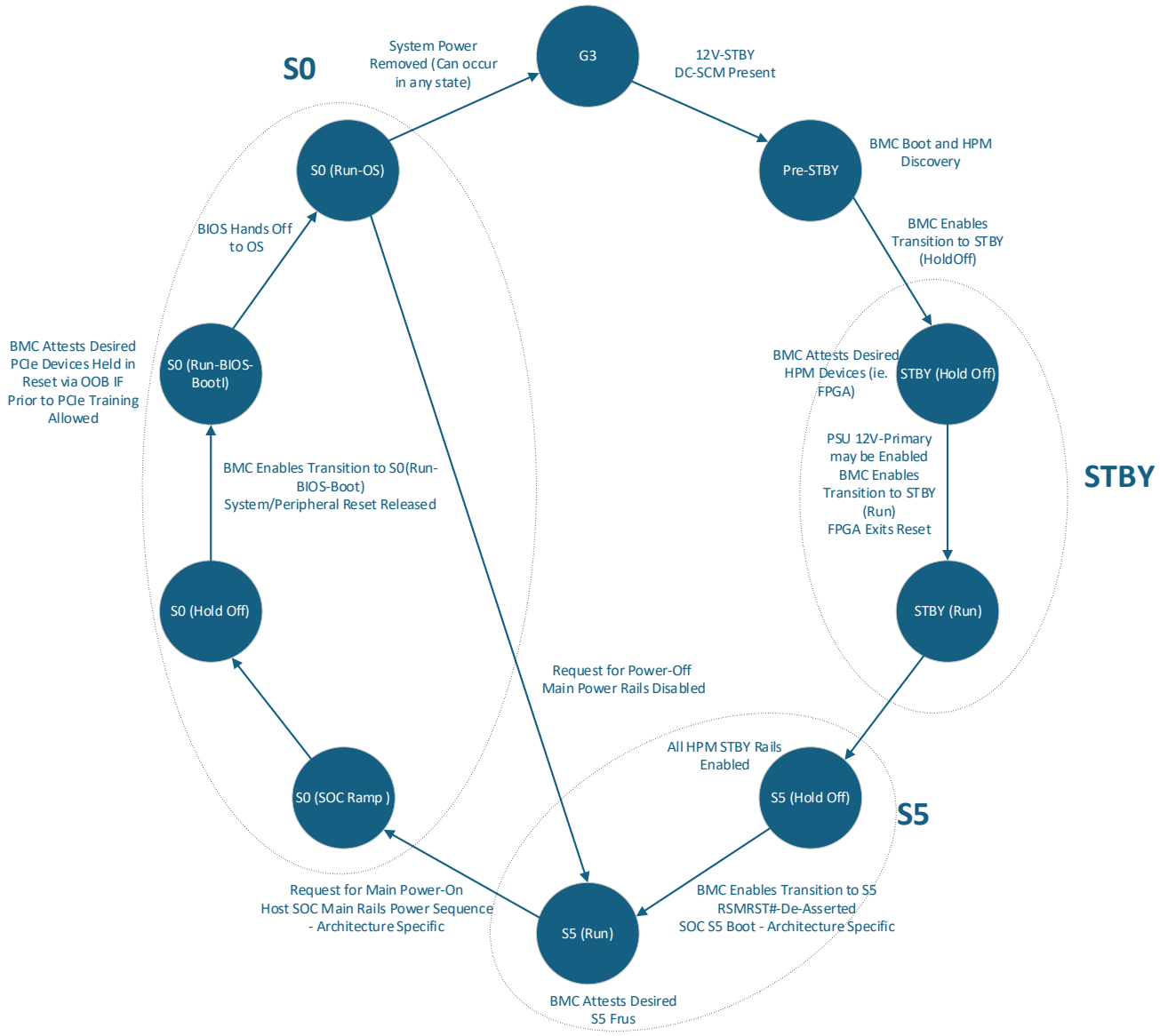


FIGURE 15: POWER STATE TRANSITION DIAGRAM

Assertion of THERMTRIP_N by any SoC in the platform when in S0 shall cause the system to immediately transition to S5 and disable all S0 enabled voltage regulators.

12.2.3.1. Power State Transition Description

The following table shows the expected power and sub power states in an M-PNP system. In addition, the table shows a high-level system power rail availability in each power state. Power

state transition details column provides information on what tasks might occur in specific states and examples of what is required to transition out of a state.

TABLE 16: POWER STATE TRANSITION TABLE

PD Power State	System Power State Description	Power State Transition Details	Power State as Defined in DC-SCM 2.x Specification
G3	Coin Cell Battery Power Only No PSU Power Present	1. DC-SCM Seated enables PSU 12V-STBY Delivery to DC-SCM 2. Transition to Pre-STBY	G3
Pre-STBY	12V-STBY to DC-SCM HPM 3.3V STBY to FRU	1. DC-SCM Powered and Boots – HPM Discovery, HPM Aux Rails Enabled 2. HPM_STBY_EN Asserted by DC-SCM 3. Transition to STBY (Hold Off)	Pre-STBY
STBY (Hold Off)	DC-SCM Powered HPM Aux Rails in Regulation	1. HPM_STBY_RDY is Asserted by HPM when PD and other relevant device rails are in regulation. 2. BMC Authentication/Attests Desired HPM Devices (Ex. HPM PD) as per security policy.[5] 3. BMC/DC-SCM de-asserts HPM_STBY_RST# 4. HPM PD exits reset and is functional. 5. Transition to STBY (Run)	STBY
STBY (Run)	DC-SCM Powered HPM Aux Rails in Regulation	1. DC-SCM<->HPM-PD Communication Available 2. BMC enables transition to S5 (Hold Off) by setting PD power management state transition CSR **OR** via LTPI / SGPIO GPIO bit field. 3. PSU 12V-Primary may be enabled based on 12V load requirements. 4. Transition to S5 (Hold Off)	STBY
S5 (Hold Off)	All Aux Rails in Regulation	1. BMC Authentication/Attests Desired HPM Devices / FRU storage devices as per security policy.[6] 2. BMC Sets Transition to S5 Run in PD power management state transition CSR **OR** via LTPI / SGPIO GPIO bit field. 3. PD de-asserts S5 specific component resets 4. RSMRST# De-Asserted 5. Transition to S5 (Run)	S5
S5 (Run)	All Aux Rails in Regulation	1. Request for Main Power-On (Ex. Power Button) 2. Main Power Rails Enabled (12V Primary) 3. Transition to S0 (SOC-Ramp)	S5
S0 (SOC-Ramp)	All (Aux/Main) Rails in Regulation	1. Host SOC Rails Ramp 2. Transition to S0 (Hold Off)	S0
S0 (Hold Off)	All (Aux/Main) Rails in Regulation	1. BMC Authentication/Attests Desired HPM Devices (Ex. Host SOCs) as per security policy.[7] 2. BMC enables transition to S0 (Run-BIOS-Boot) by setting PD power management state transition CSR **OR** via LTPI / SGPIO GPIO bit. 3. Transition to S0 (Run-BIOS-Boot)	S0
S0 (Run-BIOS-Boot)	All (Aux/Main) Rails in Regulation	1. BMC Authentication/Attests Desired Devices prior to PCIe Training (Ex. PCIe Devices using OOB IF to SOC) 2. BMC/BIOS(UEFI) FW hands off to OS (Run-OS)	S0

[5] Note: Security policy is up to system integrator.
[6] ibid
[7] ibid

S0 (Run-OS)	All Rails (Aux/Main) in Regulation	1. OS Boot	S0

12.2.3.2. Power State Transition vs. Power and Reset States by Component

The tables below outline the power and reset state per each component type in the system vs. the power state.

Table Legend

Red: No

Green: Yes

Yellow: May be either

TABLE 17: POWER AND RESET STATUS PER POWER STATE BY MAJOR SYSTEM COMPONENT TYPE

	PDB		HPM				HPM Primary PD / CPLD		SoC		
Power State	12V STBY On	12V PRIM On	STBY Power On	RSMRST# De-Asserted	Main Power On	System Reset# De-Asserted	Power On	PD Reset De-Asserted	STBY Power On	Main Power On	Reset# De-Asserted
G3											
PRE-STBY											
STBY (Hold Off)											
STBY (Run)											
S5 (Hold Off)											
S5 (Run)											
S0 (SoC Ramp)											
S0 (Hold Off)											
S0 (Run-Boot-BIOS)											
S0 (Run-OS)											

TABLE 18: POWER AND RESET STATUS PER POWER STATE BY NON-ENDPOINT COMPONENT TYPE

	Non-Endpoint Peripherals										
	Riser		Storage Backplane			Fan Board		Control Panel	PCIe Switch		
Power State	STBY Power On	Main Power On	STBY Power On	Main Power On	Reset# De-Asserted	STBY Power On	Main Power On	STBY Power On	STBY Power On	Main Power On	Reset# De-Asserted
G3											
PRE-STBY											
STBY (Hold Off)											
STBY (Run)											
S5 (Hold Off)											
S5 (Run)											
S0 (SoC Ramp)											
S0 (Hold Off)											
S0 (Run-Boot-BIOS)											
S0 (Run-OS)											

TABLE 19: POWER AND RESET STATUS PER POWER STATE BY ENDPOINT COMPONENT TYPE

	Endpoint Peripherals			
Power State	STBY Power On	STBY Reset# De-Asserted	Main Power On	Reset# De-Asserted
G3				
PRE-STBY				
STBY (Hold Off)				
STBY (Run)				
S5 (Hold Off)				
S5 (Run)				
S0 (SoC Ramp)				
S0 (Hold Off)				
S0 (Run-Boot-BIOS)				
S0 (Run-OS)				

12.2.4. Power Sequencing and Management IP Block Architecture

This section contains the general architecture for the power sequencing and management IP block.

The following figure provides a conceptual illustration of the abstract architecture for the power management IP block.

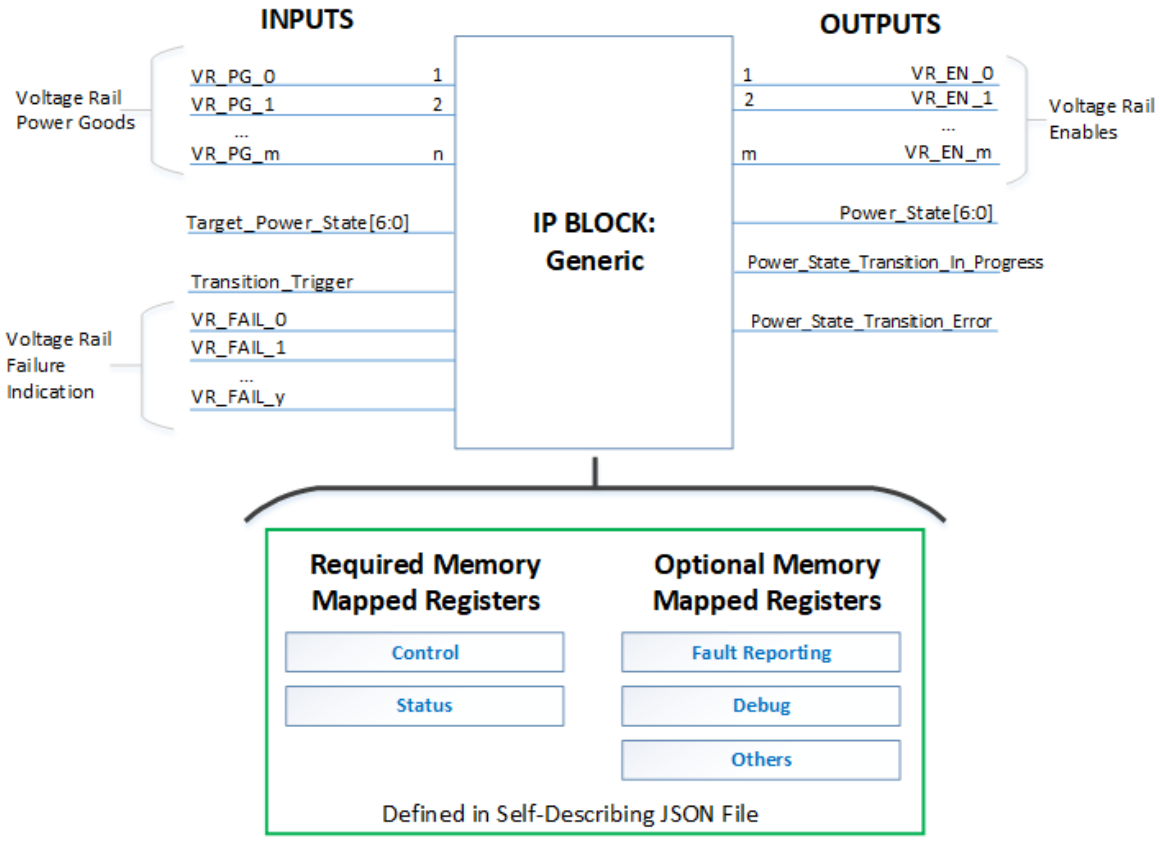


FIGURE 16: POWER MANAGEMENT IP BLOCK ABSTRACT ARCHITECTURE

The JSON self-describing information for the HPM primary PD shall provide a reference to the specification for all required and optional registers. Please refer to the JSON self-describing information section for more details on how this is accomplished.

12.2.5. Power Sequencing and Management IP Block Requirements

This section contains the general abstract architecture for the power sequencing and management IP block.

12.2.5.1. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 20: POWER MANAGEMENT IP BLOCK REQUIRED AND OPTIONAL FEATURE SET

Name	REQ	OPT	Description
System Power State Indication and Transition Status	Y		A bitfield that indicates the main state and a bitfield that indicates the sub-state for the system power as defined in this specification.
System Power State Control	Y		Contains power state transition request flag and target state.
Main Power Good Indication	Y		Aggregated main power rail power good indication signal as defined in CSRs.
Standby Power Good Indication	Y		Aggregated standby power rail power good indication signal as defined in CSRs.
Aggregated Main Voltage Failure Indication	Y		Aggregated main voltage power rail failure indication signal as defined in CSRs.
Aggregated Standby Voltage Failure Indication	Y		Aggregated standby voltage power rail failure indication signal as defined in CSRs.

12.2.5.2. BMC Visible Power Management IP Block Inputs

The table below contains the set of required and optional recommended BMC visible power management IP block inputs signals. These will be reflected in IP block CSRs and transported via tunneled GPIOs either through the LTPI or SGPIO interfaces. See the LTPI and SGPIO chapters for more information regarding tunneled GPIOs.

TABLE 21: POWER MANAGEMENT IP BLOCK REQUIRED AND OPTIONAL INPUTS

Name	REQ	OPT	Description
Power State Transition Field	Y		Indicates the power state that the BMC is requesting the PD to transition to.
Power State Transition Trigger	Y		This indicates that the BMC is requesting the PD to transition to the power state as indicated in the Power State Transition field.
Power Supply Presence Signal(s)	Y		This indicates power supply presence as sourced by HPM-PDB SGPIO bus or provided directly from PS connectors.
Power Supply AC OK Signal(s)	Y		Indication for AC input valid for one or more power supplies as sourced by HPM-PDB SGPIO bus or provided directly from PS connectors.
Power Supply DC Output OK Signal(s)	Y		Indication for DC output valid for one or more power supplies as sourced by HPM-PDB SGPIO bus or provided directly from PS connectors.
Standby Power Good	Y		System standby power good indication.
Main Power Good	Y		System main power good indication.
Voltage Rail Power Good Indication	Y		Indication for one or more voltage rail power good signals.
Main Power Fault Detection	Y		Indication of power fault for system main power.
Standby Power Fault Detection	Y		Indication of power fault for system standby power.
Power supply Fault Detection	Y		Fault detection and indication for one or more power supplies.
Voltage Rail Power Fault Detection	Y		Fault detection and indication for one or more power rails.

12.2.5.3. BMC Visible Power Management IP Block Outputs

The table below contains the set of required and optional recommended BMC visible power management IP block output signals. These will be reflected in IP block CSRs and transported via tunneled GPIOs either through the LTPI or SGPIO interfaces. See the LTPI and SGPIO chapters for more information regarding tunneled GPIOs.

TABLE 22: POWER MANAGEMENT IP BLOCK REQUIRED AND OPTIONAL OUTPUTS

Name	REQ	OPT	Description
Power State Field	Y		This field represents the current power state of the PD.
Power State Transition Indication	Y		Indication that the power state is in the process of transitioning.
Power State Transition Error Indication	Y		This indicates that an error occurred during the power state transition.
Power Management IP Block Interrupt	Y		Interrupt signal indicating that a state change or failure condition has occurred.
Main PCIe Reset	Y		Main system PCIe reset signal
Main CPU Reset	Y		Main system CPU / SoC reset signal provided to each CPU / SoC.
Resume Reset	Y		Main system resume / standby reset signal
CPU Standby Reset	Y		CPU / SoC resume / standby reset signal provided to each CPU / SoC.

12.2.6. Power Sequencing and Management CSRs Memory Map

This section contains the Control and Status Registers (CSR) memory map for standardized Power Sequencing and Management IP block functionality.

12.2.6.1. Power Management CSR Region Overview

These ranges are all offsets from the POWER_MANAGEMENT_IP_BLOCK base address.

This base address is defined by the CPLD/PD developer and is not prescribed here.

TABLE 23: POWER MANAGEMENT IP BLOCK CSR MAP

Region	Range Bottom	Range Top	Sections	Name	Description
0	0x0000	0x00FF		Global Registers	General IP block level registers
1	0x0100	0x0FFF	0x0XX0	Power Management CSR Set	Power state specific status and control.
2[8]	0x1000	0xXXXX		OEM SECTION	Undefined

The above register range ordering shall be preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offset locations and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.2.6.2. Power Management IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 24: POWER MANGEMENT IP BLOCK CSR REGION 0 REGISTER DEFINITION

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

The above register range ordering shall be preserved according to this specification.

[8] Note: Implementor can use this for additional regions as needed.

Note: All register space sizes and offsets are examples only. The register offset location and range values shown above are not required. The vendor may resize the ranges and offset locations as needed. The PDPD JSON file shall contain the register offsets and ranges for a given implementation.

12.2.6.2.1. Power Management IP Block Common Registers Subregion 0 Description

TABLE 25: POWER MANGEMENT IP BLOCK CSR REGION 0 SUBREGION 0 REGISTER DEFINITION

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each)
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state that requires a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status			Optional extended error code that is vendor defined.

				See section 12.1.1.8, “IP Block CSR: OEM Extended Status”, for details.
0x08 – 0x0F	Reserved			

12.2.6.2.2. Power Management IP Block Common Registers Subregion 1 Description

Reserved

12.2.6.3. Power Management IP Block Region 1 CSR Memory Map

The power management registers start at offset 0x100 from the base address for the power management CSR space.

This region is divided into two subregions.

TABLE 26: POWER MANGEMENT IP BLOCK CSR REGION 1 MEMORY MAP DEFINITION

Sub-Region	Range	Name	Description
0	0x0000 – 0x00FF	Power Management IP Block Required CSR Set	Set of required registers for power management functionality.
1	0x0100 – 0x0FFF	Vendor Extension	Reserved if more registers are needed

The above register range ordering shall be preserved according to this specification.

A vendor may use the vendor extension range to accommodate design specific registers. For example, if a given design requires more VR fault indication registers, then those registers can be defined in the vendor extension range. The PD JSON file should contain the details regarding the definition of the vendor extension registers so that the BMC can utilize them as needed.

Note: All register space sizes and offsets are examples only. The values shown are not required register location offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.2.6.3.1. Power Management IP Block Region 1 Subregion 0 Register List

The table below contains the register list for this IP Block region.

TABLE 27: POWER MANAGEMENT IP BLOCK REGION 1 REGISTER LIST

Offset Address	Name	Width	ATTR	Description
0x00	General Power Control	4B	RW	Power management state machine control.
0x04	General Power Status	4B	R	Power management state machine status.
0x08	Power Error Status	4B	R	Power management state machine status and error status condition.
0x0C	Power Management SM Error State	4B	R	Indicates which state the power management state machine was at when failure occurred.

0x10	Aggregated Power Good and Reset State	4B	R	Common power good and reset states.
0x14	VR Power Good State	4B	R	Indication of up to 32 voltage regulator power good states.
0x18	VR Failure State	4B	R	Indication of up to 32 voltage regulator fault states.
0x1C	SoC Presence	4B	R	Indication of presence for up to 32 SoCs.
0x20	SoC THERMTRIP	4B	R	Indication of THERMTRIP for up to 32 SoCs.
0x24	SoC THERMTRIP LATCH		RW1C	Indication of THERMTRIP_LATCH for up to 32 SoCs.
0x28	SoC POWER_THROTTLE	4B	RW	Power Throttle control for up to 32 SoCs.
0x2C – 0xFF	Reserved			

12.2.6.3.2. Power Management IP Block Region 1 Register Offset 0x00: General Power Control

The table below contains the description of the General Power Control register.

TABLE 28: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X00 DEFINITION

Bit(s)	Name	Description	ATTR
31:16	Reserved	RFU	R
15:8	SM Timeout Control	Power Management State machine timeout Control configuration register.	R/W

		0x00 = Default (Timeout Disabled) 0x01 = 30 seconds 1 minute 0x02= 1 Minute 0x03 = 2 Minutes 0x04 = 3 Minutes 0x05 = 4 Minutes 0x06 = 5 Minutes 0x07 = 10 Minutes 0x08 – 0xFF reserved	
7	Transition Trigger Flag	Setting this bit will cause the HPM Primary PD to perform the necessary functions to reach the BMC required target power state.	RW
6:0	Target Power State	This provides up to 128 possible states. States above 64 are available as vendor defined states. The HPM Primary PD cannot produce the set of pre-operational early power states. Thus, all pre-operational states are conceptually represented as 0b by the PD. The SCM shall keep a separate state variable to keep track of these states before the PDPD is operational. 0000000b: G3 (PDPD off) 0000000b: Pre-Standby (PD off) 0000000b: Standby Hold Off (PD in reset) Main State Definition 0000001b: Standby Run (PD is out of reset) 0000010b: S5 Hold Off 0000011b: S5 Run 0000101b: S0 SoC Ramp 0000100b: S0 Hold Off 0000110b: S0 Run BIOS Boot 0000111b: S0 Run OS --- 0001000b: Reserved.	RW

		… 0111111b: Reserved. 1000000b: OEM Reserved. … 1111111b: OEM Reserved.	

Note: The PD JSON file will indicate if the vendor has defined OEM specific states.

12.2.6.3.3. Power Management IP Block Region 1 Register Offset 0x04: General Power Status

The table below contains the description of the General Power Status register.

TABLE 29: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X04 DEFINITION

Bit(s)	Name	Description	ATTR
31:8	Reserved	RFU	R
7	Transition Flag	Indicates if PD is in the process of transitioning power states.	R
6:0	Current Power State	This field provides up to 128 possible states. States above 64 are available as vendor defined states. The HPM Primary PD cannot produce the pre-operational early power states. Thus, all pre-operational states are represented as 0b by the PD. The SCM shall keep a separate state variable to keep track of these states before the PD is operational. 0000000b: G3 (PD off) 0000000b: Pre-Standby (PD off)	R

		0000000b: Standby Hold Off (PD in reset) Main State Definition 0000001b: Standby Run (PD is out of reset) 0000010b: S5 Hold Off 0000011b: S5 Run 0000101b: S0 SoC Ramp 0000100b: S0 Hold Off 0000110b: S0 Run BIOS Boot 0000111b: S0 Run OS ---------- 0001000b: Reserved … 0111111b: Reserved 1000000b: OEM Reserved. … 1111111b: OEM Reserved.	

Note: The PD JSON file will indicate if the vendor has defined OEM specific states.

12.2.6.3.4. Power Management IP Block Region 1 Register Offset 0x08: Power Error Status

The table below contains the description of the Power Error Status register.

TABLE 30: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X08 DEFINITION

Bit(s)	Name	Description	ATTR
31:8	Reserved	RFU	R
7	Transition Error Flag	Indicates if transition failure has occurred due to failure to enter BMC required power state.	R

6:0	Power State Error Indication	This field provides up to 128 possible error codes. States above 64 are available as vendor defined codes. 0000001b: Error Transition Timeout 0000010b: Voltage Rail Failure (see VR fail indication register for specific voltage rail failure) 0000011b: Power State Transition Failure (see error register for details). 0000011b: Reserved. … 0111111b: Reserved. 1000000b: Reserved For Vendor Defined Usage. … 1111111b: Reserved For Vendor Defined Usage.	R

12.2.6.3.5. Power Management IP Block Region 1 Register Offset 0x0C: Power Management State Machine Error State

The table below contains the description of the Power Management state machine error state register.

TABLE 31: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X0C DEFINITION

Bit(s)	Name	Description	ATTR
31:7	Reserved	RFU	R
6:0	Power State at Fault	This field indicates which state the power management state machine failure occurred.	R

12.2.6.3.6. Power Management IP Block Region 1 Register Offset 0x10: Aggregated Power Good and Reset State

The table below contains a description of the aggregated power good and reset register.

TABLE 32: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X10 DEFINITION

Bit(s)	Name	Description	ATTR
31:8	Reserved	RFU	R
7	CPU_PWR_BTN_N	State of CPU power button signal. This is used for informational purposes. As this is controlled by the power management state machine.	R
6	CPU_RESET_N	Reset to one or more CPUs	R
5	STBY_CPU_RESET_N	Standby reset to one or more CPUs	R
4	PCIE_RESET_N	PCIe Reset to devices	R
3	RSMRST_N	Standby reset	R
2	CPU_POWERGOOD	CPU Power Good as aggregate of all CPU VR Power Good signals. 1 = valid, 0 not valid	R
1	MAIN_ POWERGOOD	Main Power Good signal as aggregate of all main VR power good signals. 1 = valid, 0 not valid	R
0	STBY_ POWERGOOD	Standy Power Good signal as aggregate of all standby VR power good signals.	R

		1 = valid, 0 not valid	

Note: The PD JSON file will indicate if the vendor has defined OEM specific states.

12.2.6.3.7. Power Management IP Block Region 1 Register Offset 0x14: VR Power Good State

The table below contains the description of the Power Good State register. This has the support for up to 32 separate voltage rail power good states based on definition within the JSON file. The JSON file will indicate which voltage rails are being monitored. It is not required to use all 32 possible voltage rail power good indication signals.

TABLE 33: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X14 DEFINITION

Bit(s)	Name	Description	ATTR
31:0	VR_PG_31 .. VR_PG_0	Indication of voltage rail Power Good indication. Bit 31: Voltage Rail Power Good VR 31 Bit 30: Voltage Rail Power Good VR 30 … Bit 1: Voltage Rail Power Good VR 1 Bit 0: Voltage Rail Power Good VR 0	R

The PD JSON file defines which voltage rail these signals map to.

12.2.6.3.8. Power Management IP Block Region 1 Register Offset 0x18: VR Power Failure State

The table below contains the description of the Power Failure State register. This has the support for up to 32 voltage rail power failure states based on definition within the JSON file.

TABLE 34: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X18 DEFINITION

Bit(s)	Name	Description	ATTR
31:0	VR_FAULT_31 .. VR_FAULT_0	Indication of voltage rail Power Good indication. Bit 31: Voltage Rail Power FAULT VR 31 Bit 30: Voltage Rail Power FAULT VR 30 … Bit 1: Voltage Rail Power FAULT VR 1 Bit 0: Voltage Rail Power FAULT VR 0	RW1C

The PD JSON file defines which voltage rail these signals map to. The JSON file will indicate which voltage rails are being monitored. It is not required to use all 32 possible voltage rail power failure indication signals.

A voltage rail failure shall be defined as one of the following:

- Sourced from a voltage regulator fault signal provided to the PD
- As determined by the PD using internal PD logic
    - One example is during a power state transition where a voltage rail never becomes valid during a given time window
        - The definition of the timeout is left to the implementor and is outside the scope of this specification
    - Another example could be a determination of a power good signal unexpected de-assertion when in a specific power state (e.g., S0)
    - This logic is vendor defined and left to the implementor and is outside the scope of this specification
- As provided by an external signal source via a bus interface
    - An example can be autonomous reading of an I2C expander by the PD
    - Another example could be sourced via shift registers
    - This source is vendor defined and is outside the scope of this specification

A system implementor may want to include further voltage regulator fault information. This can be added using the vendor specific register region if so desired. The specific definition of such registers is outside the scope of this specification. Any specific action after detection of a VR fault by the PD is defined by the vendor. Example actions could be to de-assert one or more power good CSR bits, change power state, and / or trigger an interrupt.

12.2.6.3.9. Power Management IP Block Region 1 Register Offset 0x1C: SoC Presence

The table below contains the description of the SoC presence register. This supports for up to 32 SoCs based on definition within the JSON file. The JSON file will indicate the number of

SOCs supported by the platform. Any greater than what is supported will be reserved bits in the register.

TABLE 35: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X1C DEFINITION

Bit(s)	Name	Description	ATTR
31:0	SOC_PRES_31_N ... SOC_PRES_0_N	Indication of SOC presence. Bit 31: SoC Presence 31 Bit 30: SoC Presence 30 … Bit 1: SoC Presence 1 Bit 0: SoC Presence 0	R

12.2.6.3.10. Power Management IP Block Region 1 Register Offset 0x20: SoC THERMTRIP Signal Current State

The table below contains the description of the SoC THERMTRIP register. This would indicate the current state. This supports up to 32 SoCs based on definition within the JSON file. The JSON file will indicate the number of SOCs supported by the platform. Any greater than what is supported will be reserved bits in the register.

TABLE 36: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X20 DEFINITION

Bit(s)	Name	Description	ATTR
31:0	SOC_THERMTRIP_31_N ... SOC_ THERMTRIP _0_N	Indication of SOC catastrophic thermal condition (a.k.a. THERMTRIP) assertion. Bit 31: SoC THERMTRIP 31 Bit 30: SoC THERMTRIP 30 … Bit 1: SoC THERMTRIP 1 Bit 0: SoC THERMTRIP 0	R

12.2.6.3.11. Power Management IP Block Region 1 Register Offset 0x24: SoC THERMTRIP LATCH

The table below contains the description of the SoC THERMTRIP LATCH register. This supports up to 32 SoCs based on definition within the JSON file. The JSON file will indicate the number of SOCs supported by the platform. Any greater than what is supported will be reserved bits in the register.

TABLE 37: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X24 DEFINITION

Bit(s)	Name	Description	ATTR
31:0	SOC_THERMTRIP_LATCH_31_N ... SOC_ THERMTRIP_ LATCH _0_N	Indication of SOC catastrophic thermal condition (a.k.a. THERMTRIP) assertion. Bit 31: SoC THERMTRIP LATCH 31 Bit 30: SoC THERMTRIP LATCH 30 … Bit 1: SoC THERMTRIP LATCH 1 Bit 0: SoC THERMTRIP LATCH 0	RWC

Regardless of the current state of the THERMTRIP signal, the THERMTRIP LATCH register will latch the assertion of THERMTRIP and hold until the BMC clears it.

12.2.6.3.12. Power Management IP Block Region 1 Register Offset 0x28: SoC POWER_THROTTLE

The table below contains the description of the optional SoC Power Throttle register. This supports up to 32 SoCs based on definition within the JSON file. The JSON file shall indicate the number of SOCs supported by the platform that supports the power throttle feature. Any greater than what is supported will be reserved bits in the register.

TABLE 38: POWER MANGEMENT IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0X28 DEFINITION

Bit(s)	Name	Description	ATTR

31:0	SOC_POWER_THROTTLE_31_N ... SOC_ POWER_ THROTTLE _0_N	BMC control of power throttle assertion to the SOC. Bit 31: SOC POWER_THROTTLE 31 Bit 30: SOC POWER_THROTTLE 30 … Bit 1: SOC POWER_THROTTLE 1 Bit 0: SOC POWER_THROTTLE 0	R/W

12.2.6.4. Power Management CSRs Region 1 Sub-region 1 Vendor Extension CSR Definition

Reserved for vendor extension usage.

12.2.6.5. Power Management CSRs Region 2 Memory Map

Reserved for vendor use.

12.2.7. Platform Silicon Vendor Specific Power Sequencing and Management Support

A vendor may add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support.

12.2.8. Example Implementations

Example implementations, if available, can be found in external IP examples list as described in section 7.4, “MHS HPM and Peripheral Programmable Device Specification Supplementary Materials”.

12.3. System / Device Reset Control Logical IP Block

This section contains the logical IP block description for the System / Device Reset Control requirements and descriptive abstraction for this IP block.

12.3.1. System / Device Reset Control Conceptual Overview

This section contains an overview of the typical System Management and Reset Control functionality for a typical server.

The intention of the system management IP block is to provide a common set of control and status interfaces for controlling reset functions on the HPM, peripherals, and options by the BMC. This includes system reset and individual reset signal control for peripherals and options.

12.3.2. System / Device Reset Control IP Block Architecture

This section contains the general abstract architecture for the System / Device Reset Control IP block.

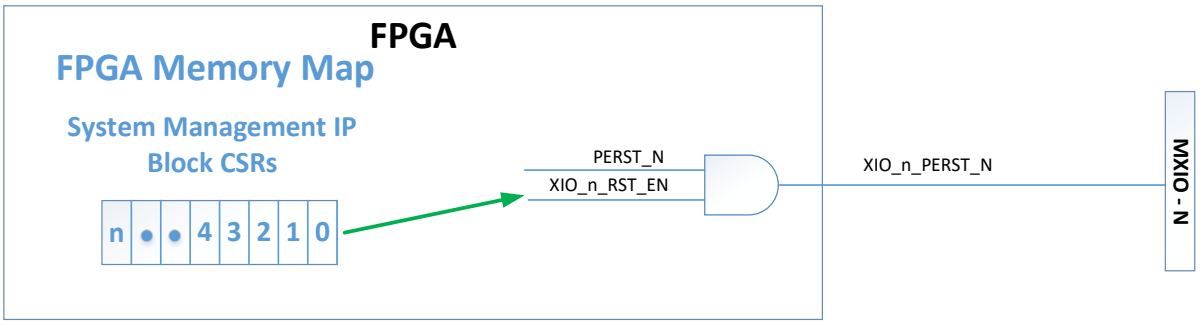


FIGURE 17: SYSTEM RESET CONTROL IP BLOCK ARCHITECTURAL DIAGRAM

The above figure represents n number of possible PCIe Reset Enable bits controlling n number of PERST_N signals for every MXIO-n connector. For MXIO connectors that have both a PERST_A_N and PERST_B_N set of signals, the system designer may use the same control signal to control both, or to break up into separate enable bits. If so, then it shall be indicated within the HPM JSON and the FGPA JSON using the internal signal virtual wire mapping framework to indicate which physical signals map to CSRs.

The PD JSON and HPM JSON files will provide the specific mapping between internal CSR to PD physical signal which, in turn, will provide the specific connection to the M-XIO connector

PERST_N signal such that the BMC will know which enable controls which PCIe reset signal on the M-XIO connector.

This example can be extended to OCP slots, HPM on-board devices, and any OEM or other connector that utilizes the PCIe reset signal.

12.3.3. System / Device Reset Control IP Block Requirements

This section contains the general abstract architecture for the System Management and Reset Control IP block.

12.3.3.1. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 39: SYSTEM RESET CONTROL REQUIRED AND OPTIONAL FEATURE SET

Name	REQ	OPT	Description
MXIO PCIe Reset Enable CSRs	Y		Controls PCIe reset enable control to N number of MXIO connectors.
OCP PCIe Reset Enable CSRs	Y		Controls PCIe reset enable control to N number of OCP connectors.
HPM On-board Device PCIe Reset Enable CSRs		Y	Vendor optional PCIe Reset Enable control logic to N number of HPM on-board devices.

12.3.3.2. Required and Optional Inputs

The table below contains the set of required and optional recommended inputs consumed by this IP block.

TABLE 40: SYSTEM RESET CONTROL REQUIRED AND OPTIONAL INPUTS

Name	REQ	OPT	Description
PCIe Reset	Y		Source reset signal for this IP block.

12.3.3.3. Required and Optional Outputs

The table below contains the set of required and optional recommended outputs produced by this IP block.

TABLE 41: SYSTEM RESET CONTROL REQUIRED AND OPTIONAL OUTPUTS

Name	REQ	OPT	Description
MXIO PCIe Reset(s)		Y	Generation of one or more MXIO PERST_N PCIe reset signals.
OCP PCIe Reset(s)		Y	Generation of one or more OCP slot PERST_N PCIe reset signals.
On-Board Device Reset(s)		Y	Generation of one or more on-board device PERST_N PCIe reset signals.

12.3.4. System / Device Reset Control CSRs Requirements

This section contains the Control and Status Registers (CSR) requirements for the standardized System Management and Reset Control IP block functionality.

12.3.4.1. System Management IP Block CSR Region Map Overview

These ranges are all offsets from the System Management IP Block base address. This base address is defined by the CPLD/PD developer and is not prescribed here.

TABLE 42: SYSTEM RESET CONTROL CSR MEMORY MAP

Region	Range Bottom	Range Top	Sections	Name	Description
0	0x0000	0x00FF		Global Registers	IP block level to cover all targets
1	0x0100	0x0FFF		System / Device Reset Control IP Block Required CSR Set	For control over PCIe reset to each MXIO connector.
2[9]	0x1000	0xXXXX		OEM SECTION	Undefined

The register range ordering shall be preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register location offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.3.4.2. Region 0: System / Device Reset Control IP Block Common Registers

This region is divided into two subregions.

[9] Note: Implementor can use this for additional regions as needed.

TABLE 43: SYSTEM RESET CONTROL IP BLOCK CSR REGION 0 MEMORY MAP DEFINITION

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

The register range ordering shall be preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register location offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.3.4.2.1. Region 0 Subregion 0 Memory Map

TABLE 44: SYSTEM RESET CONTROL IP BLOCK CSR REGION 0 SUBREGION 0 MEMORY MAP DEFINITION

Offset Address	Name	Data Width	ATTR	Description
0x00	IP Block ID	2B	R	Unique ID to identify the IP block. See section 12.1.1.1, “Required IP Block CSR: Status”, for details.
0x02	IP Block Version	2B	R	Major.Minor (1 byte each). See section 12.1.1.2, “Required IP Block CSR: Status”, for details.
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.

0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details. A global reset of this IP block will set all reset control outputs that gate PERST_N to asserted.
0x05	Reserved			Reserved for future use.
0x06	Reserved			Reserved for future use.
0x07	Extended Status	1B	R/W	Optional vendor defined status register.
0x08 – 0x0F	Reserved			Reserved for future use.

12.3.4.2.2. Region 0 Subregion 1 Memory Map

Reserved

12.3.4.3. Region 1 System / Device Reset Control IP Block Required CSR Memory Map

This region is divided into four subregions.

TABLE 45: SYSTEM RESET CONTROL IP BLOCK CSR REGION 1 MEMORY MAP DEFINITION

Sub-Region	Range	Name	Description
0	0x0000 – 0x00FF	M-XIO PCIe Reset Enable Registers	Control over PCIe reset, PERST_N, to up to 2048 logical M-XIO connectors. Provides up to 256 * 8 bits or 2048 individual enable bits.
1	0x0100 – 0x01FF	OCP Slot PCIe Reset Enable Registers	Control over PCIe reset, PERST_N, to up to 2048 logical OCP connectors. Provides up to 256 * 8 bits or 2048 individual enable bits.
2	0x0200 – 0x02FF	HPM On-board Device PCIe Reset Enable Registers	Control over PCIe reset, PERST_N, to up to 2048 logical HPM on-board devices. Provides up to 256 * 8 bits or 2048 individual enable bits.

3	0x0300 – 0x0FFF	RESERVED	Reserved if more registers are needed

The PD JSON file indicates how many Reset Enable registers are implemented in the specific PD design.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.3.4.3.1. Region 1 Subregion 0 Memory Map - MXIO PCIe Reset Enable CSRs

This sub-region contains the set of PCIe reset enables in increments of 8 MXIO x8 connectors for each byte of CSR space. Each byte represents 8 enable bits and goes from logical MXIO-0 to MXIO-n where each is a x8 MXIO.

Example:

MXIO-0 starts at bit 0 in register offset 0.

MXIO-1 starts at bit 1 in register offset 0.

…

MXIO-n starts at bit n in register offset (n modulo 8).

12.3.4.3.2. System Management PCIe Reset Control Register Offset 0x00: PCIe Reset Enable Logical MXIO-0 to MXIO-7

TABLE 46: SYSTEM RESET CONTROL IP BLOCK CSR REGION 1 REGISTER 0x00 DEFINITION

Bit(s)	Name	Description	ATTR	W	Notes
7	XIO_RST_EN_7	PCIe Reset Enable 7	R/W	1'b0	Mapped to logical MXIO-7
6	XIO_RST_EN_6	PCIe Reset Enable 6	R/W	1'b0	Mapped to logical MXIO-6
5	XIO_RST_EN_5	PCIe Reset Enable 5	R/W	1'b0	Mapped to logical MXIO-5
4	XIO_RST_EN_4	PCIe Reset Enable 4	R/W	1'b0	Mapped to logical MXIO-4
3	XIO_RST_EN_3	PCIe Reset Enable 3	R/W	1'b0	Mapped to logical MXIO-3
2	XIO_RST_EN_2	PCIe Reset Enable 2	R/W	1'b0	Mapped to logical MXIO-2
1	XIO_RST_EN_1	PCIe Reset Enable 1	R/W	1'b0	Mapped to logical MXIO-1
0	XIO_RST_EN_0	PCIe Reset Enable 0	R/W	1'b0	Mapped to logical MXIO-0

For MXIO connectors that are x16, they will be treated as two x8 connectors logically. The x16 MXIO connector has both a PERST_A_N and PERST_B_N connected. The indication of the presence of a x16 connector will be mapped using the combination of the PD and HPM JSON files. The JSON file will describe the PERST connection as logical x8s even if it is physically combined in a x16 connector.

12.3.4.3.3. System Management PCIe Reset Control Register Offset 0x01: PCIe Reset Enable Logical MXIO-8 to MXIO-15

TABLE 47: SYSTEM RESET CONTROL IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER 0x01 DEFINITION

Bit(s)	Name	Description	ATTR	W	Notes
7	XIO_RST_EN_15	PCIe Reset Enable 15	R/W	1'b0	Mapped to Logical MXIO-15

6	XIO_RST_EN_14	PCIe Reset Enable 14	R/W	1'b0	Mapped to Logical MXIO-14
5	XIO_RST_EN_13	PCIe Reset Enable 13	R/W	1'b0	Mapped to Logical MXIO-13
4	XIO_RST_EN_12	PCIe Reset Enable 12	R/W	1'b0	Mapped to Logical MXIO-12
3	XIO_RST_EN_11	PCIe Reset Enable 11	R/W	1'b0	Mapped to Logical MXIO-11
2	XIO_RST_EN_10	PCIe Reset Enable 10	R/W	1'b0	Mapped to Logical MXIO-10
1	XIO_RST_EN_9	PCIe Reset Enable 9	R/W	1'b0	Mapped to Logical MXIO-9
0	XIO_RST_EN_8	PCIe Reset Enable 8	R/W	1'b0	Mapped to Logical MXIO-8

12.3.4.3.4. System Management PCIe Reset Control Register Offset 0x02 – N: PCIe Reset Enable Logical MXIO-16 to MXIO-N

This section represents the set of PCIe Reset Control Registers that control the set of logical MXIO-16 to MXIO-N connectors in increments of 8 connectors each.

12.3.4.3.5. Region 1 Subregion 1 Memory Map– OCP PCIe Reset Enable CSRs

This sub-region contains the set of PCIe reset enables in increments of 8 OCP x8 connectors for each byte of CSR space. Each byte represents 8 enable bits and goes from logical OCP -0 to OCP -n where each is a x8 OCP. It is expected that most HPMs have a maximum of 2 on-board OCP slot connectors. Thus, it may be that there are only one or two registers required.

Example:

OCP -0 starts at bit 0 in register offset 0.

OCP -1 starts at bit 1 in register offset 0.

…

OCP -n starts at bit n in register offset (n modulo 8).

12.3.4.3.6. OCP Slot PCIe Reset Control Register Offset 0x00: PCIe Reset Enable Logical OCP-0 to OCP-7

TABLE 48: SYSTEM RESET CONTROL IP BLOCK CSR REGION 1 SUBREGION 1 REGISTER 0X00 DEFINITION

Bit(s)	Name	Description	ATTR	W	Notes
7	OCP_RST_EN_7	PCIe Reset Enable 7	R/W	1'b0	Mapped to logical OCP -7
6	OCP _RST_EN_6	PCIe Reset Enable 6	R/W	1'b0	Mapped to logical OCP -6
5	OCP _RST_EN_5	PCIe Reset Enable 5	R/W	1'b0	Mapped to logical OCP -5
4	OCP _RST_EN_4	PCIe Reset Enable 4	R/W	1'b0	Mapped to logical OCP -4
3	OCP _RST_EN_3	PCIe Reset Enable 3	R/W	1'b0	Mapped to logical OCP -3
2	OCP _RST_EN_2	PCIe Reset Enable 2	R/W	1'b0	Mapped to logical OCP -2

1	OCP _RST_EN_1	PCIe Reset Enable 1	R/W	1'b0	Mapped to logical OCP -1
0	OCP _RST_EN_0	PCIe Reset Enable 0	R/W	1'b0	Mapped to logical OCP -0

For OCP connectors that are x16, they will be treated as two x8 connectors logically. The x16 OCP connector has both a PERST_A_N and PERST_B_N connected. The indication of the presence of a x16 slot will be mapped using the combination of the PD and HPM JSON files. The JSON file shall describe the PERST connection as logical x8s even if it is physically combined in a x16.

12.3.4.3.7. OCP Slot PCIe Reset Control Register Offset 0x01 - N: PCIe Reset Enable Logical OCP-8 to OCP-N

This definition represents PCIe Reset Control Registers controlling logical OCP-8 to OCP-n in increments of 8.

12.3.4.4. Region 1 Subregion 2 Memory Map – HPM On-board Device PCIe Reset Enable CSRs

This sub-region contains the set of PCIe reset enables in increments of 8 HPM On-board device x8 connectors for each byte of CSR space. Each byte represents 8 enable bits and goes from logical HPM_DEVICE -0 to HPM_DEVICE -n. This is an optional feature and the implementor does not need to utilize this if there are no on-board devices with PCIe reset control requirements.

Example:

HPM_DEVICE -0 starts at bit 0 in register offset 0.

HPM_DEVICE -1 starts at bit 1 in register offset 0.

…

HPM_DEVICE -n starts at bit n in register offset (n modulo 8).

12.3.4.4.1. HPM_DEVICE Slot PCIe Reset Control Register Offset 0x00: PCIe Reset Enable Logical HPM_DEVICE -0 to HPM_DEVICE -7

TABLE 49: SYSTEM RESET CONTROL IP BLOCK CSR REGION 1 SUBREGION 2 REGISTER 0X00 DEFINITION

Bit(s)	Name	Description	ATTR	W	Notes
7	HPM_DEVICE _RST_EN_7	PCIe Reset Enable 7	R/W	1'b0	Mapped to logical HPM_DEVICE -7
6	HPM_DEVICE _RST_EN_6	PCIe Reset Enable 6	R/W	1'b0	Mapped to logical HPM_DEVICE -6
5	HPM_DEVICE _RST_EN_5	PCIe Reset Enable 5	R/W	1'b0	Mapped to logical HPM_DEVICE -5
4	HPM_DEVICE _RST_EN_4	PCIe Reset Enable 4	R/W	1'b0	Mapped to logical HPM_DEVICE -4
3	HPM_DEVICE _RST_EN_3	PCIe Reset Enable 3	R/W	1'b0	Mapped to logical HPM_DEVICE -3
2	HPM_DEVICE _RST_EN_2	PCIe Reset Enable 2	R/W	1'b0	Mapped to logical HPM_DEVICE -2
1	HPM_DEVICE _RST_EN_1	PCIe Reset Enable 1	R/W	1'b0	Mapped to logical HPM_DEVICE -1
0	HPM_DEVICE _RST_EN_0	PCIe Reset Enable 0	R/W	1'b0	Mapped to logical HPM_DEVICE -0

If the register is included in the PD JSON file description, then it is required that the JSON file describes the PERST connection between HPM on-board device and the PD physical pin.

12.3.4.4.2. HPM Device PCIe Reset Control Register Offset 0x01 - N: PCIe Reset Enable Logical HPM_DEVICE -8 to HPM_DEVICE - N

This definition represents PCIe Reset Control Registers controlling logical HPM_DEVICE -8 to HPM_DEVICE - n in increments of 8.

12.3.4.5. Region 1 Subregion 3 Memory Map – Reserved

Reserved

12.3.4.6. Region 2: OEM CSR Memory Map

Reserved for OEM use.

12.3.5. Vendor Specific Extensions Support

Vendors may add extensions to this IP block as needed by using OEM reserved regions for CSRs.

Vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support.

12.3.6. Example Implementation

Examples implementation, if available, can be found in external IP examples list as described in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

12.4. LTPI Standardized Logical IP Block

This section contains the logical IP block description for the LTPI interface requirements and descriptive abstraction for this IP block.

12.4.1. LTPI Interface Conceptual Overview

This section contains an overview of the LTPI interface functionality for a typical set of use cases.

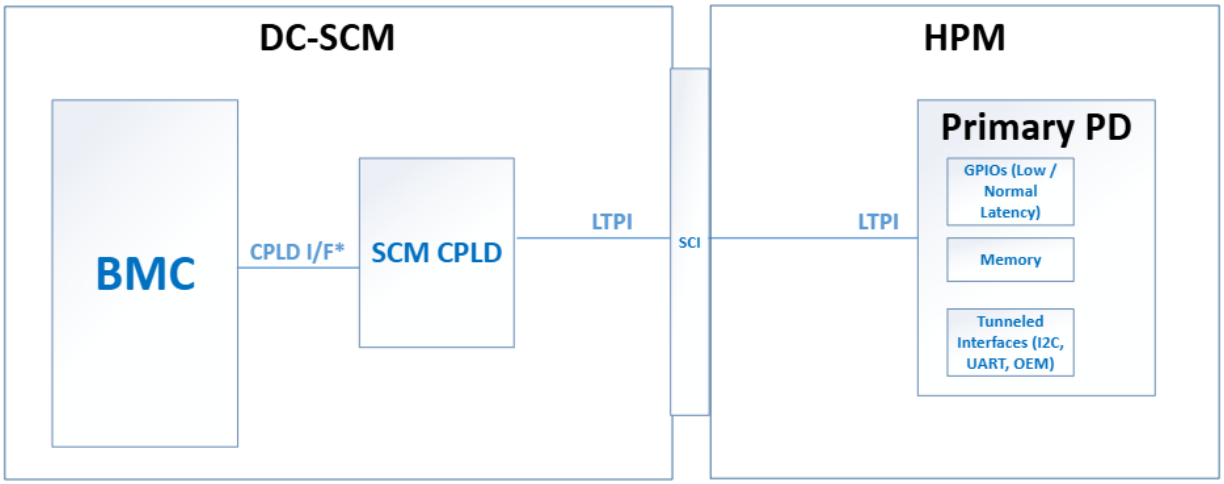


FIGURE 18: EXAMPLE LTPI INTERCONNECTION BETWEEN BMC AND HPM PD

The LTPI interface primarily provides low and normal latency virtual wire transport support. Additionally, it provides tunneling capabilities for I2C and UART buses from the HPM through the PD to the BMC via the LTPI bus. The use of I2C and / or UART tunnelling is optional. Lastly, it is intended that the LTPI interface will provide access using the data channel for random access into the PD memory map. Additional use cases are undefined and up to the HPM vendor.

Please refer to the LTPI spec for further details.

12.4.2. LTPI Interface IP Block Architecture

This section contains an overview of the internal architecture for the LTPI interface IP block within the PD and the relationship and mappings between the LTPI external interface controller within the IP block and other PD internal blocks.

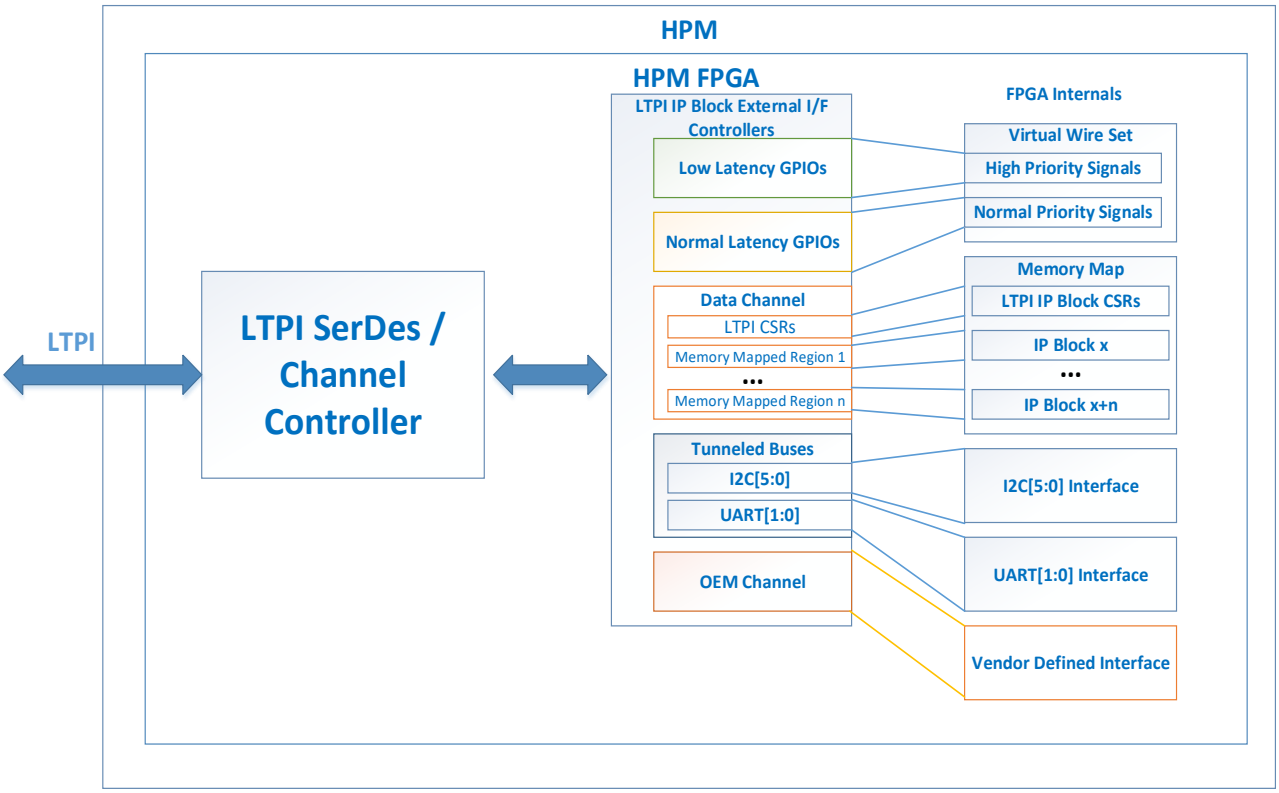


FIGURE 19: LTPI IP BLOCK ARCHITECTURE

This diagram illustrates the mapping between LTPI channels and tunneled interfaces to the PD internal structures such as virtual wires, memory mapped CSRs, and external bus interfaces. The PD JSON file shall provide a detailed description of which virtual wires are mapped to LTPI GPIO channels. The PD JSON file shall contain the mapping of memory mapped CSRs to the LTPI data channel at specified offsets and sizes. Next, specific tunneled buses are indicated within the JSON structure as to whether they are tunneled and the LTPI bus JSON representation contains the map back to those tunneled interfaces. Lastly, the OEM channel is completely vendor defined and does not have a specific schema. It is optional to implement tunnelling of system buses.

Please consult the LTPI specification for further details.

12.4.3. LTPI Interface IP Block Requirements

This section contains the general abstract architecture for the LTPI interface IP block.

12.4.3.1. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 50: LTPI REQUIRED AND OPTIONAL FEATURE SET

Name	REQ	OPT	Description
DC-SCM 2.1 LTPI Specification Rev. 1.1 Version 1.1	Y		LTPI Specification Rev. 1.1 Version 1.1 or newer.
DC-SCM 2.1 Specification	Y		DC-SCM 2.1 Specification or newer.
LTPI Data Channel Access to PD Memory Map	Y		It is required that the PD memory map is accessible via the LTPI data channel.
LTPI Required Tunneled GPIO List	Y		All GPIOs marked as required shall be tunneled via the GPIO channel as described in the tunneled GPIO list.

12.4.3.2. LTPI Error Handling and Failure Reporting

For LTPI error handling and failure reporting, please consult the LTPI Specification Revision 1.1 Version 1.1 or newer for details.

12.4.3.3. LTPI GPIO Channel Tunneled GPIOs List

The table below contains the set of required and optional recommended GPIOs tunneled by the SCM to HPM LTPI bus GPIO channel where the input direction represents from the SCM to the HPM PD and the output direction represents from the HPM PD to the SCM.

TABLE 51: LTPI TUNNELED GPIO LIST

Name	Latency	DIR	W	REQ	OPT	Description

Global Interrupt	Low	Out	1'b0	Y		Global interrupt signal. All IP block signals are wire OR'ed into this signal.
Current Power State Bit Field	Low	Out	7'b0	Y		Power state field reflective of CSR bit field.
Power Management IP Block Interrupt	Low / Normal	Out	1'b0		Y	Power management IP block interrupt. Optional recommended.
Power State Transition Flag	Normal	Out	1'b0	Y		Power state transition flag reflective of CSR bit.
Target Power State Transition Request Bit Field	Normal	In	7'b0	Y		Power state transition request bit field reflective of CSRs.
Power State Transition Trigger Flag	Normal	In	1'b0	Y		Power state transition trigger flag reflective of CSR bit.
Transition Error Flag	Normal	Out	1'b0		Y	Power state transition error flag reflective of CSR bit. Optional recommended.
Power State Error Indication Bit Field	Normal	Out	7'b0		Y	Error code indication reflective of CSR bit field. Optional recommended.
Power Supply Presence Indication Signal(s)	Normal	Out	1'b0	Y		Power supply presence indication reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD and then re-tunneled to the SCM via LTPI GPIO channel.
Power Supply AC OK	Normal	Out	1'b0	Y		Power supply AC OK indication reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD and then re-tunneled to the SCM via LTPI GPIO channel.
Power Supply DC OK	Normal	Out	1'b0	Y		Power supply DC OK indication reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD and then re-tunneled to the SCM via LTPI GPIO channel.
Power Supply PS_ON State Signal(s)	Normal	Out	1'b0	Y		State of power supply PS_ON signal OK indication reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD and then re-tunneled to the SCM via LTPI GPIO channel.
Power Supply SMBus / PMBus Alert Signals	Normal	Out	1'b0	Y		Power supply SMBUS / PMBUS alert signals indication reflective of CSR

						bits as tunneled via PDB-HPM SGPIO to PD.
CPU Presence	Normal	Out	Varies	Y		CPU presence signals mapped to normal latency GPIOs. Single bit if single CPU otherwise one bit per CPU.
RSM_RST_N	Normal	Out	1'b0	Y		Resume reset
CPU Reset	Normal	Out	1'b0	Y		Main CPU reset
PCIe Reset	Normal	Out	1'b0	Y		Main PCIe reset
Main Power Good	Normal	Out	1'b0	Y		Main power good signal generated from PD to HPM on-board devices.
CPU Standby Power Good	Normal	Out	1'b0	Y		Standby power good signal generated from PD to HPM CPUs / SoCs.

The vendor may elevate any normal latency GPIO to low latency as needed provided that there are acceptable spare low latency GPIOs available. This shall be indicated in the PD JSON file.

12.4.3.3.1. LTPI Data Channel Tunneled GPIOs Mapping PD Internal Virtual Wires Considerations

The PD implementor can map any internal PD virtual wire to an external interface using the PD Virtual Wire Framework as described in chapter 11,"Virtual Wires Support". For LTPI, these could be mapped to either low or normal latency GPIOs.

The table below contains categories of possible tunneled GPIOs mapped from virtual wires.

TABLE 52: LTPI TUNNELED GPIO PD VIRTUAL WIRE MAPPING LIST

Name	DIR	Description
IP Block Interrupt Status Signals	Out	One or more IP block interrupt signals reflective of the IP Block global interrupt CSR bit(s).
Error or Failure Indication	In / Out	One or more failure indication signals (e.g., voltage rail failure).
Flex IO and Sideband Signals	In / Out / Bidirectional	Mapped status reflective of the IP Block CSR bit(s).

MUX Control Signals	Out	Mapped control bits and bit fields reflective of the IP Block CSR bit(s).
PESTI Target Virtual Wire Control	In / Out / Bidirectional	Mapped bits and bit fields reflective of the IP Block CSR bit(s).
PDB SGPIO signals	In / Out / Bidirectional	Mapped bits and bit fields reflective of the IP Block CSR bit(s) tunneled from the HPM-PDB SGPIO IP block through the LTPI GPIO channel.
Control Panel Signal Tunneling	In / Out / Bidirectional	Mapped bits and bit fields reflective of the IP Block CSR bit(s) for either virtual wires or sideband signals.
Thermal State / Error Indication	Out	Overall state of system and component thermal status.
PD GPIOs	In / Out / Bidirectional	Mapped bits and bit fields reflective of the IP Block CSR bit(s).
Static Information	In / Out	Any static information such as IDs can be tunneled. However, it is recommended and encouraged to provide all static data via JSON files.

It is required that the PD implementor provides these mappings within the PD JSON file. Section 7.4, “MHS HPM PD Specification Supplementary Materials”, contains a list of collators which contain one or more example PD JSON files with virtual wire mapping examples.

12.4.4. LTPI Interface Control and Status Registers Memory Map

This section contains the Control and Status Registers (CSR) requirements for standardized LTPI Interface functionality.

12.4.4.1. LTPI CSR Region Overview

These ranges are all offset from the LTPI_IP_BLOCK base address. This base address is defined by the CPLD/PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 53: LTPI CSR MEMORY MAP

Region	Range Bottom	Range Top	Name	Description

0	0x0000	0x00FF	Global Registers	General IP block level registers
1	0x0100	0x02FF	LTPI CPLD/PD Information	Contents of both specification defined, and OEM defined registers.
2	0x0300	0x3FF	LTPI Bus Interface Control and Status Registers	Window into contents of LTPI Bus Interface Control and Status Registers as per LTPI Specification. See LTPI specification for further details.
3	0x0400	0x0FFF	Reserved	Reserved for future use
4[10]	0x1000	0xFFFF	OEM SECTION	Vendor defined

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.4.4.2. LTPI IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 54: LTPI IP BLOCK CSR REGION 0 REGISTER MAP DEFINITION

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

It is required that the register range ordering is preserved according to this specification.

[10] Note: The implementor can use this for additional regions as needed.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.4.4.2.1. LTPI IP Block Common Registers Sub-Region 0 Description

The table below contains the definition for the LTPI IP block common registers sub-region 0.

TABLE 55: LTPI IP BLOCK CSR REGION 0 SUBREGION 0 REGISTER MAP DEFINITION

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each).
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	R/W	This is a global IP block interrupt status indication bit. BMC can write once to clear it. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	R/W	Enable / Disable interrupt generation. Bit 0: Interrupt Enable

				See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	R/W	Optional vendor defined status register.
0x08 – 0x0F	Reserved			Reserved for future use.

12.4.4.2.2. LTPI IP Block Common Registers Sub-Region 1 Description

Reserved

12.4.4.3. LTPI IP Block Region 1: CPLD/PD Information CSR Memory Map

The LTPI region one registers that start at Offset 0x100 from the base address for the LTPI CSR space. This is defined within the LTPI specification as the CPLD/PD Information register set. The required registers are defined below.

TABLE 56: LTPI IP BLOCK CSR REGION 0 SUBREGION 1 REGISTER MAP DEFINITION

Sub-Region	Range Bottom	Range Top	Name	Description
0	0x0100	0x01FF	LTPI CPLD / PD Information Registers	CPLD/PD version, device ID, etc.
1	0x0200	0x02FF	OEM CSR Set *	OEM specific

12.4.4.3.1. LTPI Region 1 Sub-region 0 CSR Memory Map

TABLE 57: LTPI IP BLOCK CSR REGION 1 SUBREGION 0 REGISTER MAP DEFINITION

Offset Address	Name	Data Width	ATTR	Description
0x0100	Vendor ID	2B	R	Vendor ID of the PD implementor.
0x0102	Device ID	2B	R	Device ID of the PD as assigned by the PD implementor.
0x0104	Version	4B	R	Version information for the PD image.
0x0108 – 0x01FF	Reserved			Reserved for future use.

12.4.4.3.2. LTPI Region 1 Sub-region 1 CSR Memory Map

Reserved

12.4.4.4. LTPI IP Block Region 2 CSR Memory Map

The LTPI region 2 registers start at Offset 0x300 from the base address for the LTPI management CSR space. This is defined within the LTPI specification as the LTPI Control and Status register set. See the LTPI specification for further details regarding the register definition.

12.4.4.5. LTPI IP Block Region 3 CSR Memory Map

Reserved

12.4.4.6. LTPI IP Block Region 4 CSR Memory Map

Reserved for vendor defined usage.

12.4.5. LTPI Interface Data Channel Tunneling Requirements

This section outlines the data channel tunneling requirements. It is required that the vendor describe within the PD JSON file all memory mapped regions that are mapped to the LTPI data channel. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

The table below contains the set of IP blocks that are mapped to the LTPI data channel. All other IP blocks are optional and can be mapped into the LTPI data channel if access via the BMC is required.

TABLE 58: LTPI DATA CHANNEL REQUIRED TUNNELED REGIONS LIST

PD IP Block	Req?	Description
IP Block Directory	Y	Required to provide access to the PD JSON file and / or patch files if stored in PD main memory.
Power (Management / Sequencing)	Y	Required for control of system power. This is a required IP block.
System / Device Reset Control	Y	Required for control of system resets. This is a required IP block.
LTPI	Y	Required since platform design is utilizing the LTPI data channel.
M-PESTI	Y[11]	Required as this provides access to PESTI target registers for peripheral support which is needed whether the target supports M-PESTI or simple presence.
FLEX IO MXIO and M-PIC PICPWR Sideband Configuration	Y[12]	Required to configure FLEX-IO and PICPWR sideband signals for peripheral support.

[11] Note: Only required if peripherals are supported in this system.
[12] ibid

12.4.6. LTPI Interface Bus Tunneling Requirements

It is not required that the HPM vendor utilize LTPI to tunnel any buses. However, it is required that the vendor describe within the PD JSON file all tunneled bus interfaces that it does support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.4.7. LTPI Interface Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.4.8. LTPI Interface Example Implementation

Example implementations for LTPI may be found within the collateral document, “IP Block Source Code Examples List" for specific examples. See section 7.4, “MHS HPM PD Specification Supplementary Materials”, and the LTPI specification for further details.

12.5. SGPIO Logical IP Block

This section contains the logical IP block description for the DC-SCM to HPM SGPIO interface requirements and descriptive abstraction for this IP block.

12.5.1. SGPIO Interface Conceptual Overview

This section contains an overview of the SGPIO interface functionality for a typical set of use cases. The Figure 20 below contains a block diagram showing the SGPIO interconnection between the MHS components that utilize the SGPIO interface. There are 2 use cases identified in the MHS Specifications:

1. SCM CPLD working as SGPIO Initiator to the HPM PD working as SGPIO Target. In this target use case, the SGPIO bus is used as an alternative to LTPI interface for Real Time Virtual Wire access. Since LTPI supports both Real Time Virtual Wires and Random-Access Memory Mapped an additional interface shall be used together with SGPIO to make it a valid alternative for LTPI. As shown in the Figure 20 either SPI_SCMCTRL or I2C/I3C shall be used to provide Random Access Memory Mapped support for HPM CPLD. SGPIO discovery and capabilities detection shall be implemented using the SGPIO discovery mechanism as outlined in this specification. The presence of an SGPIO bus connection between the SCM and HPM primary PD shall be provided in the HPM JSON self-describing file, and the details of how this shall be implemented are defined within the Modular-Plug-n-Play (M-PNP) FRU, Discovery & Boot Design Specification. An additional interface (shown in Figure 20) such as JTAG or SPI Interface, shall allow the BMC to collect the PD JSON file. The BMC uses this interface to access HPM PD primary PD memory map during platform discovery outlined in the M-PNP FRU, Discovery & Boot Design Specification.
2. HPM PD working as SGPIO Initiator to PDB CPLD working as SGPIO Target. In this use case the SGPIO is used to provide access to the Real-Time Virtual Wires on the PDB. In case of PDB SGPIO capabilities discovery BMC uses I2C interface to read the PDB CPLD FRU data as defined by the Modular-Plug-n-Play FRU, Discovery & Boot Design Specification.

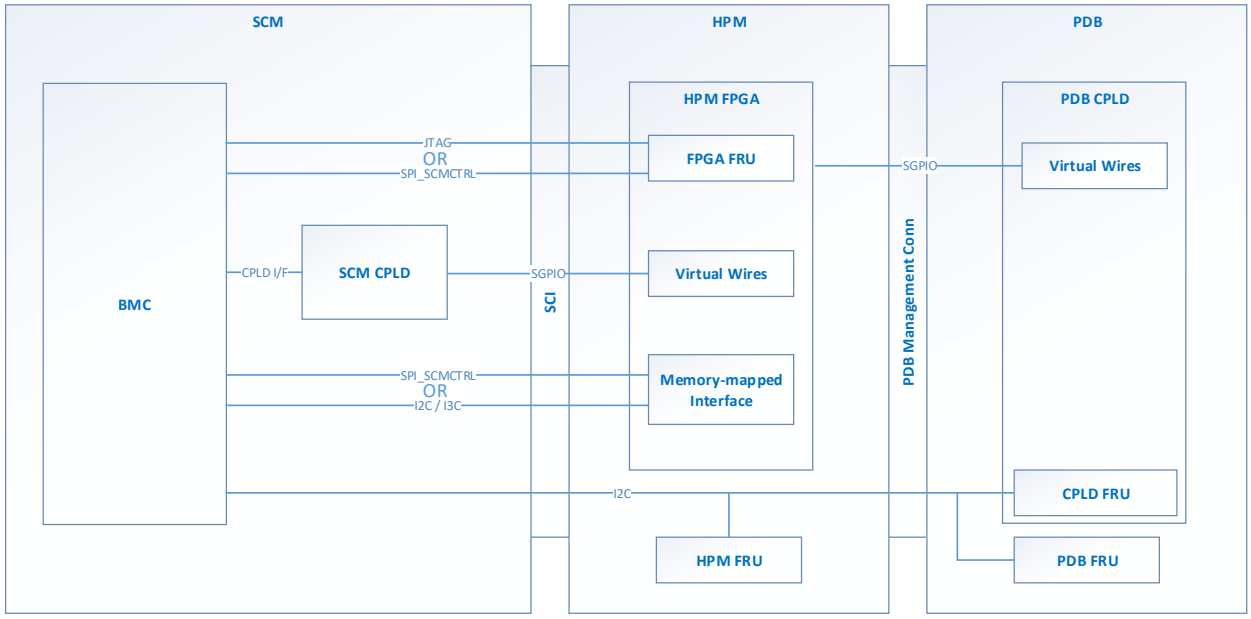


FIGURE 20: SGPIO INTERCONNECTION BLOCK DIAGRAM

The SGPIO is a bidirectional interface that allows the sending of Virtual Wires in both directions. However, due to the nature of the SGPIO interface operations, there are 2 distinctive roles that are identified on the SGPIO Bus i.e.:

1. SGPIO Initiator – this side of the SGPIO bus is responsible for driving the bus clock and initiation of all the active cycles on the bus.
2. SGPIO Target – this side of the SGPIO bus acts as an active endpoint on the SGPIO interface following bus cycles as driven by the initiator.

The diagram below shows what the different SGPIO roles for the SGPIO use cases that are defined in this specification.

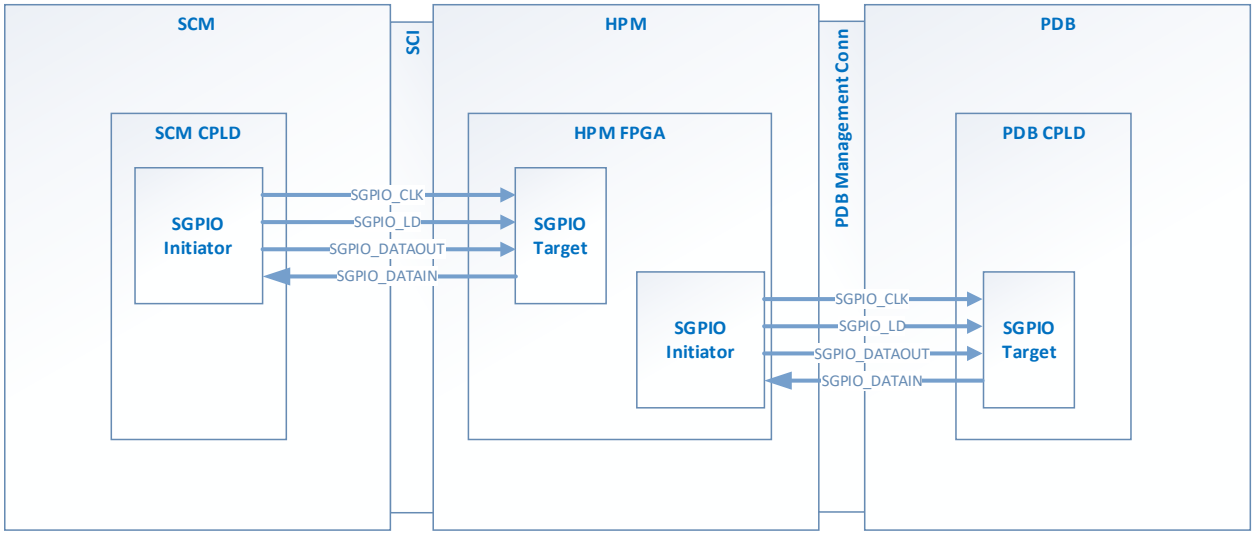


FIGURE 21 SGPIO INITIATOR AND TARGET ROLES.

12.5.2. SGPIO Interface Operation Phases

The next section outlines the details of SGPIO interface operation Phases. For both use cases defined in Figure 22 there are three main phases of the interface operation:

1. SGPIO Capabilities Discovery Phase
   This phase is used to detect SGPIO configuration and capabilities based on the FRU-based discovery flow as defined in the Modular-Plug-n-Play FRU, Discovery & Boot Design Specification. The BMC shall make sure that the SGPIO version, frequency and payloads are compliant and supported between the SGPIO Initiator and Target before it allows the SGPIO to be engaged.
2. SGPIO Handshake Phase
   This phase is used to detect SGPIO Target readiness before SGPIO can enter the Operational Phase where SGPIO Virtual Wires are sent as a payload.
3. SGPIO Operational Phase
   In this phase interface is fully operational and will use the Operational Frame with Virtual Wires as a payload.

The generic diagram covering SGPIO operation phases is presented in the following diagram.

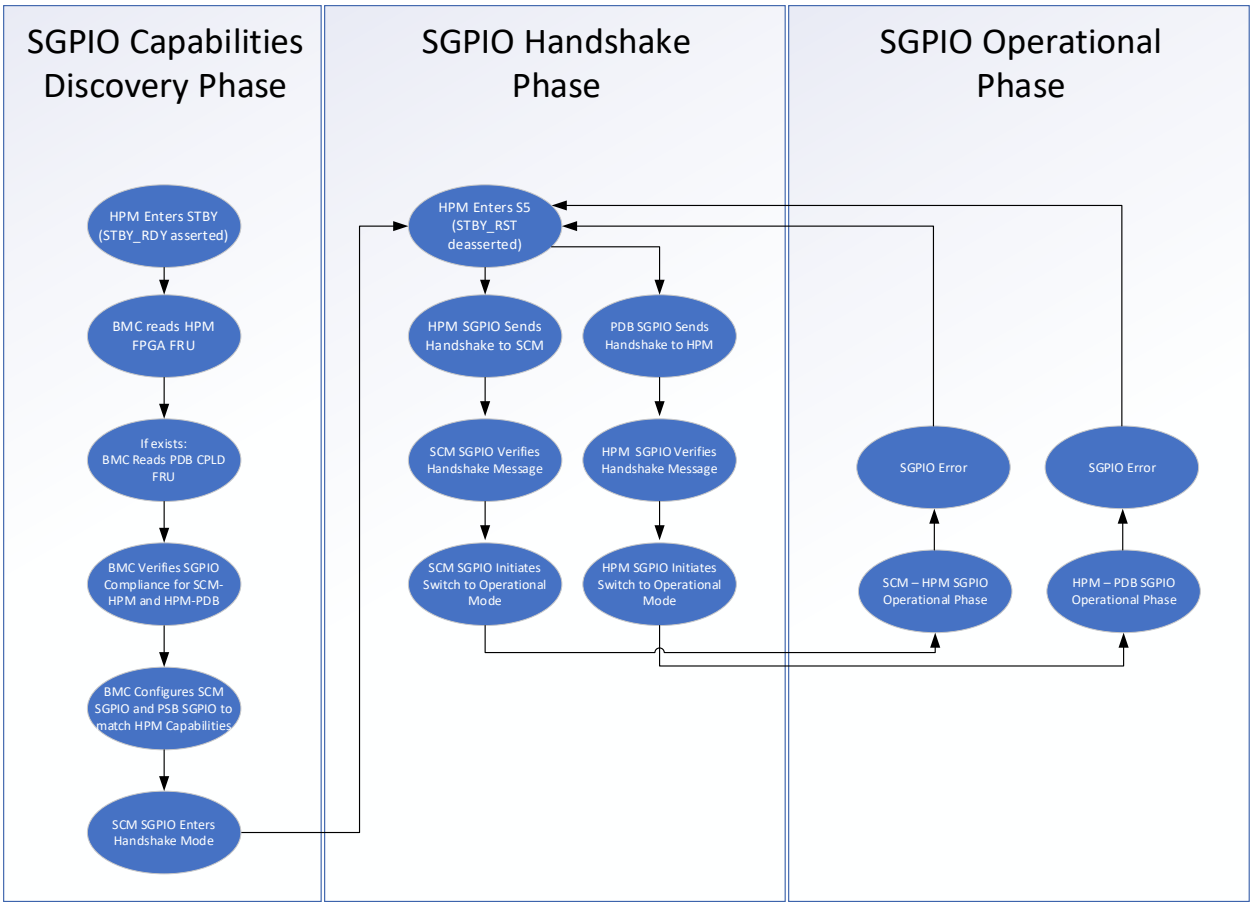


FIGURE 22 SGPIO INTERFACE OPERATION PHASES

12.5.2.1. SGPIO Capabilities Discovery Phase

The SGPIO Capabilities Discovery Phase is needed to verify the SGPIO configuration compliance between SCM CPLD and HPM PD and HPM PD and PDB CPLD which includes. During the FRU Capabilities Discovery BMC checks the following attributes based on different FRU sources:

a. SGPIO Interface Enabled State (HPM FRU, PDB FRU)
b. SGPIO Protocol Version (HPM PD FRU, PDB PD FRU)
c. SGPIO Interface Speed (HPM PD FRU, PDB PD FRU)
d. SGPIO Payload Content (HPM PD FRU, PDB PD FRU)

In order to move to the next Phase, the SGPIO Interface shall be provisioned on the HPM and SGPIO Protocol Version, Interface Speed and Payload Content. SCM SGPIO IP might expose the configuration registers to the BMC to allow for configuration of the SCM SGPIO IP to match the HPM SGPIO Capabilities e.g. by configuring the SGPIO Clock Speed or Interface Version. For the details regarding the HPM FRU, PDB FRU and SGPIO related attributes refer to the Modular-Plug-n-Play FRU, Discovery & Boot Design Specification and supplemental FRU JSON schema and PD JSON Schema as mentioned in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

12.5.2.2. SGPIO Handshake Phase

SGPIO Interface Handshake is needed for initial synchronization between SGPIO Initiator and Target as they come up at different times. After the Handshake is completed the SGPIO interface switch over to SGPIO Transport Phase in which it will remain until next PD reset. The Handshake phase shall be asynchronous and shall not depend on the SGPIO protocol directly, if possible, to allow for clean isolation between SGPIO Handshake and SGPIO Operational Phases. The MHS PnP SGPIO Protocol defines the use of UART protocol for Handshake Phase. As presented in diagram on the Figure 35 the UART interface is used in simplex mode on SGPIO_DATAIN line exclusively with SGPIO protocol.

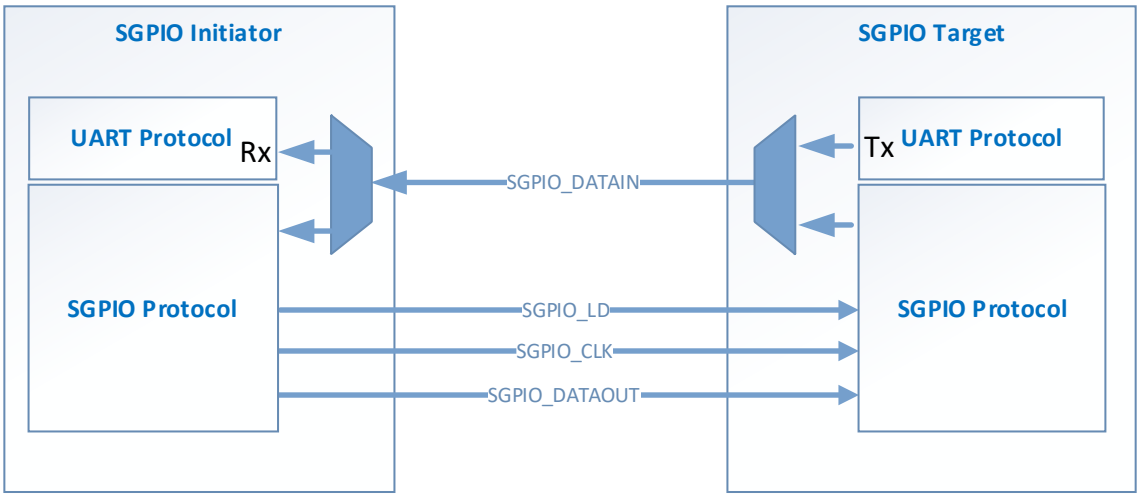


FIGURE 23 UART HANDSHAKE PHASE BLOCK DIAGRAM

The SGPIO Handshake Flow is presented in the diagram on the Figure 24 below.

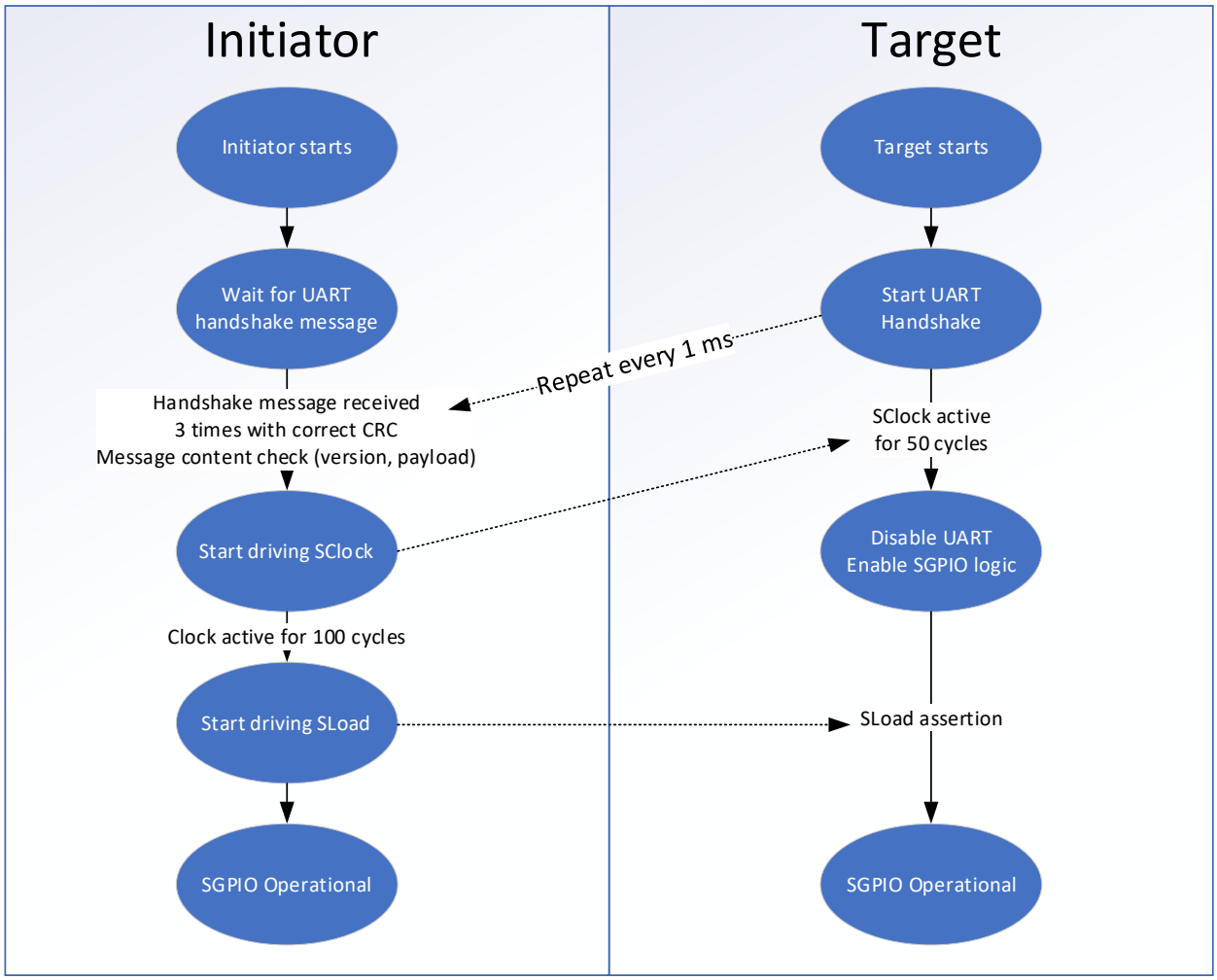


FIGURE 24 SGPIO HANDSHAKE FLOW

The SGPIO Handshake over UART always uses the following fixed UART configuration and sends messages at a fixed cadence as defined in the Table 59.

TABLE 59 SGPIO HANDSHAKE CONFIGURATION OF UART INTERFACE

Parameter	Value
Baud rate	115200
Data bits	8
Stop bits	1
Parity bits	None
Repeat message period	1 millisecond

The Handshake UART Message is defined the Table 60..

TABLE 60 SGPIO HANDSHAKE MESSAGE

Offset [bytes]	Size [bits]	Bit Field								Description
		7	6	5	4	3	2	1	0	
0	8	SGPIO Version Major				SGPIO Version Minor				BCD Encoded SGPIO Protocol Version [SGPIO Version Major]. [SGPIO Version Minor]
1	8	Max SPIO Clock Speed								Maximum Clock Speed supported by SGPIO Initiator/Target. The value of this field shall be defined in megahertz (MHz). For SGPIO 1.0 allowed values are 1 to 25.
2	8	Payload Size								Payload Size supported by SGPIO Initiator/Target. Value in this field is defined as number of payload bytes (Virtual Wires) in SGPIO Frame. The value is multiple of 16 bytes. For SGPIO 1.0 allowed values are 1 to 16.
3	16	OEM								Reserved for OEM Use
5	8	CRC								This is CRC-8 with following polynomial: x8 + x2 + x1 + 1 The polynomial initial value is defined as ‘0’. The CRC is calculated for an entire UART Handshake frame. CRC shall be calculated without input bits inversion and without result inversion/reflection.

The UART Handshake Message shall be repeated by the SGPIO Target until it has received a stable SGPIO input clock from SGPIO Initiator. The next step is the switch to SGPIO Operational Phase which is triggered by SGPIO Sload signal assertion by the Initiator. The State Transitions are defined in Table 61 for SGPIO Initiator and in Table 62 for SGPIO Target.

TABLE 61: SGPIO STATE CHANGE CONDITIONS FOR SGPIO INITIATOR

State	State Entry Condition Initiator
Wait for UART Handshake	Physical Reset, CRC Error

Start driving SGPIO Clock	Received 3 UART Handshake Messages in a row with correct CRC
SGPIO Operational	Target stopped sending UART Handshake Messages and SGPIO Clock output is stable for 100 cycles.

TABLE 62: SGPIO STATE TRANSITIONS FOR SGPIO TARGET

State	State Entry Condition Target
Start Sending UART Handshake	Physical Reset, CRC Error
Stop Sending UART Handshake	Received 50 stable clock cycles on SGPIO Clock input
SGPIO Operational	Observed SGPIO Load assertion.

12.5.2.3. SGPIO Interface Operational Phase

The SGPIO interface is required to support a transport phase as it is required to transport the Virtual Wire traffic. The SGPIO Interface operational phase follows the SGPIO timing, voltage requirements and waveform requirements as defined in the latest DC-MHS M-PIC Revision 1.0 specification and errata.

12.5.3. SGPIO Virtual Wire Packet Format

This section outlines the virtual wire transport packet format transported on the SGPIO bus. The definition of the SGPIO Operational Frame is defined in the table below. Note that the N is defined to be the maximum number of tunneled GPIOs in the frame.

TABLE 63 : SGPIO FRAME PAYLOAD DEFINITION

Offset [bytes]	Size [bits]	Bit Field								Description
		7	6	5	4	3	2	1	0	
0	8	VW 7	VW 6	VW 5	VW 4	VW 3	VW 2	VW 1	VW 0	Virtual Wires Index from 0 to 7
1	8	VW 15	VW 14	VW 13	VW 12	VW 11	VW 10	VW 9	VW 8	Virtual Wires Index from 8 to 15

…	…	…	…	…	…	…	…	…	…	…
N-1	8	VW (N-1)*8 + 7	VW (N-1)*8 + 6	VW (N-1)*8 + 5	VW (N-1)*8 + 4	VW (N-1)*8 + 3	VW (N-1)*8 + 2	VW (N-1)*8 + 1	VW (N-1)*8	Virtual Wires Indexes from ((N-1) * 8) to ((N-1) * 8) + 7
N[1]	8	VW (N)*8 + 7	VW (N)*8 + 6	VW (N)*8 + 5	VW (N)*8 + 4	VW (N)*8 + 3	VW (N)*8 + 2	VW (N)*8 + 1	VW (N)*8	Virtual Wires Indexes from ((N) * 8) to ((N) * 8) + 7
N+1	8	CRC-8								

[1]) Note that N depends on the Payload Size defined in Table 60 and N = ((Payload Size / 8) - 1)

The example of SGPIO running between SCM and HPM is presented in the Figure 25.

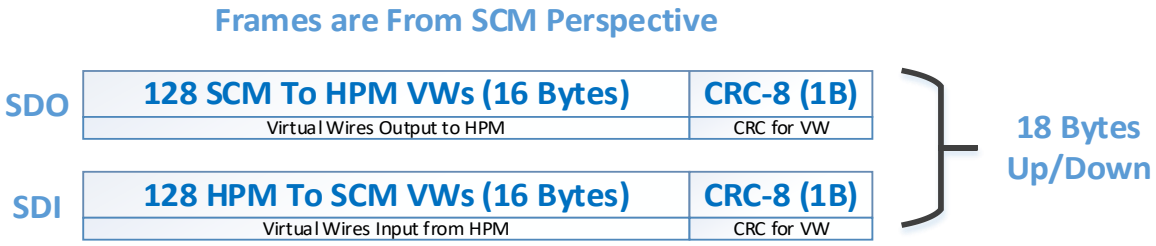


FIGURE 25 EXAMPLE OF SGPIO FRAMES EXCHANGING 128 VIRTUAL WIRES BETWEEN SCM AND HPM

12.5.4. SGPIO CRC and Error Handling

This section outlines the virtual wire transport packet format transported on the SGPIO bus.

Each SGPIO Frame contains the CRC checksum. This is CRC-8 with following polynomial:

$$x^8 + x^2 + x^1 + 1$$

The polynomial initial value is defined as '0'. The CRC is calculated for entire Virtual Wire payload. CRC shall be calculated without input bits inversion and without result inversion/reflection.

12.5.4.1. SGPIO CRC Error Handling

For the SGPIO Frames that contain CRC Error both Initiator and Target shall drop the frame and do not update the local Virtual Wire states based on the incorrect SGPIO Frame. It shall wait for the next SGPIO Frame with correct CRC error. An optional CRC Error Counter register is defined in the 12.5.7 SGPIO Interface Control and Status Registers Requirements to allow BMC to monitor the CRC error counters and apply corrective actions according to BMC FW policies.

To allow automatic IP recovery from intermittent error conditions that span across multiple SGPIO Frames the SGPIO Interface shall be reset when 3 consecutive CRC Errors are received. The reset of Initiator or Target shall trigger the switch to SGPIO Handshake state of both endpoints.

12.5.5. SGPIO Interface IP Block Architecture

The SGPIO IP Block Architecture shall be configurable at IP level to allow same IP instantiation as SGPIO Initiator in Figure 26 and SGPIO Target shown in Figure 27.

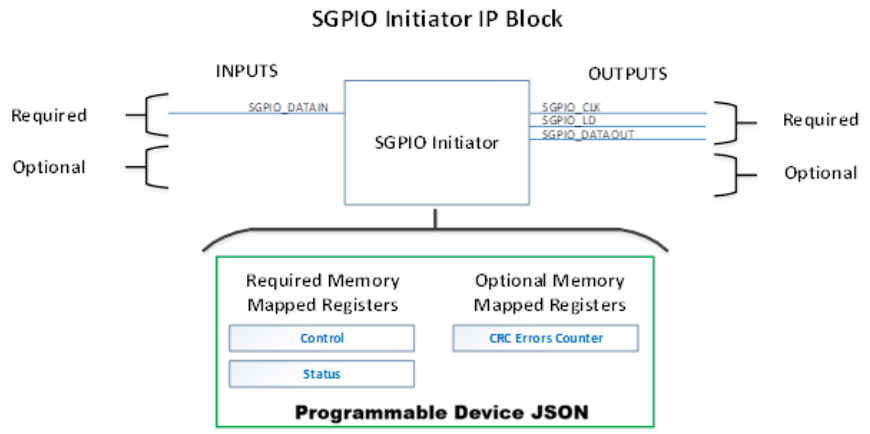


FIGURE 26: SGPIO INITIATOR IP BLOCK DIAGRAM

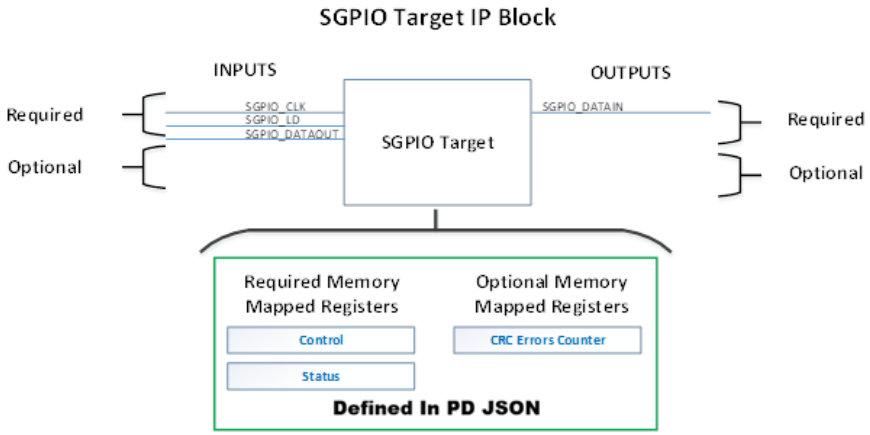


FIGURE 27: SGPIO TARGET IP BLOCK DIAGRAM

12.5.6. SGPIO Interface IP Block Requirements

This section contains the general abstract architecture for the SGPIO interface IP block.

12.5.6.1. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 64: SGPIO REQUIRED AND OPTIONAL FEATURE SET

Name	REQ	OPT	Description
SGPIO UART Handshake	Y		Required handshake protocol for discovery of the SGPIO bus.
SGPIO 1.0 Frame Compliance	Y		Required SGPIO frame protocol for basic SGPIO bus interoperability.
SGPIO CRC Error Counter CSR		Y	Required SGPIO error counter register for basic SGPIO bus interoperability.
SGPIO Required GPIO List Support		Y	Set of GPIOs that shall be tunneled to or from the PD for basic interoperability support.

12.5.7. SGPIO Interface Control and Status Registers Requirements

This section contains the Control and Status Registers (CSR) requirements for standardized SGPIO interface functionality.

12.5.7.1. SGPIO CSR Region Overview

These ranges are all offsets from the SGPIO_IP_BLOCK base address. This base address is defined by the CPLD/PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 65: SGPIO CSR MEMORY MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x00FF	Global Registers	General IP block level registers
1	0x0100	0x0FFF	SGPIO Interface CSRs	SPGIO IP block specific registers
2[13]	0x1000	0xFFFF	OEM SECTION	Vendor defined

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.5.7.2. SGPIO IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 66: SGPIO IP BLOCK CSR REGION 0 REGISTER MAP DEFINITION

Sub-Region	Range	Name	Description
0	0x00 – 0x000F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0x00FF	GLOBAL RESERVED	Save enough space if more registers are needed

[13] Note: Implementor can use this for additional regions as needed.

12.5.7.2.1. SGPIO IP Block Common Registers Sub-Region 0 Description

The table below contains the definition for the SGPIO IP block common registers sub-region 0.

TABLE 67: SGPIO IP BLOCK CSR REGION 0 SUBREGION 0 REGISTER MAP DEFINITION

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each)
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details. Additional Error Conditions: Invalid Command Invalid Address Boundary Error Bus Hang
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state with write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable

				See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	R/W	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x08 – 0x0F	Reserved			

12.5.7.2.2. SGPIO IP Block Common Registers Sub-Region 1 Description

Reserved

12.5.7.3. SGPIO IP Block Region 1 CSR Memory Map

This region is divided into two subregions.

TABLE 68: SGPIO IP BLOCK CSR REGION 0 SUBREGION 1 REGISTER MAP DEFINITION

Sub-Region	Range	Name	Description
0	0x0000 – 0x00FF	SGPIO IP Block Required CSR Set	Set of required registers for SGPIO functionality.
1	0x0100 – 0x0FFF	Vendor Extension	Reserved if more registers are needed

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as

needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.5.7.3.1. SGPIO IP Block Region 1 Registers Sub-Region 0 Description

The table below contains the list of SGPIO IP block common registers sub-region 1.

TABLE 69: SGPIO IP BLOCK CSR REGION 0 SUBREGION 1 REGISTER LIST

Name	Size	REQ	OPT	Description
SGPIO Link Status	4 B	Y		SGPIO Link Status Register
SGPIO Link Control	4 B	Y		SGPIO Link Control Register
SGPIO CRC Error Counter	4 B		Y	SGPIO Link Error Counter Register

The SGPIO CSR Registers with Offsets are defined in the Table 70.

TABLE 70 SGPIO CSR REGISTERS DEFINITION

Offset [bytes]	Size [bytes]	Name	Bitfield	Type	Definition
0x00	4	SGPIO Link Status	31:24	RO	IP SGPIO Payload Size
			23:16	RO	IP SGPIO Clock Speed
			15:12	RO	IP SGPIO Version Major
			11:8	RO	IP SGPIO Version Minor

<table>
<tr><td></td><td></td><td></td><td>7</td><td>RO</td><td>Frame Loss Error</td></tr>
<tr><td></td><td></td><td></td><td>6</td><td>RO</td><td>Clock Loss Error</td></tr>
<tr><td></td><td></td><td></td><td>5</td><td>RO</td><td>UART Handshake Timeout Error*</td></tr>
<tr><td></td><td></td><td></td><td>4</td><td>RO</td><td>CRC Error</td></tr>
<tr><td></td><td></td><td></td><td>3:0</td><td>RO</td><td>SGPIO Link State
<table><tr><th>State</th><th>Encoding</th></tr><tr><td>SGPIO Handshake</td><td>0x0</td></tr><tr><td>SGPIO Handshake Error</td><td>0x1</td></tr><tr><td>SGPIO Operational</td><td>0x2</td></tr><tr><td>SGPIO Operational Error</td><td>0x03</td></tr><tr><td>Reserved</td><td>0x4 – 0xF</td></tr></table></td></tr>
<tr><td>0x04</td><td>4</td><td>SGPIO Link Control</td><td>31:24</td><td>-</td><td>Reserved</td></tr>
<tr><td></td><td></td><td></td><td>23:16</td><td>RW</td><td>SGPIO Payload Size Override Value</td></tr>
<tr><td></td><td></td><td></td><td>15:8</td><td>RW</td><td>SGPIO Clock Speed Override Value</td></tr>
<tr><td></td><td></td><td></td><td>7:2</td><td>-</td><td>Reserved</td></tr>
<tr><td></td><td></td><td></td><td>1</td><td>RW</td><td>SGPIO Config Software Override Enable</td></tr>
<tr><td></td><td></td><td></td><td>0</td><td>RW</td><td>SGPIO Link Software Reset</td></tr>
<tr><td>0x08</td><td>4</td><td>SGPIO Handshake Data
Target: Sent,
Initiator: Received</td><td>31:24</td><td>-</td><td>Reserved</td></tr>
<tr><td></td><td></td><td></td><td>23:16</td><td>RO</td><td>SGPIO Payload Size</td></tr>
<tr><td></td><td></td><td></td><td>15:8</td><td>RO</td><td>SPIO Max Clock Speed</td></tr>
<tr><td></td><td></td><td></td><td>7:4</td><td>RO</td><td>SGPIO Version Major</td></tr>
<tr><td></td><td></td><td></td><td>3:0</td><td>RO</td><td>SGPIO Version Minor</td></tr>
<tr><td>0x0C</td><td>4</td><td>SGPIO Handshake Data OEM</td><td>31:16</td><td>-</td><td>Reserved</td></tr>
<tr><td></td><td></td><td></td><td>15:0</td><td>RO</td><td>OEM</td></tr>
</table>

		Target: Sent, Initiator: Received			
0x10	4	SGPIO CRC Error Counter	31:0	RWC	SGPIO Link CRC Error Counter
0x14	4	SGPIO Frame Loss Counter	31	RWC	Overflow Bit
			30:0	RWC	SGPIO Frame Loss Counter

*Note: The specific handshake timeout value is vendor defined. It is required that the HPM primary PD implement at least two (2) handshake retries before setting the SGPIO link state to a value indicating a handshake error and the UART handshake timeout error indication.

12.5.7.3.2. SGPIO IP Block Region 1 Registers Sub-Region 1 Description

Reserved for vendor use.

12.5.7.4. SGPIO IP Block Region 2 CSR Memory Map

Reserved for vendor use.

12.5.8. SGPIO Tunneled PD Virtual Wires Set Requirements

This section contains the set of required and optional recommended for transported GPIOs.

12.5.8.1. SCM To HPM SGPIO Tunneled GPIOs List

The table below contains the set of required and optionally recommended GPIOs tunneled by the SCM to HPM SGPIO bus. Here the input direction is represented by signaling going from the SCM to the HPM primary PD and the output direction is represented by signaling going from the HPM Primary PD to the SCM.

TABLE 71: SCM TO HPM SGPIO TUNNELED GPIO LIST

Name	DIR	W	REQ	OPT	Description
Global Interrupt	Out	1'b0	Y		Global interrupt signal. All IP block signals are wire OR'ed into this signal.
Current Power State Bit Field	Out	7'b0	Y		Power state field reflective of CSR bit field.
Power Management IP Block Interrupt	Out	1'b0		Y	Power management IP block interrupt. Optional recommended.
Power State Transition Flag	Out	1'b0	Y		Power state transition flag reflective of CSR bit.
Target Power State Transition Request Bit Field	In	7'b0	Y		Power state transition request bit field reflective of CSRs.
Power State Transition Trigger Flag	In	1'b0	Y		Power state transition trigger flag reflective of CSR bit.
Transition Error Flag	Out	1'b0		Y	Power state transition error flag reflective of CSR bit. Optional recommended.
Power State Error Indication Bit Field	Out	7'b0		Y	Error code indication reflective of CSR bit field. Optional recommended.
Power Supply Presence Indication Signal(s)	Out	1'b0	Y		Power supply presence indication reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD and then re-tunneled to the SCM via SGPIO.
Power Supply AC OK	Out	1'b0	Y		Power supply AC OK indication reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD and then re-tunneled to the SCM via SGPIO.
Power Supply DC OK	Out	1'b0	Y		Power supply DC OK indication reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD and then re-tunneled to the SCM via SGPIO.
Power Supply SMBus / PMBus Alert Signals	Out	1'b0	Y		Power supply SMBUS / PMBUS alert signals indication reflective of CSR bits as tunneled via PDB-HPM SGPIO to PD and then re-tunneled to the SCM via SGPIO.
CPU Presence	Out	Varies	Y		CPU presence signals mapped to normal latency GPIOs. Single bit if single CPU otherwise one bit per CPU.
RSM_RST_N	Out	1'b0	Y		Resume reset generated from PD to HPM on-board devices.

CPU Resume Reset	Out	1'b0	Y		Resume reset to the CPU generated from PD to HPM on-board devices.
CPU Reset	Out	1'b0	Y		Main CPU / SoC reset generated from PD to HPM on-board devices.
PCIe Reset	Out	1'b0	Y		Main PCIe reset generated from PD to HPM on-board devices.
Main Power Good	Out	1'b0	Y		Main power good signal generated from PD to HPM on-board devices.
CPU Standby Power Good	Out	1'b0	Y		Standby power good signal generated from PD to HPM CPUs / SoCs.
CPU Power Good	Out	1'b0	Y		Main power good signal generated from PD to HPM CPUs / SoCs.
Heartbeat Signal	Out	1'b0		Y	Optional Heartbeat signal that toggles to ensure HPM PD is alive. Most useful if implemented by an MCU and not programmable logic.

These are the same set of GPIOs as defined in the LTPI IP block section but with no latency designation as there is no distinction between lower and higher latency GPIOs in the SGPIO active phase definition.

Note: Default values for these signals will be provided in the PD JSON file.

12.5.8.1.1. SCM To HPM SGPIO Tunneled GPIOs Mapping PD Internal Virtual Wires Considerations

The PD implementor can map any internal PD virtual wire to an external interface, such as the SGPIO bus.

The table below contains categories of possible tunneled GPIOs mapped from virtual wires where the input direction represents from the SCM to the HPM PD and the output direction represents from the HPM PD to the SCM.

TABLE 72: SCM TO HPM SGPIO TUNNELED PD VIRTUAL WIRE CATAGORIES LIST

Name	DIR	Description
IP Block Interrupt Status Signals	Out	One or more IP block interrupt signals reflective of the IP Block global interrupt CSR bit(s).

Voltage Regulator Power Good and Error Indication Signals	Out	One or more voltage regulator power good and error / failure indication signals reflective of the IP Block CSR bit(s).
Flex IO and Sideband Signals	In / Out / Bidirectional	Mapped status reflective of the IP Block CSR bit(s).
MUX Control Signals	Out	Mapped control bits and bit fields reflective of the IP Block CSR bit(s).
PESTI Target Virtual Wire Control	In / Out / Bidirectional	Mapped bits and bit fields reflective of the IP Block CSR bit(s).
PDB SGPIO signals	In / Out / Bidirectional	Mapped bits and bit fields reflective of the IP Block CSR bit(s) tunneled from the HPM-PDB SGPIO IP block through the LTPI GPIO channel.
Control Panel Signal Tunneling	In / Out / Bidirectional	Mapped bits and bit fields reflective of the IP Block CSR bit(s) for either virtual wires or sideband signals.
Thermal State / Error Indication	Out	Overall state of system and component thermal status.
PD GPIOs	In / Out / Bidirectional	Mapped bits and bit fields reflective of the IP Block CSR bit(s).

It is required that the PD implementor provides these mappings within the PD JSON file. Section 7.4, “MHS HPM PD Specification Supplementary Materials”, contains a list of collators which contain one or more example PD JSON files with virtual wire mapping examples.

As there is no distinction between lower and higher latency GPIOs during the SGPIO active phase, it is recommended that the PD implementor takes extra care to ensure that the latency is sufficient to handle tunneling of time critical signals, such as interrupt signals and key power state fields.

12.5.8.2. SGPIO HPM To PDB Required and Optional GPIOs List

The table below contains the set of required and optionally recommended GPIOs tunneled by the HPM to PDB SGPIO bus.

TABLE 73: HPM TO PDB SGPIO TUNNELED GPIO LIST

Name	DIR	W	REQ	OPT	Description
PDB Global Interrupt	In	1’b0	Y		PDB interrupt signal. All PDB CPLD IP block signals are wire OR’ed into this signal.

Power Supply Presence Indication Signal(s)	In	1'b0	Y		Power supply presence indication reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD.
Power Supply AC OK Indication Signal(s)	In	1'b0	Y		Power supply AC OK indication reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD.
Power Supply DC OK Indication Signal(s)	In	1'b0	Y		Power supply DC OK indication reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD.
Power Supply PS_ON State Signal(s)	In	1'b0	Y		State of power supply PS_ON signal from PDB CPLD to power supplies reflective of CSR bit as tunneled via PDB-HPM SGPIO to PD.
Power Supply SMBus / PMBus Alert Signal(s)	Out	1'b0	Y		Power supply SMBUS / PMBUS alert signals indication reflective of CSR bits as tunneled via PDB-HPM SGPIO to PD.
Main Power Enable	Out	1'b0	Y		Main voltage rail (e.g., 12V IN) power on enable signal.

Note: Default values for these signals will be provided in the PD JSON file.

12.5.8.2.1. HPM To PDB SGPIO Tunneled GPIOs PD Internal Virtual Wires Mapping Considerations

The PD implementor can map any internal PD virtual wire to an external interface, such as the SGPIO bus.

The table below contains categories of possible tunneled GPIOs mapped from virtual wires where the input direction represents from the PDB to the HPM PD and the output direction represents from the HPM PD to the PDB.

TABLE 74: HPM TO PDB SGPIO TUNNELED PD VIRTUAL WIRE CATAGORIES LIST

Name	DIR	Description
IP Block Interrupt Status Signals	Out	One or more IP block interrupt signals reflective of the IP Block global interrupt CSR bit(s).

Error or Failure Indication	In	One more failure indication signals (e.g., voltage rail failure)
PICPWR Sideband Signals	In / Out / Bidirectional	Mapped status reflective of the either IP Block CSR bit(s) or pass-through from PDB programmable logic (e.g., CPLD).
MUX Control Signals	Out	Mapped MUX control bits and bit fields reflective of the PDB programmable logic (e.g., CPLD) IP Block CSR bit(s).
M-PESTI Target Virtual Wire Control	In / Out / Bidirectional	Mapped bits and bit fields reflective of the PDB programmable logic (e.g., CPLD) M-PESTI IP Block CSR bit(s).
M-PESTI Target Registers	In / Out / Bidirectional	Mapped bits and bit fields reflective of the PDB programmable logic (e.g., CPLD) M-PESTI IP Block CSR bit(s).
PDB Thermal State / Error Indication and Registers	Out	Mapped bits and bit fields reflective of the PDB state and / or component thermal status.
PDB Programmable Logic GPIOs	In / Out / Bidirectional	Mapped bits and bit fields reflective of the PDB programmable logic (e.g., CPLD) IP Block CSR bit(s).
Static Information	In / Out	Any static information such as IDs can be tunneled. However, it is recommended and encouraged to provide all static data via JSON files.

It is required that the PD implementor provides these mappings within the PD JSON file and the PDB vendor provides similar SGPIO mapping in the PDB CPLD JSON file. Section 7.4, “MHS HPM PD Specification Supplementary Materials”, contains a list of collators which contain one or more example PD JSON files with virtual wire mapping examples.

As there is no distinction between lower and higher latency GPIOs during the SGPIO active phase, it is recommended that the PD implementor takes extra care to ensure that the latency is sufficient to handle tunneling of time critical signals, such as interrupt signals and key power state fields.

12.5.9. SGPIO Interface Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.5.10. SGPIO Interface Example Implementation

Example implementations for SGPIO may be found within the collateral document, “IP Block Source Code Examples List" for specific examples. See section 7.4, “MHS HPM PD Specification Supplementary Materials”, for further details.

12.6. DC-SCM to HPM PD SPI Interface IP Logical Block

This section contains the logical IP block description for the DC-SCM to HPM PD SPI interface support requirements and descriptive abstraction for this IP block.

12.6.1. SPI Interface Conceptual Overview

This section contains an overview of the SPI interface functionality for a typical set of use cases.

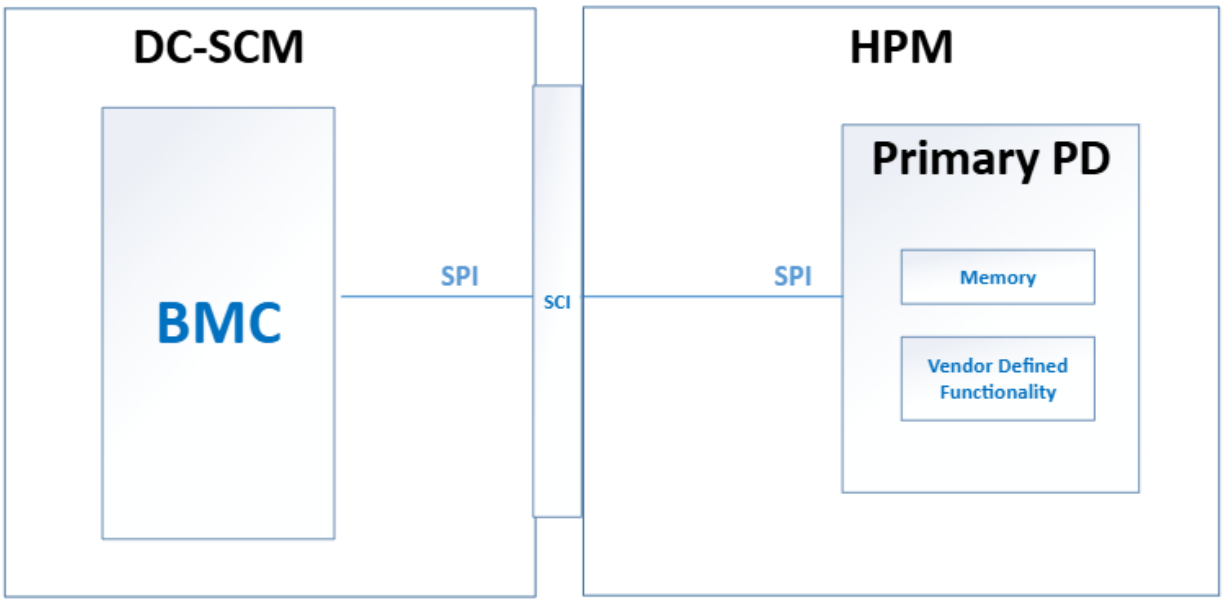


FIGURE 28: BMC SPI INTERCONNECTION TO PRIMARY HPM PD

It is intended that the SPI interface can be optionally provided by the PD as another mechanism for the BMC to get random access into the PD memory map. Additional use cases are undefined and up to the HPM vendor.

12.6.2. SPI Interface IP Block Architecture

This section contains the general abstract architecture for the SPI interface IP block. This interface operates as a SPI slave device much like a SPI flash component.

The figure below contains an architectural diagram of the SPI interface into the PD main memory.

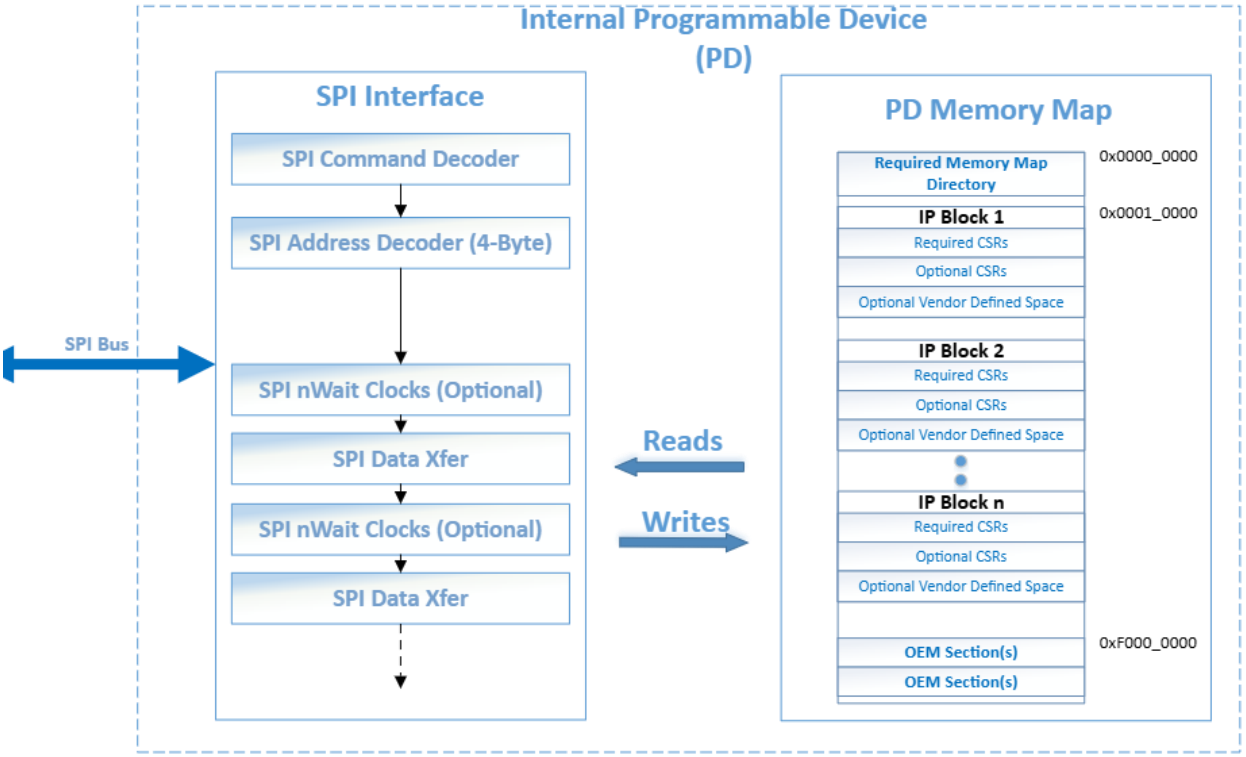


FIGURE 29: SPI ARCHITECTURAL DIAGRAM

It is the BMC's responsibility to handle synchronized atomic access via this interface and internal process and thread safety.

12.6.3. SPI Interface IP Block Requirements

This section contains the general abstract architecture for the SPI interface IP block.

12.6.3.1. SPI Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 75: SPI REQUIRED AND OPTIONAL FEATURES LIST

Name	REQ	OPT	Description
Random Access	Y		SPI IP block shall provide random access to PD main memory.
256 Byte Read and Write Transactions Support	Y		Maximum transaction size to be 256 bytes
Single Mode	Y		
Dual Mode Data	Y		SPI dual data mode support
Dual Mode Address / Command		Y	SPI dual mode to include address and command in addition to data.

12.6.3.2. SPI Required and Optional SPI Command Support

The table below contains the set of required and optional recommended SPI commands supported by this IP block.

TABLE 76: SPI REQUIRED AND OPTIONAL COMMAND LIST

Name	Code	REQ	OPT	Description
Normal WRITE	0x02	Y		Single I/O
Normal READ	0x03	Y		Single I/O – No Dummy Bytes
Fast READ	0x0B	Y		Single I/O – Dummy Byte Supported for device read latency
DUAL WRITE (1-1-2)	0x22		Y	Dual I/O for Data
DUAL WRITE (1-2-2)	0x23		Y	Dual I/O for Address and Data
DUAL READ (1-1-2)	0x3C		Y	Dual I/O for Data – Dummy Byte Supported for device read latency
DUAL READ (1-2-2)	0xBB		Y	Dual I/O for Address and Data – Dummy Byte Supported for device read latency

It is required that the PD provide the necessary safeguards against writing past boundaries and to ignore all writes that exceed a given region or sub-region boundary. Reads that exceed a region or sub-region boundary shall result in returning 0xFF when crossing the boundary.

12.6.3.3. SPI Command Timing Diagrams

The following timing diagrams below contain the timing diagram reference examples.

Normal Read Command

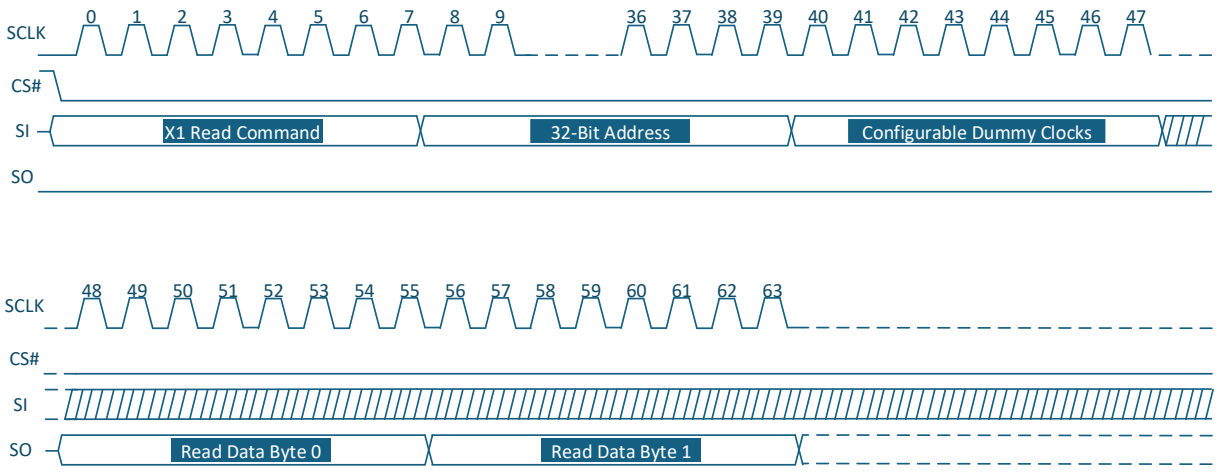


FIGURE 30: SPI INTERFACE NORMAL READ COMMAND

Dual Read Command With Configurable Dummy Clocks

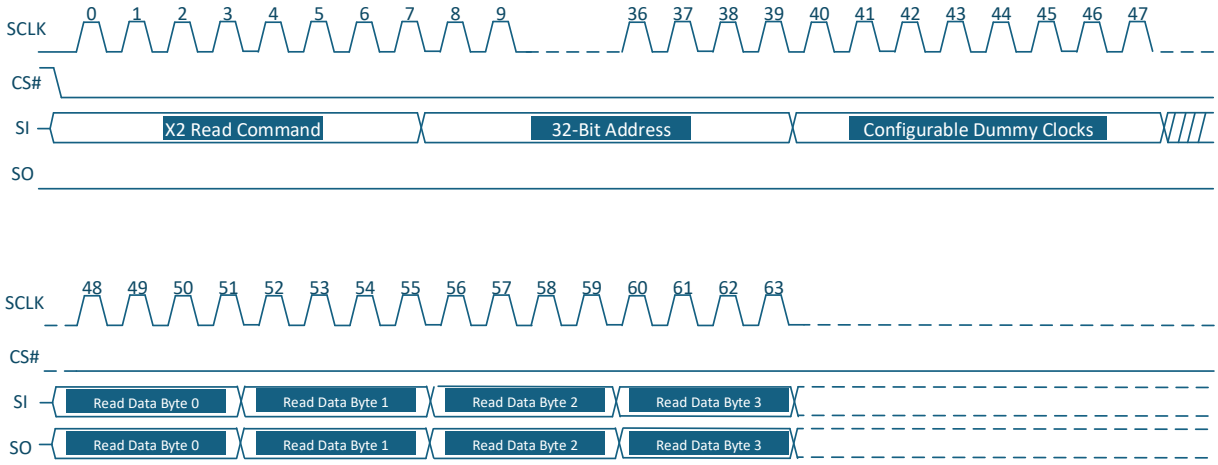


FIGURE 31: SPI INTERFACE DUAL READ COMMAND WITH DUMMY CLOCKS

Normal Write Command

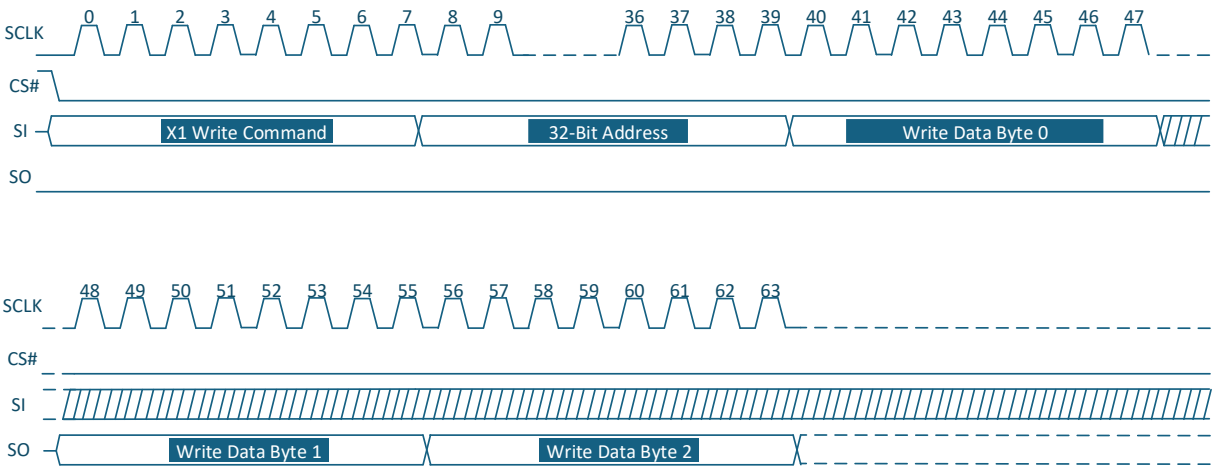


FIGURE 32: SPI INTERFACE NORMAL WRITE COMMAND

Normal Write Command With Configurable Dummy Clocks

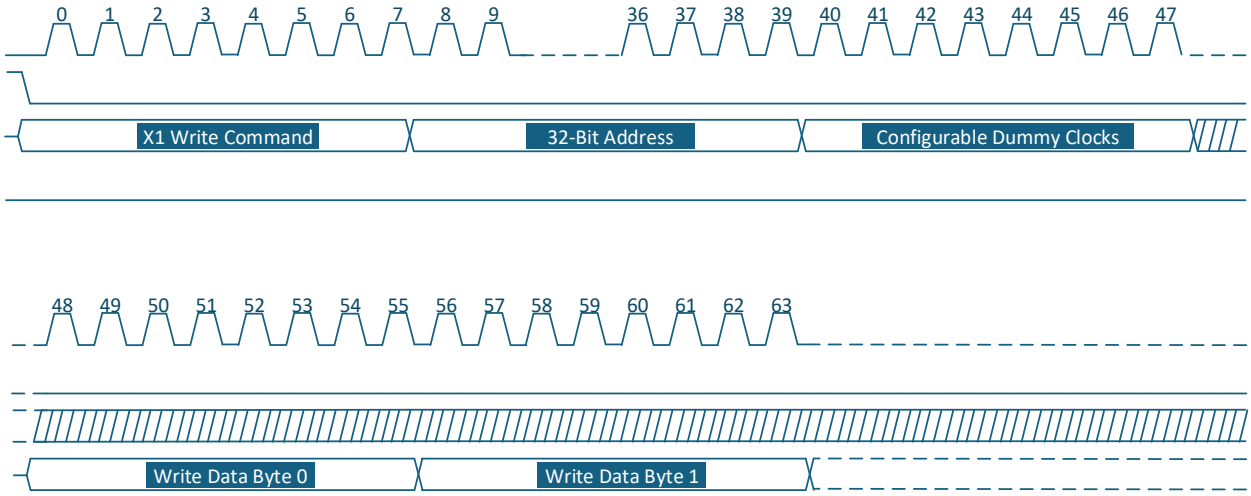


FIGURE 33: SPI INTERFACE NORMAL WRITE COMMAND WITH DUMMY CLOCKS

Dual Write Command With Configurable Dummy Clocks

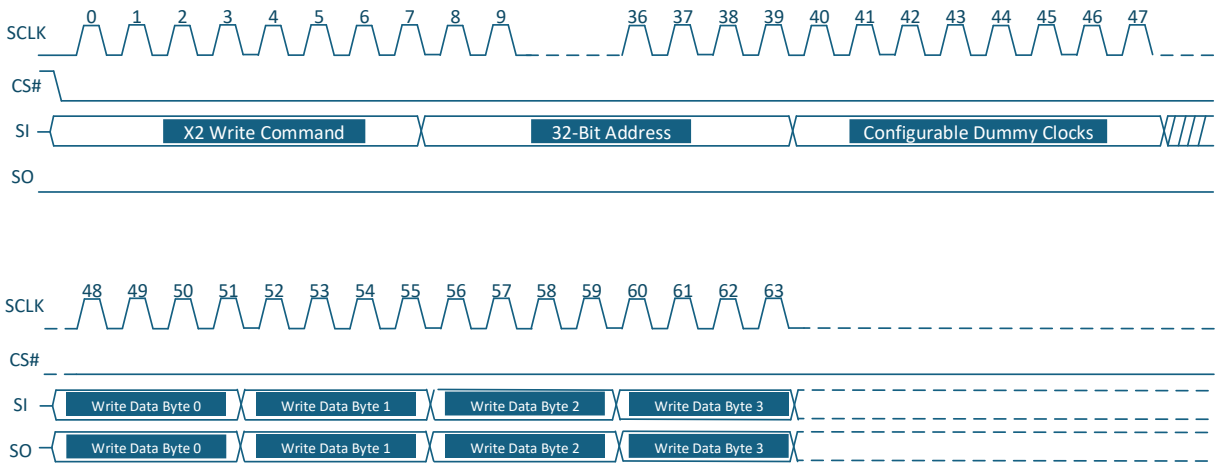


FIGURE 34: SPI INTERFACE DUAL WRITE COMMAND WITH DUMMY CLOCKS

12.6.4. SPI Interface Control and Status Registers Memory Map

This section contains the Control and Status Registers (CSR) memory map for standardized SPI Interface functionality in the PD.

12.6.4.1. SPI CSR Region Overview

These ranges are all offsets from the SPI_IP_BLOCK base address. This base address is defined by the Programmable Device (PD) developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 77: SPI CSR MEMORY MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x0FFF	Global Registers	General IP block level registers

1[14]	0x1000	0xFFFF	OEM SECTION	Vendor defined

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register location offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.6.4.2. SPI IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 78: SPI IP BLOCK CSR REGION 0 REGISTER MAP DEFINITION

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFFF	GLOBAL RESERVED	Reserved for future use.

12.6.4.2.1. SPI IP Block Common Registers Sub-Region 0 Description

The table below contains the definition for the SPI IP block common registers sub-region 0.

[14] Note: Implementor can use this for additional regions as needed.

TABLE 79: SPI IP BLOCK CSR REGION 0 SUBREGION 0 REGISTER MAP DEFINITION

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2B	R	Major.Minor (1 byte each)
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details. Additional Error Conditions: Invalid Command Invalid Address Boundary Error Bus Hang
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1B	RW1C	Interrupt state with write once to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	RW	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x08 – 0x0F	Reserved			

12.6.4.2.2. SPI IP Block Common Registers Sub-Region 1 Description

Reserved

12.6.4.3. SPI IP Block Region 1 CSR Memory Map

Reserved for vendor defined usage.

12.6.5. SPI Interface Vendor Specific Extensions Support

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support.

12.6.6. SPI Interface Example Implementation

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

Example implementations for SPI may be found within the collateral document, “IP Block Source Code Examples List" for specific examples. See section 7.4, “MHS HPM PD Specification Supplementary Materials”, for further details.

12.7. DC-SCM to HPM PD I2C / I3C Basic Interface IP Logical Block

This section contains the logical IP block description for the DC-SCM to HPM PD I2C / I3C Basic interface support requirements and descriptive abstraction for this IP block.

12.7.1. I2C / I3C Basic Interface Conceptual Overview

This section contains an overview of the I2C / I3C Basic interface functionality for a typical set of use cases.

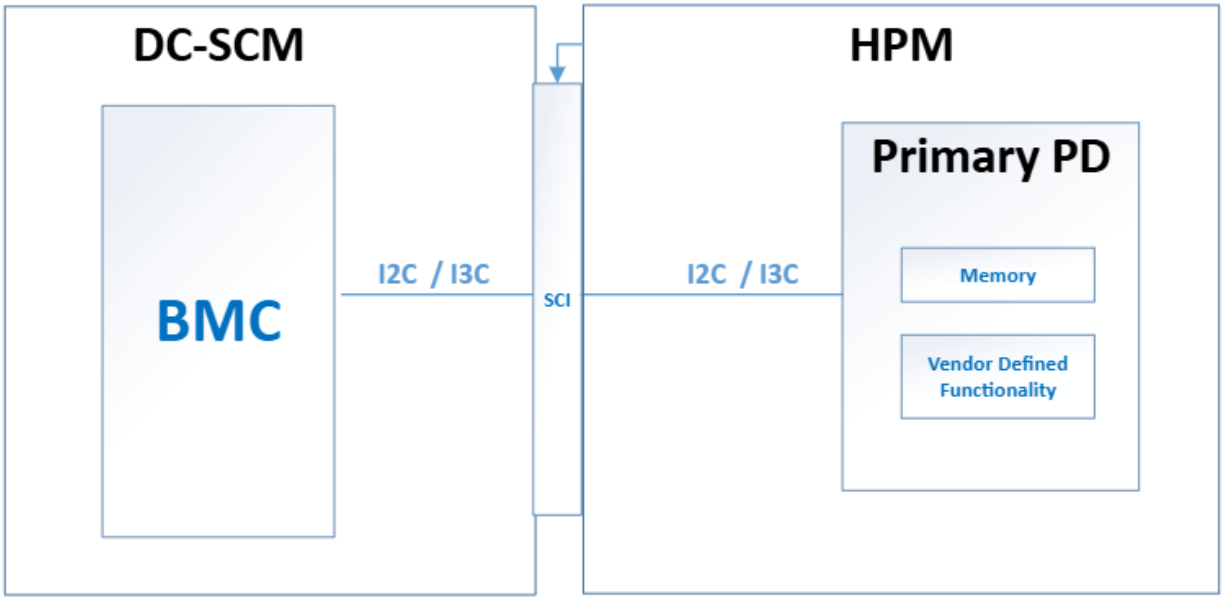


FIGURE 35: BMC I2C/I3C CONNECTION TO PD

One example use case for I2C or I3C is that it can optionally provide another mechanism for the BMC to get random access into the PD memory map. This would behave much like an I2C EEPROM device. Additional use cases are undefined and up to the HPM vendor.

12.7.2. I2C / I3C Basic Interface IP Block Architecture

This section contains the general abstract architecture for the I2C / I3C Basic interface IP block.

The figure below contains an architectural diagram of the I2C / I3C Basic mailbox interface into the PD main memory.

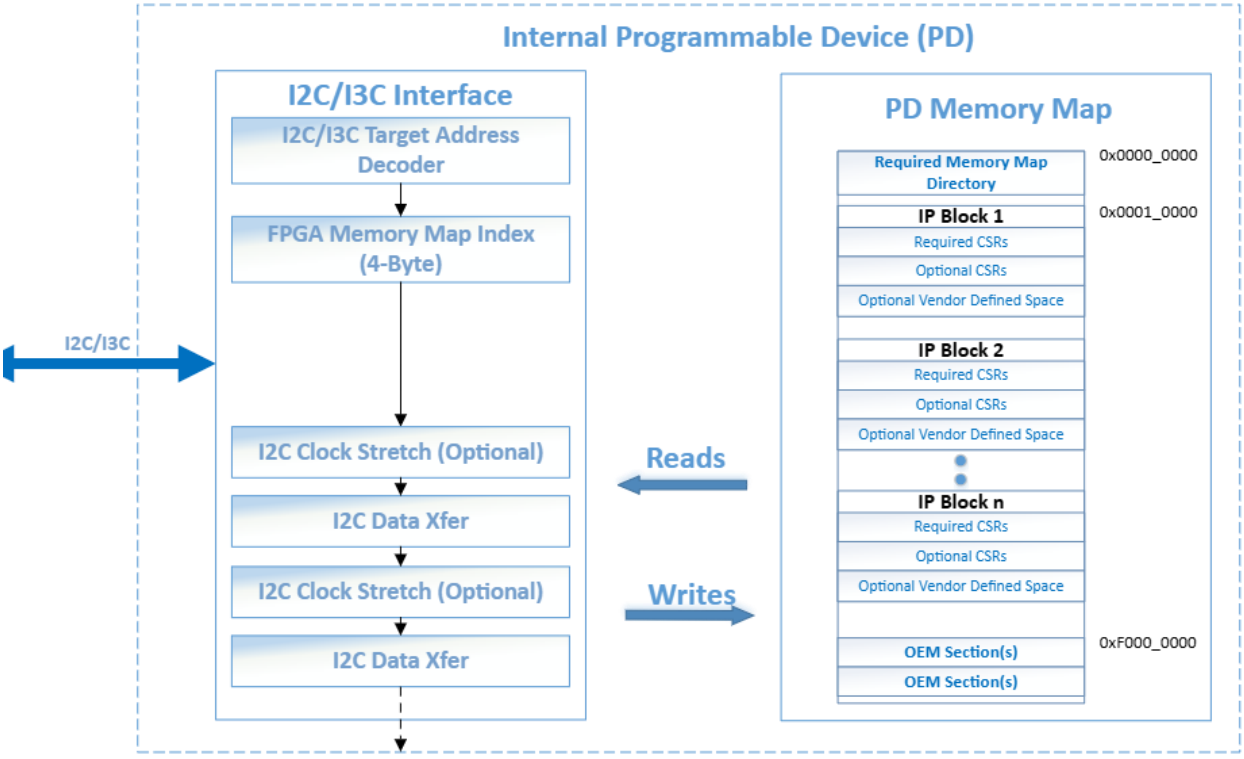


FIGURE 36: I2C MAILBOX REGISTERS ARCHITECTURAL DIAGRAM

Access via the I2C interface, as shown in the diagram, requires a 4-byte address used to direct reads and writes into the memory space much like a standard I2C EEPROM. This allows up to 256 byte reads and writes to and from the PD main memory space. It is required that the PD provide the necessary safeguards against writing past boundaries and to ignore all writes that exceed a given region or sub-region boundary. Reads that exceed a region or sub-region boundary shall result in returning 0xFF when crossing the boundary.

It is the BMC's responsibility to handle synchronized atomic access via this interface and internal process and thread safety.

12.7.3. I2C / I3C Basic Interface IP Block Requirements

This section contains the general abstract architecture for the I2C / I3C Basic interface IP block.

12.7.3.1. I2C / I3C Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 80: I2C REQUIRED AND OPTIONAL FEATURES LIST

Name	REQ	OPT	Description
I2C Specification	Y		If this interface implements I2C then this IP block shall adhere to specification version 7.0.
I3C Specification	Y		If this interface implements I3C this IP block shall adhere to specification version 1.2.
Random Access	Y[15]		Provide random access to the PD memory map only if this is not a tunneled interface.
SPDM		Y	Can optionally provide SPDM support only if this is not a tunneled interface if interface is internally connected as a multi-function endpoint.

12.7.3.2. Multi-function I2C / I3C Endpoint Considerations

The I2C or I3C interface may be internally connected within the PD as a multifunction endpoint. This would allow the PD to support SPDM on the same I2C or I3C bus by responding to a different slave address. The platform designer shall ensure there are no address conflicts, and all addresses and device descriptions shall be in the PD JSON and HPM JSON.

[15] Note: Random access provided only if this is not a tunneled interface via external bus (e.g. LTPI).

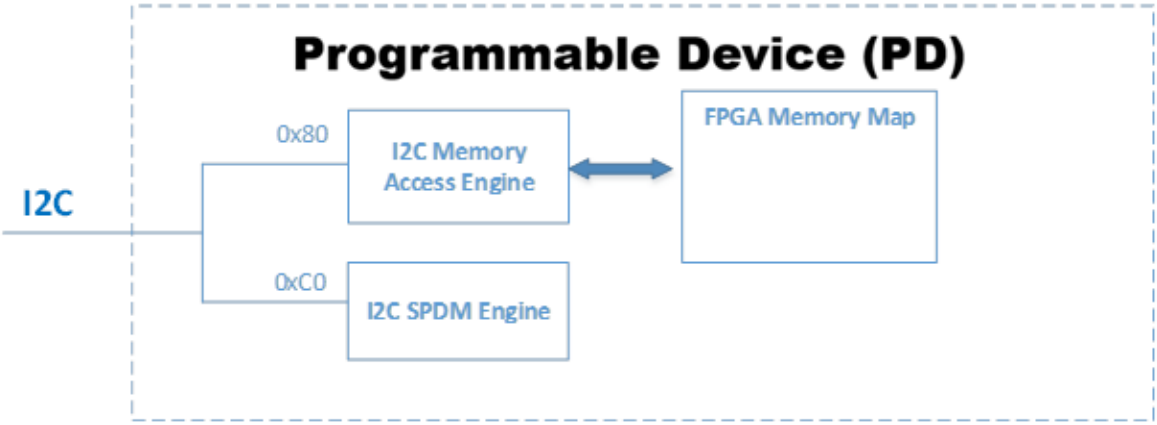


FIGURE 37: MULTI-FUNCTION I2C / I3C ENDPOINT EXAMPLE DIAGRAM

The example diagram above indicates that the same bus interface can support multiple addresses with one supporting this IP block as memory access engine and the other connected internally to SPDM engine. The I2C addresses shown above are meant as examples only.

12.7.3.3. I2C Command Timing Diagrams

The following timing diagrams below contain the timing diagram reference examples.

I2C Read Command

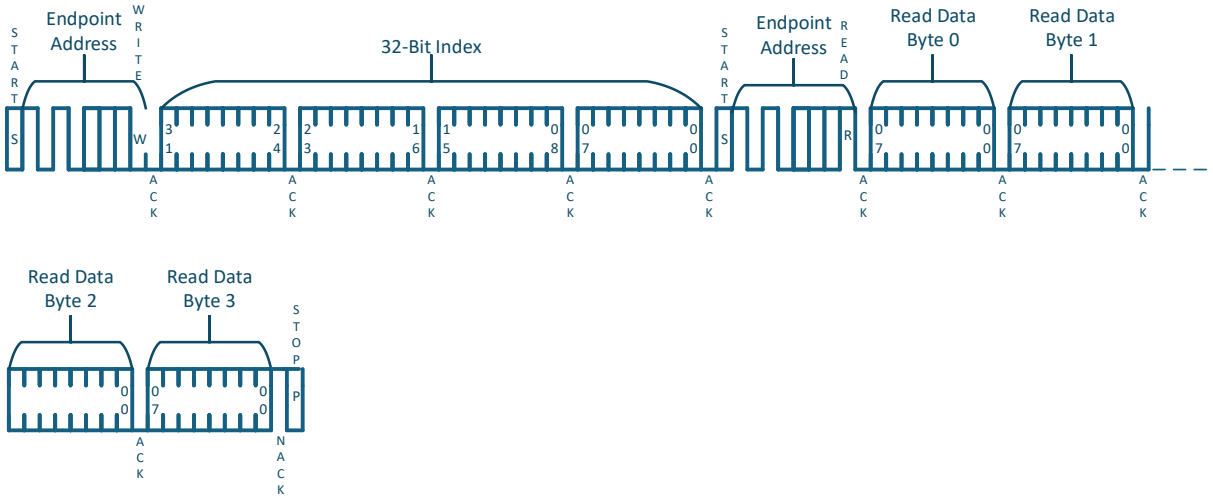


FIGURE 38: EXAMPLE I2C READ TIMING DIAGRAM

I2C Write Command

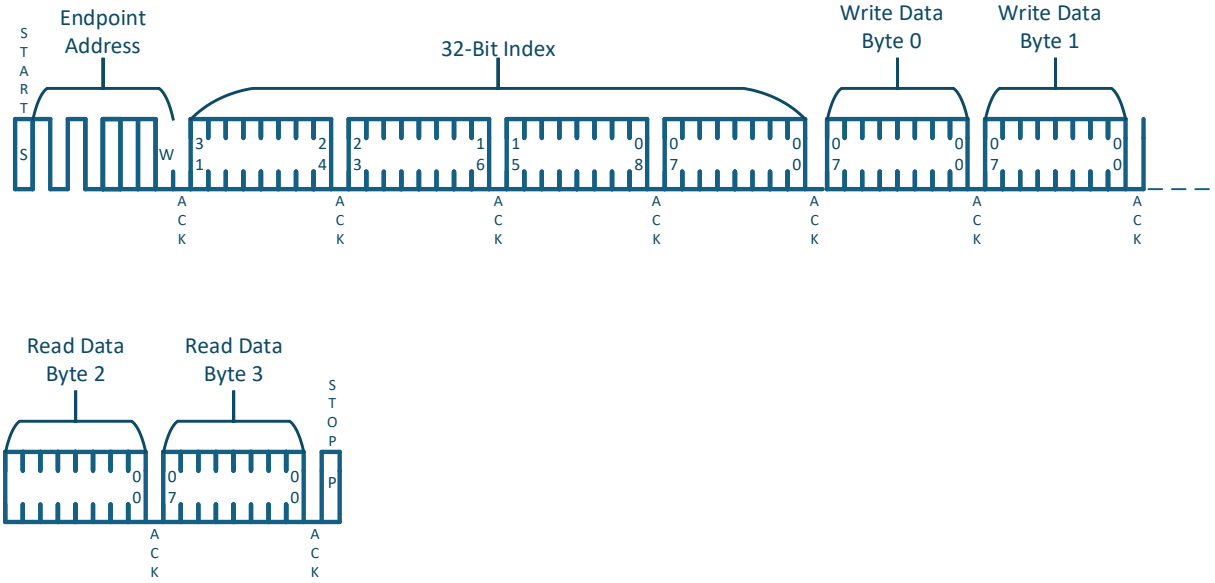


FIGURE 39: EXAMPLE I2C WRITE TIMING DIAGRAM

12.7.3.4. I3C Basic Considerations

This section explores some of the differences between an implementation using I2C and one supporting I3C basic.

Key differences between I2C and I3C Basic is outlined below:

- Faster data transfer rates: I3C's push-pull interface allows for significantly higher speeds than I2C.
- Dynamic addressing: This feature simplifies device management and reduces the risk of address conflicts.
- In-band interrupt Signals: Eliminates the need for separate interrupt pins, saving valuable I/O pins.
- Error detection and recovery: I3C includes error detection mechanisms, such as parity checks and CRC (Cyclic Redundancy Check), for data integrity.

The IP block will provide the same logical EEPROM-like functionality regardless of whether it supports I3C Basic as well as I2C.

12.7.4. I2C / I3C Basic Interface Control and Status Registers Memory Map

This section contains the Control and Status Registers (CSR) memory map for standardized I2C / I3C Basic Interface functionality.

12.7.4.1. I2C / I3C Basic CSR Region Overview

These ranges are all offsets from the I2C_I3C_IP_BLOCK base address. This base address is defined by the CPLD/PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 81: I2C CSR MEMORY MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x00FF	Global Registers	General IP block level registers

1[16]	0x1000	0xFFFF	OEM SECTION	Vendor defined

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.7.4.2. I2C / I3C Basic IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 82: I2C / I3C IP BLOCK CSR REGION 0 SUBREGION 0 REGISTER MAP DEFINITION

Sub-Region	Range	Name	Description
0	0x0000 – 0x000F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x0010 – 0x0FFF	GLOBAL RESERVED	Save enough space if more registers are needed

12.7.4.2.1. I2C / I3C Basic IP Block Common Registers Sub-Region 0 Description

[16] Note: Implementor can use this for additional regions as needed.

The table below contains the definition for the I2C / I3C Basic IP block common registers sub-region 0.

TABLE 83: I2C / I3C IP BLOCK CSR REGION 0 SUBREGION 0 SUBREGION 0 REGISTER MAP DEFINITION

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each)
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details. Additional Error Conditions: Invalid Command Invalid Address Boundary Error
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW1C	Interrupt state that utilizes a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.

0x07	Extended Status	1B	R/W	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x08 – 0x0F	Reserved			

12.7.4.2.2. I2C / I3C Basic IP Block Common Registers Sub-Region 1 Description

Reserved

12.7.4.3. I2C / I3C Basic IP Block Region 1 CSR Memory Map

Reserved for vendor defined usage.

12.7.5. I2C / I3C Basic Interface Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support.

12.7.6. I2C / I3C Basic Interface Example Implementation

Example implementations for I2C / I3C Basic may be found within the collateral document, “IP Block Source Code Examples List" for specific examples. See section 7.4, “MHS HPM PD Specification Supplementary Materials”, for further details.

12.8. M-PESTI Interface Logical IP Block

This section contains the logical IP block description for the M-PESTI requirements and descriptive abstraction for this IP block.

12.8.1. M-PESTI Interface Conceptual Overview

This section contains an overview of the M-PESTI interface functionality for a typical set of use cases.

As it is intended for M-PESTI to be the primary discovery mechanism for peripherals and options regarding presence detection, it is required that each MPESTI target shall provide cable presence functionality, at a minimum.

It is required that the PD shall provide the self-describing JSON data necessary for the BMC to locate all M-PESTI registers within the M-PESTI CSR space that are needed for presence detection for all M-PESTI connected MXIO and MPIC connectors within the system. Simple presence indication is the minimum requirement. However, additional M-PESTI capabilities are recommended. Please refer to the M-PESTI specification for additional details.

Please see "M-PNP Discovery, Boot, and FRU Specification" for the discovery process usage of M-PESTI.

12.8.2. M-PESTI M-PNP Specific Terminology

The list of specific terminology regarding M-PESTI is in the table below.

TABLE 84: M-PESTI M-PNP SPECIFIC TERMINOLOGY

Term	Description
Target	Single PESTI end device that the initiator communicates with. This is sometimes referred to as a ‘PESTER’ although

	in this proposal we will stick to the M-PESTI specification terms.
M-PESTI Engine	The PESTI initiator hardware in the PD / CPLD device.
Initiator Instance	An initiator instance is 1:1 for each PESTI bus wire. For a single PESTI wire, it may handle up to eight (8) PESTI targets through a MUX as per the M-PESTI specification. If a given vendor defined implementation supports larger than eight (8) MUXes, then it shall indicate that and provide target CSR space up to their reported maximum and report the maximum latency during the active phase for virtual wires to the BMC due to the additional MUX/target support provided in the JSON file. Support for MUXes shall be described in the PD JSON file for each initiator as well as the maximum fanout supported if it deviates above the baseline eight (8) maximum targets.
Initiator IP block	An initiator IP block handles up to eight (8) MPESTI bus initiator instances thus eight (8) individual bus wires. An initiator IP block can either be implemented using a **single state machine** and an internal MUX to control up to 8 targets through up to 8 PESTI wires. **OR** An initiator IP block can have **individual, independent state machines per PESTI wire**. This can result in up to sixty-four (64) PESTI targets if the implementor supports it via external MUXes. Support for PESTI MUXes is optional. A given implementation can combination of Engine and MUX. The number of the initiators supported by initiator IP block shall be provided PD JSON file. See section "**FI M-PESTI Initiator Instance IP Block Implementation Variations**", for more details.
Initiator IP Block Set	This is a grouping of Initiator IP blocks. It can support up to sixty-four (64) individual PESTI wires using 8 initiator IP blocks which can result in 64*8 = 256 PESTI targets via MUXes. The size of the initiator block set with the quantity of initiator IP blocks shall be provided PD JSON file.
Initiator Super Sets	More than one initiator IP Block set. A group of N number of initiator IP Block sets that allows for supporting a larger

	number of initiators IP blocks beyond 8 thus resulting in a much larger possible super set of targets beyond 256. The number of the initiator block sets supported by the PD shall be provided PD JSON file.
MPESTI IP Block	This contains all individual initiator IP block sets.

12.8.3. M-PESTI Interface IP Block Architecture

This section contains the general abstract architecture for the M-PESTI interface IP block.

The diagram below shows the relationship between targets, initiator instances, initiator IP blocks, and initiator IP block sets.

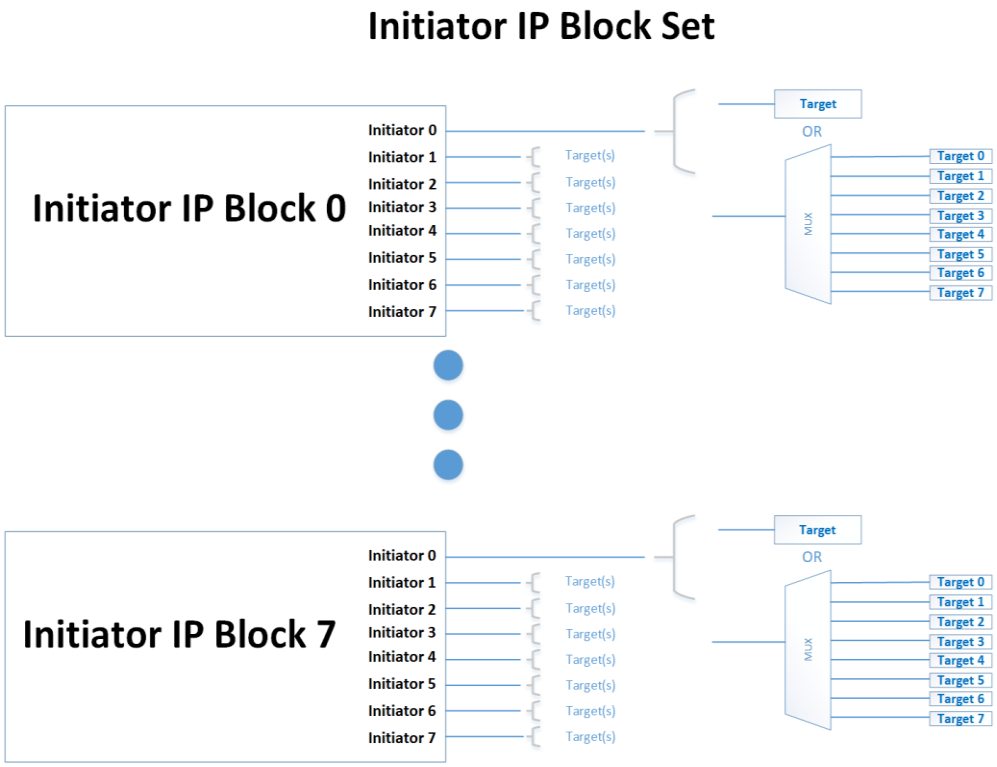


FIGURE 40: INITIATOR IP BLOCK ARCHITECTURAL BLOCK DIAGRAM

Initiator IP blocks can then be organized in a larger IP block **super-set**. This is shown in the following diagram.

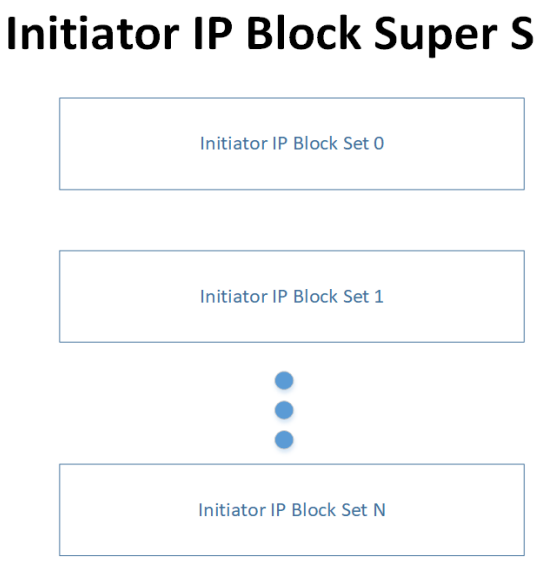


FIGURE 41: INITIATOR IP BLOCK SUPER-SET CONCEPTUAL BLOCK DIAGRAM

A given PD/CPLD design can have one or more initiator sets. Each initiator set has its own memory space allocated as per the memory map described in this document. An overall MPESTI IP block can implement a super-set to combine all MPESTI support into a single block or can subdivide into several depending on the requirements of the platform design.

12.8.3.1. M-PESTI Initiator Instance IP Block Implementation Variations

This section outlines various MPESTI initiator IP block implementation variations that can be used in a given platform design.

There are two main variations of Initiator instance IP block implementations:

- 1 state machine per 1 PESTI wire as an Initiator Instance
  - External MUXes can be supported.
- 1 state machine and multiple wires (up to 8) through an internal MUX
  - Uses MUX / Switch control command messages.
  - Cannot support external MUXes because internal MUX is target of MUX / Switch control messages.

A description of the IP block and what it is capable of shall be provided in the PD JSON file.

Initiator IP Block With Independent State Machines Per Initiator Instance / PESTI Wire

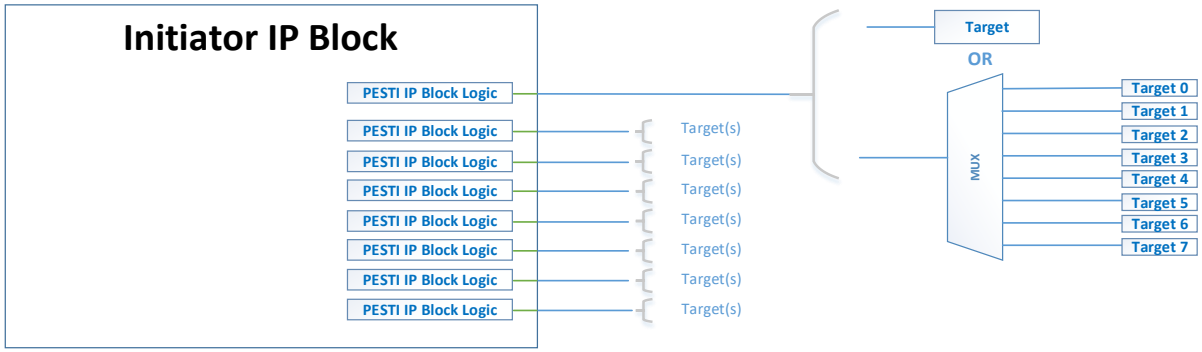


FIGURE 42: INITIATOR IP BLOCK WITH INDEPENDENT STATE MACHINES PER PESTI WIRE EXAMPLE

You can define the initiator IP block to represent a set of up to 8 independent state machines which correspond to one state machine per PESTI wire.

This is the most interoperable configuration.

Initiator IP Block With Internal MUX

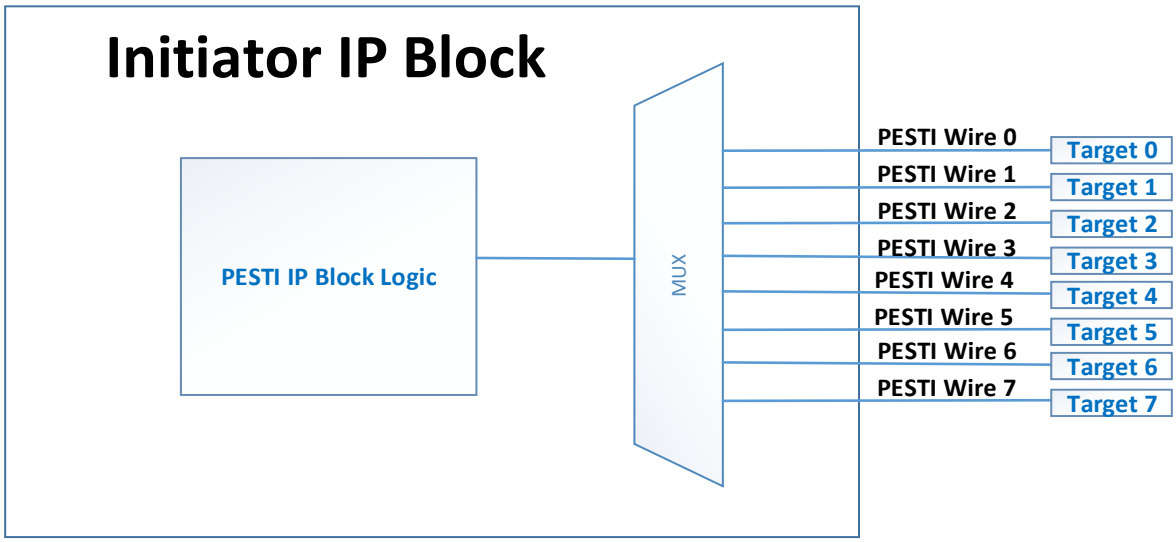


FIGURE 43: M-PESTI INITIATOR IP BLOCK WITH INTERNAL MUX

Alternatively, you can define an initiator IP block to represent a single state machine that manages up to 8 PESTI wires through an internal MUX. If so, then it will have to indicate that it is not able to support and external MUX.

An implementation can have the MUXing logic locally in the PD or remotely on the peripheral. However, if the MUX is locally implemented in the PD, then all peripherals shall be single targets such that the sum of all targets for the PESTI wire group is no larger than eight (8) for baseline interoperability.

This is considered the least interoperable configuration.

Support for a peripheral that includes a MUX on the PESTI wire is optional. is the HPM may support a peripheral implementation that includes a MUX. Choosing not to support a MUX on the peripheral is the least interoperable choice for an HPM implementation.

Initiator IP Block Maximum Target Support Rule

Intention is to create a rule for the Initiator IP block to indicate how many targets it can support. This rule would be checked by the BMC during discovery. The rule is described below.

- The system designer / integrator would create a maximum number of targets supported for each Initiator IP block to meet their latency requirements. Typically, it would be <= 8, but is free to choose another number equal to N
- BMC would parse JSON files to determine how many potential targets exist based on number of MUXes on each peripheral and number of targets due to MUX fanout.
- BMC will assert POWERON_ALLOW if:
    - The sum total of peripherals targets supported as indicated in the JSON file that is then for identified is equal to X.
    - If X is less than or equal (<=) to the target then then the rule shall be considered satisfied.
- Otherwise, BMC shall indicate a failed configuration.
    - The indication mechanism is beyond scope of this specification.

Initiator IP Block target support per PESTI wire is typically a maximum of eight due to MUX fanout restrictions in the MUX / Switch control message. It is based off the JSON file information which reflects what is the actual implementation.

12.8.4. M-PESTI Discovery Considerations

This section outlines specific discovery process considerations that are relevant to the MPESTI IP block.

To assist the BMC in the discovery process, it is required that each M-PESTI bus wire instance in the JSON file has attributes which indicate whether it supports an external MUX or is connected to an internal MUX. This indication is implemented via a specific set of attributes in

the JSON file. Consult the “HPM and Peripheral Programmable Device JSON Schema ” and schema documentation for more details regarding specific JSON attributes.

Details regarding the discovery process are outlined explicitly within the M-PNP.FRU_DISCOVERY_BOOT specification.

12.8.5. M-PESTI Interface IP Block Requirements

This section contains the general abstract architecture for the M-PESTI interface IP block.

12.8.5.1. M-PESTI Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 85: M-PESTI IP BLOCK REQUIRED AND OPTIONAL FEATURE SET

Name	REQ	OPT	Description
M-PESTI Specification Version 1.2	Y		The M-PESTI IP block shall adhere to the most recent M-PESTI specification.

12.8.6. M-PESTI Control and Status Registers Requirements

This section contains the Control and Status Registers (CSR) requirements for standardized M-PESTI interface functionality. It provides a common set of CSRs for one or more M-PESTI initiators that system firmware can interact with. The intention is to allow for flexible and simple extensibility of a group of initiators into what we are defining as an initiator IP block and initiator set. This is complementary to the discovery process usage of FRU information storage devices for MHS component discovery.

12.8.6.1. M-PESTI IP BLOCK CSR Region Map Overview

This section contains an overview of the MPESTI IP block CSR regions. These ranges are all offset from the MPESTI_IP_BLOCK base address. This base address is defined by the PD developer and is not prescribed here. The defined regions below will be accessible to the BMC via external memory mapped random access interfaces. This shall be described with the self-describing JSON file distributed as part of the PD image.

TABLE 86: M-PESTI IP BLOCK CSR MEMORY MAP

Region	Range Bottom	Range Top	Sections	Name	Description
1	0x00000	0x0FFFF		Global Registers	IP block level that affects entire set of targets. Subdivided into required and optional sections. Required 48B global range.
2	0x10000	0x1FFFF	0x1XXX0	Target Required CSR Set (4B per target).	Contains standard MPESTI registers plus the number of virtual wires in and out per each target. This has DSTAT some basic config and status. All the required registers per target are defined here. 64 targets * 4 then 256B
3	0x20000	0x2FFFF	0x2XX00	Discovery Payload Linear Address Space	128 bytes per target window
4	0x30000	0x3FFFF	0x3XXX0	VWIRE range 32B per target with 16B as input signals and 16B as output signals.	Intention is to mirror Virtual Wires into block RAM in 32B increments per target. It will contain 16B for inputs and 16B for the mirrored VWIREs outputs as well.
5	0x40000	0x4FFFF	0x4XX00	Target Optional CSR Set [Optional]	All per target optional but specified CSRs. Sets aside 256B per target.
6	0x50000	0x5FFFF		OEM* Section [Optional]	Optional and unspecified and not organized on a per target basis.

*Note: OEM defined space can be used by OEM and/or CPLD/PD developer.

The PD developer shall create a JSON file as a build artifact that will contain the MPESTI_IP_BLOCK base address. The JSON file will be used by BMC firmware to determine register locations from the BMC's perspective via the memory map. The JSON file will be collected by the BMC from the PD when the PD is powered but still in reset. There will be a predefined location for the PD JSON file, and it will be detailed in the PD spec and referenced by the MPNP.FRU_DISCOVERY_BOOT specification as part of the discovery process. Additionally, the PD JSON file is packaged with the firmware image and can be collected from that image by the BMC provided it has a cached copy of that image. It is acceptable to have an initial copy of any peripheral PLD JSON files in the peripheral FRU EEPROM provided that the BMC and/or the PLD itself has patch copies cached or in non-volatile memory. See section 15.1, "PD Image Considerations and Requirements", for details on PLD image and JSON file management.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register location offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.8.6.2. M-PESTI IP Block Global Registers Section

This section describes the register set that applies globally across the entire IP block. This space has been divided into a set of required and optional registers. Base address starts at 0 for the global space.

TABLE 87: M-PESTI IP BLOCK CSR REGION 0 REGISTER MAP DEFINITION

Sub-Region	Range	Name	Description
1	0x0000 – 0x0030	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
2	0x0030 – 0x0FFF	GLOBAL RESERVED	Save enough space if more registers are needed
3	0x1000 – 0xFFFF	GLOBAL OEM	Available space for OEM usage. This is where any optional registers (dev, debug, etc.) would be location.

12.8.6.2.1. M-PESTI IP Block Global Required Registers

TABLE 88: MPESTI IP BLOCK CSR REGION 0 SUBREGION 0 REGISTER MAP DEFINITION

Address	R/O	Name	W	ATTR	Description
0x00	REQ	IP Block ID	2B	RO	Not a GUID – just uniquely ID that this is MPESTI block to connect the dots between JSON block ID and actual HW.
0x02	REQ	IP Block Version	2B	RO	Major. Minor BCD encoded.
0x04	REQ	Global Status	1B	RW	Common status register
0x05	REQ	Global Control	1B	RW	Common control register. Allows for a hard reset to the IP block.
0x06	REQ	Interrupt Status	1B	RW	IP Block Interrupt Status
0x07	REQ	Interrupt Control	1B	RW	IP Block Interrupt Control
0x08	REQ	RESERVED	4B	R	reserved
0x0C	REQ	RESERVED	4B	R	reserved
0x10	OPT	Soft Reset	4B	RW	Optional feature to allow for a firmware reset of IP block which just soft resets the state machine for all initiator state machines in the IP block. [0] – Global Soft Reset
0x14	REQ	Discovery Payload: Page Select	1B		128B windows, 16 windows max. The rest is reserved.
0x15-0x1F		Reserved			

12.8.6.2.2. M-PESTI IP Block Global Required Registers

M-PESTI IP Block Register Offset 0x00: Unique IP Block ID

Bit	Name	Description	DIR	Size

15:0	IP_BLOCK_ID	A unique ID for this IP block which is used to map IP blocks within the FGPA JSON file.	RO	2B

M-PESTI IP Block Register Offset 0x02: IP Block Version

Bit	Name	Description	Direction	Size
15:0	IP Version	Major.Minor BCD encoded version.	RO	2B

M-PESTI IP Block Register Offset 0x04: Global Status Register

See section 12.1.1.4 for details.

M-PESTI IP Block Register Offset 0x05: Global Control Register

See section 12.1.1.5 for details.

M-PESTI IP Block Register Offset 0x06: Global IP Block Interrupt Status Register

See section 12.1.1.6 for details.

M-PESTI IP Block Register Offset 0x07: Global IP Block Interrupt Control Register

See section 12.1.1.7 for details.

M-PESTI IP Block Register Offset 0x10: Soft Reset Register

Bits	Name	Description	Direction	Size
31:1	RESERVED	Reserved	RO	30b

0	SOFT_RESET	Optional feature to allow for firmware reset of IP block which just soft resets the state machine for all initiator state machines in the IP block. [0] – Global Soft Reset	RW	1b

M-PESTI IP Block Register Offset 0x14: Discovery Page Select Register

Bit(s)	Name	Description	ATTR	Size
7:4	RESERVED	Reserved	RO	4b
3:0	DISCOVERY_PAGE_SELECT	[3:0] – Page Select (1 Page = 128 bytes). This is used when Discovery payload is greater than 128 bytes.	RW	4b

12.8.7. M-PESTI Per Target Required Registers

This section contains the M-PESTI Per Target required registers. The CSR space is organized as shown in the diagram below.

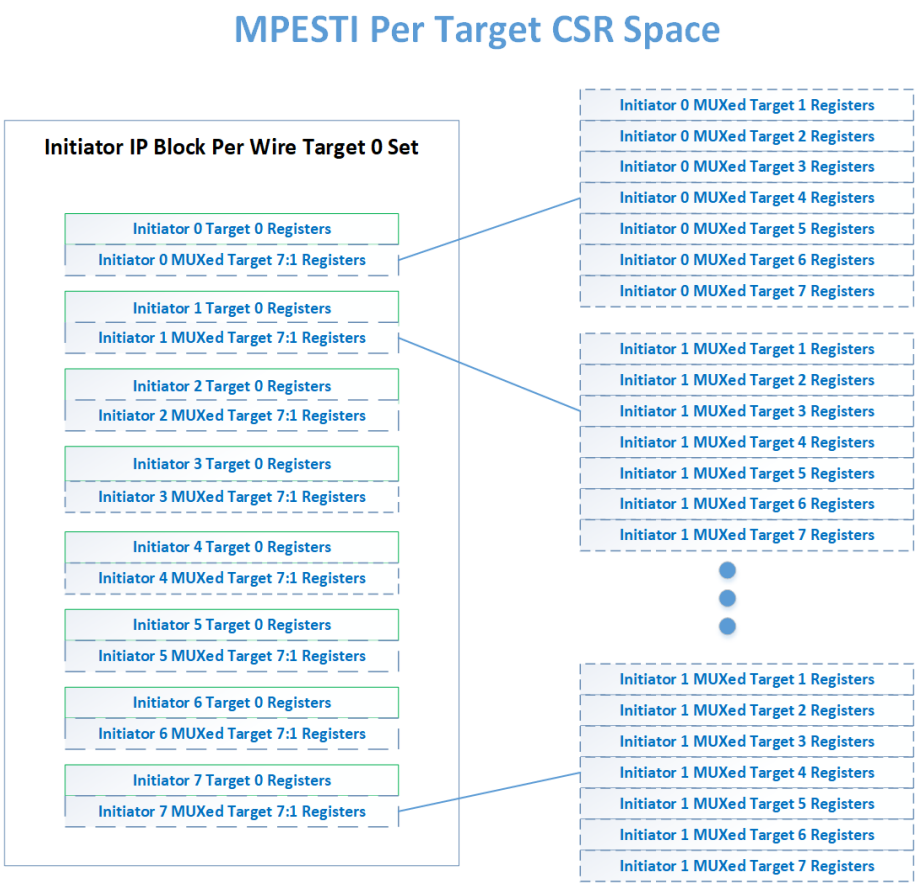


FIGURE 44: M-PESTI INITIATOR IP BLOCK PER TARGET REGISTER SET CONCEPTUAL DIAGRAM

Initiator IP block has a set of required 'Target 0' registers per PESTI wire. Each initiator is mapped 1:1 with a single PESTI wire. An Initiator IP Block can support up to 8 PESTI wires. It is NOT REQUIRED for an initiator to support targets behind a MUX. If it does, then it will indicate that a given register is implemented in support of targets behind a PESTI wire MUX. As per the MPESTI specification, the maximum standard fanout for a given MUX will be limited to 8. However, it is possible to scale beyond the fanout limit by indicating within the JSON file how many targets the CSR space supports. This is left to the vendor's discretion.

The M-PESTI per target registers map starts at Offset 0 from the base address for the per target CSR space. The base address is 0x10000 and proceeds in multiples of 4B. Thus, addresses increment starting at 'h1XXX0 where X is based on M-PESTI Target's index number. The intention is to reduce the required register footprint.

Example:
Base is at 0x10000 for target 0.
Base is at 0x10004 for target 1.
Base is at 0x10008 for target 2.

Etc.

12.8.7.1. M-PESTI Per Target Required Register Set Overview

The following table contains the required registers to manage each M-PESTI target.

TABLE 89: M-PESTI IP BLOCK CSR REGION 1 SUBREGION 1 REGISTER MAP DEFINITION

ADDR	Req / Opt.?	Name	W	ATTR	Description
0x00	REQ	MPESTI Target Error and Status 0	2B	RW	This is required per the M-PESTI specification. Defined in CSR details section
0x02	REQ	MPESTI Target Control 0	1B	RW	Defined in CSR details section Includes interrupt signal indication control.
0x03	REQ	MPESTI Target Config 0	1B	RW	Defined in CSR details section

12.8.7.2. M-PESTI Per Target Required Register Set Description

This section describes the per M-PESTI target required register set.

12.8.7.2.1. M-PESTI Per Target Register 0x00: Target Error and Status Register

The per target MPESTI target error and status register is defined in the table below.

TABLE 90: M-PESTI IP BLOCK CSR REGION 1 SUBREGION 1 REGISTER 0X00 DEFINITION

Bit	Name	Description	DIR	Size
15:7	Reserved	Reserved		

6	CHECKSUM_ERROR	Indicates that payload had a checksum error. This is an optional error indication bit.	RW1C	1’b0
5	VWIRE_UPDATED	Indicates that one or more virtual wires have been changed since last received. [OPTIONAL]	RW1C	1’b0
4	DPERR	DISCOVERY_PHASE_ERROR (R/W1C) [AKA: DPERR] [OPTIONAL] 0b: Response RX error (i.e., parity, framing) or timeout has NOT occurred. 1b: Response RX timeout or byte receive error has occurred since last cleared. Sticky bit. System management FW can write a ‘1’ to this bit to clear the error once it has been acknowledged. Latched: Write 1 to clear This will result in the Global Status register indicating an IP Block Error Code of 0x03 Communications Error. The implementor may use the extended status register to provide more specific error code indication.	RW1C	1’b0
3	APERR	ACTIVE_PHASE_ERROR (R/W1C) [AKA: APERR] 0b: Response RX error (i.e., parity, framing) or timeout has NOT occurred. 1b: Response RX timeout or byte receive error has occurred since last cleared. Sticky bit. System management FW should write a ‘1’ to this bit to clear the error once it has been acknowledged. Latched: Write 1 to clear This shall result in the Global Status register indicating an IP Block Error Code of 0x03 Communications Error. The implementor may use the extended status register to provide more specific error code indication.	RW1C	1'b0

2	ASTAT	ACTIVE_PHASE_STAT (R/W) [AKA: ASTAT] 0b: (Default) Target device is in the discovery phase. 1b: The Initiator and Target communication is in the active phase	RW	1’b0
1:0	DSTAT_00	DISCOVERY_STATUS [1:0] [AKA: DSTAT] 00b: No BREAK detected, and M-PESTI wire is static HIGH=1. 01b: M-PESTI wire is static LOW=0 (simple presence). 10b: Discovery payload has been received with a good checksum. 11b: BREAK release detected, but payload has not been successfully received.	RO	2'b00

12.8.7.2.2. M-PESTI Per Target Register 0x02: Target Control Register

The per target MPESTI target control register is defined in the table below. The target control register is applicable for peripherals that also support simple presence via cable detect signaling and is not limited to PESTI targets

TABLE 91: M-PESTI IP BLOCK CSR REGION 1 SUBREGION 1 REGISTER 0X02 DEFINITION

Bit	Name	Description	DIR	Default	Size
7	RESERVED	Reserved	R	0	1'b0
6:5	TARGET_PWR_POL	Target power policy for PWR_EN control 00 = Default Off 01 = S0 Only 10 = Always On (S5 and S0) 11 = Reserved This is combined with PWRON_ALLOW which acts as an override. The target power policy allows for autonomous control of the peripheral PWR_EN based on the system power state.	R/W	0	2’b0

4	PWRON_ALLOW	Controls power on for a given peripheral's main power. FRU, PLDs/MCUs, are powered, but all downstream slots etc. are disabled. This controls PWR_EN sideband to peripheral. This will automatically set DPEN after PWR_EN is asserted.[17] If the target does not support M-PESTI, thus only supporting cable presence, then the discovery payload will be ignored by the downstream peripheral. See M-PESTI specification for further details.	R/W	0	1'b0
3	APPECEN	Active Phase PEC Enable (APPECEN)	R/W	0	1'b0
2	DBYP	Discovery Bypass (DBYP)	R/W	0	1'b0
1	APEN	ACTIVE_PHASE_ENABLE: [AKA: APEN] 0b: Initiator shall not enter command/response phase for that M-PESTI instance. 1b: Initiator will enter command/response phase if DISCOVERY_STATUS [1:0] = 10b.	R/W	0	1'b0
0	DPEN	DISCOVERY_PAYLOAD_ENABLE [AKA: DPEN] 0b: Initiator shall not send payload request command. 1b: Initiator will send payload request command, with retries, until DISCOVERY_STATUS [1:0] = 10b.	R/W	0	1'b0

12.8.7.2.3. M-PESTI Per Target Register 0x03: Target Configuration Register

The per target MPESTI target configuration register is defined in the table below.

[17] Note: PWRON_ALLOW is a control signal which ties the PWR_EN sideband to the MPESTI target. This is only valid if the target implements PWR_EN. If sideband alternative function is not set to PWR_EN for this PICPWR then it will do nothing.

TABLE 92: M-PESTI IP BLOCK CSR REGION 1 SUBREGION 1 REGISTER 0X03 DEFINITION

Bit	Name	Description	Direction	Size
7	TINT_UPD_INT_EN	Enable Target VWIRE Input Update interrupt status generation. IP block shall generate an interrupt whenever virtual wires are updated. [OPTIONAL]	RW	1'b0
6	TPLDGD_INT_EN	Enable Target Payload Good interrupt status generation. Status indicated by DSTAT register. [OPTIONAL]	RW	1'b0
5	Reserved	Reserved	R	1'b0
4	TPRS_INT_EN	Enable Target Presence interrupt status generation. Target presence shall be indicated in the DSTAT register. [OPTIONAL]	RW	1'b0
3	ACT_ERR_INT_EN	Enable Active Phase Error interrupt status generation. Status indicated by APERR. [OPTIONAL]	RW	1'b0
2	DPHS_ERR_INT_EN	Enable Discovery Phase Error interrupt status generation. Status indicated by DPERR. [OPTIONAL]	RW	1'b0
1	DBRK_INT_EN	Enable Discovery Break Assertion Timeout interrupt status generation. Status indicated in DSTAT register. [OPTIONAL]	RW	1'b0
0	BRCST_EN	Broadcast subscription: 1'b1: Allows target to receive broadcast messages 1'b0: Disables sending broadcast messages	RW	1'b0

12.8.8. M-PESTI Per Target Optional Registers

This section contains the M-PESTI Per Target optional registers. The M-PESTI per target registers map starts at Offset 0 from the base address for the per target CSR space. The base address is 0x40000 and proceeds in multiples of 256B or 0x100. Thus, addresses increment starting at 'h4XX00 where X is based on M-PESTI Target's index number.

Example:
Base is at 0x40000 for target 0.
Base is at 0x40100 for target 1.
Etc.

Optional, per-target, registers are not defined in this proposal and can be anything that the implementor requires.

12.8.9. M-PESTI Discovery Payload Register Region

This section contains the description of the M-PESTI discovery payload registers.

There are two mechanisms that the PD may implement to store the target discovery payload.

- Non-destructive per target discovery payload mailbox window
- Destructive flat 2048-byte buffer to store the most recently discovered target's discovery payload.

12.8.9.1. Non-Destructive Per Target Discovery Mailbox Window

The per target mailbox window provides a mechanism to allow for payload windows larger than 128 bytes. However, by default we expect the maximum size of the payload from the target to be no greater than 128 bytes. This does not include any additional memory space needed for target payload overhead. Thus, the window could be larger than 128B. The page select register is used to move the window.

The payload window can be within a destructive buffer. The PD JSON file will contain an attribute that indicates whether the payload window is destructive or if it maintains discovery payload data for each target. JSON file as an attribute that indicates if the PESTI payload buffer is destructive or not.

M-PESTI Discovery 'Mailbox' Window Register Definition

Describe the flat space…

Target 0 = first 128B

Target 1 = 2nd 128B

…

Target N

The diagram below depicts the flat discovery payload space.

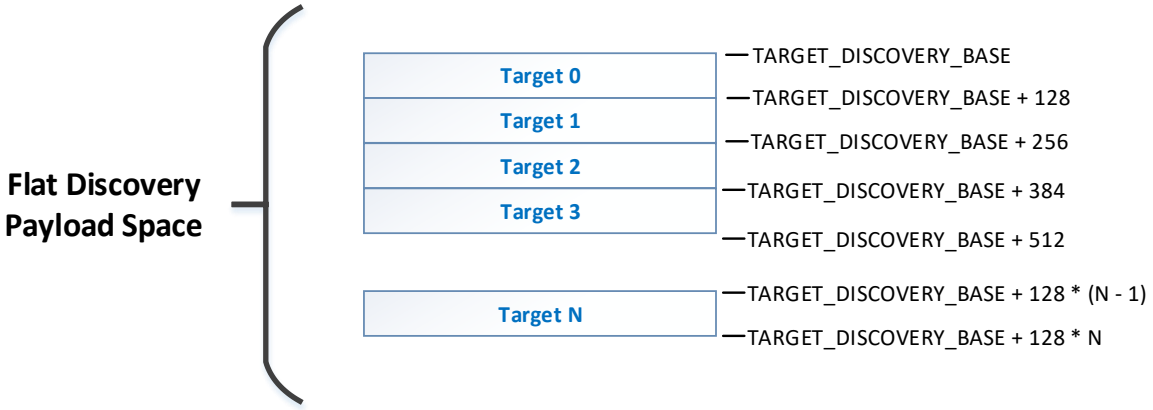


FIGURE 45: M-PESTI FLAT DISCOVERY PAYLOAD SPACE

12.8.9.1.1. Destructive Per Target Discovery Mailbox Window

This section describes an alternative discovery mailbox window implementation that the PD vendor can utilize. Rather than a windowed per target discovery mailbox, a single flat 2048B buffer will be provided to hold the discovery payload.

The diagram below contains the depiction of the flat 2048B destructive discovery payload space.

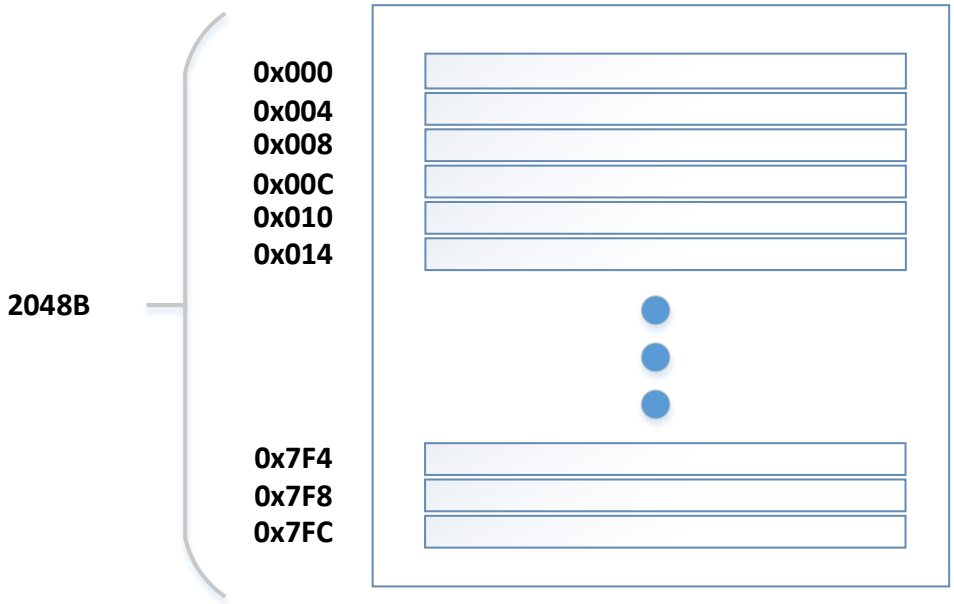


FIGURE 46: DESTRUCTIVE FIXED-SIZED M-PESTI PAYLOAD SPACE

There is no unique addressing scheme required to provide access into this buffer. Essentially, this is simply a location in the PD JSON file that indicates where the payload buffer is located. Each Discovery action overwrites it the payload buffer. The BMC is required to read it before asserting PWRON_ALLOW and/ or DPEN for the next target or that data will be lost.

12.8.10. M-PESTI Per Target Virtual Wires Register Region

This region is defined as a flat space for 15B IN / 15 OUT on a per target basis. This document caps the number of virtual wires supported to 15 in / out per target. An additional two bytes is reserved per each individual set of virtual wires on a per target basis.

Target 0 first 15B in 15B out plus two bytes reserved for a total of 32 B.

Target 1 second set of 15B in 15B out

…

Target N

The following diagram depicts the conceptual architecture of the flat per target virtual wires space.

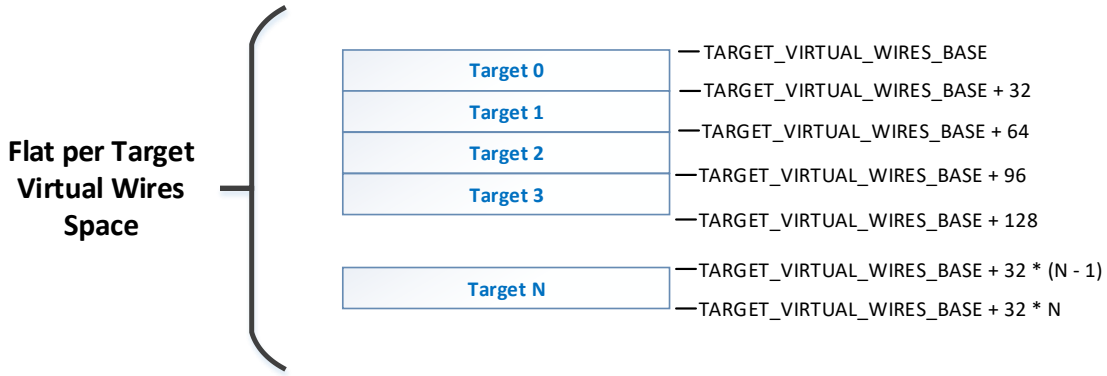


FIGURE 47: FLAT PER TARGET VIRTUAL WIRES SPACE

Per Target Virtual Wire Registers Map

TABLE 93: M-PESTI IP BLOCK CSR PER TARGET VIRTUAL WIRE REGISTER DEFINITION

Offset Address	Name	Data Width	ATTR	Description
0x00	VWIRE_IN [15:0]	16B*	R	Current state of virtual wire in. The upper Byte is reserved.
0x10	VWIRE_OUT [15:0]	16B*	RW	Can read or write the virtual wire outs. The upper byte is reserved.

Note: The register space includes the reserved byte per the virtual wire set even if the upper limit on virtual wire bytes that may be supported is 15.

The JSON self-descriptive file for the PD will provide the mapping from M-PESTI interface to the specific M-PESTI endpoint on the HPM (M-XIO connector, M-PIC connector, etc.).

12.8.11. PD M-PESTI JSON File Considerations For CSR Description

Per target register organizational scheme below.

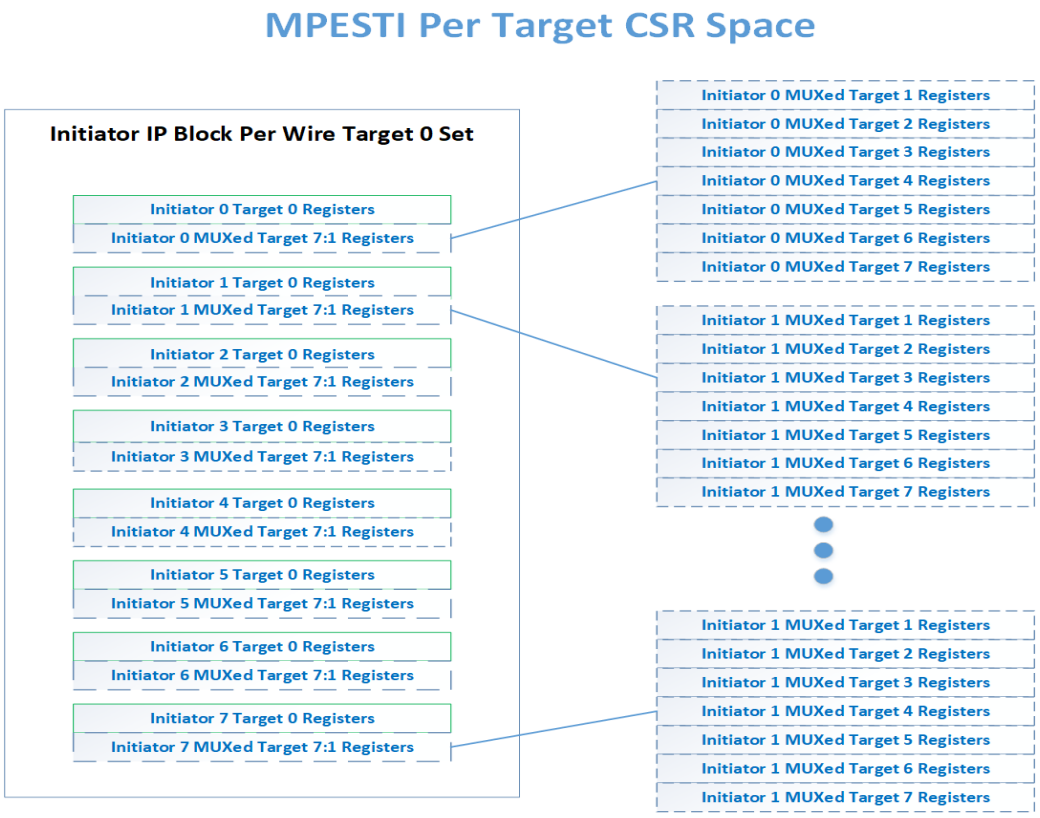


FIGURE 48: MPESTI PER TARGET CSR SPACE CONCEPTUAL DIAGRAM

The diagram above outlines the CSR space definition requirements for the set of per target registers for an initiator IP block. The PD JSON file shall describe how many (up to 8) initiators in the Initiator IP block are implemented per block. Additionally, the JSON file shall describe how many (up to 8) initiators are present in the Initiator IP block in logic such that the mapping of registers to targets is clearly defined.

Each IP block has 8 initiators and initiators are 1:1 with each physical PESTI wire. It is not required for a given PESTI wire to support a MUX. If a given PESTI wire does have a MUX attached, then it has to include the additional MUXed target registers contiguously in the CSR space after the required ‘target 0’ register. The JSON file will describe the set of optional per target registers along with target 0 register.

12.8.12. M-PESTI Interface Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.8.13. M-PESTI Interface Example Implementation

Additional materials may contain example implementations of an M-PESTI IP Block, and example JSON descriptive files. The reference document that has links to such materials is located in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

12.9. LED Control Logical IP Block

This section contains the logical IP block description for the LED requirements and descriptive abstraction for this IP block.

The intention of the LED control IP block is to provide a common set of control and status interfaces for controlling one or more LEDs on the HPM by the BMC. This may include interfaces that tunnel GPIO information to external HPM peripherals such as 7-segment indication LEDs, front panel control boards, or other vendor defined LED indicators.

This section defines the methods by which LEDs can be controlled. How the controlling entity knows when the LED should be asserted and whether or not it should be blinked at a certain rate or turned on solid (for example) is out of the scope of this document. Any platform level files or other materials that describe LED policy information are outside the scope of this document.

Support for more complex LEDs that require a bus interface are outside the scope of this specification.

It is required that the implementor add a description of this IP block in the FGPA JSON file for M-PNP interoperability.

12.9.1. LED Control Conceptual Overview

This section contains an overview of the LED Control functionality for a typical set of use cases.

There are two common use cases for LED control that are supported by this IP block.

- Basic LED Control
- Seven-segment LED Display Control

This is not an all-inclusive list, and it does not include vendor defined LED control support which can represent a myriad of additional use cases. For instance, a peripheral programmable device may use this IP block as well as an HPM primary programmable device.

12.9.2. LED Control IP Block Architecture

This section contains the general abstract architecture for the LED Control interface IP block.

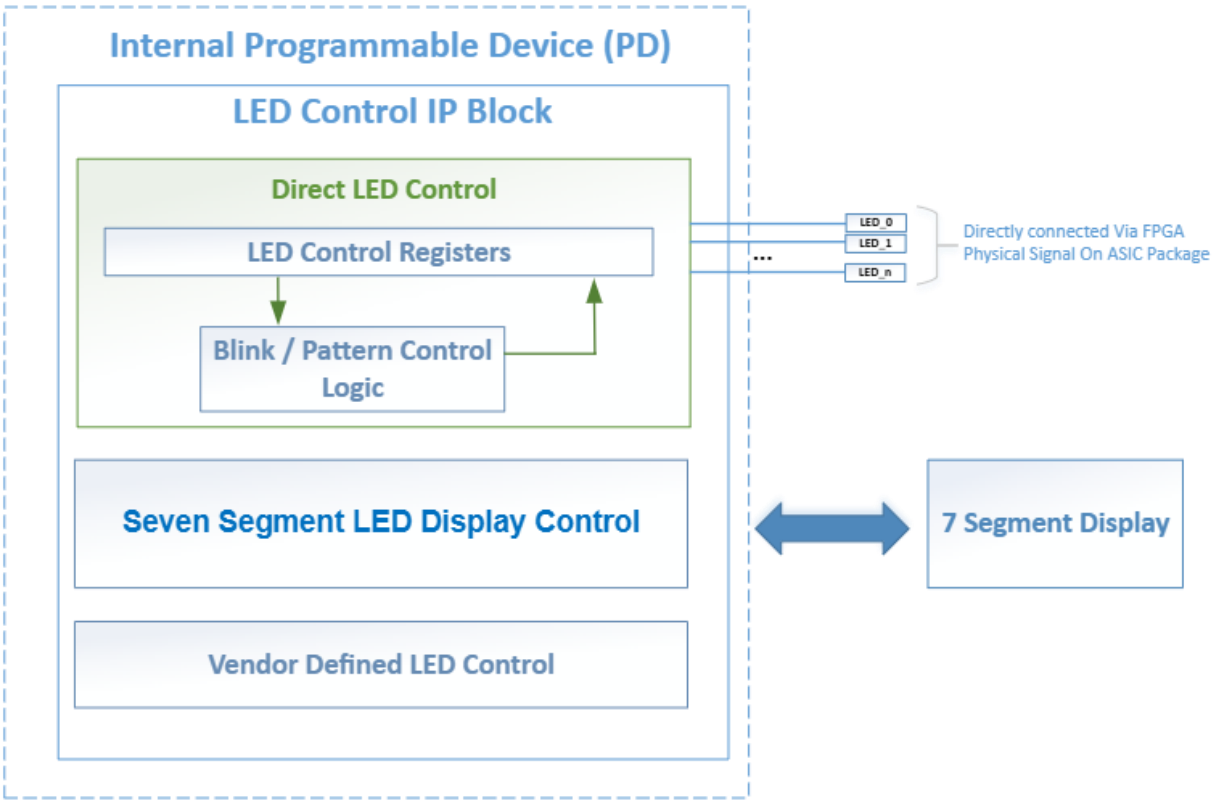


FIGURE 49: LED CONTROL ARCHITECTURAL BLOCK DIAGRAM

12.9.3. LED Control IP Block Requirements

This section contains the general abstract architecture for the LED Control interface IP block.

12.9.3.1. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 94: LED CONTROL IP BLOCK REQUIRED AND OPTIONAL FEATURE SET

Name	REQ	OPT	Description
On-board LED Control		Y	Direct control of on-board LEDs on same physical planar with PD (e.g. HPM debug LEDs)
Seven Segment Display Control		Y	Control of one or more seven segment LED displays.

12.9.4. LED Control IP Block CSR Requirements

This section contains the Control and Status Registers (CSR) requirements for standardized Light Emitting Diode (LED) Control functionality.

12.9.4.1. LED Control CSR Region Overview

This IP Block specifies a streamlined set of PD Control and Status Registers (CSRs) for controlling system LEDs and a 7-segment display on a DC-MHS HPM (Hardware Platform Module) PD. The design supports a flexible number of LED outputs (denoted LED_N for the total count of LED channels) and a four-digit 7-segment display. It provides only the essential LED features – ON, OFF, and BLINK (at 0.5 Hz, 1 Hz, 2 Hz, or 4 Hz blink rates). No additional LED control modes or features beyond these core functions are included, keeping the implementation simple. Other functions considered, but not included, are other blink rates, custom blink patterns, brightness, “Breath” function, color, fault status, lockout control, and external LED control. LED color information may be included as part of the JSON file description of the platform component hardware.

The 7-segment interface allows the Baseboard Management Controller (BMC) to display 4-character hexadecimal codes (e.g. “0000” through “FFFF”) by writing individual digit values via dedicated CSRs. Each of the four display digits is controlled independently. This simplified design attempts to add clarity and ease of integration with BMC firmware.

The LED Control CSR regions are divided into a set of ranges. These ranges are all offsets from the LED_IP_BLOCK base address. This base address is defined by the CPLD/PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 95: LED CONTROL CSR MEMORY MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x00FF	Global Registers	General IP block level registers
1	0x0100	0x3FF	LED IP Block Interface Control and Status Registers	LED IP Block Control and Status Registers.
2	0x0400	0x0FFF	Reserved	Reserved for future use
3[18]	0x1000	0xFFFF	OEM SECTION	Vendor defined

The register range ordering shall be preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown above are not explicitly required register offset locations and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.9.4.2. LED Control IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

[18] Note: Implementors can use this for additional regions as needed.

TABLE 96: LED CONTROL REGION 0 CSR MEMORY MAP

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

The register range ordering shall be preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown above are not explicitly required register location offset locations and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.9.4.2.1. LED Control IP Block Common Registers Sub-Region 0 Description

The table below contains the definition for the LED IP block common registers sub-region 0.

TABLE 97: LED CONTROL REGION 0 SUB-REGION 0 CSR DESCRIPTION

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each).
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.

0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state indication that requires a write to clear. Bit 0: Interrupt Status. See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable. See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	R/W	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x08 – 0x0F	Reserved			

12.9.4.2.2. LED Control IP Block Common Registers Sub-Region 1 Description

Reserved

12.9.4.3. LED Control IP Block Region 1 CSR Memory Map

The LED region 1 register set starts at offset 0x100 from the base address for the I2C / I3C Basic CSR space.

This region is divided into two subregions.

TABLE 98: LED CONTROL REGION 1 CSR MEMORY MAP

Sub-Region	Range	Name	Description
0	0x100 – 0x1FF	Basic LED Control CSRs	Set of required registers that handle Basic LED Control status and control functions.
1	0x200 – 0x2FF	Seven-Segment Display Control	Set of required registers that control one or more seven segment LEDs.

12.9.4.3.1. LED Control CSRs Region 1 Sub-Region 0 Description

This section contains the definition for the LED Control CSRs in Region 1 Sub-region 0.

Supported LED Functionality

- Each of the LED_N outputs can be turned OFF or ON or set to BLINK at one of four fixed rates.
- Blink rates: 0.5 Hz, 1 Hz, 2 Hz, 4 Hz
- Blink timing is handled by PD.
- The BMC writes a 3-bit state code to each LED's control register.

Note: A system implementation may choose to combine multiple LED signals to control one or more multi-colored GPIO controlled LEDs if so desired. This use case is vendor defined and outside the scope of this specification. Additionally, platform level LED control policy for this use case is outside the scope of this specification.

The table below contains the definition for the LED IP block CSRs region 1 sub-region 0.

TABLE 99: LED IP BLOCK CSRS REGION 1 SUB-REGION 0

Register Name	Bits	Access	Description
LED_n_CTRL (n = 0…N-1)	3:0 – State 7:4 – Reserved	R/W	Controls LED_n. State codes: 0000 = Off 0001 = On 0010 = Blink 0.5 Hz 0011 = Blink 1 Hz 0100 = Blink 2 Hz

			0101 = Blink 4 Hz 0110–1111 = Reserved

12.9.4.3.2. LED Control IP Block CSRs Region 1 Sub-Region 1 Description

This section contains the definition for the LED Control CSRs in Region 1 Sub-region 1. The table below contains the definition for the LED IP block CSRs region 1 sub-region 1.

TABLE 100: LED IP BLOCK CSRS REGION 1 SUB-REGION 1

Register Name	Bits	Access	Description
DISPLAY_DIGIT_m (m = 0…3)	3:0 – Value 7:4 – Reserved	R/W	Controls 7-segment digit m (0 = rightmost, 3 = leftmost). Value is 0x0–0xF for hex digits 0–F. Other values result in a blank display.

12.9.4.4. LED Control IP Block Region 2 CSR Memory Map

Reserved

12.9.4.5. LED Control IP Block Region 3 CSR Memory Map

Reserved for vendor defined usage.

12.9.5. LED Control Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.9.6. LED Control Example Implementation

Example implementations for the LED Control IP block may be found within the collateral document, “IP Block Source Code Examples List" for specific examples. See section 7.4, “MHS HPM PD Specification Supplementary Materials”, for further details.

12.10. M-XIO FLEX IO and M-PIC Sideband Control Logical IP Block

This section contains the logical IP block description for the M-XIO FLEX IO and M-PIC PICPWR Sideband Control requirements and descriptive abstraction for this IP block.

12.10.1. M-XIO FLEX IO and M-PIC PICPWR Sideband Control Conceptual Overview

This section contains an overview of the M-XIO FLEX IO and M-PIC PICPWR Sideband Control functionality for a typical set of use cases.

The intention of the M-XIO Flex-IO and M-PIC PICPWR Sideband IP block is to provide a common set of control and status interfaces for controlling the Flex-IO and sideband signaling allocation for one or more M-XIO and M-PIC PICPWR connectors on the HPM connected to the PD.

The expectation is that discovery of peripherals and options requirements for these signals will occur during the discovery process. Details for this process can be found in the "M-PNP Discovery, Boot, and FRU Specification".

12.10.1.1. M-XIO FLEX IO and M-PIC PICPWR Sideband Control Scope

This section contains the definition for a common set of CSRs for FlexIO and Sideband signal configurations that system firmware can interact with. The intention is to allow for flexible and simple extensibility for configuration of specific signals on the M-XIO and M-PIC PICPWR connectors. The end result of this proposal is to be used as part of a larger industry specification.

12.10.2. M-XIO FLEX IO and M-PIC PICPWR Sideband Control IP Block Architecture

This section contains the common architectural description for the M-XIO FLEX IO and M-PIC PICPWR Sideband Control IP blocks.

12.10.2.1. M-XIO Flex IO Architectural Block Diagram

This section contains an example block diagram for implementation of the M-XIO Flex-IO for a given FLEX-IO signal.

The following diagram shows possible alternate configuration support for alternate functions for the XIO FLEX-IO-n signals.

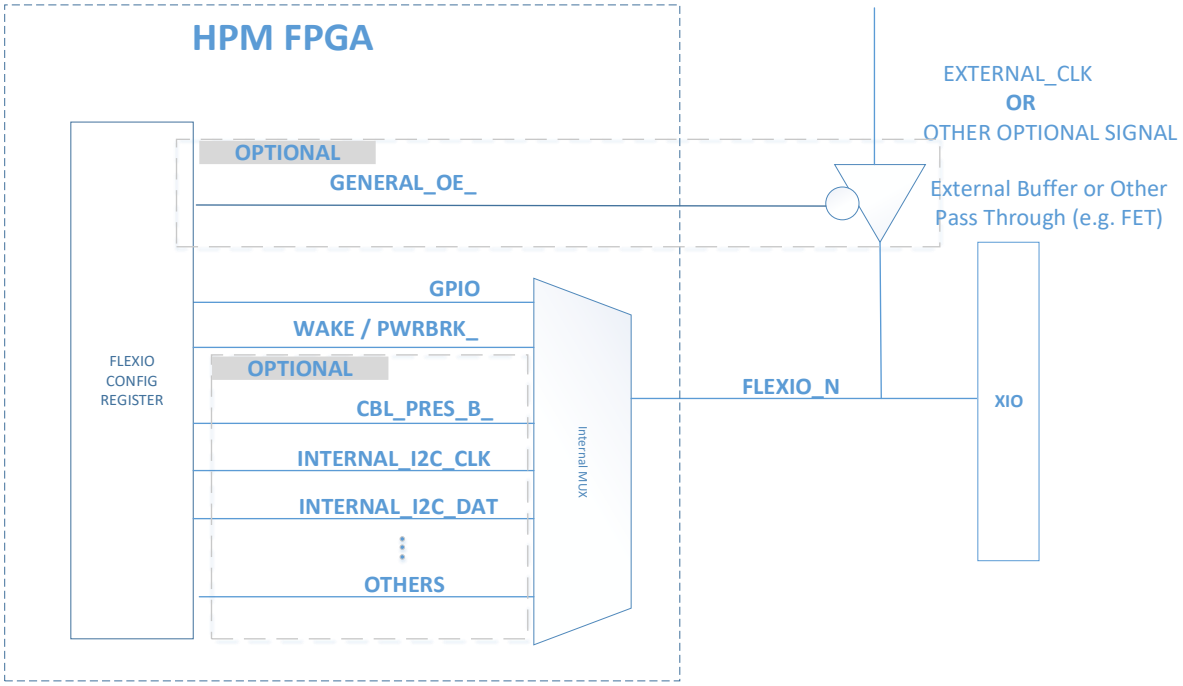


FIGURE 50: M-XIO FLEXIO ARCHITECTURAL BLOCK DIAGRAM

12.10.2.2. M-XIO Flex IO Alternate Functions List

This section contains the list of M-XIO Flex IO sideband alternate functions.

The following table contains the non-exclusive list of FlexIO signal alternate functions. Be default, all FlexIO signals shall support the basic GPIO functionality.

TABLE 101: M-XIO FLEXIO ALTERNATE FUNCTIONS LIST

Connector	Alternate Function	Default Assignment	Preferred Assignment	HPM SHALL Support	IF Supported SHALL be on	Power State Valid	Peripheral Used	Description
MXIO	WAKE_N	FLEXIO_2A	FLEXIO_2A	Y	FLEXIO_2A	S5	Riser	Shared with power brake for risers.
	PWR_BRAKE_N	FLEXIO_2A	FLEXIO_2A	Y	FLEXIO_2A	S0	Riser	Shared with wake for risers.
	CPRSNT_B_N	NC	FLEXIO_1A	N	FLEXIO_1A	S5 / S0	Backplane	Used for UBM backplanes. Can be handled by GPIO and FW. Dedicated ALT function logic is optional.
	I2C_RESETA_N	NC	FLEXIO_2A	N	FLEXIO_2A	S5 / S0	Backplane	Used for UBM backplanes. Can be handled by GPIO and FW.

								Dedicated ALT function logic is optional.
	SMSCL_B	NC	FLEXIO_5A	N	FLEXIO_5A	S5 / S0	Backplane	Used for UBM backplanes. Handled by OE_ logic to high Z the FlexIO GPIO output pin and control separate signal / pin on PD. PD does not have to support this alternative function as logic is optional.
	SMSDA_B	NC	FLEXIO_6A	N	FLEXIO_6A	S5 / S0	Backplane	Used for UBM backplanes. Handled by OE_ logic to high Z the FlexIO GPIO output pin and control separate signal / pin on PD. PD does not have to support this alternative function as logic is optional.

12.10.2.3. M-PIC PICPWR Sideband Architectural Block Diagram

This section contains an example block diagram for implementation of the M-PIC PICPWR Sideband for a given sideband signal.

The following diagram shows possible alternate configuration support for alternate functions for alternate functions on the PICPWR SBx signals.

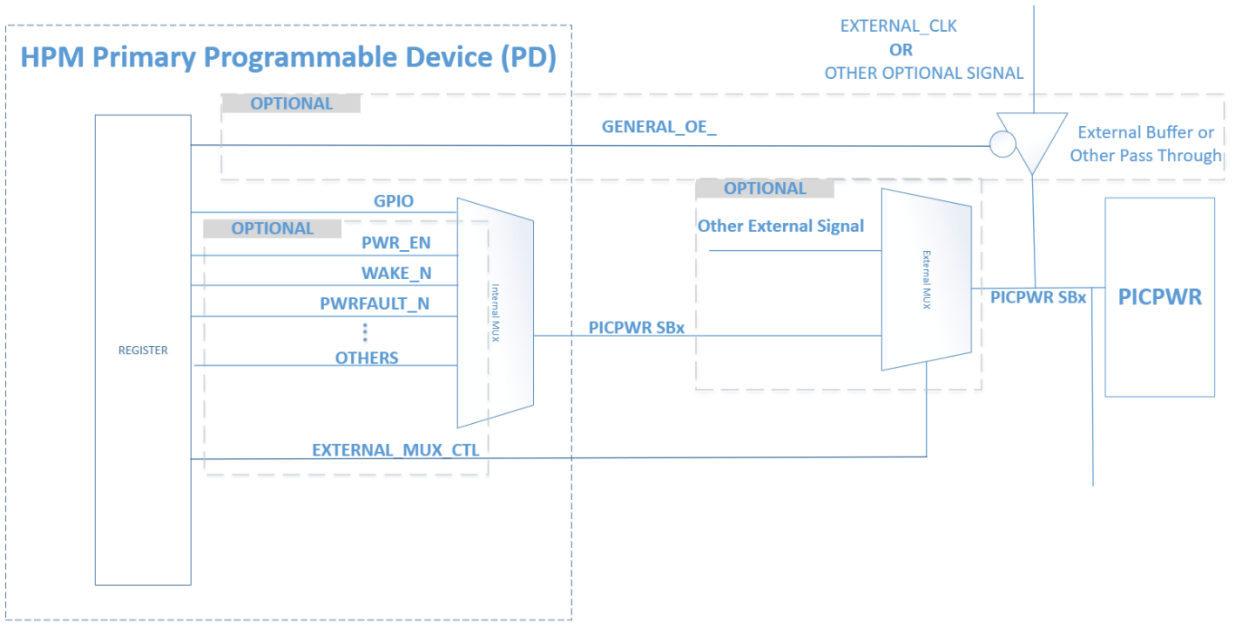


FIGURE 51: PICPWR SIDEBAND ARCHITECTURAL BLOCK DIAGRAM

12.10.2.4. M-PIC PICPWR Sideband Alternate Functions List

This section contains the list of M-PIC PICPWR sideband alternate functions.

The following table contains the non-exclusive list of PICPWR sideband signal alternate functions. Be default, all flexible PICPWR sideband signals shall support the basic GPIO functionality.

TABLE 102: PICPWR SIDEBAND ALTERNATE FUNCTIONS LIST

Connector	Alternate Function	Default Assignment	Preferred Assignment	HPM SHALL Support	IF Supported SHALL be on	Power State Valid	Peripheral Used	Description
PICPWR	PRES_A_N / PESTI_A	SB1_A	-	Y	SB1_A		Backplane / Riser	Needed by both backplanes and risers. Required.
	PWR_EN_A	SB2_A	-	N	SB2_A		Backplane / Riser	HPM does not have to have dedicated alternate function logic. Can be

								handled by FW through standard GPIO.
	PWRFAULT_N_A	SB3_A	-	N	SB3_A		Backplane / Riser	HPM does not have to have dedicated alternate function logic. Can be handled by FW through standard GPIO.
	WAKE_N_A	SB4_A	-	N	SB4_A		Riser	HPM does not have to have dedicated alternate function logic. Can be handled by FW through standard GPIO.
	PRES_B_N / PESTI_B	SB1_B	-	Y	SB1_B		Backplane / Riser	Needed by both backplanes and risers. Required.
	PWR_EN_B	SB2_B	-	N	SB2_B		Backplane / Riser	HPM does not have to have dedicated alternate function logic. Can be handled by FW through standard GPIO.
	PWRFAULT_N_B	SB3_B	-	N	SB3_B		Backplane / Riser	HPM does not have to have dedicated alternate function logic. Can be handled by FW through standard GPIO.
	WAKE_N_B	SB4_B	-	N	SB4_B		Riser	HPM does not have to have dedicated alternate function logic. Can be handled by FW through standard GPIO.

12.10.3. M-XIO FLEX IO and M-PIC PICPWR Sideband Control IP Block Requirements

This section contains the requirements list for the M-XIO FLEX IO and M-PIC PICPWR Sideband Control IP block.

12.10.3.1. M-XIO FLEX IO and M-PIC PICPWR Sideband Control IP Block Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 103: FLEXIO AND PICPWR CONFIGURATION IP BLOCK REQUIRED AND OPTIONAL FEATURE SET

Name	REQ	OPT	Description
Generic GPIO Support	Y		Every FlexIO and PICPWR sideband signal shall support generic GPIO capabilities.
Required Default Alternate Function Configurations	Y		FlexIO and PICPWR sideband signals shall support the required default alternate function configuration settings.
Optional Default Alternate Function Configurations		Y	FlexIO and PICPWR sideband signals may support the optional alternate function configuration settings.

12.10.4. M-XIO FLEX IO and M-PIC PICPWR Sideband Control IP Block CSR Requirements

This section contains the Control and Status Registers (CSR) requirements for standardized M-XIO FLEX IO PIC PICPWR Sideband control functionality.

12.10.4.1. FlexIO and PICPWR IP BLOCK CSR Region Map Overview

This section contains an overview of the M-XIO Flex IO and PICPWR sideband CSR region map. This location of the base address for this IP block shall be provided by the PD JSON file.

The following table contains the set of CSR ranges for the Flex IO and PICPWR sideband control IP block.

TABLE 104: FLEXIO AND PICPWR IP BLOCK CSR MEMORY MAP

Region	Range Bottom	Range Top	Sections	Name	Description
0	0x00000	0x0FFFF		Global Registers	IP block level to cover all targets
1	0x10000	0x1FFFF	0x1XXX0	M-XIO Target CSR Set. 4 bytes per FLEXIO, 4*7 per XIO so 28B with 4B reserved space.	Contains per MXIO target CSRs. Example: 32 XIO then 1024B total for the space.
2	0x20000	0x2FFFF	0x2XXX0	M-PIC PICPWR target CSR set. 4 bytes per sideband, 4*4 per PICPWR so 16B	4 bytes per M-PIC. Example: 32 PICPWR = 16*32 = 512B
3	0x30000	0x3FFFF	0x3XXX0	M-PIC PICPWR Control Panel target CSR set CSRs for PCP_SB1 to PCP_SB4 or SCP_SB [4:1]	4 bytes for each control panel connector. Leaving in its own section due to uniqueness. Primary and Secondary only 4 * 2 so 8B total
4	0x40000	0x4FFFF		OEM SECTION	Undefined

These ranges are all offsets from the FLEXIO_IP_BLOCK base address.
This base address is defined by the CPLD/PD developer and is not prescribed here.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as

needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.10.4.2. Region 0: FlexIO / Sideband IP Block Common Registers

This section contains the contents for the region 0 Flex IO and PICPWR sideband common registers.

This region is divided into three subregions.

TABLE 105: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 0 REGISTER MAP DEFINITION

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFFF	GLOBAL RESERVED	Save enough space if more registers are needed
2	0x1000 – 0xFFFF	GLOBAL OEM	Available space for OEM usage. This is where any optional registers (dev, debug, etc.) would be location.

12.10.4.2.1. Region 0: FlexIO / Sideband IP Block Subregion 0 Memory Map

This section contains contents for subregion 0.

TABLE 106: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 0 SUBREGION 0 REGISTER MAP DEFINITION

Offset Address	Name	Data Width	ATTR	Description
0x00	IP Block ID	2B	R	Unique ID to identify the IP block.

0x02	IP Block Version	2B	R	Major.Minor (1 byte each)
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state indication utilizing a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	R/W	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x08 – 0x0F	Reserved			Reserved for future use.

12.10.4.2.2. Region 0: FlexIO / Sideband IP Block Subregion 1 Memory Map

This section contains contents for subregion 1.

Reserved

12.10.4.2.3. Region 0: FlexIO / Sideband IP Block Subregion 2 Memory Map

This section contains contents for subregion 2.

This sub-region is currently defined for OEM / vendor specific usage.

12.10.4.3. Region 1: FlexIO Per M-XIO Target Memory Map

This section contains details regarding the region 1 memory map.

The FlexIO per target registers map starts at Offset 0 from the base address for the per target CSR space. The base address is 0x10000 and proceeds in multiples of 32 bytes or 0x20. Thus, addresses increments shall be starting at ‘h1XXX0, where X is based on M-XIO target’s index number on a 32-byte boundary in 32 B chunks.

Example:

MXIO 0 Starts at 0.

Base is at 0x10000 for target MXIO 0 FLEXIO Signal 0

Base is at 0x10004 for target MXIO 0 FLEXIO Signal 1

Base is at 0x10008 for target MXIO 0 FLEXIO Signal 2

…

0x1C – 0x1F are reserved.

MXIO 1 Starts at 0x20.

Base is at 0x10020 for target MXIO 1 FLEXIO Signal

Base is at 0x10024 for target MXIO 1 FLEXIO Signal 1

Base is at 0x10028 for target MXIO 1 FLEXIO Signal 2

…

0x2C – 0x2F are reserved.

Etc.

12.10.4.3.1. FlexIO Per M-XIO Target CSRs

This section contains details regarding region 1 per M-XIO target CSRs.

The following table outlines the required register set for each XIO target to support FlexIO configuration. The required section allocates 4 bytes per FlexIO with 4B reserved for 32B per XIO.

The JSON file will indicate which physical XIO connector maps to the block of registers.

TABLE 107: FLEX-IO MXIO CSR MEMORY MAP

XIO_FLEXIO	Offset	Signal Name	W	ATTR	Description
FLEXIO_0	0x00	FLEX_IO0_GPIO_STATUS__0	1B	RW	FlexIO GPIO status
	0x01	FLEX_IO0_GPIO _CONTROL _0	1B	RW	FlexIO GPIO config control
	0x02	FLEX_IO0_ALT_FUNCT_CONFIG_0	1B	RW	Control and configuration of FlexIO alternative functions
	0x03	FLEX_IO0_RESERVED_0	1B	RW	Reserved
FLEXIO_1	0x04	FLEX_IO0_GPIO_STATUS__1	1B	RW	FlexIO GPIO status
	0x05	FLEX_IO0_GPIO _CONTROL _1	1B	RW	FlexIO GPIO config control
	0x06	FLEX_IO0_ALT_FUNCT_CONFIG_1	1B	RW	Control and configuration of FlexIO alternative functions
	0x07	FLEX_IO0_RESERVED_1	1B	RW	Reserved
FLEXIO_2	0x08	FLEX_IO0_GPIO_STATUS_2	1B	RW	FlexIO GPIO status
	0x09	FLEX_IO0_GPIO _CONTROL _2	1B	RW	FlexIO GPIO config control
	0x0A	FLEX_IO0_ALT_FUNCT_CONFIG_2	1B	RW	Control and configuration of FlexIO alternative functions
	0x0B	FLEX_IO0_RESERVED_2	1B	RW	Reserved
FLEXIO_3	0x0C	FLEX_IO0_GPIO_STATUS_3	1B	RW	FlexIO GPIO status
	0x0D	FLEX_IO0_GPIO _CONTROL _3	1B	RW	FlexIO GPIO config control
	0x0E	FLEX_IO0_ALT_FUNCT_CONFIG_3	1B	RW	Control and configuration of FlexIO alternative functions
	0x0F	FLEX_IO0_RESERVED_3	1B	RW	Reserved
FLEXIO_4	0x10	FLEX_IO0_GPIO_STATUS_4	1B	RW	FlexIO GPIO status
	0x11	FLEX_IO0_GPIO _CONTROL _4	1B	RW	FlexIO GPIO config control

	0x12	FLEX_IO0_ALT_FUNCT_CONFIG_4	1B	RW	Control and configuration of FlexIO alternative functions
	0x13	FLEX_IO0_RESERVED_4	1B	RW	Reserved
FLEXIO_5	0x14	FLEX_IO0_GPIO_STATUS_5	1B	RW	FlexIO GPIO status
	0x15	FLEX_IO0_GPIO _CONTROL _5	1B	RW	FlexIO GPIO config control
	0x16	FLEX_IO0_ALT_FUNCT_CONFIG_5	1B	RW	Control and configuration of FlexIO alternative functions
	0x17	FLEX_IO0_RESERVED_5	1B	RW	Reserved
FLEXIO_6	0x18	FLEX_IO0_GPIO_STATUS_6	1B	RW	FlexIO GPIO status
	0x19	FLEX_IO0_GPIO _CONTROL _6	1B	RW	FlexIO GPIO config control
	0x1A	FLEX_IO0_ALT_FUNCT_CONFIG_6	1B	RW	Control and configuration of FlexIO alternative functions
	0x1B	FLEX_IO0_RESERVED_6	1B	RW	Reserved
RESERVED	0x1C – 0x1F	RESERVED	4B		

12.10.4.3.2. FlexIO Register Offset 0x00: General GPIO Status Register

This section contains the FlexIO general GPIO status register contents.

TABLE 108: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 0 REGISTER 0X00 DEFINITION

Bit(s)	Name	Description	ATTR	W	Notes
7:3	RESERVED	Reserved	R	5'b0	
2	LOGICAL_STATE	1 = Signal is Logically High 0= Signal is Logically Low	R	1'b0	Used by FW
1	PHYSICAL_STATE	1 = Signal is Physically High	R	1'b0	Useful for debug

		0 = Signal is Physically Low			
0	CONNECTED	1= Connected 0 = Not connected	R	1'b0	Whether hardware signal is physically routed between PD and XIO connector Flex IO pin

12.10.4.3.3. FlexIO Register Offset 0x01: General GPIO Control

This section contains the FlexIO general GPIO control register contents.

TABLE 109: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 0 REGISTER 0X01 DEFINITION

Bit(s)	Name	Description	ATTR	Size	Notes
7:6	INT_TRIGGER	Interrupt Behavior 0 = Both rising and falling edge 1 = rising edge only 2= falling edge only 3 =reserved	RW	1'b0	Mapping to interrupt from this source shall be indicated using a the internal virtual wire mapping framework
5	INT_EN	1 = Enabled 0 = Disabled	RW	1'b0	
4	LOGICAL_POLARITY	GPIO Logical Polarity 1= Active High 0 = Active Low	RW	1'b0	Used by logic to trigger interrupt behavior.
3:2	PHYSICAL_POLARITY	GPIO Polarity 0 = IN, 1 = OUT, 2 = BIDIR, 3 = high-Z	RW	2'b0	
1	DRIVE	GPIO Drive 0 = Open Drain	RW	1'b0	Select drive type for GPIO

		1 = Push Pull			
0	GPIO State	1 = Asserted 0 = De-Asserted	RW	1'b0	The GPIO output signal state indication if it is in GPO mode, or configuration dependent if configured as output or bidirectional.

12.10.4.3.4. FlexIO Register Offset 0x02: Alternate Function Configuration

The register description below provides the BMC with the FlexIO alternate function configuration. It is optional for a given FlexIO to support alternative function capabilities and support is PD implementation specific. If there is a capability mismatch, it is required for system BMC firmware to detect and report configuration incompatibilities. For instance, if a given peripheral is requesting an alternative function support on a specific XIO FLEXIO signal, but the PD does not support it then BMC FW will indicate a mismatch and then perform whatever the vendor's policy is that handles configuration mismatches. It is recommended that the BMC read the registers to verify that the desired configuration was accepted and valid.

The JSON descriptive file for the PD shall contain the possible set of alternate functions supported by the PD for every FlexIO signal connected to HPM XIOs. It shall indicate whether a given FlexIO signal is connected to an HPM XIO connector. If it is not connected, then the status bit and function configuration will be set to 0s and will not be configurable.

TABLE 110: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 0 REGISTER 0X02 DEFINITION

Bit(s)	Name	Description	ATTR	Size	Notes
7:5	RESERVED	Reserved for future use.		3'b0	
4:0	FLEXIO_FUNC [4:0]	FLEXIO Function Configuration 00000b: NC if not connected as per JSON file. OR	RW	5'b00	GPIO basic function is required if connected. All alternative functions are optional. BUFFER_OE_ for signals that cannot

		00000b: GPIO (Default) if indicated in JSON file as connected. 00001b: CBL_PRES_B_ [OPTIONAL] 00010b: WAKE / PWRBRK_ [OPTIONAL FOR ALL BUT FLEXIO2] 00011b: BUFFER_OE_ [OPTIONAL] 00100b: SMSCL_MUX_EN [OPTIONAL] 00101b: SMSDC_MUX_EN [OPTIONAL] 00110b: Reserved. … 11111b: Reserved.			be driven into a MUX. For UBM backplanes, I2C_RESETA_N can be handled by GPIO functionality. Five bits gives us 32 possible alternative functions. It is required that the optional configurations are encoded into the JSON file.

12.10.4.3.5. FlexIO Register 0x03: Reserved

Reserved for future use.

12.10.4.4. Region 0: FlexIO / Sideband IP Block Region 2: M-PIC Per PICPWR Sideband CSR Memory Map

This section provides the per PICPWR connector sideband registers set definition. The MPIC PICPWR per target registers map starts at Offset 0 from the base address for the per target CSR space. The base address is 0x20000 and proceeds in multiples of 4 bytes or 0x04 per sideband or 16B per PICPWR. Thus, address increments shall start at h2XXX0, where X is based on PICPWR Target's index number on a 16B byte boundary.

Example:

Base is at 0x20000 for target MPIC PICPWR 0.

Base is at 0x20010 for target MPIC PICPWR 1.

Base is at 0x20020 for target MPIC PICPWR 2.

Etc.

MPIC PICPWR 0 Starts at 0

Base is at 0x20000 for target MPIC PICPWR 0 Sideband Signal

Base is at 0x20004 for target MPIC PICPWR 0 Sideband Signal 1

Base is at 0x20008 for target MPIC PICPWR 0 Sideband Signal 2

…

0x0C – 0x0F are reserved,

MPIC PICPWR 1 Starts at 0x20.

Base is at 0x20010 for target MPIC PICPWR 1 Sideband Signal 0

Base is at 0x20014 for target MPIC PICPWR 1 Sideband Signal 1

Base is at 0x20018 for target MPIC PICPWR 1 Sideband Signal 2

…

0x1C – 0x1F are reserved.

Etc.

12.10.4.4.1. PICPWR Sideband Per M-PIC Connector 2x3 Register Definition

This section provides the per PICPWR connector sideband registers set definition overview.

The table below contains the per PICPWR connector sideband registers.

TABLE 111: SIDEBAND PICPWR CSR MEMORY MAP

PICPWR Sideband	Offset Address	Name	W	ATTR	Description
SB_0	0x00	SB_GPIO_STATUS_0	1B	RW	Sideband GPIO status
	0x01	SB_GPIO_CONTROL_0	1B	RW	Sideband GPIO config / control
	0x02	SB_ALT_FUNCT_CONFIG_0	1B	RW	Configuration of sideband alternative functions
	0x03	SB_RESERVED_0	1B	RO	Reserved
SB_1	0x04	SB_GPIO_STATUS_1	1B	RW	Sideband GPIO status
	0x05	SB_GPIO_CONTROL_1	1B	RW	Sideband GPIO config / control
	0x06	SB_ALT_FUNCT_CONFIG_1	1B	RW	Configuration of sideband alternative functions
	0x07	SB_RESERVED_1	1B	RO	Reserved
SB_2	0x08	SB_GPIO_STATUS_2	1B	RW	Sideband GPIO status
	0x09	SB_GPIO_CONTROL_2	1B	RW	Sideband GPIO config / control
	0x0A	SB_ALT_FUNCT_CONFIG_2	1B	RW	Configuration of sideband alternative functions
	0x0B	SB_RESERVED_2	1B	RO	Reserved
SB_3	0x0C	SB_GPIO_STATUS_3	1B	RW	Sideband GPIO status
	0x0D	SB_GPIO_CONTROL_3	1B	RW	Sideband GPIO config / control
	0x0E	SB_ALT_FUNCT_CONFIG_3	1B	RW	Configuration of sideband alternative functions
	0x0F	SB_RESERVED_3	1B	RO	Reserved

12.10.4.4.2. PICPWR Sideband Register Offset 0x00: General GPIO Status

This section provides the per PICPWR connector sideband registers set definition for the general GPIO status register.

TABLE 112: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 1 REGISTER 0X00 DEFINITION

Bit(s)	Name	Description	ATTR	Size	Notes
7:3	RESERVED	Reserved	R	5'b0	

2	LOGICAL_STATE	1 = Signal is Logically High 0= Signal is Logically Low	R	1'b0	Used by FW
1	PHYSICAL_STATE	1 = Signal is Physically High 0 = Signal is Physically Low	R	1'b0	Useful for debug
0	CONNECTED	1= Connected 0 = Not connected	R	1'b0	Whether hardware signal is physically routed between PD and PICPWR connector SBn IO pin

12.10.4.4.3. PICPWR Sideband Register Offset 0x01: General GPIO Control

This section provides the per PICPWR connector sideband registers set definition for the general GPIO control register.

TABLE 113: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 1 REGISTER 0X01 DEFINITION

Bit(s)	Name	Description	ATTR	Size	Notes
7:6	INT_TRIGGER	Interrupt Behavior 0 = Both rising and falling edge 1 = rising edge only 2 = falling edge only 3 = reserved	RW	1'b0	Mapping to interrupt from this source shall be implemented using the internal virtual wire mapping framework.
5	INT_EN	1 = Enabled 0 = Disabled	RW	1'b0	

4	LOGICAL_POLARITY	1 = Active High 0 = Active Low	RW	1'b0	Used by logic to trigger interrupt behavior.
3:2	PHYSICAL_POLARITY	IN, OUT, BIDIR, high-Z	RW	2'b0	
1	DRIVE	0 = Open Drain 1 = Push Pull	RW	1'b0	Select drive type for GPIO
0	GPIO State	1 = Asserted 0 = De-Asserted	RW	1'b0	GPIO output states if in GPO mode, or configuration dependent if configured as output or bidirectional.

12.10.4.4.4. PICPWR Side Band Register 0x02: Alternate Function Configuration

The register description below provides the BMC with the PICPWR sideband alternate function configuration. It is optional for a given PICPWR sideband to support alternative function capabilities and support is PD implementation specific. If there is a capability mismatch, it is required for system BMC firmware to detect and report configuration incompatibilities. For instance, if a given peripheral is requesting an alternative function support on a specific PICPWR sideband signal, but the PD does not support it then BMC FW will indicate a mismatch and then perform whatever vendor policy that handles configuration mismatches. It is recommended that the BMC read the registers to verify that the desired configuration was accepted and valid.

It is required that that JSON descriptive file for the PD contain the possible set of alternate functions supported by the PD for every sideband signal connected to HPM M-PIC PICPWR connector.

TABLE 114: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 1 REGISTER 0X02 DEFINITION

Bit(s)	Name	Description	ATTR	Size	Notes
7:5	RESERVED	Reserved	RO	3'b0	

4:0	SB_FUNC [4:0]	Sideband Function Configuration 00000b: NC if not connected thus not configurable OR 00000b: GPIO (Default) 00001b: PWR_EN_A [OPTIONAL except for SB2] 00010b: PWRFAULT_N_A [OPTIONAL except for SB3] 00011b: WAKE_N_A [OPTIONAL except for SB4] 00100b: PWR_EN_B [OPTIONAL] 00101b: PWRFAULT_N_B [OPTIONAL] 00111b: WAKE_N_B [OPTIONAL] 01000b: EXTERNAL_OE_ [OPTIONAL] 01001b: Reserved. … 11111b: Reserved.	RW	5'b00	GPIO basic function is required if connected. All alternative functions are optional. Five bits will provide up to 32 possible alternative functions.

12.10.4.4.5. PICPWR Sideband 0x03: Reserved

This register is reserved for future use.

12.10.4.5. Region 3: Control Panel Sideband CSR Memory Map

The MPIC Control Panel per target registers map starts at Offset 0 from the base address for the per target CSR space. The base address is 0x30000 and proceeds in multiples of 16 bytes

or 0x10. Thus, addresses increment starting at ‘h3XXX0 where X is based on PICPWR Target’s index number on a 16-byte boundary.

Example:

Base is at 0x30000 for target MPIC Primary Control Panel

Base is at 0x30010 for target MPIC Secondary Control Panel

Base is at 0x30020 for target MPIC Additional Control Panel

Etc.

MPIC Control Panel 0 Starts at 0

Base is at 0x30000 for target MPIC PICPWR 0 Sideband Signal

Base is at 0x30004 for target MPIC PICPWR 0 Sideband Signal 1

Base is at 0x30008 for target MPIC PICPWR 0 Sideband Signal 2

…

0x0C – 0x0F are reserved.

MPIC Control Panel 1 at 0x10

Base is at 0x30010 for target MPIC PICPWR 1 Sideband Signal 0

Base is at 0x30014 for target MPIC PICPWR 1 Sideband Signal 1

Base is at 0x30018 for target MPIC PICPWR 1 Sideband Signal 2

…

0x1C – 0x1F are reserved.

Etc.

12.10.4.5.1. Control Panel Sideband Register Definition

This section contains the control panel sideband register definition.

The table below contains the per Control Panel connector sideband registers list.

TABLE 115: CONTROL PANEL SIDEBAND PICPWR CSR MEMORY MAP

Control Panel Sideband	Offset Address	Name	W	ATTR	Description
SB_0	0x00	SB_GPIO_STATUS_0	1B	RW	Sideband GPIO status
	0x01	SB_GPIO_CONTROL_0	1B	RW	Sideband GPIO config / control
	0x02	SB_ALT_FUNCT_CONFIG_0	1B	RW	Configuration of sideband alternative functions
	0x03	SB_RESERVED_0	1B	RO	Reserved
SB_1	0x04	SB_GPIO_STATUS_1	1B	RW	Sideband GPIO status
	0x05	SB_GPIO_CONTROL_1	1B	RW	Sideband GPIO config / control
	0x06	SB_ALT_FUNCT_CONFIG_1	1B	RW	Configuration of sideband alternative functions
	0x07	SB_RESERVED_1	1B	RO	Reserved
SB_2	0x08	SB_GPIO_STATUS_2	1B	RW	Sideband GPIO status
	0x09	SB_GPIO_CONTROL_2	1B	RW	Sideband GPIO config / control
	0x0A	SB_ALT_FUNCT_CONFIG_2	1B	RW	Configuration of sideband alternative functions
	0x0B	SB_RESERVED_2	1B	RO	Reserved
SB_3	0x0C	SB_GPIO_STATUS_3	1B	RW	Sideband GPIO status
	0x0D	SB_GPIO_CONTROL_3	1B	RW	Sideband GPIO config / control
	0x0E	SB_ALT_FUNCT_CONFIG_3	1B	RW	Configuration of sideband alternative functions
	0x0F	SB_RESERVED_3	1B	RO	Reserved

12.10.4.5.2. Control Panel Sideband Register Offset 0x00: General GPIO Status

This section provides the per control panel connector sideband registers set definition for the general GPIO status register.

TABLE 116: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 2 REGISTER 0X00 DEFINITION

Bit(s)	Name	Description	ATTR	Size	Notes
7:3	RESERVED	Reserved	R	5'b0	
2	LOGICAL_STATE	1 = Signal is Logically High 0= Signal is Logically Low	R	1'b0	Used by FW
1	PHYSICAL_STATE	1 = Signal is Physically High 0 = Signal is Physically Low	R	1'b0	Useful for debug
0	CONNECTED	1= Connected 0 = Not connected	R	1'b0	Whether hardware signal is physically routed between PD and PICPWR connector SBn IO pin

12.10.4.5.3. Control Panel Sideband Register Offset 0x01: General GPIO Control

This section provides the per control panel connector sideband registers set definition for the general GPIO control register.

TABLE 117: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 2 REGISTER 0X01 DEFINITION

Bit(s)	Name	Description	ATTR	Size	Notes
7:6	INT_TRIGGER	Interrupt Behavior 0 = Both rising and falling edge 1 = rising edge only 2= falling edge only	RW	1'b0	Mapping to interrupt from this source shall be implemented using internal virtual wire mapping framework.

		3 =reserved			
5	INT_EN	1 = Enabled 0 = Disabled	RW	1'b0	
4	LOGICAL_POLARITY	1= Active High 0 = Active Low	RW	1'b0	Used by logic to trigger interrupt behavior.
3:2	PHYSICAL_POLARITY	IN, OUT, BIDIR, high-Z	RW	2'b0	
1	DRIVE	0 = Open Drain 1 = Push Pull	RW	1'b0	Select drive type for GPIO
0	GPIO State	1 = Asserted 0 = De-Asserted	RW	1'b0	GPIO output state if in GPO mode, or configuration dependent if configured as output or bidirectional.

12.10.4.5.4. Control Panel Sideband Register 0x02: Alternate Function Configuration

The register description below provides the BMC with the Control Panel Sideband alternate function configuration. It is optional for a given Control Panel sideband to support alternative function capabilities and support is PD implementation specific. If there is a capability mismatch, it is required for system BMC firmware to detect and report configuration incompatibilities. For instance, if a given peripheral is requesting an alternative function support on a specific Control Panel sideband signal, but the PD does not support it then BMC FW will indicate a mismatch and then perform whatever action is required by policy to handle the configuration mismatch. It is recommended that the BMC read the registers to verify that the desired configuration was accepted and valid.

It is required that that JSON descriptive file for the PD contain the possible set of alternate functions supported by the PD for every sideband signal connected to HPM M-PIC Control Panel connector.

TABLE 118: FLEXIO AND SIDEBAND IP BLOCK CSR REGION 2 REGISTER 0X02 DEFINITION

Bit(s)	Name	Description	ATTR	Size	Notes
7:5	RESERVED	Reserved	RO	3'b0	
4:0	SB_FUNC [4:0]	Sideband Function Configuration 00000b: NC if not connected thus not configurable. OR 00000b: GPIO (Default) 00001b: PWR_EN 00010b: Reserved … 01111b: Reserved ---------- 10000b: Reserved For OEM Use … 11111b: Reserved For OEM Use	RW	5'b00	GPIO basic function is required. All alternative functions are optional. Up to 32 possible alternative functions are definable. The last 16 functions are reserved for OEM usage.

12.10.4.5.5. Control Panel Sideband 0x03: Reserved

This register is reserved for future use.

12.10.5. M-XIO FLEX IO and M-PIC PICPWR Sideband Control Vendor Specific Extensions Support

This section contains a description of how vendor-defined extensions are integrated into the overall M-XIO FLEX IO and M-PIC Sideband Control IP block. This includes the architecture from a conceptual standpoint and implementation guidelines.

12.11. MUX Control Logical IP Block

This section contains the logical IP block description for MUX Control requirements and descriptive abstraction for this IP block.

12.11.1. MUX Control Conceptual Overview

This section contains an overview of the MUX Control functionality for a typical set of use cases.

The intention of the MUX control IP block is to provide the BMC with a common set of control and status interfaces for controlling one or more MUXes on either the HPM or connected peripherals and options. Primary use case examples are for controlling I2C or UART muxes.

12.11.2. MUX Control Bus Target Types

The following buses may have MUXes on them. Certain buses, such as I2C have MUX requirements to ensure addressing collisions do not occur or for isolation from unpowered circuitry.

TABLE 119: MUX CONTROL TARGET BUS TYPES

Bus	Target	Required (Y/N)	Reason
I2C	PICPWR	Y	All PICPWR connectors shall have isolation to ensure peripheral FRU address collision does not occur.
I2C	MXIO	Y	All MXIO connectors shall have isolation to ensure peripheral FRU address collision does not occur.
I2C	M-CRPS Power Supplies	Y	Needed for M-CRPS power supply isolation.
I2C	MISC Devices	N	MUXes used in vendor defined applications.
I3C	Single device – point to point	N	DIMM I3C without SoC sharing

I3C	Shared bus topology without hub	Y	Example is DIMM and SoC sharing of I3C bus
UART	Single PD / PLD Only	N	UART is a point-to-point bus and is only defined to support two devices
UART	Shared bus topology	Y	If UART bus topology has more devices than two then isolation is necessary
JTAG	Single PD / PLD Only	N	If bus topology only has a single PD / PLD device endpoint
JTAG	Shared bus topology	Y	If bus topology has multiple PD / PLD device endpoints
SPI	Shared bus topology	Y	If the SPI bus requires multiple SPI devices sharing the same Chip Select signal or more than two Chip Select signals.
Misc	Vender defined	N	This shall be described in the JSON file.

12.11.3. MUX Control IP Block Architecture

This section contains the general abstract architecture for MUX Control IP block.

The following diagram shows a possible configuration support for controlling the GPIO based MUX control signals.

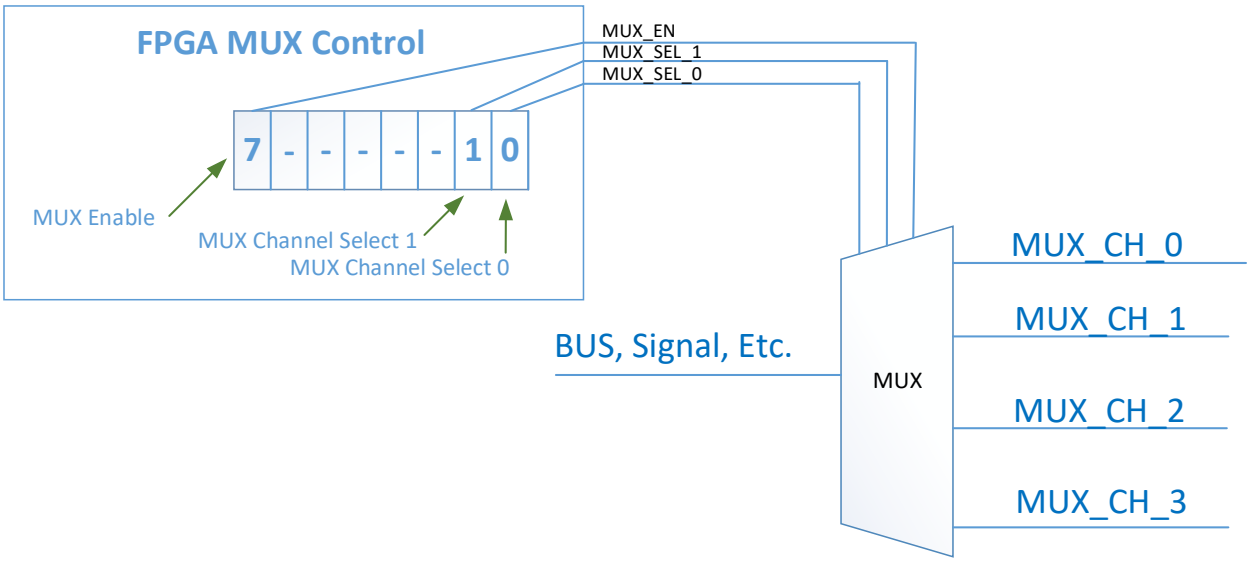


FIGURE 52: PD GPIO BASED MUX CONTROL DIAGRAM

As shown in the diagram above, the MUX Control IP block implemented within the Programmable Device (PD), here shown as an FPGA, This contains one or more registers that control the specific signals to select a given MUX's channel control GPIO signals as well as an enable signal. The PD JSON file indicates which physical signals that the register controls on the PD package, and the HPM JSON file indicates how that physical signal connects to the specific MUX device. Thus, this allows the BMC to determine the CSR to MUX mapping.

12.11.4. MUX Control IP Block Requirements

This section contains the general abstract architecture for the MUX Control IP block.

12.11.4.1. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 120: MUX CONTROL IP BLOCK REQUIRED AND OPTIONAL FEATURE SET

Name	REQ	OPT	Description
MUX Channel Control	Y		
MUX Output Enable Control	Y		

12.11.5. MUX Control CSRs

This section contains a description of the MUX IP block CSRs.

12.11.5.1. MUX Control CSR Region Map Overview

These ranges are all offsets from the MUX_CONTROL_IP_BLOCK base address.
This base address is defined by the CPLD/PD developer and is not prescribed here.

TABLE 121: MUX CONTROL IP BLOCK CSR MEMORY MAP

Region	Range Bottom	Range Top	Sections	Name	Description
0	0x0000	0x00FF		Global Registers	IP block level to cover all targets
1	0x0100	0x0FFF	0x0XX0	MUX Control Target CSR Set	For all HPM / peripheral MUXes (I2C, I3C, UART, SPI, JTAG, etc.)
2[19]	0x1000	0xXXXX		OEM SECTION	Undefined

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

[19] Note: Implementor can use this for additional regions as needed.

12.11.5.2. Region 0: MUX Control IP Block Common Registers

This region is divided into two subregions.

TABLE 122: MUX CONTROL IP BLOCK COMMON REGISTERS

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

12.11.5.2.1. Region 0: Sub-region 0 Memory Map

TABLE 123: MUX CONTROL REGION 0 SUB-REGION 0 MEMORY MAP

Offset Address	Name	Data Width	ATTR	Description
0x00	IP Block ID	2B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2B	R	Major.Minor (1 byte each).
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.

0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x5 – 0x0F	Reserved			

12.11.5.2.2. Region 0: Sub-region 1 Memory Map

Reserved

12.11.5.3. Region 1: MUX Control Per I2C Bus MUX Target Memory Map

The I2C per target MUX registers map starts at Offset 0 from the base address for the per target CSR space. The base address is set at address 0x0100 for this sub-region, and each individual target MUX register will proceed in single byte increments.

Example:

I2C MUX 0 Starts at 0x0100.

I2C MUX 1 Starts at 0x0101.

Etc.

12.11.5.4. Mux Control Register Offset 0x00: General MUX Channel Control

This section outlines the specific definition for the MUX control register at offset 0. The flowing table provides the bit definition.

TABLE 124: MUX CONTROL REGISTER OFFSET 0X00

Bit(s)	Name	Description	ATTR	W	Notes
7	MUX_EN	MUX enable / disable signal for isolation	R/W	1	Tri-states the MUX output
6:4	RESERVED	Reserved	R	3'b0	
3:0	MUX_SEL [3:0]	MUX Selection Channel Control 0000b: Channel 0 (Default) 0001b: Channel 1 0010b: Channel 2 (optional) 0011b: Channel 3 (optional) … 1111b: Channel 15 (optional)	RW	4'b0	Up to 16 channels can be supported per MUX. MUX channel quantity is vendor defined, and maximum number of channels shall be provided in the PD JSON file. At least two channels are required.

The PD JSON file will indicate how many channels are supported per MUX control register. The PD JSON file will indicate which MUX control register maps to which physical channel control pins on the PD. This is required so that the board (HPM, peripheral, etc.) JSON file can map physical MUXes on the board to specific PD pins. Thus, the BMC can use this information to know which MUX is controlled by which CSR.

The JSON file will indicate what the usage will be for a given MUX for reference purposes. Thus, if the MUX control register is mapped to a MUX on an I2C bus then it will identify this as such.

The PD shall not indicate a configuration error if the BMC sets an invalid configuration. It is recommended that the BMC read the registers to verify that the desired configuration was accepted and valid.

Note: If a given mux is controlled via a set of GPIOs such as LTPI normal latency GPIOs as well as the CSR, it is required that the state of this is reflected in the CSR if the GPIO selects a given MUX channel.

12.11.6. MUX Control Vendor Specific Extensions Support

Vendors may add extensions to this IP block as needed by using OEM reserved regions for CSRs.

Vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.11.7. MUX Control Example Implementation

Example implementations, if available, may be found in supplementary materials referenced in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

12.12. Cooling Subsystem Control Logical IP Block

This section contains the logical IP block description for Cooling Subsystem Control requirements and descriptive abstraction for this IP block. This entire IP block is fully optional.

The system cooling sub-system can represent any cooling technology such as fans or direct liquid cooling (DLC). This is at the system implementor’s discretion to identify the IP block using the common framework and provide the appropriate JSON description for BMC firmware to consume so it can interface with this IP block.

The cooling subsystem can include one or more sensors specifically used by the cooling subsystem, such as a fan tachometer used by an air-based cooling system. The PD may have direct or indirect control over for the purpose of providing cooling system specific telemetry information to the BMC. These sensors could be directly on the HPM planar, or it can be connected to the HPM via external wiring. The PD could provide a mechanism by which the BMC can access sensors, or the PD can access the sensor in an autonomous manner to collect telemetry data on behalf of the BMC.

There are several use cases that the Cooling IP block can be used for.

- Direct fan control for one or more system fans.
- Advanced fan control using I2C or another system bus to interface with one or more fans.
- Support for controlling Direct Liquid Cooling (DLC) hardware.
- Vendor defined cooling support.

Additional use cases are also possible as this is not an exclusive list.

This section will outline what is required to expose these capabilities and provide this through the use of the self-describing JSON schema framework.

12.12.1. Cooling Subsystem Control Conceptual Overview

This section contains an overview of the Cooling Subsystem Control functionality for a typical set of use cases.

The intention of the cooling system control IP block is to provide the BMC with a common set of control and status interfaces for controlling one or more cooling subsystem components on either the HPM or connected peripherals and options.

The figure below provides a simple example for direct fan control use.

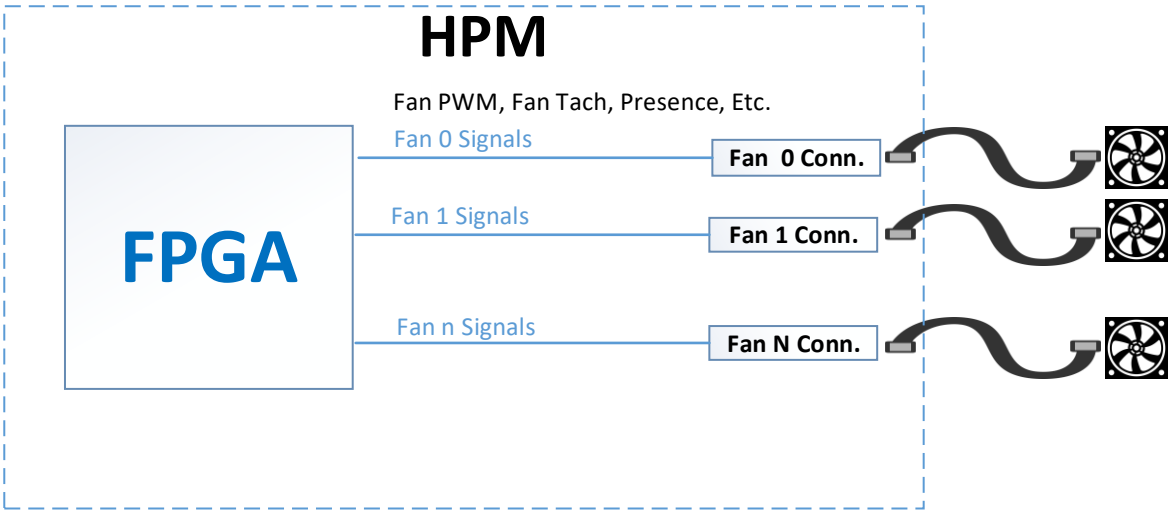


FIGURE 53: DIRECT FAN CONTROL USE CASE

In this figure, the fan control IP block implemented within the PD, here shown as an FPGA, This simply controls a set of directly attached fans to the HPM. Fan control signals such as PWM, tachometer, and presence signals provide simple fan control.

More complex fans may include control logic on the fan itself which requires interfacing through I2C or some other system bus. For more advanced use cases, it may be necessary to implement an I2C master or multi-master capability and some state machine logic to support autonomic control over these more advanced fans.

It is required that the PD implementor describe the usage and control of cooling hardware via the PD JSON file.

12.12.2. Cooling Subsystem Control IP Block Architecture

This section contains the general abstract architecture for Cooling Subsystem Control IP block.

This cooling IP block is designed to support controlling of multiple cooling technologies. The block architecture supports modularized sub-sections such that each is responsible for a different cooling technology. Each module is optional and only included as needed by the designer and required use case.

The diagram below shows how this the Cooling IP block is partitioned to support each use case.

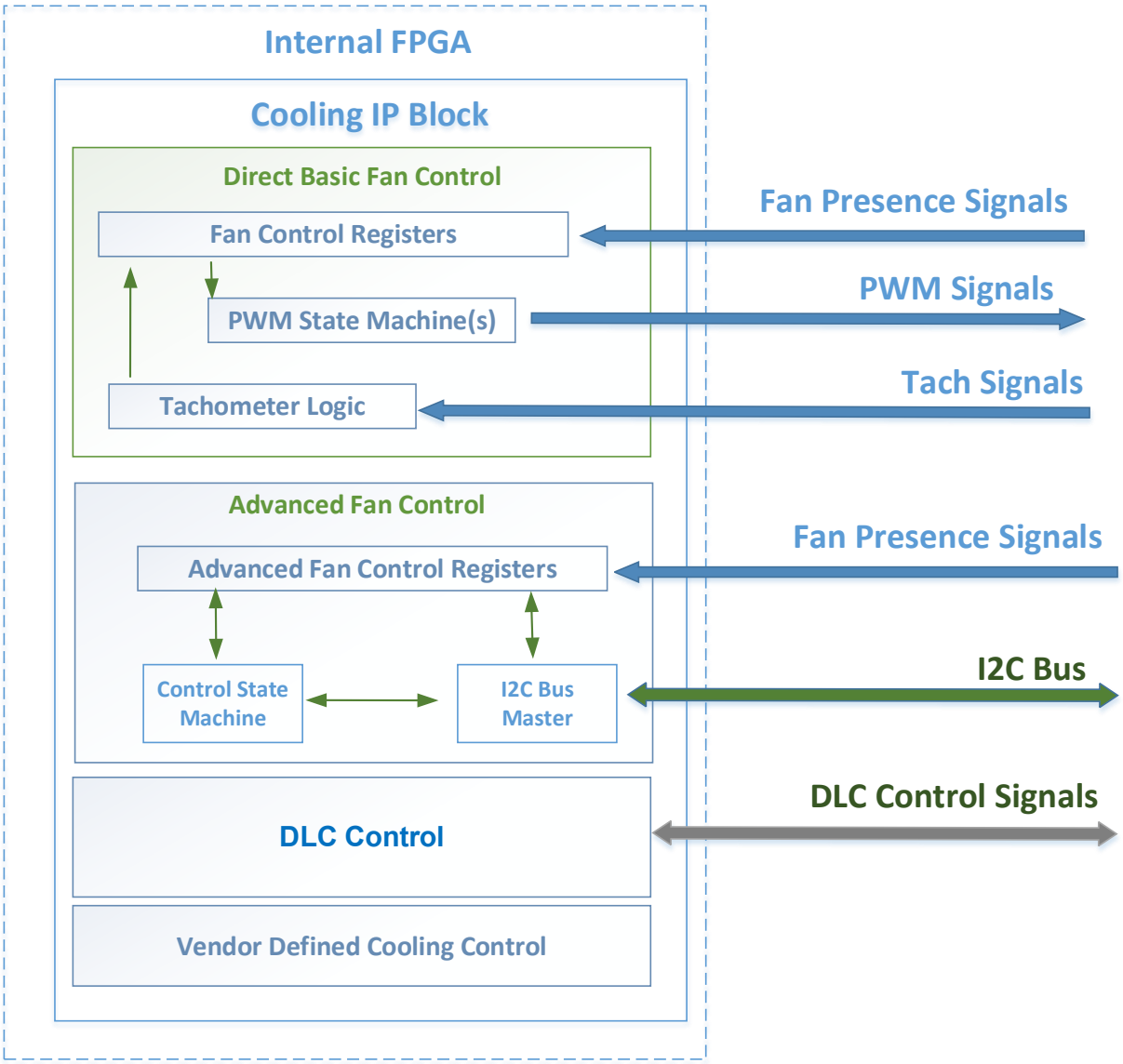


FIGURE 54: COOLING CONTROL IP BLOCK ARCHITECTURAL DIAGRAM

As shown in Figure 32, the Cooling Control IP block within the PD, here shown as an FPGA, This contains one or more sub-modules which control different cooling technologies. Each sub-module is optional as is the entire Cooling Control IP block.

The first sub-module is intended to support basic fans that utilize a standard PWM, tachometer, and presence indication signal. The next sub-module interfaces with advanced fans using a simple bus interface, such as I2C, to access a standard set of control and status registers that allow for the collection of data for control of the fan. The I2C bus master, in this example, is used by the control state machine to periodically collect data via a polling method and / or on-demand via an interrupt method if the advanced fan supports such a feature. Thus, the PD offloads the data collection and control of the set of fans on behalf of the BMC.

Lastly, the DLC control sub-module is used to interface with the DLC subsystem utilized by the system.

Vendor defined sub-modules can be supported by the IP block, but their definition is outside the scope of this specification.

12.12.3. Cooling Subsystem Control IP Block Requirements

This section contains the general abstract architecture for the Cooling Subsystem Control IP block.

12.12.3.1. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 125: COOLING SUB-SYSTEM CONTROL IP BLOCK REQUIRED AND OPTIONAL FEATURE SET

Name	REQ	OPT	Description
Direct Basic Fan Control		Y	Support for controlling one or more basic fans.
Advanced Fan Control		Y	Support for controlling one or more advanced fans.
DLC Control		Y	Support for controlling Direct Liquid Cooling hardware.
Vendor Defined Cooling Solution Control		Y	Support for controlling vendor-defined cooling solutions.

12.12.3.2. Required Inputs

The table below contains the set of required and optional recommended inputs consumed by this IP block.

TABLE 126: COOLING CONTROL IP BLOCK REQUIRED AND OPTIONAL INPUTS LIST

Name	REQ	OPT	Description
Fan Presence	Y		
Fan Tachometer Signal(s)		Y	Required by the direct basic fan control module.

12.12.3.3. Required Outputs

The table below contains the set of required and optional recommended outputs produced by this IP block.

TABLE 127: COOLING CONTROL IP BLOCK REQUIRED AND OPTIONAL OUTPUTS LIST

Name	REQ	OPT	Description
Fan PWM Signal(s)		Y	Required by the direct basic fan control module.

12.12.3.4. Required Bus Support

The table below contains the set of required and optional recommended bus support consumed by this IP block.

TABLE 128: COOLING CONTROL IP BLOCK REQUIRED AND OPTIONAL BUS SUPPRT LIST

Name	REQ	OPT	Description
I2C		Y	Required by the advanced fan control module.

12.12.4. Cooling Subsystem Control IP Block CSR Requirements

This section contains the Control and Status Registers (CSR) requirements for standardized Cooling Subsystem Control functionality.

12.12.4.1. Cooling Control CSR Region Overview

The Cooling Control CSR regions are divided into a set of ranges. These ranges are all offsets from the COOLING_IP_BLOCK base address. This base address is defined by the CPLD/PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 129: COOLING CONTROL IP BLOCK CSR MEMORY MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x00FF	Global Registers	General IP block level registers
1	0x0100	0x0BFF	Cooling Control IP Block Interface Control and Status Registers	Cooling Control IP Block Control and Status Registers.
2	0x0C00	0x0FFF	Reserved	Reserved for future use
3[20]	0x1000	0xFFFF	OEM SECTION	Vendor defined usage.

[20] Note: Implementor can use this for additional regions as needed.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.12.4.2. Cooling Control IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 130: COOLING SUBSYSTEM REGION 0 SUBREGIONS

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

12.12.4.2.1. Cooling Control IP Block Common Registers Sub-Region 0 Description

The following section contains the description for the cooling control IP block common register set.

The table below contains the definition for the Cooling IP block common registers sub-region 0.

TABLE 131: COOLING SUBSYSTEM REGION 0 SUBREGIONS 0 DESCRIPTION

Offset Address	Name	Width	ATTR	Description

0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each).
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state indication that requires a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	1	FAN_WATCHDOG_CONTROL	RW	Watchdog enable and service
0x08	1	FAN_WATCHDOG_STATUS	RO	Watchdog status
0x09	2	FAN_WATCHDOG_TIMEOUT_MS	RW	Watchdog timeout in milliseconds. PD may enforce a minimum allowed timeout (implementation-defined)

				Internal timeout countdown resets to this value on WD_KICK
0xB-0xF	6	Reserved	RO	

To prevent thermal runaway in the event of BMC failure (crash, hang, reset, or loss of communication), the PD Cooling IP implements a supervision watchdog that is part of the Global Register block. The BMC is required to periodically service this watchdog. If the watchdog expires, the PD autonomously forces all enabled fans into a failsafe full-speed mode until the watchdog is cleared and normal operation is restored. These registers are added to end of the common global registers for the IP block.

12.12.4.2.1.1. Register Offset 0x7: FAN_WATCHDOG_CONTROL

The following table contains the description for the fan watchdog control register.

TABLE 132: COOLING SUBSYSTEM FAN WATCHDOG CONTROL REGISTER

Bit(s)	Name	Description	ATTR
0	WD_ENABLE	1 = Enable BMC supervision watchdog	RW
1	WD_KICK	Write-1-to-service watchdog (self-clearing)	RW
2	WD_CLEAR	Clear watchdog-expired condition	W1C
3:7	Reserved	Reads 0.	RO

Notes:

WD_KICK resets the internal watchdog timer.
WD_CLEAR is used after a watchdog expiration to allow recovery.
WD_ENABLE shall be set for the watchdog to be active.

12.12.4.2.1.2. Register Offset 0x8: FAN_WATCHDOG_STATUS

The following table contains the description for the fan watchdog status register.

TABLE 133: COOLING SUBSYSTEM FAN WATCHDOG STATUS REGISTER

Bit(s)	Name	Description	ATTR
0	WD_EXPIRED	1=Watchdog timeout occurred	RO
1	WD_ACTIVE	1=Watchdog enabled and counting	RO
2:7	Reserved	Reads 0.	RO

WD_EXPIRED is latched and cleared by writing 1 to WD_CLEAR.

When WD_EXPIRED = 1, fans are in failsafe mode.

12.12.4.2.2. Cooling Control IP Block Common Registers Sub-Region 1 Description

Reserved

12.12.4.3. Cooling Control IP Block Region 1 CSR Memory Map

The Cooling region one registers starts at offset 0x100 from the base address for the I2C / I3C Basic CSR space.

This region is divided into four subregions.

TABLE 134: COOLING SUBSYSTEM REGION 1 CSR MEMORY MAP

Sub-Region	Range	Name	Description
0	0x100 – 0x2FF	Direct Basic Fan Control CSRs	Set of registers that handle Basic Fan control PWM, tachometer, and presence functions.
1	0x300 - 0x4FF	Advanced Fan Control CSRs	Set of registers that handle remote advanced fan status, I2C payload, and control functions.
2	0x500 – 0x6FF	Direct Liquid Cooling (DLC) Support	Set of registers that handle controlling Direct Liquid Cooling (DLC) hardware.
3	0x700 – 0xBFF	Vendor Defined Cooling Subsystem Control	Set of registers reserved for vendor defined use.

It is required that the PD implementor specifies within the JSON file which ‘sub-regions are present and supported.

12.12.4.4. Cooling Control CSRs Region 1 Sub-Region 0 Description

The table below contains the definition for the Cooling IP block CSRs region 1 sub-region 0.

TABLE 135: DIRECT BASIC FAN CONTROL CSR TABLE

Offset [Bytes]	Size [Bytes]	Name	Type	Description
0x100	1	FAN_COUNT	RO	Reports the number of basic fans supported by this block. Fans are indexed from 0 to FAN_COUNT-1.
0x101	1	FAN_CAPABILITIES	RO	Bit-mapped capabilities. See register table for bit definitions.
0x102	2	GLOBAL_PWM_FREQUENCY	RW	Defines the global PWM base frequency used by all basic fans (typically 25 kHz).
0x104	12	Reserved	RO	Reserved
0x110+16n	16	Fan n Register Block	-	See Individual Fan register definitions below

TABLE 136: DIRECT BASIC FAN CONTROL - FAN N REGISTER BLOCK (0X110+16N)

Offset [Bytes]	Size [Bytes]	Name	Type	Description
0x110+ 16n	1	FANn_CONTROL	RW	Control flags (Enable, Mode, Ramp).
0x111+ 16n	1	FANn_STATUS	ROW 1C	Status flags (Faults, Presence).
0x112+ 16n	1	FANn_PWM_TARGET	RW	Target Duty Cycle (used in Manual/Open-Loop Mode).
0x113+ 16n	1	FANn_PWM_MIN	RW	Safety Floor for PWM output.
0x114+ 16n	1	FANn_PWM_MAX	RW	Safety Ceiling for PWM output.
0x115+ 16n	2	FANn_TACH_READING	RO	Current RPM measurement (16-bit).
0x117+ 16n	2	FANn_TACH_TARGET	RW	Target RPM (used in Automatic/Closed-Loop Mode).
0x119+ 16n	2	FANn_TACH_MIN	RW	Minimum RPM threshold for Under-Speed fault.
0x11B+ 16n	2	FANn_TACH_MAX	RW	Maximum RPM threshold for Over-Speed fault.
0x11D+ 16n	1	FANn_RAMP_RATE	RW	Max PWM change per step (Acoustic smoothing).
0x11E+ 16n	2	Reserved	RO	Reserved.

12.12.4.5. Cooling Control IP Block CSRs Region 1 Sub-Region 0 Register Definitions

12.12.4.5.1. Register Offset 0x101: FAN_CAPABILITIES

Cooling IP Block Region 1 Sub-region 0 Register Offset 0x101: FAN_CAPABILITIES

TABLE 137: COOLING IP BLOCK REGION 1 SUB-REGION 0 REGISTER OFFSET 0X101: FAN_CAPABILITIES

Bit(s)	Name	Description	ATTR
0	PWM_AVAIL	1 = Hardware supports PWM output pins.	RO
1	TACH_AVAIL	1 = Hardware supports Tachometer input capture.	RO
2	PRES_AVAIL	1 = Hardware supports dedicated Presence detection pins.	RO
3	CLOSED_LOOP_AVAIL	1 = Hardware supports internal RPM PI control (Mode=1). If 0, only Manual PWM (Mode=0) is supported, and FANn_TACH_TARGET is ignored.	RO
4	RAMP_AVAIL	1 = Hardware supports acoustic ramp rate control. If 0, changes are always instantaneous and FANn_RAMP_RATE is ignored.	RO
5	POLARITY_CFG	1 = Hardware supports configurable PWM polarity (Active High/Low).	RO
6:7	Reserved	Reads 0.	RO

12.12.4.5.2. Register Offset 0x110+16n: FANn_CONTROL

Cooling IP Block Region 1 Sub-region 0 Register Offset 0x110+16n: FANn_CONTROL

TABLE 138: COOLING IP BLOCK REGION 1 SUB-REGION 0 REGISTER OFFSET 0X110+16N: FANN_CONTROL

Bit(s)	Name	Description	ATTR
0	ENABLE	**Master Enable**. 1 = Fan active. 0 = Fan driver disabled (PWM=0).	RW
1	MODE	**Control Loop Mode**. 0 = **Open Loop (Manual PWM)**. Hardware drives	RW

		FANn_PWM_TARGET. 1 = **Closed Loop (Auto RPM)**. Hardware Adjusts PWM to match FANn_TACH_TARGET.	
2	RAMP_ENABLE	**Ramp Enable**. 1 = Changes to output PWM are smoothed by FANn_RAMP_RATE. 0 = Changes are applied instantly (Step).	RW
3	TACH_MON	**Tach Monitor**. 1 = Enable faults if TACH borders (Min/Max) are crossed.	RW
4	PRES_MON	**Presence Monitor**. 1 = Enable faults if Presence signal is lost.	RW
5:7	Reserved	Shall be 0.	RO

12.12.4.5.3. Register Offset 0x111+16n: FANn_STATUS

Cooling IP Block Region 1 Sub-region 0 Register Offset 0x111+16n: FANn_STATUS

TABLE 139: COOLING IP BLOCK REGION 1 SUB-REGION 0 REGISTER OFFSET 0X111+16N: FANN_STATUS

Bit(s)	Name	Description	ATTR
0	PRESENT	1 = Fan physically present (if supported).	RO
1	TACH_FAULT	1 = Tachometer reading outside valid range (Min/Max).	W1C
2	STALL	1 = Fan commanded ON but RPM is effectively zero.	W1C
3	PWM_OOR	1 = Calculated/Requested PWM clipped by Min/Max limits.	W1C
4	PRES_FAULT	1 = Presence signal inconsistent.	W1C
5:7	Reserved	Reads 0.	RO

12.12.4.5.4. Cooling Modes

Manual PWM Mode (MODE = 0)

In Manual PWM Mode, the BMC directly sets the fan speed by writing a duty cycle. The hardware does not regulate RPM.

- FANn_PWM_TARGET: Sets the desired duty cycle (0-255).
- Control: The hardware drives the pin at this duty cycle, subject to FANn_PWM_MIN / FANn_PWM_MAX clamping.
- Ramping: If RAMP_ENABLE is set, changes to the output duty cycle are applied gradually at FANn_RAMP_RATE steps per second to reduce acoustic noise. If clear, changes are instantaneous.
- Monitoring: FANn_TACH_READING is updated with real RPM for host monitoring and stall detection (if TACH_MON is set), but it does not affect the output.

Closed-Loop (Automatic) Mode (MODE = 1)

In Closed-Loop Mode, the Fan Controller hardware actively regulates the PWM output to maintain a specific RPM.

- FANn_TACH_TARGET: Sets the desired RPM.
- Control: The hardware PI (Proportional-Integral) loop adjusts the internal PWM duty cycle to minimize the error (TACH_TARGET - TACH_READING).
- Safety Bounds: The loop is constrained by FANn_PWM_MIN/MAX. If the target RPM cannot be achieved within these PWM limits (e.g., fan blocked or aging), the PWM_OOR status bit is set.
- Monitoring: FANn_TACH_MIN/MAX act as absolute safety thresholds. If the RPM leaves this window despite the loop's best efforts, TACH_FAULT is asserted.

12.12.4.6. Cooling Control IP Block CSRs Region 1 Sub-Region 1 Description

In this model, the PD provides a simplified abstraction layer for controlling intelligent fans that expose an I²C interface. All fans managed through this sub-region are assumed to be directly addressable I²C devices residing on a single shared bus.

The PD does not perform automatic device discovery, bus scanning, or topology inference. Instead, the BMC is responsible for explicitly configuring each fan instance by programming its I²C address and supported capabilities through dedicated configuration registers.

The PD state machine executes I²C transactions on behalf of the BMC based on this static configuration. It applies per-fan control policies (such as RPM targets and ramping), retrieves fan telemetry, and mirrors status information into memory-mapped CSRs.

External fan controllers, if present in the system, are assumed to be managed directly by the BMC and are outside the scope of this PD interface

The advanced fan control CSRs are listed below over multiple tables where each table contains a logical functional grouping

Optionally, the PD may expose a BMC-driven I²C pass-through interface to allow the BMC to issue ad-hoc I²C transactions directly to an I²C fan for debug and bring-up (e.g., reading device

capabilities or accessing registers not supported by the PD's current logic). When enabled, the pass-through engine executes a single I²C transaction described by the BMC and returns completion status and read data via CSRs. This interface is intended for low-rate debug use and is not a replacement for the normal fan polling/control state machine.

The table below contains the definition for the Cooling IP block CSRs region 1 sub-region 1.

Global Control & Status (0x300–0x31F)

TABLE 140: GLOBAL CONTROL & STATUS (0X300–0X31F)

Offset [Bytes]	Size [Bytes]	Name	Type	Description
0x300	1	FAN_COUNT	RO	Number of I²C fans supported
0x301	1	FAN_CONTROL	RW	Global enable and reset
0x302	1	FAN_STATUS	RO	Global engine status
0x303	2	FAN_POLL_PERIOD_MS	RW	SM telemetry scan period in milliseconds.
0x305	1	Reserved	RO	Read 0
0x306	1	DBG_I2C_CONTROL	RW	enable/start/abort + basic formatting controls
0x307	1	DBG_I2C_STATUS	ROW1C	busy/done + error latches like NACK/timeout
0x308	4	DBG_I2C_ADDR_CMD	RW	Holds the transaction description—which device address to talk to and what kind of transaction to run (read vs write, and typically the byte counts / protocol options depending on how you encode it). Think of it as the "command header".
0x30C	2	DBG_I2C_REG	RW	Holds an optional register index/offset that the engine will send before the data phase (common for "read register X" style devices). Your USE_REG / REG_16BIT bits in DBG_I2C_CONTROL control whether this is used and whether it's 8- or 16-bit.
0x30E	1	DBG_I2C_CLKDIV	RW	Optional knob to set the I²C SCL rate (implementation-defined). Useful for bring-up if a fan is sensitive to bus speed or if you need slower debug transactions.

0x310	4	DBG_I2C_WDATA	RW	The write payload bytes the BMC wants sent to the device (e.g., value to write into the target register).
0x314	4	DBG_I2C_RDATA	RO	The read payload bytes captured from the device after a read transaction completes.

Valid fan indices (n) are 0 … FAN_COUNT−1.

Per-Fan Register Block (0x320+ (n * 32))

TABLE 141: PER-FAN REGISTER BLOCK (0X320+ (N * 32))

Offset [Bytes]	Size [Bytes]	Name	Type	Description
0x320+32n	1	FANn_I2C_ADDRESS	RW	I2C Address of fan
0x321+32n	1	FANn_CAPABILITIES	RO	Capability bitmap
0x322+32n	1	FANn_CONTROL	RW	Enable and ramp control
0x323+32n	1	FANn_STATUS	ROW 1C	Fan status and faults
0x324+32n	2	FANn_RPM_TARGET	RW	The primary setpoint written by the BMC (or thermal loop).
0x326+32n	2	FANn_RPM_MIN	RW	Lower bound for safety/faulting.
0x328+32n	2	FANn_RPM_MAX	RW	Upper bound for safety/faulting.
0x32a+32n	2	FANn_RAMP_RATE	RW	Slew limit (RPM/s or %/s).
0x32c+32n	2	FANn_LAST_RPM	RO	Last measured RPM mirrored from device.
0x32e+32n	2	FANn_LAST_PWM	RO	Diagnostic feedback. Shows the duty cycle the device reports it is running, which helps debug mismatches between requested RPM and actual drive effort
0x330+32n	2	FANn_VENDOR_ID	RO	Optional Vendor ID read from Fan
0x332+32n	2	FANn_PRODUCT_ID	RO	Optional Product ID read from Fan
0x334+32n	12	Reserved	RO	Read 0

12.12.4.7. Cooling Control IP Block CSRs Region 1 Sub-Region 1 Register Definitions

12.12.4.7.1. Register Offset 0x301: FAN_CONTROL

Cooling IP Block Region 1 Sub-region 1 Register Offset 0x301: FAN_CONTROL

TABLE 142: COOLING IP BLOCK REGION 1 SUB-REGION 1 REGISTER OFFSET 0X301: FAN_CONTROL

Bit(s)	Name	Description	ATTR
0	ENABLE	Enable advanced fan control engine	RW
1	RESET	Reset internal state machine (self-clearing)	RW
2:7	Reserved	reads 0	RO

12.12.4.7.2. Register Offset 0x302: FAN_ STATUS

Cooling IP Block Region 1 Sub-region 1 Register Offset 0x302: FAN_STATUS

TABLE 143: COOLING IP BLOCK REGION 1 SUB-REGION 1 REGISTER OFFSET 0X302: FAN_STATUS

Bit(s)	Name	Description	ATTR
0	BUSY	The State Machine is actively processing a background sequence.	RO
1	ERROR_LATCHED	A critical error occurred (e.g., bus stuck, invalid opcode).	RO
2:7	Reserved	Reads 0	RO

12.12.4.7.3. Register Offset 0x306: DBG_I2C_CONTROL

Cooling IP Block Region 1 Sub-region 1 Register Offset 0x306: DBG_I2C_CONTROL

TABLE 144: COOLING IP BLOCK REGION 1 SUB-REGION 1 REGISTER OFFSET 0X306: DBG_I2C_CONTROL

Bit(s)	Name	Description	ATTR
0	DBG_I2C_ENABLE	1 = Enable BMC-driven I²C pass-through engine. 0 = Disabled.	RW
1	START	Write-1 to start a single debug transaction described by DBG_I2C_ADDR_CMD / DBG_I2C_REG / DBG_I2C_WDATA. Self-clearing.	W1S
2	ABORT	Write-1 to abort an in-flight debug transaction (best-effort). Self-clearing.	W1S
3	USE_REG	1 = Send a register/offset value (DBG_I2C_REG) prior to data phase. 0 = Raw I²C read/write without register index.	RW
4	REG_16BIT	When USE_REG=1: 1 = DBG_I2C_REG is 16-bit (sent MSB then LSB). 0 = DBG_I2C_REG is 8-bit.	RW
5:7	Reserved	Shall be 0.	RO

12.12.4.7.4. Register Offset 0x307: DBG_I2C_ STATUS

Cooling IP Block Region 1 Sub-region 1 Register Offset 0x307: DBG_I2C_STATUS

TABLE 145: COOLING IP BLOCK REGION 1 SUB-REGION 1 REGISTER OFFSET 0X307: DBG_I2C_STATUS

Bit(s)	Name	Description	ATTR
0	BUSY	1 = Debug I²C engine busy executing a transaction.	RO
1	DONE	1 = Transaction completed (success or failure). Clear by writing 1 to DONE.	RO W1C
2	NACK	1 = Address or data NACK observed. Clear by writing 1 to NACK.	RO W1C
3	ARB_LOST	1 = Arbitration lost (multi-master). Clear by writing 1 to ARB_LOST.	RO W1C

4	TIMEOUT	1 = Implementation-defined timeout. Clear by writing 1 to TIMEOUT.	RO W1C
5	BAD_CMD	1 = Invalid command/length encoding in DBG_I2C_ADDR_CMD. Clear by writing 1 to BAD_CMD.	RO W1C
6:7	Reserved	write 0, read 0	WR

12.12.4.7.5. Register Offset 0x308: DBG_I2C_ ADDR_CMD

Cooling IP Block Region 1 Sub-region 1 Register Offset 0x308: DBG_I2C_ADDR_CMD

TABLE 146: COOLING IP BLOCK REGION 1 SUB-REGION 1 REGISTER OFFSET 0X308: DBG_I2C_ADDR_CMD

Bit(s)	Name	Description	ATTR
0:6	SLAVE_ADDR	7-bit I²C address.	RW
7	ADDR_MODE	0=7-bit (SLAVE_ADDR valid), 1=10-bit (if 1, SLAVE_ADDR is low 7 bits; upper bits come from SLAVE_ADDR_10B field below)	RW
8:9	XFER_TYPE	• 0 = WRITE • 1 = READ • 2 = WR_RD (combined: write phase then repeated-start read phase) • 3 = reserved	RW
10:13	WLEN	number of data bytes to write from DBG_I2C_WDATA (0–4) • For XFER_TYPE=WRITE: this is the write payload length • For XFER_TYPE=WR_RD: this is the “write payload length” after the optional DBG_I2C_REG bytes (those are implied by DBG_I2C_CONTROL.USE_REG/REG_16BIT)	RW
14:17	RLEN	number of bytes to read into DBG_I2C_RDATA (0–4) • Valid for XFER_TYPE=READ or WR_RD	RW
18	SEND_STOP	1=send STOP at end (for WR_RD, STOP after read); 0=implementation-defined (typically still STOP; or allow chaining)	RW

19	PEC_ENABLE	optional SMBus PEC (CRC-8) enable; if unsupported, treat as 0	RW
20:21	SPEED_SEL	0=use default, 1=use DBG_I2C_CLKDIV, others reserved	RW
22:31	Reserved	Read 0	RO

12.12.4.7.6. Register Offset 0x321+32n:FANn_CAPABILITIES

Cooling IP Block Region 1 Sub-region 1 Register Offset 0x321+32n: FANn_CAPABILITIES

TABLE 147: COOLING IP BLOCK REGION 1 SUB-REGION 1 REGISTER OFFSET 0X321+32N: FANN_CAPABILITIES

Bit(s)	Name	Description	ATTR
0	PWM_AVAIL	Fan supports PWM control	RO
1	TACH_AVAIL	Fan reports RPM	RO
2	CLOSED_LOOP_AVAIL	Fan supports internal RPM regulation	RO
3	FAULT_AVAIL	Fan reports fault conditions	RO
4:7	Reserved	Reads 0	RO

12.12.4.7.7. Register Offset 0x322+32n: FANn_CONTROL

Cooling IP Block Region 1 Sub-region 1 Register Offset 0x322+32n: FANn_CONTROL

TABLE 148: COOLING IP BLOCK REGION 1 SUB-REGION 1 REGISTER OFFSET 0X322+32N: FANN_CONTROL

Bit(s)	Name	Description	ATTR
0	ENABLE	Enable control for this fan	RW
1	RAMP_ENABLE	Enable slew-limited target stepping	RW
2:7	Reserved	Reads 0	RO

12.12.4.7.8. Register Offset 0x323+32n: FANn_STATUS

Cooling IP Block Region 1 Sub-region 1 Register Offset 0x323+32n: FANn_STATUS

TABLE 149: COOLING IP BLOCK REGION 1 SUB-REGION 1 REGISTER OFFSET 0X323+32N: FANN_STATUS

Bit(s)	Name	Description	ATTR
0	PRESENT	Fan present and responding	RO
1	OVERSPEED	Latched: Measured RPM above FANn_RPM_MAX. (latched)	W1C
2	UNDERSPEED	Latched: Measured RPM below FANn_RPM_MIN. (latched)	W1C
3	FAULT	Latched: Device reported a fault. (latched)	W1C
4:7	Reserved	Reads 0	RO

12.12.4.8. Cooling Control Example Usage

This section provides usage examples that follow the operation rules as outlined in the section below.

Operating Rules

- The PD advertises supported fan count via FAN_COUNT.
- The BMC shall only access fan indices less than FAN_COUNT.
- The BMC programs FANn_I2C_ADDRESS and limits before enabling FANn_CONTROL.ENABLE.
- The PD periodically polls fan telemetry and updates status registers.

The following section below illustrates a set of concrete examples using the CSRs: DBG_I2C_CONTROL / STATUS / ADDR_CMD / REG / WDATA / RDATA registers.

Example 1: DBG I2C Read (read 2 bytes from device reg 0x10)

Goal: Read 2 bytes from I2C device address 0x2A register 0x10.

1. **Enable + set options**
    - DBG_I2C_CONTROL.DBG_I2C_ENABLE = 1

- DBG_I2C_CONTROL.USE_REG = 1
- DBG_I2C_CONTROL.REG_16BIT = 0
- (optional) speed select:
    - DBG_I2C_CONTROL.SPEED_SEL = 0 (default) or 1 (use DBG_I2C_CLKDIV)

2. **Program command header (DBG_I2C_ADDR_CMD)**
    - SLAVE_ADDR = 0x2A
    - ADDR_MODE = 0 (7-bit)
    - XFER_TYPE = 2 (WR_RD)
    - WLEN = 0 (no extra payload bytes after the register index)
    - RLEN = 2 (read 2 bytes)
    - SEND_STOP = 1 (recommended for single-shot debug)
    - PEC_ENABLE = 0 (unless you need SMBus PEC and it's supported)
3. **Program register index**
    - DBG_I2C_REG = 0x0010 (engine will send 0x10 because REG_16BIT=0)
4. **Start transaction**
    - Write 1 to DBG_I2C_CONTROL.START
5. **Poll for completion**
    - Poll DBG_I2C_STATUS.BUSY until it becomes 0
    - Then check DBG_I2C_STATUS.DONE == 1
6. **Check errors**
    - If DBG_I2C_STATUS.NACK == 1 → target didn't ACK (wrong address/reg, device not present, etc.)
    - If DBG_I2C_STATUS.TIMEOUT == 1 → bus stuck/timeout
    - If DBG_I2C_STATUS.BAD_CMD == 1 → invalid encoding (e.g., WLEN/RLEN out of range)
7. **Read the returned data**
    - Read DBG_I2C_RDATA
        - Byte packing is implementation-defined; commonly:
            - RDATA[7:0] = first byte read
            - RDATA[15:8] = second byte read
8. **Clear sticky status bits (W1C fields)**
    - Write 1 to clear DONE, and clear NACK/TIMEOUT/BAD_CMD/ARB_LOST if they latched.

Example 2: DBG I2C Write (write 1 byte value 0x55 to device reg 0x20)

Goal: Write 0x55 to I2C device 0x2A, register 0x20.

1. **Enable + set options**
   - DBG_I2C_CONTROL.DBG_I2C_ENABLE = 1
   - DBG_I2C_CONTROL.USE_REG = 1
   - DBG_I2C_CONTROL.REG_16BIT = 0
2. **Program command header (DBG_I2C_ADDR_CMD)**
   - SLAVE_ADDR = 0x2A
   - ADDR_MODE = 0
   - XFER_TYPE = 0 (WRITE)
   - WLEN = 1 (one payload byte)
   - RLEN = 0
   - SEND_STOP = 1
3. **Program register index**
   - DBG_I2C_REG = 0x0020
4. **Program payload**
   - DBG_I2C_WDATA = 0x00000055 (with 0x55 in the lowest byte, assuming little-endian packing)
5. **Start transaction**
   - Write 1 to DBG_I2C_CONTROL.START
6. **Poll and check result**
   - Wait for BUSY=0, then check DONE=1
   - If NACK=1, the device rejected address or a byte in the sequence (register index or data)
7. **Clear W1C status bits**
   - Clear DONE (and any error flags) by writing 1 to those bits.

12.12.4.9. Cooling Control IP Block CSRs Region 1 Sub-Region 2 Description

This section contains the definition for the Cooling IP block CSRs region 1 sub-region 2 which provides monitoring and safety enforcement for liquid-cooled systems. In addition to leak detection, this sub-region supports liquid flow monitoring for systems that integrate liquid cooling hardware within the server (e.g., LAAC).

The table below contains the definition for the Cooling IP block CSRs region 1 sub-region 2.

TABLE 150: COOLING SUBSYSTEM IP BLOCK CSRS REGION 2 SUB-REGION 2

Offset [Bytes]	Size [Bytes]	Name	Type	Description
0x500	2	DLC_CAPABILITIES	RO	Bitmap: LEAK_ZONES_CNT, SHUTDOWN_LOGIC.
0x502	2	DLC_GLOBAL_STATUS	RO	Master summary: LEAK_DETECTED.
0x504	2	DLC_GLOBAL_CONTROL	RW	ENABLE, CLEAR_LATCHED_FAULTS, AUTO_SHUTDOWN_ENABLE.
0x506	2	DLC_SHUTDOWN_MASK	RW	Selects which Leak Zones trigger an immediate hardware power-down.
0x508	2	DLC_FLOW_STATUS		DLC Flow Status Register
0x50A	2	DLC_FLOW_CONFIG		Optional Flow Configuration (Policy Only)
0x50C	4	DLC_FLOW_MIN_THRESHOLD		Minimum acceptable flow threshold Units are implementation-defined (e.g., pulses/sec, L/min × scale) Used only for fault detection
0x510+n	1	DLC_LEAK_ZONE[n]	RO	Status of individual leak ropes/spot sensors (Wet/Dry/Disconnect).
...	...	Reserved	RO	Reserved

All sensing and safety enforcement is performed in hardware. The BMC consumes status and configures policy through memory-mapped CSRs. Active control of pumps, valves, or flow rate is out of scope.

At the top level, the sub-region provides:

- Capabilities register to indicate supported DLC features, such as the number of leak zones and the presence of shutdown logic
- A global status register summarizing leak detection state across all zones
- A global control register to enable DLC logic, clear latched faults, and control automatic shutdown behavior
- A shutdown mask register to select which leak zones trigger an immediate hardware power-down
- Per-zone status registers that report the condition of individual leak sensors (e.g., dry, wet, open, short), including latched leak indications that persist until explicitly cleared

12.12.4.9.1. Register Offset 0x500: DLC_CAPABILITIES

Cooling IP Block Region 1 Sub-region 2 Register Offset 0x500: DLC_CAPABILITIES

TABLE 151: COOLING IP BLOCK REGION 1 SUB-REGION 2 REGISTER OFFSET 0X500: DLC_CAPABILITIES

Bit(s)	Name	Description	ATTR
0:3	LEAK_ZONES_CNT	Number of supported DLC leak zones (0–15) with bit 0 mapped to Zone0. The number of per-zone status registers implemented equals this value.	RO
4	SHUTDOWN_LOGIC	1 = Automatic hardware shutdown logic is implemented.	RO
5	FLOW_MONITORING	Liquid flow monitoring supported	RO
6	FLOW_AUTO_SHUTDOWN	Auto shutdown on flow fault supported	RO
7:15	Reserved	Reads 0	RO

12.12.4.9.2. Register Offset 0x502: DLC_GLOBAL_STATUS

Cooling IP Block Region 1 Sub-region 2 Register Offset 0x502: DLC_GLOBAL_STATUS

TABLE 152: COOLING IP BLOCK REGION 1 SUB-REGION 2 REGISTER OFFSET 0X502: DLC_GLOBAL_STATUS

Bit(s)	Name	Description	ATTR
0	LEAK_DETECTED	1 = At least one leak zone reports a wet or latched-wet condition.	RO
1	FLOW_FAULT	Liquid flow fault detected	RO
2	FLOW_PRESENT	Liquid flow detected	RO
3	FLOW_SENSOR_ERROR	Flow sensor fault	RO
4:15	Reserved	Reads 0	RO

12.12.4.9.3. Register Offset 0x504: DLC_GLOBAL_CONTROL

Cooling IP Block Region 1 Sub-region 2 Register Offset 0x504: DLC_GLOBAL_CONTROL

TABLE 153: COOLING IP BLOCK REGION 1 SUB-REGION 2 REGISTER OFFSET 0X504: DLC_GLOBAL_CONTROL

Bit(s)	Name	Description	ATTR
0	ENABLE	1 = Enable DLC monitoring logic. When 0, DLC status registers may remain readable but leak evaluation is disabled.	RW
1	CLEAR_LATCHED_FAULTS	Write 1 to clear all latched leak indications in all zones. Self-clearing.	RW
2	AUTO_SHUTDOWN_ENABLE	1 = Enable automatic hardware shutdown when a masked leak zone detects a leak.	RW
3	FLOW_MON_ENABLE	Enable liquid flow monitoring	RW
4	CLEAR_FLOW_FAULT	Clear latched flow fault	RW
5:15	Reserved	Reads 0	RO

12.12.4.9.4. Register Offset 0x506: DLC_SHUTDOWN_MASK

Cooling IP Block Region 1 Sub-region 2 Register Offset 0x506: DLC_SHUTDOWN_MASK

TABLE 154: COOLING IP BLOCK REGION 1 SUB-REGION 2 REGISTER OFFSET 0X506: DLC_SHUTDOWN_MASK

Bit(s)	Name	Description	ATTR
n	ZONE_SHUTDOWN_ENABLE	1 = Leak zone *n* triggers hardware shutdown when AUTO_SHUTDOWN_ENABLE is set and the zone reports a wet or latched condition	RW
n+1:15	Reserved	Reads 0	RO

12.12.4.9.5. Register Offset 0x508: DLC_FLOW_STATUS

Cooling IP Block Region 1 Sub-region 2 Register Offset 0x508: DLC_FLOW_STATUS

TABLE 155: COOLING IP BLOCK REGION 1 SUB-REGION 2 REGISTER OFFSET 0X508: DLC_FLOW_STATUS

Bit(s)	Name	Description	ATTR
0	FLOW_PRESENT	Flow detected above minimum threshold	RO
1	FLOW_BELOW_MIN	Flow below acceptable threshold (latched)	W1C
2	FLOW_SENSOR_ERROR	Sensor fault or missing	W1C
3	FLOW_UNKNOWN	Flow status invalid or not yet sampled	RO
4:15	Reserved	Reads 0	RO

12.12.4.9.6. Register Offset 0x50A: DLC_FLOW_CONFIG

Cooling IP Block Region 1 Sub-region 2 Register Offset 0x50A: DLC_FLOW_CONFIG

TABLE 156: COOLING IP BLOCK REGION 1 SUB-REGION 2 REGISTER OFFSET 0X50A: DLC_FLOW_CONFIG

Bit(s)	Name	Description	ATTR
0	FLOW_MON_ENABLE	Enable flow monitoring	RW
1	AUTO_SHUTDOWN_ON_FLOW	Shutdown if flow fault occurs	W1C
2	CLEAR_FLOW_FAULT	Clear latched flow fault	W1C
3:15	Reserved	Reads 0	RO

Note: This register defines policy only. It does not control pumps, valves, or flow rate.

12.12.4.9.7. Register Offset 0x510+n: DLC_LEAK_ZONE[n]

Cooling IP Block Region 1 Sub-region 2 Register Offset 0x510+n: DLC_LEAK_ZONE[n]

TABLE 157: COOLING IP BLOCK REGION 1 SUB-REGION 2 REGISTER OFFSET 0X510+N: DLC_LEAK_ZONE[N]

Bit(s)	Name	Description	ATTR
0:2	STATUS	000=**DRY** (Normal), 001=**WET** (Leak Detected), 010=**OPEN** (Cable Disconnected/Cut), 011=**SHORT** (Sensor Fault).	RO
3	LATCHED_WET	1 = Leak occurred and is latched until CLEAR_LATCHED_FAULTS is written.	RO
4:7	Reserved	Reads 0	RO

Operating Rules for Flow Monitoring

- Flow monitoring is optional and advertised via DLC_CAPABILITIES.
- When enabled, the PD periodically samples flow sensor inputs.
- Flow loss while the system is enabled shall assert FLOW_FAULT.
- The BMC may clear latched faults after correcting the condition.

2.12.4.10. Cooling Control IP Block CSRs Region 1 Sub-Region 3 Description

This sub-region is vendor defined.

2.12.4.11. Cooling Control IP Block Region 2 CSR Memory Map

Reserved

2.12.4.12. Cooling Control IP Block Region 3 CSR Memory Map

Reserved for vendor defined usage.

12.12.5. Cooling Subsystem Control Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.12.6. Cooling Subsystem Control Example Implementation

Example implementations, if available, may be found in supplementary materials referenced in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

12.13. System Telemetry Collection Logical IP Block

This section contains the logical IP block description for System Telemetry Management requirements and descriptive abstraction for this IP block. This entire IP block is fully optional.

System telemetry can include one or more sensors, virtual or physical, that it has direct or indirect control over for the purpose of providing telemetry information to the BMC. These sensors could be directly on the HPM planar, or it can be connected to the HPM via external wiring. The PD could provide a mechanism by which the BMC can access sensors, or the PD can access the sensor in an autonomous manner to collect telemetry data on behalf of the BMC.

There are several use cases that the Telemetry IP block can be used for.

- BMC triggered (e.g., one-shot, as needed) collection of telemetry by the PD via one or more interfaces to one or more sensors (e.g., power, temperature, etc.) either on board the HPM or remotely connected to peripherals.
- Support for autonomous PD collection of telemetry via one or more interfaces to one or more sensors (e.g., power, temperature, etc.) either on-board the HPM or remotely connected to peripherals.
- Vendor defined telemetry support.

Additional use cases are also possible as this is not an exclusive list.

This section will outline what is required to expose these capabilities and provide this through the use of the self-describing JSON schema framework.

12.13.1. System Telemetry Collection Conceptual Overview

This section contains an overview of the System Telemetry Management functionality for a typical set of use cases.

The intention of the system telemetry management IP block is to provide the BMC with a common set of control and status interfaces for controlling one or more system telemetry components on either the HPM or connected peripherals and options.

The figure below provides a simple example telemetry collection via the PD (shown as an FPGA).

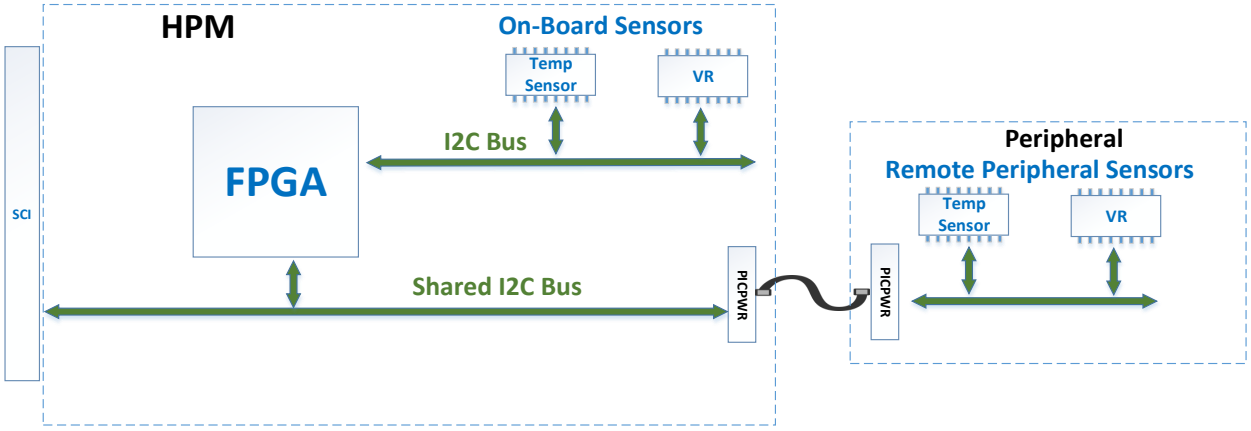


FIGURE 55: TELEMETRY COLLECTION IP BLOCK USE CASES

In this figure, the IP block provides telemetry collection and potential aggregation on behalf of the BMC. Collection can occur via a BMC triggering mechanism, or it can occur through the use of a telemetry collection state machine to drive autonomous data collection from one or more sensors. If data collection occurs via a shared interface, such as I2C connected to both the BMC and PD, then it is required that both support multi-master capabilities.

It is required that the PD implementor describe the usage and control of this telemetry hardware interfaces and that it indicates the I2C master supports multi-master mode via the PD JSON file.

12.13.2. System Telemetry Collection IP Block Architecture

This section contains the general abstract architecture for System Telemetry Management IP block.

The diagram below shows how the Telemetry Collection IP block is used to support the I2C bus as primary interface example to sensors.

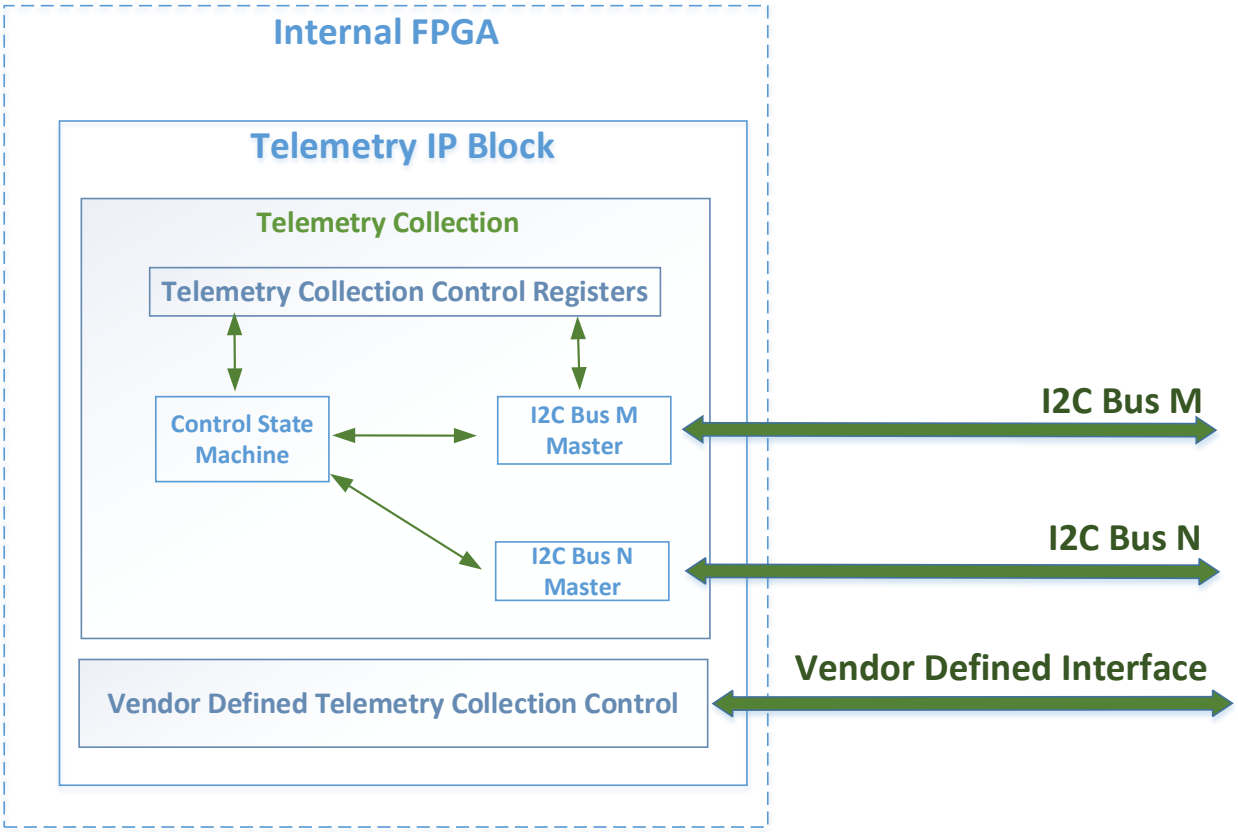


FIGURE 56: TELEMETRY COLLECTION IP BLOCK ARCHITECTURAL BLOCK DIAGRAM

As shown in the figure above, the Telemetry Collection IP block within the PD, here shown as an FPGA, example shows the usage of I2C as the primary bus interface to one or more sensors. The Telemetry Collection control registers handle the configuration and either enables triggered or continuous autonomous polling of the I2C connected sensors. This example uses I2C as the system bus that interconnects sensors with the PD. The choice of bus connected to sensors is up to the implementer. The main requirement is that the specific bus that is utilized shall be disclosed in the JSON self-describing file for both the PD and HPM.

12.13.3. System Telemetry Collection IP Block Requirements

This section contains the general abstract architecture for the System Telemetry Management IP block.

12.13.3.1. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 158: TELEMETRY IP BLOCK REQUIRED AND OPTIONAL FEATURES LIST

Name	REQ	OPT	Description
Triggered Telemetry Collection		Y	PD collection of telemetry from one or more sensors via BMC manually triggering PD to perform data collection.
Autonomous Telemetry Collection		Y	PD autonomous collection of telemetry from one or more sensors via
Shared Bus Multi-master Support		Y	Shared PD and BMC bus interface (e.g., I2C) to one or more sensors shall have multi-master support if both are possible bus masters.
Vendor Defined Telemetry Collection		Y	Support for controlling vendor defined telemetry data collection via non-standard vendor defined interface(s).

12.13.4. System Telemetry Collection IP Block CSR Requirements

This section contains the Control and Status Registers (CSR) requirements for standardized System Telemetry Management functionality.

12.13.4.1. System Telemetry Collection CSR Region Overview

The System Telemetry Collection CSR regions are divided into a set of ranges. These ranges are all offset from the TELEMETRY_IP_BLOCK base address. This base address is defined by the PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 159: TELEMETRY IP BLOCK CSR MEMORY MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x00FF	Global Registers	General IP block level registers

1	0x0100	0x0BFF	System Telemetry Collection IP Block Interface Control and Status Registers	System Telemetry Collection IP Block Control and Status Registers.
2	0x0C00	0x0FFF	Reserved	Reserved for future use
3[21]	0x1000	0xFFFF	OEM SECTION	Vendor defined usage.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.13.4.2. System Telemetry Collection IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 160: SYSTEM TELEMETRY IP BLOCK REGION 0 REGISTER MAP

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

[21] Note: Implementor can use this for additional regions as needed.

12.13.4.2.1. System Telemetry Collection IP Block Common Registers Sub-Region 0 Description

The table below contains the definition for the System Telemetry Collection IP block common registers sub-region 0.

TABLE 161: SYSTEM TELEMETRY IP BLOCK REGION 0 SUB-REGION 0 REGISTER MAP

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each).
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state indication that requires a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	R/W	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x08 – 0x0F	Reserved			

12.13.4.2.2. System Telemetry Collection IP Block Common Registers Sub-Region 1 Description

Reserved

12.13.4.3. System Telemetry Collection IP Block Region 1 CSR Memory Map

The System Telemetry Collection region 1 registers start at offset 0x100 from the base address for the System Telemetry Collection CSR space.

This region is divided into two subregions.

TABLE 162: SYSTEM TELEMETRY IP BLOCK REGION 1 REGISTER MAP

Sub-Region	Range	Name	Description
0	0x0100 – 0x07FF	System Telemetry Collection CSRs	Set of registers that handle System Telemetry Collection functions.
1	0x0800 – 0x0FFF	Reserved	Set of registers reserved for future use.

It is required that the PD implementor specifies within the JSON file which ‘sub-regions are present and supported.

12.13.4.3.1. System Telemetry Collection CSRs Region 1 Sub-Region 0 Description

The table below contains the definition for the Cooling IP block CSRs region 1 sub-region 0.

This document specifies a streamlined set of PD Control and Status Registers (CSRs) for controlling PD telemetry collection. The System Telemetry Collection IP block is an optional module that allows the HPM PD to gather and expose sensor data (telemetry) to the BMC. It can interface with physical or virtual sensors located on the HPM board or connected via external wiring. The PD can provide a single read or automated continuous readings and provides mins, maxes, and an accumulator/sample count to accommodate different BMC data usage models while keeping the PD logic simple.

Register Map

TABLE 163: SYSTEM TELEMETRY IP BLOCK REGION 1 SUB-REGION 0 REGISTER MAP

Register Name	Size	Access	Description
Sensor_n_CTRL (n = 0…N-1)	8 Bytes	R/W	Byte 0 Control. Bit field encoded. 0xXXXXXXX1 = Perform a 1 time read. 0xXXXXXX1X = Reset Minimum Value. 0xXXXXX1XX = Reset Maximum Value. 0xXXXX1XXX = Reset Failure Count. Bytes 1-4 0x00000001 to 0xFFFFFFFF =Sensor periodic read interval in milliseconds. 0x00000000 = do not periodically read Bytes 5-7 Reserved
Sensor_n_STATUS (n = 0…N-1)	24 Bytes	R	Byte 0 Status. 0x00 = Valid. 0x01 = Failed. 0x02 = Missing. 0x03 to 0xFF = Reserved Byte 1-2 Current Value Byte 3-4 Minimum Value Byte 5-6 Maximum Value Byte 7-14 Accumulator Byte 15-20 Sample Count. Incremented on every successful sensor read. Byte 21-22 Failure count. Incremented on every failed read attempt. This includes both failed and missing. Maxes out at 0xFFFF Bytes 23 Reserved

Implementation Notes

- The Sample Count, Current Value, Minimum Value, Maximum Value, and Accumulator are updated on every successful sensor read. Both for periodic and 1-time reads.

    - The control byte (byte 0) is bit encoded. If Bit 0 is a 1, it means "perform a 1 time read". If Bit 1 is a 1, it means "Reset the minimum value". If Bit 2 is a 1, it means "reset the maximum value". If Bit 4 is a 1, it means "reset the failure count". With a single write, any or all of the 4 allowed actions can be indicated. This is standard bit encoded notation.
    - Examples: Bytes 1-4 specify the periodic read interval in milliseconds, as stated. With 0x00000000 meaning do not read. As an example, if the desire is to read the sensor once every second, then 0x000003E8 would be loaded. If the sensor is to be read once every minute, then 0x0000EA60 would be loaded. If the sensor is to be read once every day, then 0x05265C00 would be loaded.

- The 1 time read bit should be cleared by the PD when the read is complete and the status bytes have been updated.
- The Min/Max/failure count reset bit should be cleared by the PD when the min/max/failure count values have been cleared.
- The PD shall ensure that the values being returned in the status are coherent. Meaning that the status bytes are not in the process of being updated.
    - For instance, the PD could have 2 copies of these registers and always return the values from the currently stable structure.
    - This tick/tock buffer design should be transparent to the sensor reader (eg. BMC)
- To perform a 1-time sensor read:
    - Set control byte 0 to 0x01
    - Read control byte 0 until bit 0 is 0
    - Read sensor status
- To know that the sensor status has been updated, compare the current sample count to the prior sample count
- To know the total number of reads attempted over an interval, use the sample count plus failure count differences over that interval.
- To get an average for a periodically read senor value over a period of X ms:
    - Read the status and store the sample count and accumulator
    - Wait X ms
    - Read the status again
    - Average = Accumulator difference/Sample Count difference

12.13.4.3.2. System Telemetry Collection IP Block CSRs Region 1 Sub-Region 1 Description

Reserved

12.13.4.4. System Telemetry Collection IP Block Region 2 CSR Memory Map

Reserved

12.13.4.5. System Telemetry Collection IP Block Region 3 CSR Memory Map

Reserved for vendor defined usage.

12.13.5. System Telemetry Collection Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.13.6. System Telemetry Collection Example Implementation

Example implementations, if available, may be found in supplementary materials referenced in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

12.14. General Use GPIOs IP Block

This section contains the general use GPIOs IP block description for a generalized PD GPIO control use case. This entire IP block is fully optional.

12.14.1. General Use GPIOs IP Block Overview

This section contains an overview of the general use PD GPIOs IP block functionality for a typical set of use cases.

The intention of the general use PD GPIOs IP block is to provide a generically controlled set of GPIOs connected to physical PD signals on the HPM that the BMC can directly manipulate.

12.14.2. General Use PD GPIOs IP Block Architecture

This section contains the general abstract architecture for a general use PD GPIOs IP block.

The diagram below contains a conceptual diagram of how the General Use PD GPIOs IP block can be implemented.

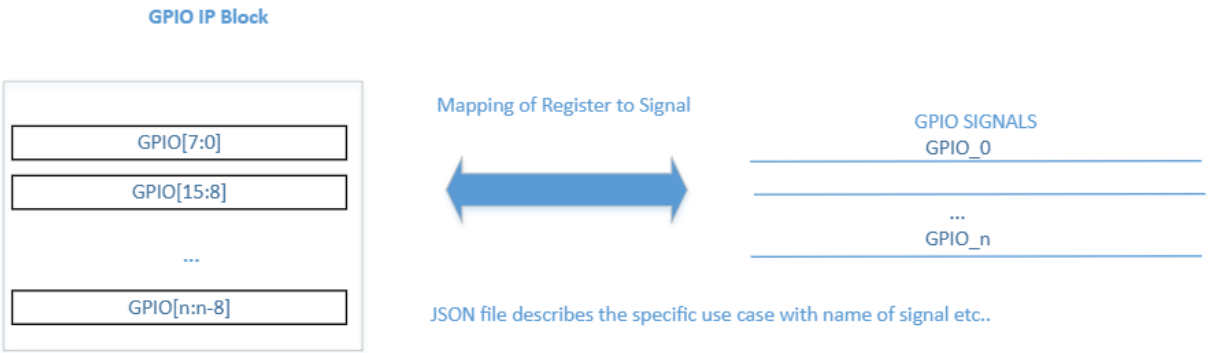


FIGURE 57: GPIO IP BLOCK ARCHITECTURAL DIAGRAM

As shown in the figure above, the General Use PD GPIOs IP block contains a set of registers that are mapped to one or more physical signals. Each register contains the necessary configuration settings to configure the GPIO for general purpose usage. The specific definition and usage of those signals are wholly defined. The vendor may define these within the JSON file.

12.14.3. General Use PD GPIOs IP Block Requirements

This section contains the general abstract architecture for the General Use PD GPIOs IP block.

12.14.3.1. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 164: GENERAL PURPOSE I/O IP BLOCK REQUIRED AND OPTIONAL FEATURES LIST

Name	REQ	OPT	Description
GPIO Directional Configuration	Y		It is required that this IP block implements GPIO directional control.
GPIO Drive	Y		It is required that this IP block implements GPIO drive control.
GPIO State	Y		It is required that this IP block implements GPIO state indication.
GPIO As Interrupt Source		Y	The GPIO may be used as trigger for interrupts from this IP block.

12.14.3.2. Special Purpose Signals Implemented Within GPIO IP Block

The table below contains the set of required and optional special purpose signals implemented by this IP block.

TABLE 165: SPECIAL PURPOSE SIGNALS IMPLEMENTED IN GPIO IP BLOCK

Name	DIR	W	REQ	OPT	Description
HPM FRU WP Override	Out	1'b0		Y	The HPM FRU write protection override signal control bit.

12.14.4. General Use PD GPIOs IP Block CSR Requirements

This section contains the Control and Status Registers (CSR) requirements for standardized General Use case PD GPIOs functionality.

12.14.4.1. General Use PD GPIOs CSR Region Overview

The General Use PD GPIOs CSR regions are divided into a set of ranges. These ranges are all offsets from the GPIOS_IP_BLOCK base address. This base address is defined by the CPLD/PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 166: GENERAL USE GPIO IP BLOCK CSRS MEMORY MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x00FF	Global Registers	General IP block level registers
1	0x0100	0x0BFF	General Use PD GPIOs IP Block Interface Control and Status Registers	General Use PD GPIOs IP Block Control and Status Registers.
2	0x0C00	0x0FFF	Reserved	Reserved for future use
3[22]	0x1000	0xFFFF	OEM SECTION	Vendor defined usage.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

[22] Note: Implementor can use this for additional regions as needed.

12.14.4.2. General Use PD GPIOs IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 167: GENERAL USE GPIO IP BLOCK REGION 0 REGISTER MAP

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

12.14.4.2.1. General Use PD GPIOs IP Block Common Registers Region 0 Sub-Region 0 Description

The table below contains the definition for the General Use PD GPIOs IP block common registers sub-region 0.

TABLE 168: GENERAL USE GPIO IP BLOCK REGION 0 SUBRGION 0 REGISTER MAP

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each).

0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state indication that requires a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	R/W	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x08 – 0x0F	Reserved			Reserved for future use.

12.14.4.2.2. General Use PD GPIOs IP Block Common Registers Region 0 Sub-Region 1 Description

Reserved

12.14.4.3. General Use PD GPIOs IP Block Region 1 CSR Memory Map

The General Use PD GPIOs region one registers start at offset 0x100 from the base address for General Use PD GPIOs CSR space.

This region is divided into two subregions.

TABLE 169: GENERAL USE GPIO IP BLOCK REGION 1 REGISTER MAP

Sub-Region	Range	Name	Description
0	0x0100 – 0x07FF	General Use PD GPIOs CSRs	Set of registers that handle General Use PD GPIOs functions.
1	0x0800 – 0x0FFF	Reserved	Set of registers reserved for future use.

It is required that the PD implementor specifies within the JSON file which ‘sub-regions are present and supported.

12.14.4.3.1. General Use PD GPIOs CSRs Region 1 Sub-Region 0 Description

The table below contains the definition for the General Use PD GPIOs IP block CSRs region 1 sub-region 0.

12.14.4.3.2. General Use GPIO Register Offset 0x00: General GPIO Status Register

This section contains the general GPIO status register contents.

TABLE 170: GENERAL USE GPIO IP BLOCK REGION 1 SUBRGION 0 REGISTER 0X00

Bit(s)	Name	Description	ATTR	W	Notes
7:3	RESERVED	Reserved	R	5'b0	
2	LOGICAL_STATE	1 = Signal is Logically High 0= Signal is Logically Low	R	1'b0	Used by FW

1	PHYSICAL_STATE	1 = Signal is Physically High 0 = Signal is Physically Low	R	1'b0	Useful for debug
0	RESERVED	Reserved	R	1'b0	

12.14.4.3.3. General Use GPIO Register Offset 0x01: General GPIO Control

This section contains the general GPIO control register contents.

TABLE 171: GENERAL USE GPIO IP BLOCK REGION 1 SUBRGION 0 REGISTER 0X01

Bit(s)	Name	Description	ATTR	Size	Notes
7:6	INT_TRIGGER	Interrupt Behavior 0 = Both rising and falling edge 1 = rising edge only 2= falling edge only 3 =reserved	RW	1'b0	Mapping to interrupt from this source shall be implemented using the internal virtual wire mapping framework.
5	INT_EN	1 = Enabled 0 = Disabled	RW	1'b0	
4	LOGICAL_POLARITY	GPIO Logical Polarity 1= Active High 0 = Active Low	RW	1'b0	Used by logic to trigger interrupt behavior.
3:2	PHYSICAL_POLARITY	GPIO Polarity 0 = IN, 1 = OUT, 2 = BIDIR, 3 = high-Z	RW	2'b0	
1	DRIVE	GPIO Drive 0 = Open Drain 1 = Push Pull	RW	1'b0	Select drive type for GPIO

0	GPIO State	1 = Asserted 0 = De-Asserted	RW	1'b0	The GPIO output signal state indication if it is in GPO mode, or configuration dependent if configured as output or bidirectional.

12.14.4.3.4. General Use PD GPIOs IP Block CSRs Region 1 Sub-Region 1 Description

Reserved

12.14.4.4. General Use PD GPIOs IP Block Region 2 CSR Memory Map

Reserved

12.14.4.5. General Use PD GPIOs IP Block Region 3 CSR Memory Map

Reserved for vendor defined usage.

12.14.5. General Use PD GPIOs Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.14.6. General Use PD GPIOs Example Implementation

Example implementations, if available, may be found in supplementary materials referenced in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

12.15. Host Access IP Block

This section contains the general use Host Access IP block description for a generalized host exclusive and BMC shared memory use case. This entire IP block is fully optional.

12.15.1. Host Access IP Block Architecture

This section contains the general abstract architecture for a general use Host Access IP block. The diagram below contains a block conceptual diagram of the Host Access IP block.

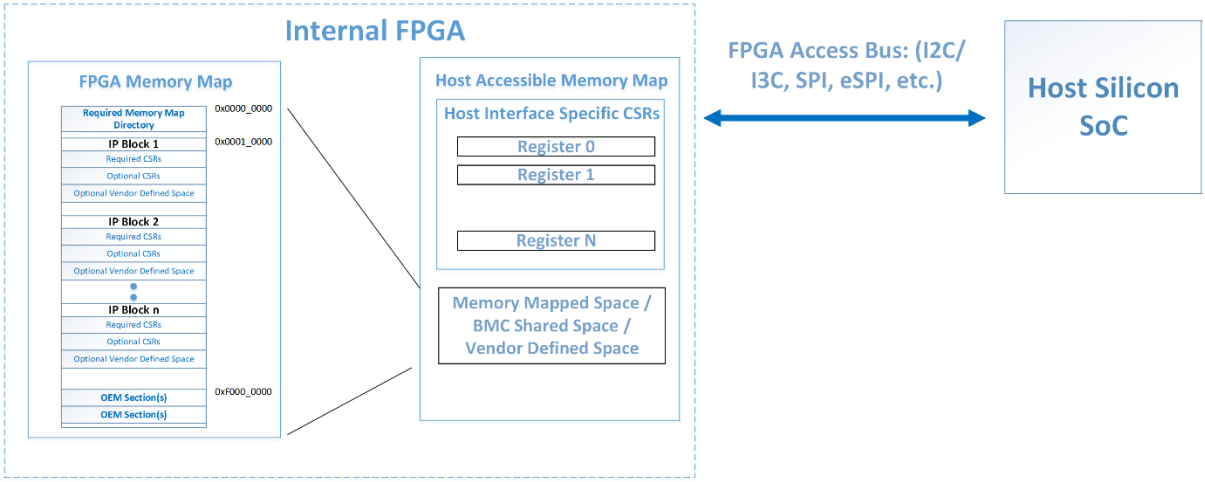


FIGURE 58: HOST ACCESS IP BLOCK ARCHITECTURAL DIAGRAM

As shown in the figure above, the Host Access IP block within the programmable device, here shown as an FPGA, contains a set of registers that are mapped to either BMC exclusive, BIOS exclusive, or shared regions.

12.15.2. Host Access IP Block Requirements

This section contains the general abstract architecture for the Host Access IP block.

12.15.2.1. General Requirements

The following are a set of general requirements for host register access.

There exists two classes of bus interfaces that can provide PD access:

- Low level that is available pre-boot and does not require BIOS configuration
  - Examples: I2C, eSPI, SPI, etc.
- Higher level that requires some BIOS configuration and may not be fully available pre-boot
  - Examples: PCIe, USB, etc.

It is recommended that system designers carefully consider boot phase availability considerations when choosing a particular interface. It is also important to consider bandwidth requirements. If the amount of traffic is minimal then a heavier weight interface may not be appropriate.

The next section outlines specific requirements for each of the commonly used buses for PD interface.

12.15.2.2. eSPI Interface Usage Requirements

The following section outlines requirements for host access to the PD via the eSPI bus.

- Act as eSPI slave device as outlined in this specification*
- At a minimum the PD shall provide a memory window from eSPI host accessible space into host accessible CSRs
- Optional ALERT signal is recommended
- It is required that the PD JSON indicate which buses are host access capable

*Note: See the eSPI IP block chapter 12.16.

12.15.2.3. I2C / I3C Interface Usage Requirements

The following section outlines requirements for host access to the PD via the I2C/I3C bus.

- Act as I2C / I3C slave device as outlined in this specification*
- At a minimum the PD shall provide a memory window from I2C / I3C host accessible space into host accessible CSRs
- Optional ALERT signal is recommended
    - This can be a standard SMB_ALRT_ signal or a designated vendor defined GPIO from the PD
    - It is required to indicate a specific signal designated for this mechanism in both the HPM and PD JSON file
- It is required that the PD JSON indicates which buses are host access capable
- It is required that the HPM JSON file provides the I2C / I3C bus address within the bus description.
    - It is required that the BMC provides this address during early boot via payload described in the boot process via MMBI. See section M-PNP.FRU_DISCOVERY_BOOT specification for details.

*Note: See the I2C IP block chapter 12.7.

12.15.2.4. SPI Requirements

The following section outlines requirements for host access to the PD via the SPI bus.

- Act as SPI slave device as outlined in this specification*
- At a minimum the PD shall provide a memory window from SPI host accessible space into host accessible CSRs
- Optional ALERT signal is recommended
  - It is left to the vendor to choose a alert signaling mechanism to the host
  - It is required to indicate a specific signal designated for this mechanism in both the HPM and PD JSON file
- It is required that the PD JSON indicates which buses are host access capable

*Note: See the SPI IP block chapter 12.6.

12.15.2.5. PCIe Requirements

The following section outlines requirements for host access to the PD via the PCIe bus.

- Shall follow the PCIe common endpoint compliant to the PCIe spec.
  - Specific parameters such as bus width are vendor defined
- At a minimum the PD shall provide a memory window from PCIe host accessible space into host accessible CSRs
- Optional ALERT signal is recommended
- It is required that the PD JSON indicates which buses are host access capable
- All other PCIe bus parameters, such as bus width, are vendor defined but shall be indicated in both the HPM JSON and the PD JSON file.

This specification does not describe the usage of a PCIe endpoint as an IP block. Addition of PCIe as an IP block is vendor defined.

12.15.2.6. USB Requirements

The following section outlines requirements for host access to the PD via the USB bus.

- Shall follow the USB common endpoint compliant to the USB spec.
- At a minimum the PD shall provide a memory window from USB host accessible space into host accessible CSRs
- Optional ALERT signal is recommended
- It is required that the PD JSON indicates which buses are host access capable
- All other USB bus parameters, such as bus version, are vendor defined but shall be indicated in either the HPM JSON and / or the PD JSON file.

This specification does not describe the usage of a USB endpoint as an IP block. Addition of USB as an IP block is vendor defined.

12.15.2.7. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

Required:

- Global registers set for BMC access
- One or more regions that are host accessible via a host accessible bus

Optional:

- Shared memory space between host and BMC
- Vendor defined space

12.15.3. Host Access IP Block CSR Requirements

This section contains the Control and Status Registers (CSR) requirements for standardized Host Access functionality.

12.15.3.1. Host Access CSR Region Overview

The Host Access CSR regions are divided into a set of ranges. From a BMC viewpoint of the PD memory map, these ranges are all offsets from the HOST_ACCESS_IP_BLOCK base address. This base address is defined by the CPLD/PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 172: HOST ACCESS IP BLOCK REGION REGISTER MAP

Region	Range Bottom	Range Top	Sections	Name	Description
0	0x0000	0x0FFF	2	Global Registers	Read/Write for BMC Not accessible for BIOS
1	0x1000	Vendor Defined	1	Host Accessible CSRs	Read Only for BMC Read/Write for BIOS Host BAR maps to this region
2	Vendor Defined	Vendor Defined	Vendor Defined	Shared CSR Space	Read/Write for BMC Read/Write for BIOS
3*	Vendor Defined	Vendor Defined	Vendor Defined	Vendor Defined Space	Vendor Defined

*Note: Implementors can use this for additional regions as needed.

All vendor defined ranges shall be provided in the JSON file memory map.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.15.3.2. Host Access IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 173: HOST ACCESS IP BLOCK REGION 0 REGISTER MAP

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

12.15.3.2.1. Host Access IP Block Common Registers Sub-Region 0 Description

The table below contains the definition for the Host Access IP block common registers sub-region 0.

TABLE 174: HOST ACCESS IP BLOCK REGION 0 SUBREGION 0 REGISTER MAP

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each).

0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state indication that requires a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	R/W	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x07 – 0x0F	Reserved			Reserved for future use.

12.15.3.2.2. Host Access IP Block Common Registers Sub-Region 1 Description

Reserved

12.15.3.3. Host Access IP Block Region 1 CSR Memory Map

The Host Access region one registers start at offset 0x1000 from the base address Host Access CSR space.

It is required that the PD implementor specifies within the JSON file which ‘sub-regions are present and supported.

12.15.3.3.1. Host Access CSRs Region 1 Sub-Region 0 Description

This region is designated as a primarily host accessible CSR space. The BMC can read these CSRs but not write to them.

The definition of these registers is vendor defined.

12.15.3.3.2. Host Access IP Block CSRs Region 1 Sub-Region 1 Description

Reserved

12.15.3.4. Host Access IP Block Region 2 CSR Memory Map

This region is designated as shared memory space between the host and BMC and is optional.

Usage details between Host / BIOS and BMC are outside the scope of this specification.

12.15.3.5. Host Access IP Block Region 3 CSR Memory Map

Reserved for vendor defined usage.

12.15.4. Host Access JSON Requirements

The following section outlines the information required in the HPM and PD JSON files to adequately advertise the PD host access capabilities.

- It is required that the HPM JSON file in HPM FRU contains attributes indicating that the Host can access PD registers via each bus with that supported functionality.
- It is required that the PD JSON file contains this information:
  - Specific host access IP block
  - Indication of accessibility to the IP block for every bus that can access it
  - Provide CSR list in memory map for the host access IP block
  - Indicate which registers are host exclusive vs. the mapping of shared space with BMC within the memory map
  - Host accessible registers description
  - Shared registers description
- The BIOS shall be informed by the BMC during the boot process of the bus and access mechanisms for the PD register space
  - BAR advertisement will be covered in the subsequent section
  - The PD will provide the BAR within the JSON file
- The PD implementor should ensure that the host only accessible registers to have a definition in the PD JSON file.

12.15.5. BAR Advertisement

The BMC shall provide the BIOS during early boot which bus the BIOS can use to access the PD. The BMC shall provide the BAR and access mechanism for the PD, below shown as an FPGA, memory space. It is required that the BAR maps directly to the host accessible register space, Region 1.

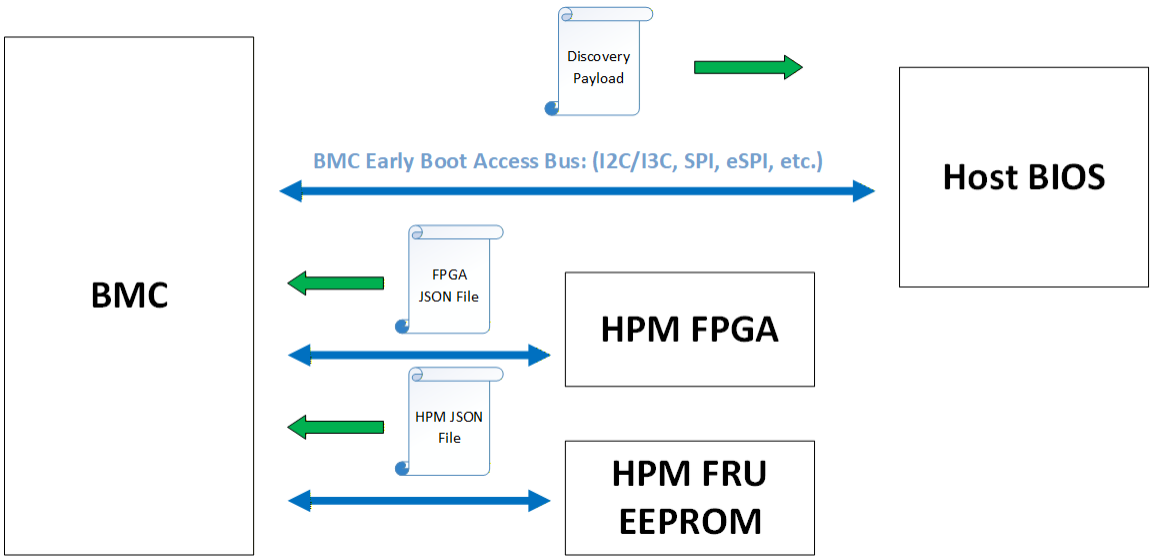


FIGURE 59: EARLY BOOT DISCOVERY PAYLOAD SIMPLIFIED FLOW DIAGRAM

The diagram above provides a simplified depiction for how the host access interface BAR and other information is provided to the host BIOS by the BMC during the early boot phase. During discovery, the JSON files are collected by the BMC prior to the boot process. During the early boot phase, the BIOS will initiate communications with the BMC over an interface that is available during early boot to collect the discovery payload. This is facilitated using the host - BMC MMBI protocol. See the M-PNP.FRU_DISCOVERY_BOOT specification for details on the discovery process and usage of MMBI payloads to transport early boot information between the host and BMC.

12.15.6. Host Access Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.15.7. Host Access Example Implementation

Example implementations, if available, may be found in supplementary materials referenced in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

12.16. eSPI IP Block

This section contains the general use eSPI IP block description for a bus interface for host or BMC access to the PD. This entire IP block is fully optional.

12.16.1. eSPI IP Block Architecture

This section contains the general abstract architecture for a general use eSPI IP block. The diagram below contains a block conceptual diagram of the eSPI IP block.

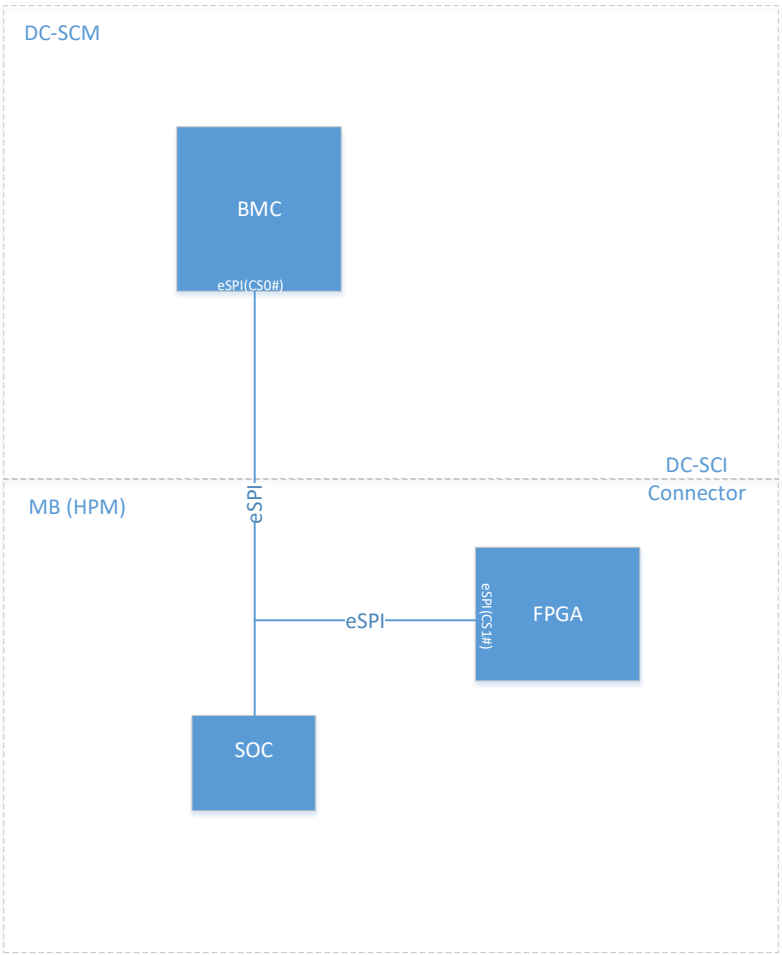


FIGURE 60: ESPI SYSTEM ARCHITECTURAL DIAGRAM

In the figure above, the eSPI Chip select 0 (CS0) connects to the BMC located on the DC-SCM and chip select 1 (CS1) connects to the Programmable Device, here shown as an FPGA, located on the HPM.

The block diagram below illustrates the eSPI interface internal legacy style Low Pin Count (LPC) style registers from the perspective of accessing this interface via the host CPU.

LPC (Low Pin Count) style register set definitions have been used by eSPI devices to emulate the legacy register sets as described in LPC based Super-IO specifications and this follows a specific definition. Please refer to those example specifications for more historical information.

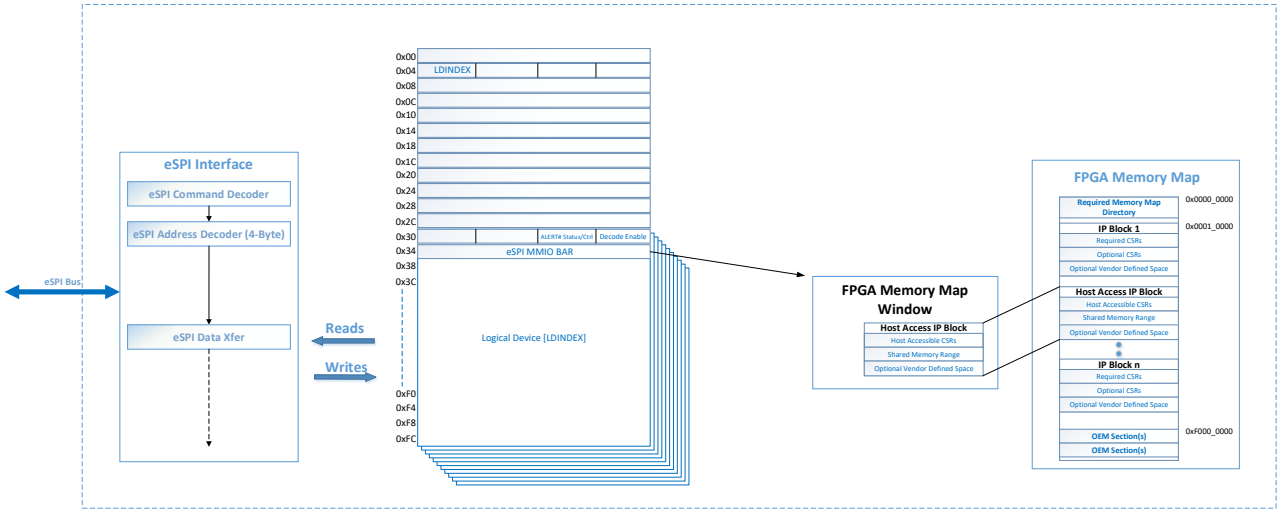


FIGURE 61: ESPI IP BLOCK INTERNAL ARCHITECTURE

As shown in the figure above, the eSPI IP block maps a set of registers using the BAR as referenced in the logical device 0 registers for the purpose of allowing the host to access the PD memory map. These registers are internal to the PD, here shown as an FPGA, and are referenced through the eSPI interface of the PD.

12.16.2. eSPI IP Block Requirements

This section contains the general abstract architecture for the eSPI IP block.

12.16.2.1. General Requirements

The following are a set of general requirements for the eSPI IP block.

- For Host access, the eSPI interface can provide a window into the Host Access IP Block CSR space or provide full access to the PD memory map. It is recommended that this range be carefully chosen and limited from a security perspective.

- The BMC can view the PD IP block registers via the memory map accessible using another interface such as LTPI data channel, I2C, SPI or other mechanism.
- Host access to PD registers will use eSPI bus CS1#.
- System BIOS (e.g., UEFI) will set up the SOC's architectural specific registers to enable a memory range (MMIO) to be forwarded down to eSPI CS1# using peripheral channel. This host-based mechanism is SoC / CPU specific and is outside the scope of this specification.
- The ALERT# signaled by the PD to the host will use in-band eSPI signaling. ALERT# support (status/enable) is defined in the eSPI PD IP block.
- The PD will use LPC Device 0 for the base address register (BAR), BAR enable, and the ALERT# enable / status register.
- The host (e.g., UEFI BIOS) will use index I/O 0x2E/0x2F, 0x4E/0x4F to set up the 32-bit BAR (LPC Device 0 – 0x34) and enable the decode (LPC Device 0 – 0x30 -Bit 0). The assignment of a suitable I/O address used for mapping CS1# is left to the vendor / system integrator.
- For all practical purposes, the eSPI MMIO BAR behaves much like a PCIe BAR. Writing 0's and then reading that register back will determine the size / range of the address space. System BIOS (e.g., UEFI) will use this value to comprehend the decode range.
- It is left to the vendor / system integrator to decide what internal PD registers are accessible via the eSPI interface.

12.16.2.2. Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

Required:

- Global registers set within BMC registers space
- LPC style common register set for host CPU access
- LPC style device 0 register set mapping to host accessible space using a BAR mechanism

Optional:

- Additional usage of an eSPI bus to other system components. This is outside the scope of this specification.
- Vendor defined space

For required eSPI capabilities, configuration features, and requirements, please consult the eSPI specification for details. For error handling and failure indication functionality please consult the eSPI specification for details.

12.16.3. eSPI IP Block CSR Requirements

This section contains the Control and Status Registers (CSR) requirements for standardized eSPI functionality.

12.16.3.1. eSPI CSR Region Overview

The eSPI CSRs are categorized by either using the interface from the host or BMC perspective.

- Host accessible
- BMC accessible

Accessing register space via the eSPI interface requires usage of on an ‘LPC’ like legacy register set. The next section describes this register set.

12.16.3.2. eSPI LPC Style Host Accessible Register Set Overview

This section provides and overview of the eSPI LPC style registers as usable by the host CPU. The figure below contains a conceptual diagram of this register set.

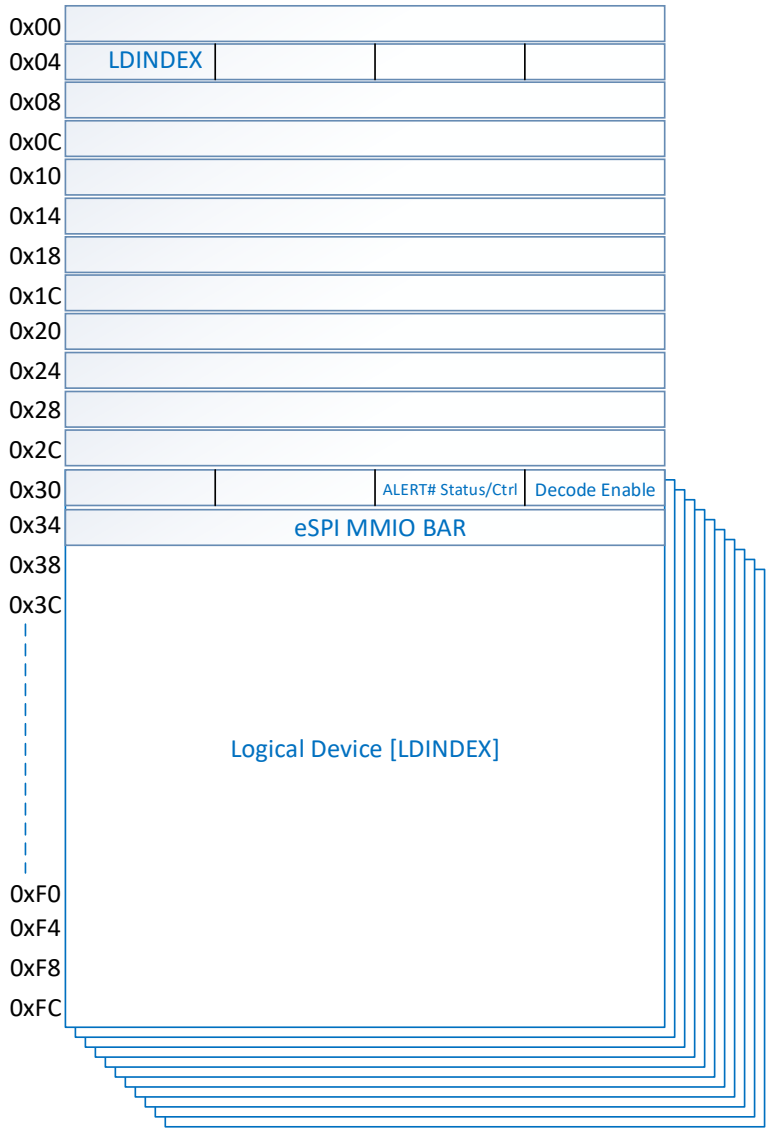


FIGURE 62: ESPI DEVICE REGISTER SPACE

Host will use standard LPC programming practices to program eSPI MMIO BAR and ALERT# control/status bits. LDINDEX register at index 0x07 shall be set to device 0(0x00), then device 0 index 0x30/0x34 can be configured.

12.16.3.2.1. Example Programmatic Flow

This section outlines an example programmatic flow for configuring the eSPI LPC-style registers for usage by the host.

Example: Host sets MMIO BAR with a 64K size to 0xF8000000 and enables the decode range.

//Set LDINDEX register to 0x00

Host writes to I/O address 0x2E (in host I/O space) a value of 0x07

Host writes to I/O address 0x2F (in host I/O space) a value of 0x00

// Host writes 0's to the BAR Register to determine BAR register decode size

Host writes to I/O address 0x2E (in host I/O space) a value of 0x34

Host writes to I/O address 0x2F (in host I/O space) a value of 0x00

Host writes to I/O address 0x2E (in host I/O space) a value of 0x35

Host writes to I/O address 0x2F (in host I/O space) a value of 0x00

Host writes to I/O address 0x2E (in host I/O space) a value of 0x36

Host writes to I/O address 0x2F (in host I/O space) a value of 0x00

Host writes to I/O address 0x2E (in host I/O space) a value of 0x37

Host writes to I/O address 0x2F (in host I/O space) a value of 0x00

// Read BAR Register to get size

Host writes to I/O address 0x2E (in host I/O space) a value of 0x34

Host reads from I/O address 0x2F (in host I/O space)

Host writes to I/O address 0x2E (in host I/O space) a value of 0x35

Host reads from I/O address 0x2F (in host I/O space)

Host writes to I/O address 0x2E (in host I/O space) a value of 0x36

Host reads from I/O address 0x2F (in host I/O space)

Host writes to I/O address 0x2E (in host I/O space) a value of 0x37

Host reads from I/O address 0x2F (in host I/O space)

// Host programs BAR Register

Host writes to I/O address 0x2E (in host I/O space) a value of 0x36

Host writes to I/O address 0x2F (in host I/O space) a value of 0x00

Host writes to I/O address 0x2E (in host I/O space) a value of 0x37

Host writes to I/O address 0x2F (in host I/O space) a value of 0xF8

// Host Enables BAR Register

Host writes to I/O address 0x2E (in host I/O space) a value of 0x30

Host writes to I/O address 0x2F (in host I/O space) a value of 0x01

12.16.3.3. eSPI LPC Style Register Set Definition

This section defines the eSPI LPC style registers as used by the host CPU.

12.16.3.3.1. eSPI Global Register Set Definition

This section defines the eSPI LPC style global registers set.

LPC Global Index 0x10: Vendor ID Register

Name	Bitfield	Type	Bit Field Definition
Vendor ID Register	31:0	R	Shall follow PCIe Vendor ID definition

This register is required.

LPC Global Index 0x14: Device ID Register

Name	Bitfield	Type	Bit Field Definition

Device ID Register	31:0	R	Shall follow PCIe Device ID definition

This register is required.

LPC Global Index 0x20: LPC Device ID Register

Name	Bitfield	Type	Bit Field Definition
LPC Device ID Register	7:0	R	Vendor defined

This register is required.

LPC Global Index 0x21: LPC Device Revision Register

Name	Bitfield	Type	Bit Field Definition
LPC Device Revision Register	7:0	R	Major revision of device as assigned by vendor.

This register is required.

12.16.3.3.2. eSPI Logical Device 0 Register Set Definition

This section defines the eSPI LPC style logical device 0 register set.

LPC Device 0 Index 0x30: Host BAR Decode Enable

Name	Bitfield	Type	Bit Field Definition
eSPI Host BAR Decode Enable	7:1	RW	Reserved – writes shall not take any effect.
	0	RW	**1 = Enabled, 0 Disabled**

This register is required.

LPC Device 0 Index 0x31: Host ALERT# Status/Control Register

Name	Bitfield	Type	Bit Field Definition

eSPI Host In-Band ALERT# Status/Control Register	7:1	RW	Reserved – writes shall not take any effect.
	1	RW	**Latched ALERT# Interrupt Status Indication - Write 1 to clear**
	0	RW	**1 = Enabled, 0 Disabled**

This register is required.

LPC Device 0 Index 0x34: Host BAR Decode Enable

Name	Bitfield	Type	Bit Field Definition
eSPI Host BAR Register	31:4	RW	Defines the high-order bits of the physical address. The number of writable bits determines the region size (e.g., bits 31:12 writable = 4 KB region)
	3:0	R	4'b0000

This register is required.

The eSPI MMIO BAR register is used to provide a mapping window into PD memory map. The implementor should limit the mapping of the Host Access IP block CSR space for security reasons. Usage of logical devices 1 – N is vendor defined.

The diagram below depicts the mapping of the memory map window as located by the BAR in logical device 0 to the host accessible region within the PD memory map.

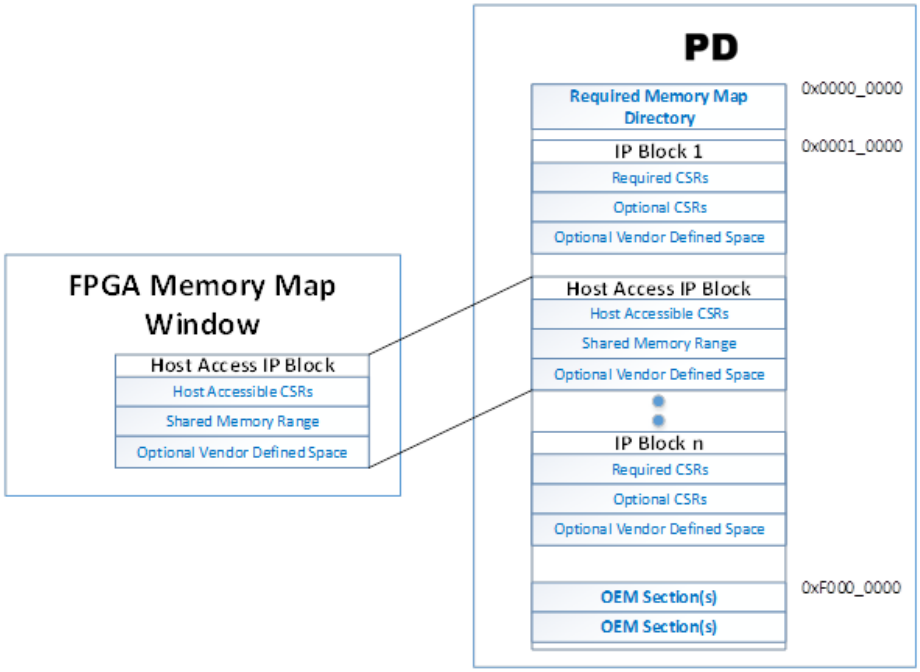


FIGURE 63: PD MEMORY MAP WINDOW

Note: It is recommended that the host only has access to the host access IP block.

12.16.3.4. eSPI BMC Accessible Register Set Overview

The BMC accessible eSPI IP block CSRs are part of the PD memory map which is accessible via other interfaces such as LTPI or other general PD memory map access interfaces such as I2C or SPI.

Access register ranges below:

TABLE 175: ESPI IP BLOCK REGISTER MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x00FF	Global Registers	General IP block level registers

1	0x0100	0x01FF	Mirror Host LPC Registers	This mirrors the host LPC register space for BMC to access. Mostly would use this for debug purposes.
2	0x0200	0x0FFF	Reserved	Reserved for future use
3[23]	0x1000	0xFFFF	OEM SECTION	Vendor defined usage.

*Note: Implementor can use this for additional regions as needed.

It is required that all vendor defined ranges be provided in the JSON file memory map.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.16.3.4.1. eSPI IP Block BMC Accessible Region 0 CSR Memory Map

This region is divided into two subregions.

TABLE 176: ESPI IP BLOCK REGION 0 REGISTER MAP

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

[23] Note: Implementor can use this for additional regions as needed.

12.16.3.4.1.1. eSPI CSRs Region 0 Sub-Region 0 Description

The table below contains the definition for the eSPI IP block common registers sub-region 0.

TABLE 177: ESPI IP BLOCK REGION 0 SUBREGION 0 REGISTER MAP

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each).
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state indication that requires a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	R/W	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x08 – 0x0F	Reserved			Reserved for future use.

12.16.3.4.2. eSPI IP Block BMC Accessible Region 1 CSR Memory Map

The eSPI IP block region 1 registers start at offset 0x100 from the base address for the eSPI IP block CSR space.

This region is divided into two subregions.

TABLE 178: ESPI IP BLOCK REGION 1 REGISTER MAP

Sub-Region	Range	Name	Description
0	0x0100 – 0x01FF	Mirror Host LPC Registers	Only for BMC to read the registers for debug purposes.
1	0x0200 – 0x0FFF	Reserved	Set of registers reserved for future use.

It is required that the PD implementor specifies within the JSON file which ‘sub-regions are present and supported.

12.16.3.4.2.1. eSPI CSRs Region 1 Sub-Region 0 Description

The eSPI CSRs region 1 sub-region 0 is a mirror of the LPC host registers.

It is required that the BMC only have read access as this is for debug only.

12.17. M-PESTI Target IP Block

This section contains the M-PESTI Target IP block description for peripheral programmable device use case. This entire IP block is fully optional.

12.17.1. M-PESTI Target IP Block Overview

This section contains an overview of the M-PESTI Target IP block functionality for a typical set of use cases.

The intention of the general use M-PESTI Target IP block is to provide a definition for peripheral programmable devices a M-PNP specific target payload, a deterministic definition for peripheral virtual wires, and set of common CSRSs that the BMC can directly manipulate via a bus that provides CSR memory space access. This is complementary to the discovery process usage of I2C for MHS component discovery.

12.17.2. M-PESTI Target IP Block Architecture

This section contains the general abstract architecture for a M-PESTI Target IP block.

The diagram below contains an example conceptual diagram of how the an M-PESTI target IP block is connected within an peripheral CPLD.

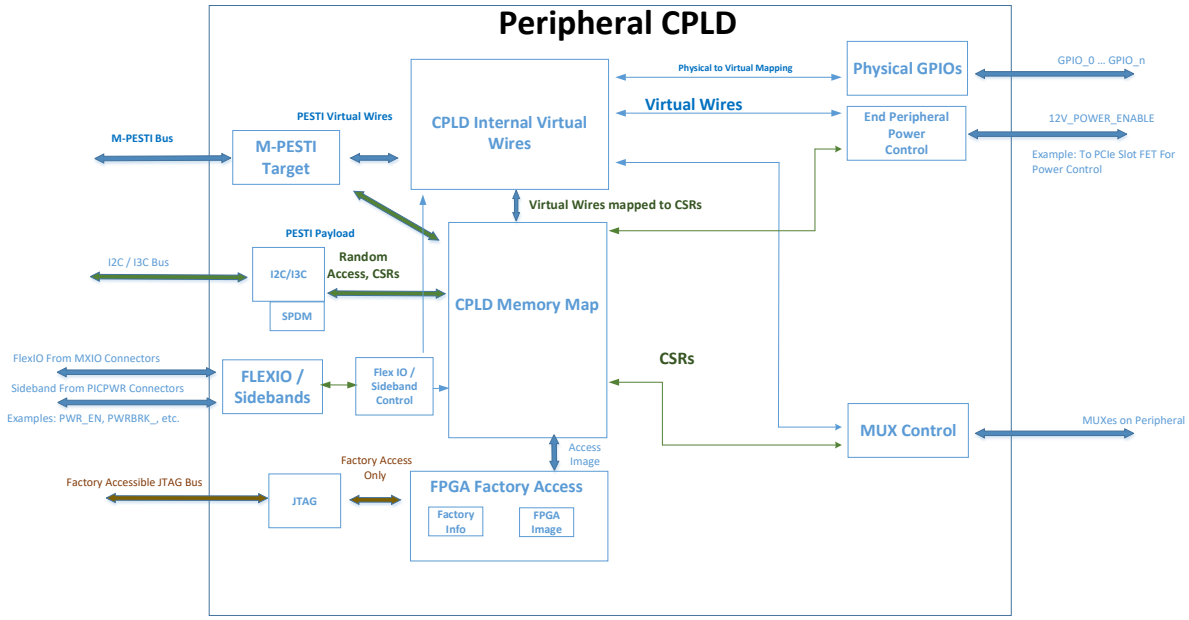


FIGURE 64: EXAMPLE PERIPHERAL CPLD BLOCK DIAGRAM WITH M-PESTI TARGET IP BLOCK

As shown in the figure above, the M-PESTI IP block contains a set of registers that contain M-PESTI target payload data, virtual wires, and other CSRs.

12.17.3. M-PESTI Target IP Block Requirements

This section contains the general abstract architecture for the M-PESTI Target IP block.

The common set of goals for this proposal is outlined below.

- Create common PD interface for MPESTI initiator to interact with M-PESTI target in peripheral PLD
- Common virtual wire format
- Common JSON description of virtual wires
- register set guidance

12.17.3.1. M-PESTI Target Required and Optional Feature Set

The table below contains the set of required and optional recommended features implemented by this IP block.

TABLE 179: MPESTI TARGET REQUIRED AND OPTIONAL FEATURES

Name	REQ	OPT	Description
M-PESTI Target Payload Version 3	Y		M-PESTI specific target payload version 3 supporting M-PNP features.
Virtual Wires Type 1		Y	Only required if supporting virtual wires for specifically named peripheral device classes and subclasses.

12.17.4. M-PESTI Target Discovery Payload Version 3

This section contains the M-PNP required discovery payload details for M-PESTI targets.

TABLE 180: MPESTI DISCOVERY PAYLOAD VERSION 3

Byte #	Byte Offset	7	6	5	4	3	2	1	0
1	00h	Payload Format Version [7:0] = 03h for MPNP							
2	01h	Discovery Payload Size [7:0]							
3	02h	Vendor ID [15:8]							
4	03h	Vendor ID [7:0]							
5	04h	Device Class [7:0]							
6	05h	Device ID [15:8]							
7	06h	Device ID [7:0]							
8	07h	Reserved					AP_PEC_SUPPORT	VENDOR_ID_FORMAT	
9	08h	Device Subclass [7:0]							
10	09h	Reserved			VWIRE_FMT_TYPE[3:0]				
11	0Ah	Reserved			VWIRE_FMT_VERSION[3:0]				
12	0Bh	Reserved			NUM_VIRTUAL_WIRE_OUTPUT_BYTES [3:0]				
13	0Ch	Reserved			NUM_VIRTUAL_WIRE_INPUT_BYTES [3:0]				
14	0Dh	Device Version [7:0]							
15	0Eh	MCU/CPLD Major Version [7:0]							
16	0Fh	MCU/CPLD Minor Version [7:0]							
17	10h	Reserved						PCA Revision [3:0]	
18-23	11h – 16h	Reserved							
24	17h	Padding / Checksum							

12.17.4.1. M-PESTI Target Discovery Payload Version 3 Definitions

Payload field definition provided in the following table.

TABLE 181: MPESTI PAYLOAD VERSION 3 DEFINITION

Field Name	Description
Discovery Payload Size [7:0]	Value in multiples of 8 bytes.
Vendor ID [15:0]	Same as PCI vendor id. This is needed because some earlier MPESTI payload versions did not include this.
Device Class [7:0]	Device Class ID As Per M-PNP FRU_DISCOVERY_BOOT specification.
Device Subclass [7:0]	Device Subclass ID As Per M-PNP FRU_DISCOVERY_BOOT specification.
Device ID[15:0]	
AP_PEC_SUPPORT	Some MPESTI payloads did not include this so this is added.
VENDOR_ID_FORMAT	Some MPESTI payloads did not include this so this is added.
VWIRE_FMT_TYPE [3:0]	Virtual Wire Payload Type: Type 0 indicates it is vendor defined in JSON file Type 1 is single byte low latency defined as per this spec Type 2 – 31 Reserved
NUM_VIRTUAL_WIRE_OUTPUT_BYTES [3:0]	Indicates the number of virtual wire output bytes supported by the peripheral.
VWIRE_FMT_VERSION [3:0]	Version of virtual wire type definition
NUM_VIRTUAL_WIRE_INPUT_BYTES [3:0]	Indicates the number of virtual wire input bytes supported by the peripheral.
Device Version [7:0]	Programmable Device Hardware Version

MCU/CPLD Major Version [7:0]	Programmable Device Image Major Version
MCU/CPLD Minor Version [7:0]	Programmable Device Image Minor Version
PCA Revision [3:0]	Peripheral board version

12.17.5. M-PESTI Target Virtual Wire Format Type 1 Formatting Definition

This section contains the M-PESTI Target virtual wire Format Type 1 formatting definition and requirements for standardized M-PESTI Target functionality.

12.17.5.1. M-PESTI Target Virtual Wire Format Type 1 General Format Definitions

The general format definition is outlined below

OUT = FROM HPM

IN = TO HPM

VWIRE OUT

7	6	5	4	3	2	1	0
VWO_3	VWO_2	VWO_1	VWO_0	RSVD	Command		

Up to 8 commands

VWO_[3:0] = Virtual Wire Output Signal Bits

Definition changes per command encoding and per device class / subclass.

JSON file defines which commands give you which virtual wires

VWIRE IN

7	6	5	4	3	2	1	0
VWI_3	VWI_2	VWI_1	VWI_0	HB	Command		

HB = Heartbeat (optional for CPLDs, mandatory for MCU)

VWI_[3:0] = Virtual Wire Input Signal Bits

Definition changes per command encoding and per device class / subclass.

Command is echoed back

MCU heartbeat needed in event of firmware failure causing one or more cores to fail

12.17.5.2. M-PESTI Target Virtual Wire Format Type 1 General Command List

The general command list definition is outlined below

TABLE 182: MPESTI VIRTUAL WIRE TYPE 1 FORMAT

Command	Command	Description
0x00	Echo Command	The initiator sends a pattern to the target in the upper nibble and then the target repeats this pattern in the upper nibble along with toggling the heartbeat bit.
0x01	Power Management	Virtual wires defined to perform power related functionality. Examples include power rail control and power fault signaling.
0x02	Fault Detect	Virtual wires defined to provide more fault information from the target.
0x03	Cable Detect or Vendor Defined	Virtual wires defined to provide cable detect functionality for peripherals such as PCIe CEM Risers, OCP NIC floating risers, Backplanes, and others. This is reserved as vendor defined for certain device classes.
0x04	Configuration	Virtual wires defined to provide configuration functionality.
0x05	Location Index	Vendor defined index within a box or section specific to a device class (e.g. backplane #, box # section #)

0x06	Device Class / Subclass Specific	device class / subclass specific or vendor defined
0x07	Device Class / Subclass Specific	device class / subclass specific or vendor defined

12.17.5.3. M-PESTI Target Virtual Wire Format Type 1 Command List Per Peripheral Device Class and Subclass

The command list definition is outlined in this section per device class and subclass.

12.17.5.3.1. Fixed Riser Device Class 0x00 Subclass 0x00 Command List and Definition

The fixed riser virtual wire command list and definitions are outlined in the tables below.

TABLE 183: MPESTI VIRTUAL WIRE TYPE 1 FORMAT COMMAND LIST

Command	Command Code (Nibble)	Req?	Description
Echo	0x00	Y	echo returns the same pattern
Power	0x01	Y	
Reserved	0x02		cable detect was removed as not needed for fixed riser
Configuration	0x03	Y	JTAG / USB Timing or Timing Sync MUX selectors. All others reserved.
Location	0x04		Location index for where IO device is positioned in a specific box or section within the chassis. Selects what value is returned by the target.

			Location_Nibble_Index[3:0]: Index 0: Box Lower Nibble Index 1: Box Upper Nibble Index 2: Section Lower Nibble Index 3: Section Upper Nibble 4 – 15 reserved
Vendor Defined	0x05		
Vendor Defined	0x06		
Vendor Defined	0x07		

TABLE 184: MPESTI TARGET VIRTUAL WIRE FORMAT TYPE 1 VOUT COMMAND DEFINITION

VOUT [0] Command Definition									
Command	**Command Code (Nibble)**	**Bit 7**	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power	0x01	V RSVD	V RSVD	SLOT_AUX_EN	MAIN_EN 1 = ON 0 = OFF	RSVD	Command		
Reserved	0x02						Command		
Configuration	0x03	JTAG(0), USB2 MODE(1)	EEPROM_WP 1 = ON 0 = OFF	RFU	RFU	RSVD	Command		
Location Index	0x04	LOCATION_INDEX[3:0] (see command definition)				RSVD	Command		
Vendor Defined	0x05						Command		

Vendor Defined	0x06						Command
Vendor Defined	0x07						Command

TABLE 185: MPESTI TARGET VIRTUAL WIRE FORMAT TYPE 1 VIN COMMAND DEFINITION

VIN [0] Virtual Wire Definition									
		Bit 7	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power	0x01	Mappable Fault	TEMP_ALERT	3V_AUX_Fault	Main Fault	Heartbeat	Command		
Reserved	0x02						Command		
Configuration	0x03	JTAG(0), USB2 MODE(1)	EEPROM_WP 1 = ON 0 = OFF	RFU	RFU	RSVD	Command		
Location Value	0x04	LOCATION[3:0] Box or Section Nibble (see command definition)				RSVD	Command		
Vendor Defined	0x05						Command		
Vendor Defined	0x06						Command		
Vendor Defined	0x07						Command		

12.17.5.3.2. Floating Riser Device Class 0x00 Subclass 0x01 Command List and Definition

The floating riser virtual wire command list and definitions are outlined in the tables below.

TABLE 186: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X00 SUBCLASS 0X01 COMMAND DEFINITION

Command	Cmd Code (Nibble)	Req?	Description
Echo	0x00	Y	echo returns the same pattern
Power	0x01		
Cable Detect	0x02		You map CBL_DET_X to the appropriate slot / cable you want to in your JSON file.
Configuration	0x03		Reserved for Future Use
Location	0x04		Location index for where IO device is positioned in a specific box or section within the chassis. Selects what value is returned by the target. Location_Nibble_Index[3:0]: Index 0: Box Lower Nibble Index 1: Box Upper Nibble Nibble Index 2: Section Lower Nibble Index 3: Section Upper
Vendor Defined	0x05		
Vendor Defined	0x06		
Vendor Defined	0x07		

TABLE 187: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X00 SUBCLASS 0X01 VOUT COMMAND DEFINITION

VOUT [0] Command Definition

Command	Command Code (Nibble)	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power	0x01	V RSVD	V RSVD	SLOT_AUX_EN	MAIN_EN	RSVD	Command		
Cable Detect	0x02	CBL_DET_D	CBL_DET_C	CBL_DET_B	CBL_DET_A	RSVD	Command		
Configuration	0x03	JTAG(0), USB2 MODE(1)	EEPROM_WP 1 = ON 0 = OFF	RFU	RFU	RSVD	Command		
Location Index	0x04	Location_Nibble_Index[3:0]				RSVD	Command		
Vendor Defined	0x05						Command		
Vendor Defined	0x06						Command		
Vendor Defined	0x07						Command		

TABLE 188: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X00 SUBCLASS 0X01 VIN COMMAND DEFINITION

VIN [0] Virtual Wire Definition									
		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	B 2	B 1	B 0
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power	0x01	Mappable Fault	TEMP_ALERT	3V_AUX_Fault	Main Fault	RSVD	Command		
Cable Detect	0x02	SLOTB_CBL_DET	SLOTA_CBL_DET	SLOTD_CBL_DET	SLOTC_CBL_DET	RSVD	Command		
Configuration	0x03	JTAG(0), USB2 MODE(1)	EEPROM_WP 1 = ON 0 = OFF	RFU	RFU	RSVD	Command		

Location Index	0x04	Reserved				RSVD	Command
Vendor Defined	0x05						Command
Vendor Defined	0x06						Command
Vendor Defined	0x07						Command

12.17.5.3.3. OCP NIC Floating Riser Device Class 0x00 Subclass 0x02 Command List and Definition

The OCP NIC floating riser virtual wire command list and definitions are outlined in the tables below. This is for a single x16 that can be bifurcated.

TABLE 189: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X00 SUBCLASS 0X02 COMMAND DEFINITION

Command	Cmd Code (Nibble)	Req?	Description
Echo	0x00	Y	echo returns the same pattern
Power	0x01		
Cable Detect	0x02		You map CBL_DET_X to the appropriate slot / cable you want to in your JSON file.
Configuration	0x03		Reserved for Future Use - are there any other bits we should define?
Location	0x04		Location index for where IO device is positioned in a specific box or section within the chassis. Selects what value is returned by the target. Location_Nibble_Index[3:0]: Index 0: Box Lower Nibble

			Index 1: Box Upper Nibble Index 2: Section Lower Nibble Index 3: Section Upper
Slot ID	0x05		
Vendor Defined	0x06		
Vendor Defined	0x07		

TABLE 190: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X00 SUBCLASS 0X02 VOUT COMMAND DEFINITION

VOUT [0] Command Definition									
Command	**Command Code (Nibble)**	**Bit 7**	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power	0x01	V RSVD	V RSVD	SLOT_AUX_EN	MAIN_EN	RSVD	Command		
Cable Detect	0x02	CBL_DET_D	CBL_DET_C	CBL_DET_B	CBL_DET_A	RSVD	Command		
Configuration	0x03	JTAG(0), USB2 MODE(1)	EEPROM_WP 1 = ON 0 = OFF	RFU	RFU	RSVD	Command		
Location Index	0x04	Location_Nibble_Index[3:0]				RSVD	Command		
Slot ID / Bifurcation	0x05	Bifurcation [1:0]		Slot ID [1:0]			Command		
Vendor Defined	0x06						Command		
Vendor Defined	0x07						Command		

TABLE 191: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X00 SUBCLASS 0X02 VIN COMMAND DEFINITION

VIN [0] Virtual Wire Definition								
		Bit 7	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**B2**	**B1**
Echo	0x00	Echo[3:0]				Heartbeat	Command	
Power	0x01	Mappable Fault	TEMP_ALERT	3V_AUX_Fault	Main Fault	RSVD	Command	
Cable Detect	0x02	SLOTB_CBL_DET	SLOTA_CBL_DET	SLOTD_CBL_DET	SLOTC_CBL_DET	RSVD	Command	
Configuration	0x03	JTAG(0), USB2 MODE(1)	EEPROM_WP 1 = ON 0 = OFF	RFU	RFU	RSVD	Command	
Location Index	0x04	Reserved				RSVD	Command	
Slot ID / Bifurcation	0x05	Bifurcation [1:0]		Slot ID [1:0]		RSVD	Command	
Vendor Defined	0x06						Command	
Vendor Defined	0x07						Command	

12.17.5.3.4. Double Wide OCP NIC Floating Riser Device Class 0x00 Subclass 0x03 Command List and Definition

The Double Wide OCP NIC floating riser virtual wire command list and definitions are outlined in the tables below. This is for a two x16 combined as a double wide slot that also can be bifurcated.

TABLE 192: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X00 SUBCLASS 0X03 COMMAND DEFINITION

Command	Cmd Code (Nibble)	Req?	Description
Echo	0x00	Y	echo returns the same pattern
Power	0x01		
Cable Detect	0x02		You map CBL_DET_X to the appropriate slot / cable you want to in your JSON file.
Configuration	0x03		Reserved for Future Use
Location	0x04		Location index for where IO device is positioned in a specific box or section within the chassis. Selects what value is returned by the target. Location_Nibble_Index[3:0]: Index 0: Box Lower Nibble Index 1: Box Upper Nibble Index 2: Section Lower Nibble Index 3: Section Upper
Slot ID / Bifurcation	0x05		
Vendor Defined	0x06		
Vendor Defined	0x07		

TABLE 193: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X00 SUBCLASS 0X03 VOUT COMMAND DEFINITION

VOUT [0] Command Definition									
Command	**Command Code (Nibble)**	**Bit 7**	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		

Power	0x01	V RSVD	V RSVD	SLOT_AUX_EN	MAIN_EN	RSVD	Command
Cable Detect	0x02	CBL_DET_D	CBL_DET_C	CBL_DET_B	CBL_DET_A	RSVD	Command
Configuration	0x03	JTAG(0), USB2 MODE(1)	EEPROM_WP 1 = ON 0 = OFF	RFU	RFU	RSVD	Command
Location Index	0x04	Location_Nibble_Index[3:0]				RSVD	Command
Slot ID / Bifurcation	0x05	Bifurcation [1:0]		Slot ID [1:0]			Command
Vendor Defined	0x06						Command
Vendor Defined	0x07						Command

TABLE 194: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X00 SUBCLASS 0X03 VIN COMMAND DEFINITION

VIN [0] Virtual Wire Definition									
		Bit 7	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**B 2**	**B 1**	**B 0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power	0x01	Mappable Fault	TEMP_ALERT	3V_AUX_Fault	Main Fault	RSVD	Command		
Cable Detect	0x02	SLOTB_CBL_DET	SLOTA_CBL_DET	SLOTD_CBL_DET	SLOTC_CBL_DET	RSVD	Command		
Configuration	0x03	JTAG(0), USB2 MODE(1)	EEPROM_WP 1 = ON 0 = OFF	RFU	RFU	RSVD	Command		
Location Index	0x04	Reserved				RSVD	Command		
Slot ID / Bifurcation	0x05	Bifurcation [1:0]		Slot ID [1:0]		RSVD	Command		

Vendor Defined	0x06						Command
Vendor Defined	0x07						Command

12.17.5.3.5. Non-UBM Backplane Device Class 0x02 Subclass 0x00 Command List and Definition

The Non-UBM Backplane virtual wire command list and definitions are outlined in the tables below.

TABLE 195: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X02 SUBCLASS 0X00 COMMAND DEFINITION

Command	Cmd Code (Nibble)	Req?	Description
Echo	0x00	Y	echo returns the same pattern
Power	0x01		
Cable Detect	0x02		You map CBL_DET_X to the appropriate slot / cable you want to in your JSON file.
Configuration	0x03		
Location Index	0x04		
Vendor Defined	0x05		
Vendor Defined	0x06		
Vendor Defined	0x07		

TABLE 196: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X02 SUBCLASS 0X00 VOUT COMMAND DEFINITION

VOUT [0] Command Definition									
Command	**Command Code (Nibble)**	**Bit 7**	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power	0x01	V RSVD	V RSVD	SLOT_AUX_EN	MAIN_EN	RSVD	Command		
Cable Detect	0x02	CBL_DET_D	CBL_DET_C	CBL_DET_B	CBL_DET_A	RSVD	Command		
Configuration	0x03	V RSVD	EEPROM_WP 1 = ON 0 = OFF	V RSVD	V RSVD	RSVD	Command		
Location Index	0x04	Location_Nibble_Index[3:0]				RSVD	Command		
Vendor Defined	0x05						Command		
Vendor Defined	0x06						Command		
Vendor Defined	0x07						Command		

TABLE 197: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X02 SUBCLASS 0X00 VIN COMMAND DEFINITION

VIN [0] Virtual Wire Definition									
		Bit 7	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		

Power	0x01	Mappable Fault	TEMP_ALERT	3V_AUX_Fault	Main Fault	RSVD	Command
Cable Detect	0x02	CBL_DET_D	CBL_DET_C	CBL_DET_B	CBL_DET_A	RSVD	Command
Configuration	0x03	V RSVD	EEPROM_WP 1 = ON 0 = OFF	SLOT_AUX_EN	MAIN_EN	RSVD	Command
Location Index	0x04	Location[3:0]				RSVD	Command
Vendor Defined	0x05						Command
Vendor Defined	0x06						Command
Vendor Defined	0x07						Command

12.17.5.3.6. UBM Backplane Device Class 0x02 Subclass 0x01 Command List and Definition

The UBM Backplane virtual wire command list and definitions are outlined in the tables below.

TABLE 198: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X02 SUBCLASS 0X01 COMMAND DEFINITION

Command	Cmd Code (Nibble)	Req?	Description
Echo	0x00	Y	echo returns the same pattern
Power	0x01		
Cable Detect	0x02		You map CBL_DET_X to the appropriate slot / cable you want to in your JSON file.

Configuration	0x03		
Location Index	0x04		
Vendor Defined	0x05		
Vendor Defined	0x06		
Vendor Defined	0x07		

TABLE 199: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X02 SUBCLASS 0X01 VOUT COMMAND DEFINITION

VOUT [0] Command Definition									
Command	**Command Code (Nibble)**	**Bit 7**	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power	0x01	V RSVD	V RSVD	SLOT_AUX_EN	MAIN_EN	RSVD	Command		
Cable Detect	0x02	CBL_DET_D	CBL_DET_C	CBL_DET_B	CBL_DET_A	RSVD	Command		
Configuration	0x03	V RSVD	EEPROM_WP 1 = ON 0 = OFF	V RSVD	V RSVD	RSVD	Command		
Location Index	0x04	Location_Nibble_Index[3:0]				RSVD	Command		
Vendor Defined	0x05						Command		
Vendor Defined	0x06						Command		
Vendor Defined	0x07						Command		

TABLE 200: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X02 SUBCLASS 0X01 VIN COMMAND DEFINITION

VIN [0] Virtual Wire Definition									
		Bit 7	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power	0x01	Mappable Fault	TEMP_ALERT	3V_AUX_Fault	Main Fault	RSVD	Command		
Cable Detect	0x02	CBL_DET_D	CBL_DET_C	CBL_DET_B	CBL_DET_A	RSVD	Command		
Configuration	0x03	V RSVD	EEPROM_WP 1 = ON 0 = OFF	V RSVD	V RSVD	RSVD	Command		
Location Index	0x04	Location[3:0]				RSVD	Command		
Vendor Defined	0x05						Command		
Vendor Defined	0x06						Command		
Vendor Defined	0x07						Command		

12.17.5.3.7. Primary Control Panel Device Class 0x05 Subclass 0x00 Command List and Definition

The Primary Control Panel virtual wire command list and definitions are outlined in the tables below.

Control Panel Transfer Type 1 Single Byte Definition

TABLE 201: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X05 SUBCLASS 0X00 COMMAND DEFINITION

Command	Command Code (Nibble)	Req?	Description
Echo	0x00	Y	echo returns the same pattern
Power Button and LED	0x01	Y	Power button state and power LED control / status
Health LED	0x02	Y	health LED control / status
Configuration	0x03	Y	
Location	0x04		Location index for where IO device is positioned in a specific box or section within the chassis. Selects what value is returned by the target. Location_Nibble_Index[3:0]: Index 0: Box Lower Nibble Index 1: Box Upper Nibble Index 2: Section Lower Nibble Index 3: Section Upper Nibble 4 – 15 reserved
UID Control and LED	0x05	Y	UID button and LED control / status
USB Control	0x06		Not required if USB is not supported by the control planel.
Vendor Defined	0x07		

OUT = From HPM To Peripheral

This set of virtual wires are sent to the primary control panel by the HPM.

TABLE 202: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X05 SUBCLASS 0X00 VOUT COMMAND DEFINITION

VOUT [0] Command Definition									
Command	**Command Code (Nibble)**	**Bit 7**	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power Button and LED	0x01	RSVD	RSVD	Power LED[2:0] Three Color LED PowerLED[0] = Green 0 = OFF 1 = ON PowerLED[1] = Amber 0 = OFF 1 = ON PowerLED[2] = Red 0 = OFF 1 = ON			Command		
Health LED	0x02	RSVD	RSVD	Health LED[2:0] Health LED[0] = Green 0 = OFF 1 = ON Health LED[1] = Amber Health LED[2] = Red			Command		
Configuration	0x03	RSVD	RSVD	RSVD	RSVD	RSVD	Command		
Location Index	0x04	See Location Definition For Details					Command		
UID Control and LED	0x05	RSVD	RSVD	RSVD	RSVD	UID LED Control 0 = Off 1=On	Command		
USB Control	0x06	RSVD	RSVD	RSVD	RSVD	RSVD	Command		
Vendor Defined	0x07						Command		

IN = From Peripheral To HPM

TABLE 203: MPESTI VIRTUAL WIRE PAYLOAD FORMAT TYPE 1 DEVICE 0X05 SUBCLASS 0X01 VIN COMMAND DEFINITION

VIN [0] Command Definition									
Command	**Command Code (Nibble)**	**Bit 7**	**Bit 6**	**Bit 5**	**Bit 4**	**3**	**2**	**1**	**0**
Echo	0x00	Echo[3:0]				Heartbeat	Command		
Power Button and LED	0x01	RSVD	Power LED[2:0] State			Power Button IN 0=not pressed 1= button pressed	Command		
Health LED	0x02	RSVD	Health LED[2:0] State			RSVD	Command		
Configuration	0x03	RSVD	RSVD	Alert 0 not asserted 1= asserted	SB3_STATE Current state of sideband 3	SB2_STATE Current state of sideband 2	Command		
Location Index	0x04	See Location Definition For Details					Command		
UID Control and LED	0x05	RSVD	RSVD	RSVD	UID LED State	UID Button IN 0=not pressed 1= button pressed	Command		
USB Control	0x06	RSVD	RSVD	USBC_PD_DET USBC PD Controller Device Detection 0 = No Device 1 =Device Detected	USBC_PD_FAULT_N USBC PD Controller Fault Detection 0 = Fault 1 = No Fault	USBC_PD_I2C_IRQ_N USBC PD I2C Interrupt 0 = Asserted 1 = Not Asserted	Command		
Vendor Defined	0x07						Command		

12.17.5.3.8. Unspecified Peripherals Usage Of Format Type 1

All other peripheral usage of Virtual Wires Format Type 1 is vendor defined and optional.

12.17.6. M-PESTI Target IP Block CSR Requirements

This section contains the Control and Status Registers (CSR) requirements for standardized M-PESTI Target functionality.

12.17.6.1. M-PESTI Target CSR Region Overview

The M-PESTI Target CSR regions are divided into a set of ranges. These ranges are all offsets from the MPESTI_TARGET_IP_BLOCK base address. This base address is defined by the CPLD/PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 204: M-PESTI TARGET CSR MEMORY MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x00FF	Global Registers	General IP block level registers
1	0x0100	0x0BFF	M-PESTI Target IP Block Interface Control and Status Registers	M-PESTI Target IP Block Control and Status Registers.
2	0x0C00	0x0FFF	Reserved	Reserved for future use
3[24]	0x1000	0xFFFF	OEM SECTION	Vendor defined usage.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as

[24] Note: Implementor can use this for additional regions as needed.

needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.17.6.2. M-PESTI Target IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 205: MPESTI TARGET REGION 0 MEMORY MAP

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

12.17.6.2.1. M-PESTI Target IP Block Common Registers Sub-Region 0 Description

The table below contains the definition for the M-PESTI Target IP block common registers sub-region 0.

TABLE 206: MPESTI TARGET REGION 0 SUBREGION 0 MEMORY MAP

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each).

0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state indication that requires a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07 – 0x0F	Reserved			

12.17.6.2.2. M-PESTI Target IP Block Common Registers Sub-Region 1 Description

Reserved

12.17.6.3. M-PESTI Target IP Block Region 1 CSR Memory Map

The M-PESTI target region one registers start at offset 0x100 from the base address for M-PESTI target CSR space.

This region is divided into two subregions.

TABLE 207: M-PESTI TARGET REGION 1 MEMORY MAP

Sub-Region	Range	Name	Description
0	0x0100 – 0x07FF	M-PESTI target CSRs	Set of registers that handle M-PESTI target functions.
1	0x0800 – 0x0FFF	Reserved	Set of registers reserved for future use.

It is required that the PD implementor specifies within the JSON file which ‘sub-regions are present and supported.

12.17.6.3.1. M-PESTI Target CSRs Region 1 Sub-Region 0 Description

The table below contains the definition for the M-PESTI Target IP block CSRs region 1 sub-region 0.

Definition TBD.

12.17.6.3.2. M-PESTI Target IP Block CSRs Region 1 Sub-Region 1 Description

Reserved

12.17.6.4. M-PESTI Target IP Block Region 2 CSR Memory Map

Reserved

12.17.6.5. M-PESTI Target IP Block Region 3 CSR Memory Map

Reserved for vendor defined usage.

12.17.7. M-PESTI Target Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.17.8. M-PESTI Target Example Implementation

Example implementations, if available, may be found in supplementary materials referenced in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

12.18. Reliability Availability and Serviceability (RAS) IP Block

This section contains the Reliability Availability and Serviceability (RAS) IP block description for a specialized interface with one or more SOCs. This entire IP block is fully optional.

12.18.1. Reliability Availability and Serviceability (RAS) IP Block Overview

This section contains an overview of the Reliability Availability and Serviceability (RAS) functionality for a typical set of use cases.

The intention of the Reliability Availability and Serviceability (RAS) IP block is to provide a black box style interface that connects specific physical PD signals on the HPM to the SoC complex that the BMC can directly manipulate. This specification does not define the specific functions and signals that this IP block will implement. Rather, that definition will come from the HPM vendor and will be documented in the JSON file due to the differences between SoCs and SoC vendors.

12.18.2. Reliability Availability and Serviceability (RAS) IP Block Architecture

This section contains the general abstract architecture for the Reliability Availability and Serviceability (RAS) IP block.

The diagram below contains a conceptual diagram of how the Reliability Availability and Serviceability (RAS) block interfaces with the HPM SoC. The SoC vendor in this figure is meant as an example for illustration purposes only.

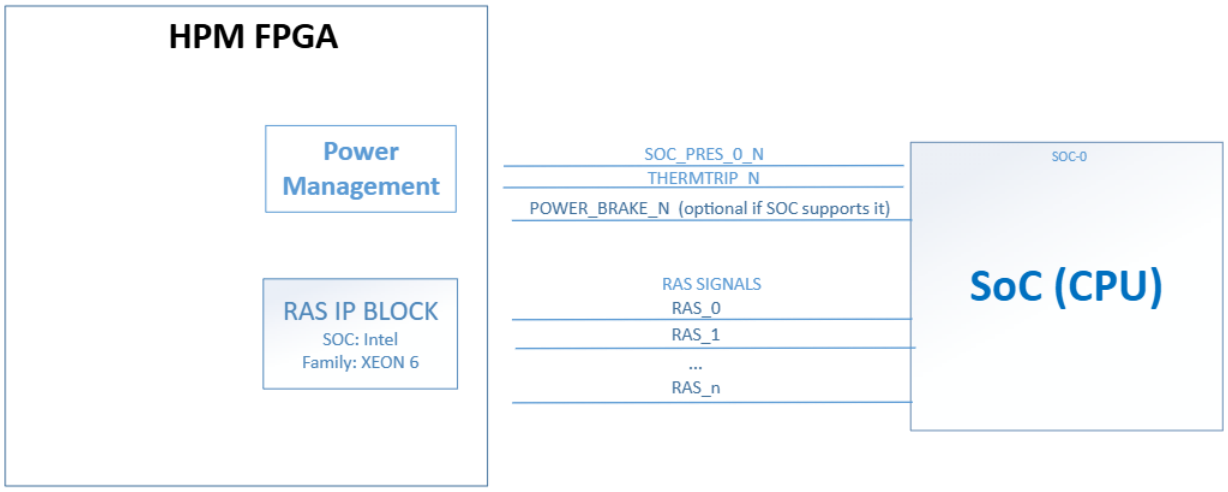


FIGURE 65: RAS IP BLOCK CONCEPTUAL ARCHITECTURE

The register set inside the IP block would support a set of RAS signals with vendor defined functionality.

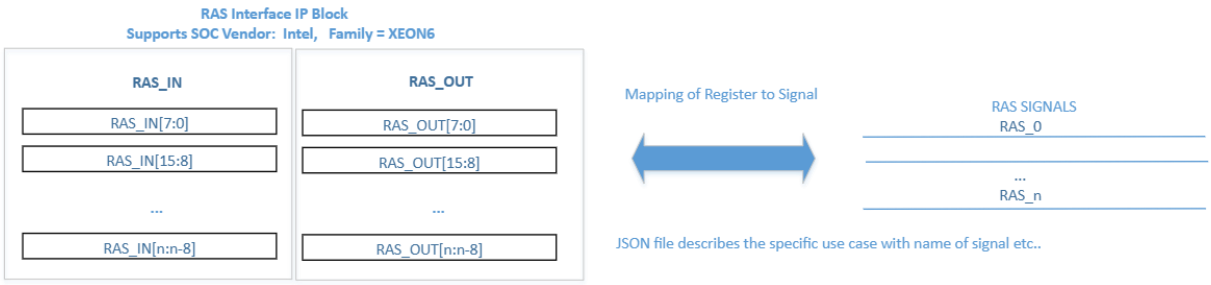


FIGURE 66: RAS IP BLOCK CONCEPTUAL EXAMPLE

As shown in the figure above, the Reliability Availability and Serviceability (RAS) IP block contains a set of registers that are mapped to one or more physical signals. The definition of those registers and the interface signals to the SoC shall be vendor defined. The SoC vendor mentioned in this figure is meant as an example for illustration purposes only.

12.18.3. Reliability Availability and Serviceability (RAS) IP Block Requirements

This section contains the general abstract architecture for the Reliability Availability and Serviceability (RAS) IP block.

12.18.3.1. Required and Optional Feature Set

As this IP block is fully vendor defined, specific feature requirements are outside the scope of this specification.

12.18.4. Reliability Availability and Serviceability (RAS) IP Block CSR Requirements

This section contains the Control and Status Registers (CSR) requirements for the Reliability Availability and Serviceability (RAS) IP Block functionality.

12.18.4.1. Reliability Availability and Serviceability (RAS) IP Block Region Overview

The Reliability Availability and Serviceability (RAS) IP Block CSR regions are divided into a set of ranges. These ranges are all offset from the Reliability Availability and Serviceability (RAS) IP Block base address. This base address is defined by the CPLD/PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 208: RAS IP BLOCK MEMORY MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x00FF	Global Registers	General IP block level registers
1	0x0100	0xFFFF	Reliability Availability and Serviceability (RAS) Control and Status Registers	Specific vendor defined registers used for SoC RAS functionality. This shall be defined in the JSON file.
2^{25}	0x1000	0xFFFF	OEM SECTION	Optional region that contains vendor defined usage.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.18.4.2. Reliability Availability and Serviceability (RAS) IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

[25] Note: Implementor can use this for additional regions as needed.

TABLE 209: RAS IP BLOCK REGION 0 MEMORY MAP

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.18.4.2.1. Reliability Availability and Serviceability (RAS) IP Block Common Registers Sub-Region 0 Description

The table below contains the definition for the Reliability Availability and Serviceability (RAS) IP block common registers sub-region 0.

TABLE 210: RAS IP BLOCK REGION 0 SUBREGION 0 MEMORY MAP

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each).
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.

0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state indication that requires a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	R/W	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x08 – 0x0F	Reserved			Reserved for future use.

12.18.4.2.2. Reliability Availability and Serviceability (RAS) IP Block Common Registers Sub-Region 1 Description

Reserved

12.18.4.3. Reliability Availability and Serviceability (RAS) IP Block Region 1 CSR Memory Map

The Reliability Availability and Serviceability (RAS) IP block region one registers shall start at offset 0x100 from the base address for General Use PD GPIOs CSR space.

This region is divided into two subregions.

TABLE 211: RAS IP BLOCK REGION 1 MEMORY MAP

Sub-Region	Range	Name	Description
0	0x0100 – 0xFFFF	RAS CSRs	Set of registers that handle Reliability Availability and Serviceability (RAS) functions.

It is required that the PD implementor specifies within the JSON file which 'sub-regions are present and supported.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.18.4.3.1. Reliability Availability and Serviceability (RAS) CSRs Region 1 Sub-Region 0 Description

The Reliability Availability and Serviceability (RAS) region 1 sub-region 0 is fully vendor defined. The vendor shall describe the register set and signal interconnection in the JSON file if this IP block is present.

12.18.4.4. Reliability Availability and Serviceability (RAS) IP Block Region 2 CSR Memory Map

Reserved for vendor defined usage.

12.18.5. Reliability Availability and Serviceability (RAS) Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.18.6. Reliability Availability and Serviceability (RAS) IP Block Example Implementation

Example implementations, if available, may be found in supplementary materials referenced in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

12.19. Universal Serial Bus (USB) IP Block

This section contains the Universal Serial bus (USB) IP block description for a bus. This entire IP block is fully optional.

12.19.1. Universal Serial Bus (USB) IP Block Overview

This section contains an overview of the Universal Serial bus (USB) IP block functionality for a typical set of use cases. It is required that the USB IP block support is limited to a single target USB 2.0 endpoint.

The following use cases for this interfaces are as follows:

- Support for connection to random access PD memory interface
- Bridging to interfaces for tunneling purposes such as I2C, UART, etc.
- Generalized GPIO tunneling support

This is not an exhaustive list. Other use cases are possible and left to the vendor to support.

12.19.2. Universal Serial Bus (USB) IP Block Architecture

This section contains the general abstract architecture for the Universal Serial bus (USB) IP block.

The diagram below contains a conceptual diagram of how the Universal Serial bus (USB) block may be used within a given system design.

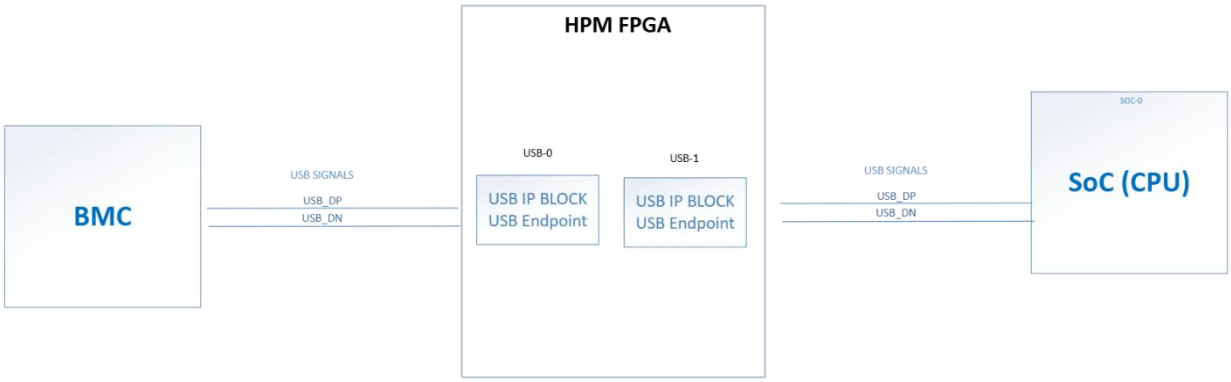


FIGURE 67: USB IP BLOCK CONCEPTUAL ARCHITECTURE

As shown in the above figure, the implementor can include multiple USB endpoint IP blocks to interface with different USB bus masters. This implementation is shown as an example only and not required.

The register set implemented by the IP block shall support a single USB endpoint. A vendor may implement vendor defined functionality using the extended register set space as outlined in the CSR requirements section. The Universal Serial bus (USB) IP block contains a set of registers that implement USB endpoint support as per the USB 2.0 specification (see specification for details).

12.19.3. Universal Serial Bus (USB) IP Block Requirements

This section contains the general abstract architecture for the Universal Serial bus (USB) IP block.

12.19.3.1. Required and Optional Feature Set

As this IP block is fully vendor defined, specific feature requirements are outside the scope of this specification.

12.19.4. Universal Serial Bus (USB) IP Block CSR Requirements

This section contains the Control and Status Registers (CSR) requirements for the Universal Serial Bus (USB) IP Block functionality.

12.19.4.1. Universal Serial Bus (USB) IP Block Region Overview

The Universal Serial Bus (USB) CSR regions are divided into a set of ranges. These ranges are all offset from the Universal Serial Bus (USB) IP block base address. This base address is defined by the CPLD/PD developer and is not prescribed here. It is required that the JSON file contains the base address which shall be defined as an IP Block region within the memory map.

TABLE 212: USB IP BLOCK MEMORY MAP

Region	Range Bottom	Range Top	Name	Description
0	0x0000	0x00FF	Global Registers	General IP block level registers
1	0x0100	0x0FFF	Universal Serial Bus (USB) IP block Control and Status Registers	Industry standard register set. This shall be defined in the JSON file.
2^{26}	0x1000	0xFFFF	OEM SECTION	Optional region that contains vendor defined usage.

It is required that the register range ordering is preserved according to this specification.

[26] Note: Implementor can use this for additional regions as needed.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.19.4.2. Universal Serial Bus (USB) IP Block Region 0 Common Registers Memory Map

This region is divided into two subregions.

TABLE 213: USB IP BLOCK REGION 0 MEMORY MAP

Sub-Region	Range	Name	Description
0	0x00 – 0x0F	GLOBAL REQUIRED	Set of required registers that globally apply to all targets and IP block
1	0x10 – 0xFF	GLOBAL RESERVED	Save enough space if more registers are needed

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.19.4.2.1. Universal Serial Bus (USB) IP Block Common Registers Sub-Region 0 Description

The table below contains the definition for the Universal Serial Bus (USB) IP block common registers sub-region 0.

TABLE 214: USB IP BLOCK REGION 0 SUBREGION 0 REGISTER LIST

Offset Address	Name	Width	ATTR	Description
0x00	IP Block ID	2 B	R	Unique ID to identify the IP block.
0x02	IP Block Version	2 B	R	Major.Minor (1 byte each).
0x03	Global Status	1B	RW	Common status register. See section 12.1.1.4, “Required IP Block CSR: Status”, for details.
0x04	Global Control	1B	RW	Common control register. See section 12.1.1.5, “Required IP Block CSR: Control”, for details.
0x05	IP Block Interrupt Status	1 B	RW	Interrupt state indication that requires a write to clear. Bit 0: Interrupt Status See section 12.1.1.6, “Required IP Block CSR: Interrupt Status”, for details.
0x06	IP Block Interrupt Control	1 B	RW	Enable / Disable interrupt generation. Bit 0: Interrupt Enable See section 12.1.1.7, “Required IP Block CSR: Interrupt Control”, for details.
0x07	Extended Status	1B	R/W	Optional vendor defined status register. See section 12.1.1.8, “Required IP Block CSR: Extended Status”, for details.
0x08 – 0x0F	Reserved			Reserved for future use.

12.19.4.2.2. Universal Serial Bus (USB) IP Block Common Registers Sub-Region 1 Description

Reserved

12.19.4.3. Universal Serial Bus (USB) IP Block Region 1 CSR Memory Map

The Universal Serial Bus (USB) IP block region one registers shall start at offset 0x100 from the base address for General Use PD GPIOs CSR space.

This region is divided into two subregions.

TABLE 215: USB IP BLOCK CSR MEMORY MAP

Sub-Region	Range	Name	Description
0	0x0100 – 0x7FFF	USB CSRs	Set of registers that handle Universal Serial Bus (USB) functions.
1	0x8000 – 0xFFFF	OEM CSRs	Vendor defined

It is required that the PD implementor specifies within the JSON file which ‘sub-regions are present and supported.

It is required that the register range ordering is preserved according to this specification.

Note: All register space sizes and offsets are examples only. The values shown are not required register offsets and ranges. The vendor may resize the ranges and offset locations as needed. The PD JSON file shall contain the register offsets and ranges for a given implementation.

12.19.4.3.1. Universal Serial Bus (USB) CSRs Region 1 Sub-Region 0 Description

The USB IP Block region 1 sub-region 0 shall implement the required register to support a single USB 2.0 endpoint. The vendor shall describe this register set and signal interconnection in the JSON file if this IP block is present.

12.19.4.4. Universal Serial Bus (USB) IP Block Region 2 CSR Memory Map

Reserved for vendor defined usage.

12.19.5. Universal Serial Bus (USB) Vendor Specific Extensions Support

Vendors are allowed to add extensions to this IP block as needed by using OEM reserved regions for CSRs.

It is required that any vendor specific features and extensions shall be described in the PD JSON file for BMC consumption if a vendor requires plug-and-play support. See PD JSON file description in chapter 10, “PD JSON Schema Representation”, for details.

12.19.6. Universal Serial Bus (USB) IP Block Example Implementation

Example implementations, if available, may be found in supplementary materials referenced in section 7.4, “MHS HPM PD Specification Supplementary Materials”.

13. HPM PD Security Requirements and Considerations

This section outlines the PD security requirements and vendor implementation considerations. This includes attestation and authentication of the PD and HPM connected components via typical communication interfaces using industry standard mechanisms such as SPDM.

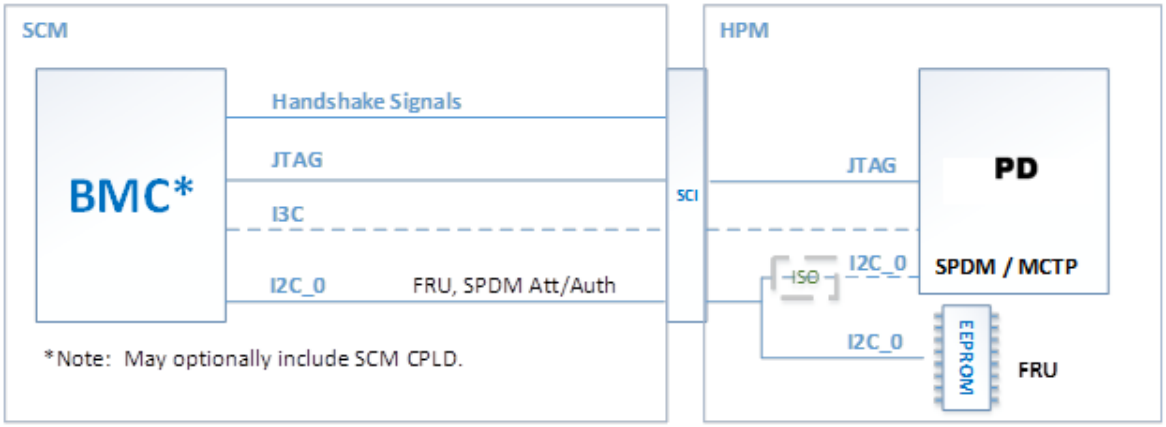


FIGURE 68: I2C / I3C INTERCONNECTION EXAMPLE FOR SPDM SUPPORT

HPM vendors that support additional security features, such as SPDM, may utilize the optionally connected I2C or I3C bus for this function or another bus that supports the SPDM protocol.

Common use cases for SPDM are to attest and authenticate the PD, and transitively, the HPM. The usage of SPDM can assist in the authentication of the HPM FRU EEPROM contents. This is outlined in the discovery flow process as described in the "M-PNP Discovery, Boot, and FRU Specification". Please consult that specification for further details.

14. PD Vendor Extensibility Requirements and Considerations

This section provides guidelines and considerations for PD vendor defined extensibility to the existing M-PNP framework. This framework allows HPM vendors to leverage the descriptive capabilities of the JSON self-describing data to describe custom IP blocks and associated CSRs for BMC firmware consumption. Additionally, vendor specific virtual wires can be defined and used for transport or IP block usage.

15. HPM PD General Requirements and Considerations

This section outlines the general PD requirements and considerations for the HPM vendor. This includes PD image considerations, JTAG requirements, and any other considerations for the HPM vendor.

15.1. PD Image Considerations and Requirements

This section provides guidance to HPM vendors regarding the PD image.

It is required that the PD provides a JSON self-describing file typically created on PD build time. This file is fundamental to the M-PNP plug-and-play framework and is necessary for BMC consumption. It is optional but recommended that this file also be available and located within the PD memory map and accessible and referenced via the PD memory map directory.

For details regarding the contents of this file, please consult the section, "PD JSON Schema Representation".

15.1.1. PD Self-Describing JSON File Considerations

It is required that the PD image contains the PD self-describing JSON file. However, to reduce the memory footprint needed to contain such a file, the PD may keep a series of “patch files” that provides the differences between JSON files from one image version to another. For this to work, the BMC would need to keep a copy of the initially factory programmed JSON file in immutable storage to use a baseline for use in applying patch files to re-create the JSON file that represents the current PD version.

It may store the initially programmed PD JSON file within the HPM FRU information device to assist in this effort. Thus, subsequent programming of the PD may store these patch files in PD memory for collection by the BMC during the discovery phase.

The figure below provides an example diagram for storage accessibility of PD JSON file patch files.

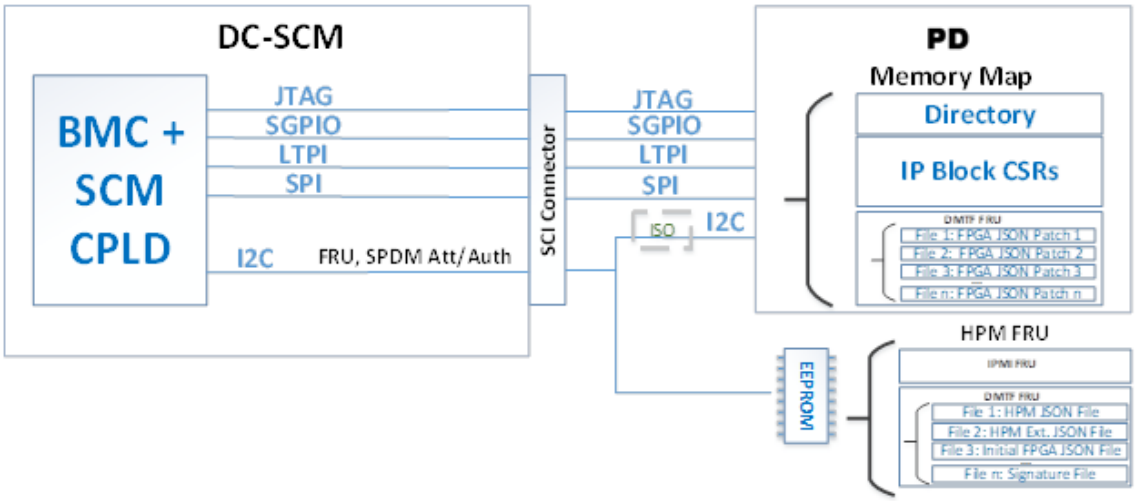


FIGURE 69: EXAMPLE PD JSON PATCH OPTIONAL DELIVERY

See M-PNP.FRU_DISCOVERY_BOOT specification for further details regarding usage of JSON files during the discovery process.

15.2. PD Programming Requirements

It is required that the HPM vendor connect JTAG or other early access interface for PD image manipulation as well as collection of key PD data such as image version, component hardware model, and component hardware manufacturing information. This information is needed as part of the discovery process. This process is outlined in the "M-PNP Discovery, Boot, and FRU Specification". Please consult that specification for more details.

The table below outlines the required information needed for the discovery process collected via JTAG or other early access interface.

TABLE 216: PD REQUIRED INFORMATION

Name	REQ	OPT	Description
Version	Y		Image version of the PD.
Manufacturer	Y		Name of manufacturer of PD silicon hardware.
Model	Y		Model name of the PD silicon hardware.

Part Number	Y		Part Number of the PD silicon hardware.
Silicon Version	Y		Silicon stepping version of the PD hardware.

It is required that whatever early access interface used to collect this data shall NOT depend on the PD running any user generated image data or code.

The DC-SCM 2.1 and HPM common circuits specifications describe the electrical requirements for the JTAG connection between SCM and HPM. Please consult those specifications for more details.

15.3. Electrical Considerations

This section covers all electrical considerations for the PD.

Please consult DC-SCM 2.1 and the HPM common circuits specifications for further electrical requirements.

16. Specification Compliance Checklist

The following table is intended to summarize the list of attributes and requirements for a design to be MHS HPM PD Specification compliant. Compliance requirements throughout this document can be identified by underlined text including the word “shall”.

TABLE 217: COMPLIANCE CHECKLIST

#	Technical Specification	Document Reference
1		

17. References

This specification relies on the following specifications:

- OCP DC-SCM 2.1 Specification
- OCP LTPI 2.1 Specification
- OCP HPM Common Circuits Type 1 Specification
- OCP MHS M-PNP Discovery, Boot, and FRU Specification
- OCP MHS M-PIC Specification
- OCP MHS M-XIO Specification
- OCP MHS M-FLW Specification
- OCP MHS M-DNO Specification
- OCP MHS M-CRPS Specification
- OCP MHS M-PESTI Specification

18. Trademarks

Names and brands may be claimed as trademarks by their respective companies. I3C is a trademark of MIPI Alliance. PMBus name and logo are trademarks of SMIF, Incorporated. PCIe® and PCI Express® are the registered trademarks of PCI-SIG.

19. Future Material

Note: This section is for reference only and intends to move into the main document body when further refined by the workgroups after Release 1.0. The implementation is highly subject to change.